ERPCLEAN2 — User Manual

Modular-monolith ERP — Spring Boot · Angular · PostgreSQL

Contents

Getting Started

- Signing In
- The App Layout
- The Branch Switcher
- How Permissions Shape What You See
- Common UI Patterns
- Signing In as rootadmin

Administration and Access

- Organisation, Companies, and Branches
- Users
- Roles
- Assigning Companies to Users
- Assigning Roles to Users
- Assigning Branches to Users
- Sales Settings — sales-order approval and stock control
- Fiscal / EFD Configuration
- Audit Trail
- Key Concepts Reference

Master Data

- Customers
- Suppliers
- Other Parties
- Sales Agents
- Products
- Units of Measure
- Price Lists
- Pricing Rules
- Currencies and FX Rates
- Tax Rates
- Distribution Routes
- Bulk Import — mass create & update from Excel

Sales and Point of Sale

- Overview
- 1. Quotations
- 2. Sales Orders
- 3. Deliveries
- 4. Sales Invoices

- 5. Sales Returns (RMA)
- 6. Blanket Orders
- 7. Standing Orders (Recurring)
- 8. Pricing Rules
- 9. Point of Sale
- 10. Sales analysis reports

Procurement (Procure-to-Pay)

- Overview
- 1. Purchase Requisitions
- 2. RFQ (Request for Quotation)
- 3. Purchase Orders
- 4. Goods Receipt
- 5. Landed Costs
- 6. Supplier Bills and 3-Way Bill Match
- 7. Purchase Returns
- 8. Purchase Settings
- 9. End-to-end procure-to-pay example
- 10. Purchase Reports

Inventory and Manufacturing

- 1. Permissions quick reference
- 2. Stock on-hand
- 3. Stock locations
- 4. Stock transfers
- 5. Stock counts
- 6. Van-stock reconciliation
- 7. Batches and lot tracking
- 8. Serial number tracking
- 9. Inventory valuation
- 10. Bills of Materials
- 11. Work Orders
- 12. WIP reconciliation
- 13. Frequently asked questions

Fixed Assets

- 1. Permissions quick reference
- 2. Asset categories
- 3. Asset register
- 4. Placing an asset in service
- 5. Transferring an asset

- 6. Depreciation
- 7. Revaluing an asset
- 8. Disposing of an asset
- 9. FA to GL reconciliation
- 10. Frequently asked questions

Projects

- 1. Permissions quick reference
- 2. Project lifecycle
- 3. Creating a project
- 4. Project detail
- 5. Project tasks
- 6. Timesheets
- 7. Issuing materials to a project
- 8. Project P&L
- 9. Cross-project WIP report
- 10. Frequently asked questions

Finance & Accounting

- General Ledger
- Accounts Receivable
- Accounts Payable
- Cash & Bank
- Tax
- Foreign Exchange (FX)

CRM — Customer Relationship Management

- Leads
- Opportunities
- Pipeline Dashboard
- Pipeline Stages (Settings)
- Activities

Reporting and Business Intelligence

- Financial Statement Reports
- Business Intelligence Dashboard
- Analytical Reports

HR & Payroll, Budgeting, and Platform Services

- Part 1 — Human Resources and Payroll
- Part 2 — Budgeting
- Part 3 — Platform Services

Getting Started

Welcome to the ERP system. This chapter explains how to sign in, find your way around, and use the common patterns you will meet on every screen.

Signing In

What signing in is. Authentication is the process of proving your identity to the system. You present a username and password; the system checks them, issues you a short-lived access token (a cryptographically signed credential, invisible to you), and loads your personal session. Everything you do after this is recorded against your identity.

Why it exists. Without authentication, anyone who could reach the URL could read, create, or delete business data. The system needs to know *who* you are before it can decide *what* you are allowed to do.

When it happens. Every time you open the application in a fresh browser tab, after your session expires (timeout is set by your administrator), or after you sign out.

How it works. On successful sign-in the system reads your default branch assignment and resolves your effective permissions for that branch. The main menu is built from those permissions — screens you cannot access are never shown. The access token is refreshed automatically in the background; when it cannot be refreshed (for example, your account was disabled mid-session) you are returned to the login page.

1. Open your browser and navigate to the URL your administrator gave you (for example, `http://erp.yourcompany.com`). The login page appears automatically.
2. Enter your **username** in the first field. Usernames are not case-sensitive.
3. Enter your **password** in the second field.
4. Click **Sign in**.

If your credentials are correct you are taken straight to the main dashboard. The system reads your assigned permissions and builds your personal menu — you will only see the sections you are allowed to use.

Sign-in problems

What you see	What to do
"Invalid username or password."	Check your username and password. The message is the same whether the username or the password is wrong — this is intentional.
"Account is locked. Try again later or contact an administrator."	Your account was locked after too many failed attempts. Ask an administrator to unlock it for you.
"Your session has expired. Please sign in again."	Your session timed out. Sign in again. Any unsaved work will be lost.

> Your account is automatically locked for 15 minutes after 5 consecutive wrong-password attempts. A successful sign-in resets the counter.

Signing out

Click your name or initials in the top-right corner of the screen to open the account menu, then click **Sign out**. (On wider screens a dedicated **Sign out** icon button is also shown in the top bar, so you can sign out in one click.) You are returned to the login page and your session is ended immediately.

The App Layout

Once signed in you see three main areas.

Top bar

The horizontal bar across the top of every screen contains:

- **Brand / logo** on the left.
- **Active branch indicator** — a chip showing the name of the branch you are currently working in (with its branch code shown beside the name on wider screens). Click it to switch to a different branch if you are assigned to more than one (see the Branch Switcher section below). If you have no assigned branch the chip reads **No branch**.
- A small coloured dot showing whether the system service is reachable (the indicator reads **API: UP** in green when healthy).
- **Your name / initials** on the right. Click to open the account menu, which shows your display name, your **@username**, a **Root administrator** badge if you are signed in as `rootadmin`, and a **Sign out** action. On medium and larger screens a separate **Sign out** icon button also sits in the top bar for one-click sign-out.

Sidebar navigation

A dark slate panel on the left (or opened by the menu icon on small screens) groups all available screens by business area — for example **Administration**, **Sales**, **Inventory**, **Accounting**, and so on — with each screen listed under its group heading. Click a screen name to navigate there.

The sidebar is **personalised** — items you do not have permission to see are simply not shown, and a group whose every item is hidden disappears entirely. If you cannot find a screen you expect, you probably lack the required permission. Contact your system administrator. (If a screen is part of the system but not yet released, it appears greyed-out with a small **soon** badge instead of a working link.)

Press **Escape** at any time to close the sidebar on a small screen.

Main content area

The large area to the right of the sidebar is where each screen loads. The current route is reflected in the browser address bar.

The home page

When you sign in — or are redirected after trying to open a screen you cannot access — you land on the **home page** (`/admin/home`).

What you see depends on your account:

- **System administrators (`rootadmin`)** see a **System setup** panel — its heading reads "System setup", under the page subtitle "System administrator — configure the platform below to get started." It presents an ordered set of configuration steps as cards — *1 Companies & branches* → *2 Roles & permissions* → *3 Users* → *4 Audit log* — for standing the platform up. Each card links straight to that area.
- **Everyone else** sees a personalised **launchpad**: the page subtitle reads "Quick links to your most-used screens," and below it a **Quick launch** grid shows one card per screen you actually hold the permission to open, drawn from a curated set spanning the dashboard, sales, purchasing, stock, finance, and manufacturing (for example Dashboard, Sales orders, POS sell, Stock on-hand, Post journal). Each card is checked against the exact permission its target screen requires, so a card you can see is always a card you can open — there is no dead-end tile. The **Dashboard** card is always shown first when you hold `BI.VIEW`. If none of the curated destinations are open to you, you instead see a brief, calm placeholder ("Your workspace is ready.") prompting you to use the menu on the left. Either way, the home page does **not** repeat the full sidebar menu — the sidebar remains the complete map of everywhere you can go.

The home page never requires a permission and never loads business data, so it is always safe to land on; this is why the system uses it as the silent redirect target for screens you cannot access.

The Branch Switcher

What a branch is. A branch is a physical or logical operating unit within a company — for example, a shop, a warehouse, a regional office, or a cost centre. Every transaction you create is stamped with the branch you were working in at the time.

Why branch switching exists. A single user may work across multiple locations or departments. Rather than logging in and out with different accounts, you stay logged in and tell the system which branch context to use for your current task. The system then shows you data scoped to that branch and enforces the permissions that apply there.

When it matters. On login the system activates your **default branch** automatically. If you are assigned to more than one branch, you can switch before performing a transaction to ensure it is recorded in the correct location.

How it works. Switching branches does not re-issue your login token. Instead the system records your active branch choice and attaches it to every subsequent request. Your effective permissions are re-resolved for the new branch's company on the very next call — permissions can differ between branches if your roles are scoped differently.

- On login the system activates your **default branch** automatically.
- If you are assigned to more than one branch, click the branch chip in the top bar to open a dropdown. Click any branch in the list to switch to it. The list shows only your active, assigned branches; each row shows the branch **name** with its short branch **code** beside it (the code is a human-readable identifier, never a raw internal identifier).
- Switching branches takes effect immediately; your permissions and the data you see may change depending on your role grants.
- Selecting the branch you are already in is a no-op — the menu simply closes.

> If you have no active branch (for example, your only branch was removed or archived), you will be in a read-only state. Contact your administrator to be assigned to an active branch.

How Permissions Shape What You See

What permissions and roles are. A permission is a named capability — for example, `USER.MANAGE` or `GL.POST`. A role is a named bundle of permissions: for instance, an "Accountant" role might bundle all the GL, AR, AP, and cash permissions that a typical accountant needs. Users are not assigned permissions directly; they are assigned roles, and those roles carry the permissions.

Why this model exists. Assigning individual permissions to each user does not scale when you have dozens of staff and hundreds of capabilities. Roles let you define a job function once and then grant or revoke it from any number of people in a single action. It also makes auditing straightforward: you can read a role's permission set and know exactly what every holder can do.

When permissions are evaluated. Every time you navigate to a screen or perform an action, the system checks your effective permissions for your active branch. If you switch branches, your permissions are re-checked against the roles you hold in the new branch's context.

How it works. Your administrator creates roles, assigns specific permissions to each role, and then grants those roles to you scoped to a company (and optionally a specific branch). The system computes the union of all permissions from all your active role grants in the current branch's context.

- **Nav items** you lack permission for are hidden entirely — you will not see a greyed-out item, just no item at all.
- If you type a URL directly for a screen you cannot access, the system redirects you to the home page quietly.

- If you are on a screen and try to perform an action you lack permission for, the system declines it without an alarming pop-up — the relevant part of the screen shows a calm "no permission" message instead. (See "When something goes wrong" under Common UI Patterns.)

The special account **rootadmin** is the system superuser and sees every screen and every action regardless of role assignment. This account is used by IT administrators only.

Common UI Patterns

Resource pickers — choosing a related record by name

Throughout the system, whenever one record links to another (for example, a sales order links to a customer, a journal line links to an account, or a user is assigned to a branch), you choose the related record **by name** from a list. You **never type a code or an internal identifier**. This applies everywhere — fiscal year, customer, supplier, product, account, branch, agent, task, and so on.

The picker is a dropdown labelled with what you are choosing (for example **Customer**):

1. Click or focus the picker field.
2. Pick the matching item from the dropdown list.
3. For longer lists (more than 12 options) a **"Type to filter by name…"** box appears automatically above the dropdown — type part of the name to narrow the list, then pick the item.

Because raw identifiers are never accepted as input, you cannot pick a record that does not exist or mistype an identifier. Internal identifiers (the long machine codes the system uses behind the scenes) are not shown anywhere in the interface; records are always presented by their human-readable name, code, or a link.

The Currency Picker — choosing a currency on a form

Wherever a form asks you for a currency — for example when entering a supplier bill, recording a customer receipt, setting an opening balance, raising a credit note, creating a sales order, invoice, quotation, blanket or standing order, ringing up a POS sale, raising a purchase order, setting up pricing tiers or a customer price, entering a product's cost or selling price, an opportunity value, an HR loan, a customer credit limit, or purchase settings — you choose it from a **Currency Picker**, not by typing a 3-letter code.

The Currency Picker is a dropdown that lists currencies as **"CODE — Name"** (for example, `TZS — Tanzanian Shilling`, `USD — US Dollar`):

- It shows **only the currencies enabled for your company** (and, where relevant, your branch). You cannot choose a currency that has not been enabled.
- It **defaults to your company's default document currency**, so for everyday single-currency work you can simply leave it as-is.
- If the list is long, a small filter box lets you type part of the code or name to narrow it.

The set of enabled currencies and the default document currency are configured by your administrator (per company, and optionally per branch). The **base currency** — the home currency your accounts are kept in — is set once for the company and **cannot be changed once any journal entries exist**, so plan it carefully at setup time. If you ever need a currency that is not in the list, ask your administrator to enable it.

> You will never be asked to "type a 3-letter currency code" or told that a field "defaults to TZS" — those are out of date. Always pick the currency from the dropdown.

List screens — search and pagination

Every list screen (for example, **Sales Orders** or **Users**) behaves the same way:

- A **search** or filter bar at the top lets you narrow results by keyword, status, date range, or other criteria relevant to that list.
- A **pager** at the bottom shows page numbers and **First / Previous / Next / Last** controls. If all results fit on one page the pager hides itself.
- Column headings may be clicked to sort the list (where supported).

Using the system on a phone or tablet

The system is usable on a phone or tablet, not just a desktop browser. On a narrow screen the sidebar becomes the slide-out menu described above, and wide transaction list tables (for example Sales Orders, Purchase Orders, Customers, or Products) scroll horizontally within their own frame rather than squeezing every column to fit. The row action in the last column (**Open**, **Edit**, **Ledger**, **Adjust**, and so on) stays pinned to the right edge of that frame as you scroll, so it is always reachable — you are never stranded with the action for a row scrolled off-screen. A soft shadow on the right edge of a table hints that more columns are available if you scroll. The interface has been verified for accessibility at phone and tablet screen sizes as well as desktop — no serious or critical issues were found across the main screens when it was last checked.

The four screen states

Every data screen can be in one of four states. The system displays a distinct visual for each:

State	What you see
Loading	A spinner or skeleton while data is being fetched. A thin progress stripe also runs across the top of the screen whenever the system is talking to the server.
Empty	A clear message that there are no records matching your criteria. This is not an error.
Error	A message explaining that something went wrong, with a prompt to try again.
No access	If you navigate directly to a screen you cannot use, you are redirected to the home page silently. If you can open a screen but lack permission for a particular *action* on it, that part of the screen shows a calm "no permission" message rather than an alarming

	pop-up.

When something goes wrong — errors and conflicts

The system aims to tell you plainly what happened and what to do, rather than showing a raw technical failure:

- A genuine failure (the server is unreachable, or an unexpected error) appears as a centered **"Something went wrong"** alert that you acknowledge with **OK**. The message is written in plain language.
- A **validation** problem (a missing required field, a value out of range, an unsupported file) is reported clearly so you can correct it and try again — you will not see a raw internal error.
- A **conflict** — for example, two people edited the same record at the same time — appears through the same centered **"Something went wrong"** alert, with the body explaining that the record was modified by another transaction and asking you to reload and try again. Reopen the record to get the latest version, re-apply your change, and save again.
- A session that has expired returns you to the login page with the toast "Your session has expired. Please sign in again." (see the Signing In section).

Money and date formats

What currency-aware money means. Every monetary value in this system is stored and displayed as a pair: an amount and its currency code (for example, `TZS 1,234.56` or `USD 200.00`). A bare number with no currency is never used. This matters because a figure of `1500` means something completely different in TZS than in USD.

Why this design. The system is built for organisations that may trade in multiple currencies. Attaching the currency to every amount from the start prevents a class of errors where amounts in different currencies are accidentally compared or summed. It also allows a second company under the same organisation to operate in a different base currency without any data migration.

- **Money** is always shown with the currency code and two decimal places, for example `TZS 1,234.56` or `USD 200.00`. You never type a currency symbol or a free-text currency code — when a form needs a currency you choose it from the **Currency Picker** (see above), which is pre-set to your company's default.
- **Dates** are shown in your local timezone. When entering dates use the date picker provided — never type raw date strings.

Creating, editing, and saving records

The general flow for creating or editing a record is:

1. Click **Create** (or open an existing record).
2. Fill in the form. Required fields are marked. The system validates as you go and shows inline messages if something is wrong.
3. Click **Save** (or the specific action button, for example **Confirm** for a sales order).

4. A brief success notification (a "toast") appears at the top of the screen to confirm the action was saved. If something went wrong, an error message appears in the form itself or as an alert — read it, correct the issue, and try again.

> **Company and Branch default to where you are working.** Where a create screen has its own **Company** and/or **Branch** picker (for example New Stock Count, New Stock Transfer, or Register New Asset), it opens pre-set to your currently **active** branch and company — the same one shown in the top bar — not simply the first company in the organisation. You can still change it before saving if you are creating the record for a different branch; only the starting default changed.

Record status and soft-delete

What soft-delete means. The system does not permanently erase records. When you deactivate a user, archive a product, or cancel a transaction, the record's status changes to `INACTIVE` or `ARCHIVED` but the record itself remains in the database and in audit history. This is called a soft-delete.

Why this exists. Business records have legal and operational significance beyond their active life. A cancelled invoice must still be traceable; a former employee's username must still appear in audit logs. Keeping the record preserves that history. It also means mistakes can be corrected by re-enabling a record rather than recreating it.

How statuses work. Most master records (users, roles, companies, branches) follow the `ACTIVE → INACTIVE / ARCHIVED` lifecycle. An `INACTIVE` record cannot be used in new transactions. An `ARCHIVED` record is additionally excluded from selection pickers and branch-switching lists. You can view inactive and archived records in the relevant administration screens by adjusting the status filter.

A record's status is shown throughout the system as a small coloured **status pill** (a status tag) — for example a green pill for active records and a muted pill for inactive or archived ones — so you can see at a glance where each record stands in a list.

> The system does not hard-delete records. Deactivating a user, archiving a product, or cancelling an order leaves the record in the system in an inactive or historical state. You can always review past records.

Signing In as rootadmin

What rootadmin is. `rootadmin` is the system superuser — a special account created once during deployment. It bypasses all permission checks and operates across every company and branch without needing role assignments.

Why it exists. Every system needs a recovery mechanism. If all administrator accounts were somehow locked or misconfigured, `rootadmin` is the account that can restore access. It is also used

for the initial bootstrapping of the organisation, companies, and first administrative users before any role grants exist.

When to use it. Exclusively for initial system setup and emergency recovery. Every action taken as `rootadmin` — including any cross-company operations — is recorded in the audit trail. Normal day-to-day work must use named user accounts with appropriate roles so that audit logs are meaningful and access is scoped correctly.

The `rootadmin` account bypasses all permission checks and sees every screen and every action in every company and branch. It is reserved for initial system setup and emergency recovery. Normal day-to-day work should use named user accounts with appropriate roles.

Administration and Access

This chapter is for administrators — typically users who hold the **ORG_ADMIN** role. It covers how to set up the organisation hierarchy (companies and branches), manage user accounts, define roles and their permissions, assign roles and branches to users, and review the audit trail.

> **Permissions required.** You need specific permissions for each section below. If a menu item or button is not visible to you, your role does not include that permission. See the table in each section.
>
> **The sidebar adapts to your permissions.** Navigation items you cannot use are hidden, and a navigation group whose every item is hidden disappears entirely — you will never see an empty group header. If a whole section of this chapter seems to be missing from your sidebar, your role lacks the permissions for it.

Organisation, Companies, and Branches

What the three-level hierarchy is. The system is structured in three nested levels. An **organisation** is the top-level entity representing your entire business group — think of it as the holding entity that owns everything else. A **company** is a distinct legal entity under the organisation (for example, a registered limited company or a subsidiary with its own tax ID). A **branch** is a physical or logical operating unit under a company — a shop, warehouse, regional office, or cost centre.

Why this structure exists. Different businesses have different legal and operational structures. A retail chain might operate a single registered company with many store branches. A group of businesses might operate several legally distinct companies sharing one ERP platform. Separating these levels allows each company to have its own data, its own base currency, and its own set of branches, while all of them are administered through one system.

When each level matters.

- The organisation is set up once during deployment and is resolved automatically — you never need to type an organisation identifier.
- Companies are created by an administrator with `COMPANY.MANAGE` permission, typically during the initial rollout or when a new legal entity is formed.
- Branches are created whenever a new location or operating unit is needed.

How the levels connect. Every transaction in the system is tagged with the branch it was performed in. A branch belongs to exactly one company, and from there the company's base currency, fiscal calendar, and legal details apply. A user's session is always scoped to an active branch; switching branches changes the company context as well (the active company is always the company that owns the active branch).

The system is structured in three levels:

- **Organisation** — the top-level entity representing your business group. It is configured by IT during deployment and is resolved automatically; you never type an organisation ID.
- **Company** — a legal entity under the organisation (for example, a registered company or subsidiary). Each company has its own data.
- **Branch** — a physical or logical office, store, or cost centre under a company. Transactions are scoped to a branch.

Viewing companies

Required permission: `COMPANY.VIEW`

Navigate to **Administration › Companies** (`/admin/companies`) in the sidebar.

The list shows each company's code, name, status (Active or Archived), and a **Manage branches** link in the last column. Above the list, the organisation name for this deployment is shown. The list is scoped to your access: a **root administrator** sees every company in the organisation, while everyone else (even holders of `COMPANY.VIEW`) sees only the companies they belong to — those they are assigned to via company membership, a branch, or a role. This keeps the company list from revealing companies you have no access to.

> The Companies screen is intentionally lean: it lets you create a company (code and name), rename it inline (see below), re-provision its default data (see below), and open each company's branches. There is no separate company detail screen — the company code can never be changed after creation.

Creating a company

What a company record is. A company record represents one legal entity — a separately registered business with its own identity, tax registration, and (optionally) its own functional currency. Creating a company in the system establishes the container under which that entity's branches, users, and transactions will live.

Why you would create one. You create a new company when a new legal entity joins your group, or when you first deploy the system and need to register your existing companies. Without at least one company, no branches can exist and no users can be assigned a working context.

Required permission: `COMPANY.MANAGE`

The create form is an inline card at the top of the Companies list (it appears only if you hold `COMPANY.MANAGE`).

1. On the Companies list, in the create card fill in:
    - **Code** — a short, unique code. Cannot be changed after creation.
    - **Name** — the company display name.
2. Click **Add company**.

The new company appears in the list with status **Active**. The organisation is resolved automatically; you do not choose it. Code and name are the only fields you enter — there is no Legal Name, Tax ID, or Timezone field on this screen.

> A duplicate code is rejected with a conflict (409) error shown beneath the form.

Renaming a company

When to rename. The system ships with a seeded **Default Company**. When you go live you will usually want to give it your real trading name. Renaming lets you correct that — and any later name change — without touching the code (which stays fixed forever).

Required permission: `COMPANY.MANAGE`

Each company row carries an **Edit** button (it appears only if you hold `COMPANY.MANAGE` — a view-only user with only `COMPANY.VIEW` sees no Edit control). To rename a company:

1. On the Companies list, click **Edit** on the company's row. The Name cell turns into an editable text box.
2. Type the new name.
3. Click **Save** to keep the change, or **Cancel** to discard it.

Only the name changes — the code, status, and other attributes are untouched. There is no Archive control on the Companies list. (Archiving exists in the underlying API but is not exposed in the admin UI.)

Provisioning a company's default data

What "Provision defaults" does. A newly created company is automatically seeded, in the same transaction as its creation, with the baseline operational and reference data every module expects to find — default tax rates, GL accounts, units of measure, enabled currencies, and the like. **Provision defaults** re-runs that same seeding for an existing company, on demand. It is idempotent: anything the company already has is left alone — the action only fills in whatever is missing.

When to use it. Use it if a company is missing expected reference data — for example, a company created before automatic provisioning existed, one whose original provisioning only partially completed, or one you simply want to heal after noticing a gap (a missing tax rate, an unset default account). It is safe to click on a company that is already fully provisioned; nothing is duplicated.

Required permission: `COMPANY.MANAGE`

Each company row carries a **Provision defaults** button alongside **Edit** (visible only if you hold `COMPANY.MANAGE`).

1. On the Companies list, click **Provision defaults** on the company's row.
2. Confirm the prompt — *"Restore the default setup (tax rates, accounts, units, …) for "\<company name\>"?"*.
3. The button switches to **Provisioning…** while the request is in flight. The Provision-defaults and Edit actions are interlocked across the **whole** company list, not per row: while any row is being renamed the **Provision defaults** button is disabled on every row, and while a Provision-defaults request is in flight **Edit** is disabled on every row.
4. On success, a confirmation alert ("Default setup restored") appears. On failure, an error alert explains what went wrong.

Viewing branches

Required permission: `BRANCH.VIEW`

There are two ways to reach a company's branch list:

- **From the sidebar.** Navigate to **Administration › Branches** (`/admin/branches`). This standalone page is reachable with `BRANCH.VIEW` alone — you do not need `COMPANY.VIEW`. It first shows a **Company** picker listing only the companies you can act in; choose one to load its branches. If you can act in exactly one company, the page selects it for you automatically and goes straight to that company's branch list.
- **From the Companies screen.** Navigate to **Administration › Companies** (`/admin/companies`), then click the **Manage branches** link on the company's row to open its branch list at `/admin/companies/<companyUid>/branches`. (This path requires `COMPANY.VIEW` to reach the Companies screen.)

Either way, the branch list shows each branch's code, name, default flag, and status.

Creating a branch

What a branch record is. A branch is the smallest organisational unit in the system. It represents one operating location or logical division — a physical store, a warehouse, a regional sales office, or an accounting department. Every transaction (a sales invoice, a goods receipt, a payment) is tagged to the branch that was active when it was created.

Why branches exist. Without branches, all transactions for a company would be lumped together with no way to separate one location's performance from another's. Branches let management analyse stock levels by store, compare revenue by region, control what each staff member can see and do (a storekeeper at one branch cannot access another branch's data), and ensure that GL postings land in the right cost centre.

When to create one. Create a branch when a new physical location opens, when a new department or cost centre is established, or during initial setup to represent your existing offices and stores.

How a branch interacts with users. A user must be assigned to at least one branch to have an active session. One of their assigned branches is marked as their **default branch** — this is the branch that activates automatically on login. A user can be assigned to many branches and can switch between them during a session without logging out.

Required permission: `BRANCH.MANAGE`

The create form is an inline card at the top of the branch list.

1. From the branch list of a company, in the create card fill in:

    - **Code** — a unique code within this company.
    - **Name** — the branch display name.
    - **Set as default** — check this to make the new branch the company's default branch. If another branch was already the default, that branch's default flag is cleared automatically.

2. Click **Add branch**.

There is no Timezone field on the form — branches inherit the company's settings.

Setting the company default branch

What the company default branch is. Each company has exactly one branch flagged as its default. This default is the branch that new users in this company start in when they have no personal default of their own, and it is used by the system in contexts where a company-level default is needed (for example, documents that auto-populate a branch).

Why only one default per company. Having more than one "default" is ambiguous — the system would not know which one to apply. The uniqueness rule (enforced at the database level) ensures there is always one unambiguous answer.

Only one branch per company can be the default. The default branch is the one users are taken to on login when no other preference is active.

1. On the branch list, find the branch you want to make default.
2. Click **Make default** on that row.

The current default branch shows a **Default** status tag instead of a button. The previously default branch is cleared automatically.

Renaming a branch

When to rename. A new deployment seeds a **Head Office** branch. Rename it (and any other branch) to match how your locations are actually called — a store name, a depot, a regional office. As with companies, the code is fixed at creation; only the name can change.

Required permission: `BRANCH.MANAGE`

Each branch row carries an **Edit** button (it appears only if you hold `BRANCH.MANAGE` — a view-only user with only `BRANCH.VIEW` sees no Edit control). To rename a branch:

1. On the branch list, click **Edit** on the branch's row. The Name cell turns into an editable text box.
2. Type the new name.
3. Click **Save** to keep the change, or **Cancel** to discard it.

Archiving a branch

A branch's code is fixed at creation; the per-row actions are **Edit** (rename), **Make default**, and **Archive**.

To archive a branch, click **Archive** on its row. The branch status changes to **Archived** and it is removed from branch-selector lists. Any users whose default was this branch will lose their active branch on next login.

> The current default branch cannot be archived directly — its row shows the **Default** tag and no Archive button. Make another branch the default first, then archive the former default.

Users

What a user account is. A user account is a named login identity in the system. It consists of a username (the login name), a display name (shown in the interface and in audit logs), a password, and optional contact details (email, phone). A user is an organisation-wide record — the same account can be active in multiple companies, though its permissions depend on which company and branch it is working in at any given moment.

Why user accounts exist. Shared logins (for example, a single "accountant" password passed around the team) make audit trails meaningless — the log shows "accountant did X" and you cannot know who actually did it. Named individual accounts mean every action is attributable to a real person, accounts can be individually disabled without disrupting others, and each person can be given exactly the permissions their job requires.

When to create a user. Create a user when a new employee joins, when a contractor needs access, or when a role needs an automated service account. If you create the user while acting as a non-root company administrator, the system automatically makes the new user a member of your active company in the same step — you do not need a separate action for this common case. You then still need to assign at least one branch and grant at least one role; a user with no branches and no roles can log in but will see no menu and have no active branch. (The `rootadmin` account has no single

active company, so a user created while signed in as root is left with no company membership — assign one explicitly first; see Assigning Companies to Users.) Branches and roles can only be assigned within a company the user already belongs to.

How the user lifecycle works. A user starts `ACTIVE`. An administrator can `DISABLE` the account (status becomes `INACTIVE`) to prevent login while preserving the account and its history — for example, during a leave of absence or pending investigation. `ENABLE` restores it to `ACTIVE`. The `rootadmin` account cannot be disabled. If too many wrong-password attempts are made, the account is automatically locked for 15 minutes; an administrator can clear this with **Unlock**. User accounts are never hard-deleted.

Required permission to view: `USER.VIEW` **Required permission to create/edit/disable/unlock/reset password:** `USER.MANAGE`

Navigate to **Administration › Users** (`/admin/users`) in the sidebar.

The users list

The list shows columns for **Username**, **Display name**, **Status**, **Locked**, and **Root** (a marker on the `rootadmin` account), plus a per-row action area. The actions on each row are **Disable**/**Enable**, **Unlock** (only when the account is locked), **Password** (an inline set-password form), and **Assignments** (a link that opens the user's Assignments page at `/admin/users/uid/<uid>`, where you manage the user's companies, branches, and roles). Root accounts do not show a Disable action.

The list is scoped to your access: a **root administrator** sees every user in the organisation, while everyone else sees only the (non-root) users who are members of their active company — including any user they have just created, since creating a user makes it a member of the creator's active company automatically (see below, and Assigning Companies to Users).

Creating a user

The create form is an inline card (**Add User**) at the top of the Users list.

1. On the Users list, in the **Add User** card fill in:
    - **Username** — must be unique. Stored in lowercase. May contain only letters, digits, dots (`.`), underscores (_), and hyphens (`-`); spaces and other characters are rejected. Up to 80 characters.
    - **Display name** — the name shown in the UI. Up to 160 characters.
    - **Temporary password** — must be at least 8 characters and contain at least one letter and one number. Common passwords (such as `password1` or `admin123`) are rejected.
2. Click **Add user**.

The user is created with status **Active** and no role or branch assignments. The create form captures only the username, display name, and temporary password — there are no email or phone fields here. Assign roles and branches next (see below). If you created the user while acting as a non-root administrator, the user is also automatically made a member of your active company — no separate step is needed, and the user immediately appears in your Users list.

> Usernames are compared case-insensitively. `Alice.Smith` and `alice.smith` refer to the same account. Creating a user whose username already exists is rejected with a conflict (409) error.

Disabling and enabling a user

- **Disable** — click **Disable** on the user's row. The user's persisted status changes to **Inactive** and they can no longer sign in. The account and its history are preserved. (Immediately after you click, the row may briefly show **Disabled** as an optimistic in-memory label; on the next refresh it settles to the stored **Inactive** status.)
- **Enable** — click **Enable** on the row to restore the user to **Active** status.

The **rootadmin** account cannot be disabled.

Unlocking a locked account

What account locking is. After 5 consecutive wrong-password attempts, the system locks the account for 15 minutes — a security measure to slow down automated guessing attacks. While locked, even the correct password is rejected and the user sees a specific message.

When to unlock manually. If a user cannot wait 15 minutes (for example, during a time-sensitive business operation), an administrator with `USER.MANAGE` can clear the lockout immediately. The failed-attempt counter resets on unlock.

If a user has been locked out after too many failed sign-in attempts, a locked indicator appears on their row. Click **Unlock** to clear the lockout. The user can then sign in again with the correct password.

Resetting a user's password

When to reset. Reset a password when a user forgets theirs, when you suspect a password has been compromised, or when a new user needs to change the temporary password set on account creation.

1. On the Users list, click **Password** on the user's row to expand the inline set-password form.
2. Enter a new password that meets the policy (at least 8 characters, at least one letter and one number, not a common password).
3. Click **Save**.

The user can sign in immediately with the new password. Passwords are never stored in plain text and are not shown in audit logs.

The user Assignments page

Click **Assignments** on a user's row to open their Assignments page (`/admin/users/uid/<uid>`). This page shows a **read-only header** (username, display name, and status, locked, and root tags) followed by three management cards in order: **Companies**, **Branch Assignments**, and **Role Assignments** (see the sections below). It does not contain a form for editing display name, email, or phone — there is no contact-details edit screen in the current interface.

> **Assign a company first.** Company membership is the prerequisite for the other two cards. Until the user belongs to at least one company, the Branch and Role cards show a hint to assign a company first, and their company pickers are empty. Once you assign a company, that company becomes selectable when assigning the user's branches and roles. (See Assigning Companies to Users below.)

Roles

What a role is. A role is a named, reusable bundle of permissions. For example, an `ACCOUNTANT` role might include permissions such as `GL.POST`, `AR.VIEW`, `AP.BILL.ENTER`, and `CASH.RECONCILE`. Once defined, the role can be granted to any number of users. If the business needs to change what accountants can do, the administrator updates the role once and the change takes effect for every holder immediately.

Why roles exist — RBAC. This design is called Role-Based Access Control (RBAC). It exists because managing permissions per-user does not scale: a company with 50 staff and over 220 permission codes would require thousands of individual permission grants, each needing manual maintenance. With roles you manage a small set of job functions, not a large matrix of individual grants. RBAC also makes compliance simpler: you can demonstrate to an auditor exactly which capabilities any given role confers.

When roles are created. Roles are created during initial setup (to match the job functions in your organisation) and updated whenever those functions evolve. A small set of **system roles** (such as `ORG_ADMIN`) are seeded during deployment and cannot be archived; custom roles can be freely created and modified.

How a role's permissions take effect. When a role's permission set is saved, the system invalidates its permission cache. Users who hold the role see the change on their very next request — there is no need to log out and back in.

The effective permission set. A user may hold multiple roles. Their effective permissions at any moment are the **union** of all permissions from all their active role grants in the current company and branch context. If Role A grants `GL.VIEW` and Role B grants `GL.POST`, a user with both roles has both.

Required permission to view: `ROLE.VIEW` **Required permission to create / edit / set permissions:** `ROLE.ADMIN` (the role catalogue itself is guarded by `ROLE.ADMIN`; the separate `ROLE.MANAGE` permission governs assigning existing roles to users — see *Assigning Roles to Users* below)

Navigate to **Administration › Roles** (`/admin/roles`) in the sidebar.

Roles are named bundles of permissions. A user can be granted one or more roles; the effective permissions are the union of all granted roles in the active branch context.

The roles list

The list shows each role's code, name, a permission count, and its status, plus a marker on **system** roles (pre-defined and cannot be archived). The code and name cells are plain text — they are not clickable. To open a role's edit page at `/admin/roles/uid/<uid>`, click the **Edit** button in the row's actions column.

Built-in operational roles

What they are. The system ships with **12 ready-made operational roles** — one for each common job function — so you can put staff to work without composing a permission set from scratch. They appear in the Roles list from day one, alongside `ORG_ADMIN`, ready to grant to users in the Assigning Roles to Users section below. Each is a **system role**: it is marked as **system** in the list, cannot be deleted or archived, and its permission set is fixed (view it on the role edit page, but you cannot change it — see Setting a role's permissions).

How they differ from ORG_ADMIN. `ORG_ADMIN` holds *every* permission in the system — it is the full administrator. The 12 operational roles are **least-privilege**: each holds only the permissions its job actually needs, so a Cashier cannot manage users, a Salesperson cannot void invoices or change pricing settings, and an Accountant cannot close the fiscal year. This is deliberate — you assign people the narrowest role that lets them do their work.

How to use them. Grant one (or more) of these roles to a user for a company, exactly as you would any other role (see Assigning Roles to Users). If none of them fits a person's responsibilities exactly, use them as a starting reference and create your own **custom** role (custom roles are freely editable; the 12 shipped roles are not).

Code	Name	What it can do
`SALESPERSON`	Salesperson	Quotes, orders, invoices, deliveries and returns; issue fiscal receipts; onboard customers. No overrides, voids, or settings.
`CASHIER`	Cashier	POS till (open, sell, close, reconcile), customer receipting, end-of-day cash count and petty cash. Front-line cash only.
`FIELD_SALES_AGENT`	Field Sales Agent	Route/van selling: capture orders, invoice and deliver on a route, collect cash receipts, reconcile van stock.
`STOREKEEPER`	Storekeeper	Stock control: view, adjust, count, post, transfer, locations; receive goods against a PO; supplier returns; batch/serial/expiry.
`ACCOUNTANT`	Accountant	GL journals (not close), AR and AP sub-ledgers, cash & bank, VAT preparation (not filing), WHT, cost tagging and financial reports.
`SALES_MANAGER`	Sales Manager	Full sales including overrides, voids,

		credit override and settings; pricing rules, CRM, agents and routes, and sales approvals.
`BRANCH_MANAGER`	Branch Manager	Broad single-branch oversight: cross-module views, approvals and operational sign-offs (POS reconcile, cash count, stock post, PO/requisition approve). Usually assigned branch-scoped.
`PROCUREMENT_OFFICER`	Procurement Officer	Requisition, RFQ, purchase order, receive and returns; landed cost and suppliers. No approve, void, or settings.
`PROCUREMENT_MANAGER`	Procurement Manager	Everything the officer has, plus PO/requisition approval, PO/goods-receipt void, and purchase settings.
`HR_PAYROLL_MANAGER`	HR & Payroll Manager	Employees, leave and loans; pay components; the full payroll run/approve/post/disburse/reverse cycle; statutory deductions.
`FINANCE_DIRECTOR`	Finance Director	Everything the Accountant has, plus GL configuration, period/year close, VAT filing, FX revaluation, fixed assets, budgeting and approvals policy. Not user or company administration.
`PRODUCTION_MANAGER`	Production Manager	Work-order lifecycle, BOMs, material movements and WIP/costing views.

> The `SALESPERSON`/`SALES_MANAGER`, `PROCUREMENT_OFFICER`/`PROCUREMENT_MANAGER`, and `ACCOUNTANT`/`FINANCE_DIRECTOR` pairs are split on purpose: the manager/director role adds the approve, void, close and settings capabilities that the day-to-day role deliberately lacks (segregation of duties). Assign the day-to-day role to front-line staff and the manager role to whoever signs off their work.

Creating a role

The create form is an inline card at the top of the Roles list.

1. In the create card, fill in:
   - **Code** — a short identifier, unique within the organisation. Cannot be changed after creation.
   - **Name** — a human-readable label.
   - **Description** — optional notes.
2. Click **Add role**.

The new role is created with no permissions. Assign permissions next.

> A duplicate role code is rejected with a conflict (409) error.

Editing a role's name or description

Open the role's edit page (`/admin/roles/uid/<uid>`) and update the name or description fields, then click **Save details**. The code cannot be changed.

Setting a role's permissions

What the permission catalogue is. A permission is the finest-grained unit of access control in the system — a named capability that says "the holder may perform this specific action." Permissions are grouped by module (for example, all `GL.*` permissions belong to the General Ledger module). The full catalogue contains over 220 codes covering every module.

> **System roles cannot have their permissions changed.** Built-in system roles (such as **ORG_ADMIN**) are marked with a notice on their edit page — *"This is a built-in system role. The code cannot be changed."* Their permission set is fixed: attempting to save a changed permission selection on a system role is rejected with a conflict (409) error (*"System role permissions cannot be modified"*). Only custom roles can have their permissions edited.

How the "replace" save works. When you click **Save permissions**, the system replaces the role's entire permission set with exactly the codes you have checked. Unchecking a box removes that permission. This means saving an empty selection leaves the role with no permissions — which is valid and means the role grants no access.

On the role edit page, the permissions panel lists every available permission grouped by module (for example, **iam**, **sales**, **accounting**).

1. Check the boxes for the permissions this role should have.
2. Uncheck any permissions to remove them.
3. Click **Save permissions**.

Saving replaces the role's entire permission set with the checked selections. Removing a permission takes effect for users who hold this role on their next request (the system re-resolves permissions promptly after changes).

> The permission catalogue contains over 220 codes across all modules. An empty permission set is valid — it means the role grants no access.

Read-closure advisory

What it is. Below the Permissions card, the role edit page can show a **Read-closure advisory** panel. It flags screens the role's checked permissions let a user *open* (because the role holds that screen's primary action permission) where the role is still missing one or more supporting **read** permissions the same screen needs on load — for example, a role that can create a sales order but was not also given the customer and product picker reads. The panel is advisory only: it never blocks **Save permissions**, and saving an incomplete selection is always allowed.

Why it exists. A screen's primary action and its reference-data pickers are separate permission checks. Without this advisory, a role that is missing a picker's read looks complete to the

administrator composing it (everything saves without error) but a real user holding only that role hits a partial, confusing block once they open the screen — the button they can see, but a supporting dropdown 403s. Testing as `rootadmin` never reveals this, because root holds every permission. The advisory surfaces the gap to the administrator at grant time instead.

When you see it. The panel appears automatically on the role edit page (`/admin/roles/uid/<uid>`) whenever the role's current permission set leaves at least one reachable screen with a missing required read; it is empty and hidden otherwise. It refreshes each time you click **Save permissions**, so you can immediately confirm a fix.

Each gap reads in the form:

> This role can open **\<screen name\>** but is missing `<CODE>`, `<CODE>` — users with only this role will be blocked on that screen.

Grant the listed codes on the Permissions panel above and click **Save permissions** again to clear the gap.

Archiving a role

There is no Archive control in the current interface. The role edit page offers only **Save details** and **Save permissions**, and the roles list has no Archive action — its only per-row action is **Edit**. (Archiving exists in the underlying API but is not exposed in the admin UI; system roles such as **ORG_ADMIN** cannot be archived in any case.)

Assigning Companies to Users

What company membership is. Company membership is an explicit record that a user belongs to a specific company. It is the foundation of a user's access: before a user can be given any branch or role in a company, they must first be made a member of that company. A user can be a member of several companies.

Why membership is required first (assign-company-first). Tying branch and role assignment to an explicit company membership makes "which companies is this person part of?" a single, deliberate decision rather than a side effect of the first role grant. It also keeps the company pickers on the Branches and Roles cards focused — they list only the companies the user actually belongs to, so you cannot accidentally grant access in the wrong company.

When to assign a company. A non-root administrator's active company is assigned automatically the moment they create the user (see Creating a user), so you will usually only need this screen when a user created by `rootadmin` needs their first company, or whenever an existing user takes on responsibilities in an additional company.

How removal is protected. You cannot remove a user's company membership while they still hold any branch assignment or role grant in that company — the system blocks it with a message asking

you to remove those first. This prevents leaving a user with branches or roles in a company they are no longer a member of.

Required permission: `USER.COMPANY.MANAGE`

> The **Companies** card appears on every user's Assignments page, but the **Assign company** and **Remove** controls are shown only to administrators who hold `USER.COMPANY.MANAGE`. Other administrators can see which companies a user belongs to but cannot change them.

Assigning a user to a company

1. Open the user's Assignments page (**Assignments** on the user's row in **Administration › Users**).
2. In the **Companies** card, choose a company from the **Assign Company** picker. (Root and full administrators can choose any company in the organisation; other administrators choose from the companies they can act in.)
3. Click **Assign**.

The company appears in the user's Companies list. Its branches and roles can now be assigned in the cards below.

Removing a company membership

In the **Companies** card, click **Remove** on the company's row.

- If the user still has any branch assignment or role grant in that company, the removal is **rejected** — remove those branches and roles first (see the two sections below).
- Otherwise the membership is removed. The user is no longer a member of that company and can be re-assigned later if needed.

Assigning Roles to Users

What a role grant is. A role grant is a record that links a specific user to a specific role, scoped to a company and optionally to a single branch. It is not a permanent property of the user; it is an explicit, revocable assignment. A user can hold many grants, and each grant has its own scope.

Why grants are scoped to a company (and optionally a branch). A user might be an accountant in the Head Office branch and a read-only viewer in the Mwanza branch. Scoping the grant to a company-and-branch means the right permissions apply in the right context. A company-wide grant (no branch restriction) gives the role's permissions in every branch of that company. A branch-scoped grant gives those permissions only when the user is active in that specific branch.

When to grant a role. After creating a new user, before they can do meaningful work. Also when an existing user takes on new responsibilities. Role grants take effect immediately on the user's next request — no re-login required.

How revocation works. Revoking a grant marks it as revoked (the record is kept for audit purposes) and removes its permissions from the user's effective set immediately. The user's currently open session will lose those permissions on their next request.

Required permission: `ROLE.MANAGE`

Navigate to **Administration › Role Grants** (`/admin/role-grants`) in the sidebar.

This screen lets you grant a role to a user for a specific company, optionally restricted to a single branch.

> You can also manage one user's role grants directly from their Assignments page. The **Role Assignments** card on `/admin/users/uid/<uid>` (reached via **Assignments** on the user's row) has the same **Grant Role** form (Role, Company, optional Branch) and a per-row **Revoke** action, scoped to that single user. On this card the **Company** picker lists only the companies the user is a member of (see Assigning Companies to Users).
>
> **Company membership is required.** A role can only be granted in a company the user already belongs to. If the user is not yet a member of the target company, assign the company first — otherwise the grant is rejected. (On the user's Assignments page the Role card simply will not offer a company the user does not belong to.)

Granting a role

1. On the Role Grants screen (`/admin/role-grants`), choose the **User** in the picker (placeholder *Select user*).
2. Choose the **Role** from the dropdown — each option is shown as `code — name` (for example `ACCOUNTANT — Accountant`).
3. Choose the **Company** in the picker. It starts empty (placeholder *Select company*) and is required — there is no default company; you must pick one.
4. Optionally choose a **Branch** to restrict the grant to one branch. Leave it blank (the picker reads *Leave blank for company-level*) to grant the role across all branches of that company.
5. Click **Grant**.

The grant appears in the grants list. The user's effective permissions update on their next request.

> You can only grant roles within a company you are active in. The `rootadmin` account can grant across companies.

Revoking a role grant

On the Role Grants screen, look up the user's current grants by selecting their name. Each active grant is listed. Click **Revoke** on the grant row you want to remove.

The grant is revoked immediately. The user loses those permissions on their next request.

> Revoking a grant that was already revoked is a no-op.

Assigning Branches to Users

What a branch assignment is. A branch assignment is a record that links a user to a specific branch, granting them the ability to switch to that branch and operate within it. Without at least one branch assignment, a user has no active branch after login and can only view limited screens.

Why branch assignments are separate from role grants. Branch access (which branches you can switch to) and permission access (what you can do in each branch) are two different concerns that can be managed independently. You might assign a user to a branch for data-visibility reasons without granting them any elevated permissions there, or you might grant a company-wide role that applies to all branches simultaneously while still controlling which branches the user can physically switch to.

The default branch rule. Each user has exactly one default branch — the branch that becomes active on login. The system enforces this at the database level: you cannot have two defaults for the same user simultaneously. If you set a new default, the previous one is cleared automatically. If a user's default branch is removed or archived, the system automatically promotes the earliest-assigned remaining branch as the new default. If no branches remain, the user has no active branch (read-only session) until an administrator assigns one.

When to use "Make default". Tick **Make default** when assigning the branch that should be this user's primary working location — typically their home branch or the branch they will use most often. You can change the default at any time.

Required permission to view branch assignments: `USER.VIEW` **Required permission to assign / change default / remove:** `BRANCH.ASSIGN`

Branch assignments control which branches a user can switch to and which data they can access. Open a user's Assignments page (`/admin/users/uid/<uid>`) by clicking **Assignments** on the user's row in the **Administration › Users** (`/admin/users`) list. A branch can only be assigned within a company the user is already a member of (see Assigning Companies to Users).

Assigning a user to a branch

1. On the user's Assignments page, find the **Branch Assignments** card.
2. Choose the **Company** by name. The picker lists only the companies this user is a member of — if the company you want is not listed, assign it first in the **Companies** card above.
3. Once a company is selected, the **Branch** picker loads that company's branches. Choose one by name.
4. Check **Make default** if you want this branch to become the user's new default.
5. Click **Assign**.

The branch appears in the user's assignments list. If **Make default** was checked and the user already had a different default branch, the old default is cleared automatically — a user can only have one default at a time.

Changing the default branch

In the user's branch assignments list, click **Set Default** on the row for the branch you want to make default. The previous default is cleared automatically.

Removing a branch assignment

Click **Remove** on the assignment row. If you remove the user's current default branch:

- If other branches remain, the system automatically promotes the earliest-assigned remaining branch as the new default.
- If no branches remain, the user will have no active branch on their next login (read-only session).

> You can only assign or remove branches within a company you are active in.

Example — Create the ACCOUNTANT role and assign it to a new user scoped to the Head Office branch

This example walks through the complete new-staff onboarding flow for Amina Juma, who joins as an accountant at the Head Office branch of Orbix Trading Co.

Step 1 — Create the role (if it does not already exist)

1. Navigate to **Administration › Roles** (`/admin/roles`).
2. In the create card, enter Code `ACCOUNTANT`, Name `Accountant`, Description `GL posting, AR, AP, Cash & Bank, Tax`.
3. Click **Add role**. The role is created with no permissions.
4. Click **Edit** on the `ACCOUNTANT` row to open `/admin/roles/uid/<uid>`.
5. In the permissions panel, check the following codes: `GL.VIEW`, `GL.POST`, `AR.VIEW`, `AR.RECEIPT.RECORD`, `AR.STATEMENT.VIEW`, `AP.VIEW`, `AP.BILL.ENTER`, `AP.PAYMENT.RUN`, `CASH.VIEW`, `CASH.ENTRY.RECORD`, `CASH.TRANSFER`, `CASH.RECONCILE`, `VAT.VIEW`, `TAXRATE.VIEW`, `REPORT.PL.VIEW`, `REPORT.BS.VIEW`, `REPORT.CASHFLOW.VIEW`, `REPORT.LEDGER.VIEW`.
6. Click **Save permissions**. The panel refreshes showing 18 of the total codes selected.

Step 2 — Create the user account

1. Navigate to **Administration › Users** (`/admin/users`).
2. In the **Add User** card, enter Username `amina.juma`, Display name `Amina Juma`, Temporary password `Amina2024#`.
3. Click **Add user**. The row appears with status **Active**. (Email and phone are not collected on this form.)

Step 3 — Assign the Head Office branch

1. Click **Branches** on Amina Juma's row to open `/admin/users/uid/<uid>`.

2. In the **Branch Assignments** panel, select Company `Orbix Trading Co.`, Branch `HO — Head Office`.
3. Check **Make default**.
4. Click **Assign**. The branch appears in the list marked as default.

Step 4 — Grant the ACCOUNTANT role

1. Navigate to **Administration › Role Grants** (`/admin/role-grants`).
2. Select User `Amina Juma`, Role `ACCOUNTANT`, Company `Orbix Trading Co.`, Branch `HO — Head Office`.
3. Click **Grant**. The grant row appears immediately.

Step 5 — Verify

1. Navigate to **Administration › Audit** (`/admin/audit`).
2. Filter by Actor `rootadmin` (or whichever admin performed these steps). Confirm five audit entries: `ROLE.CREATE` and `ROLE.PERMISSIONS_SET` (Step 1), `USER.CREATE` (Step 2), `BRANCH.ASSIGN` (Step 3), and `ROLE.GRANT` (Step 4).
3. Sign in as `amina.juma` / `Amina2024#`. Confirm the **Accounting** sidebar group is visible and items such as **Chart of Accounts** (`/admin/gl/accounts`) and **Payables** (`/admin/ap/supplier-bills`) are accessible.

Sales Settings — sales-order approval and stock control

What this screen is. Sales Settings is a per-company configuration screen that holds one setting group per company. It controls two independent things: whether large **sales orders** must be approved before they can be confirmed (and above what value that approval kicks in), and whether sales that would take **stock negative** are blocked or allowed.

Why it exists. Many businesses want a manager to sign off on unusually large orders — an unusually big customer order can tie up stock, extend a lot of credit, or signal an error in data entry. Rather than force approval on every order (which slows the counter down), you set a **threshold**: orders at or above it route to approval automatically, while smaller everyday orders confirm straight through. The check happens automatically at the moment an order is confirmed, so nobody has to remember to submit large orders manually.

When to configure it. During initial setup for each company that wants order approval, or whenever the threshold needs to change. The workflow is **off by default** — a brand-new company confirms every sales order without approval until you turn this on.

Required permission: `SALES.SETTINGS.MANAGE` (the same permission both opens the screen and allows saving — there is no view-only tier). If you lack it the menu item is hidden and the screen shows *"You don't have permission to view sales settings."*

Navigate to **Sales › Sales Settings** (`/admin/sales-settings`) in the sidebar.

Setting the sales-order approval threshold

1. On the Sales Settings screen, choose the **Company** from the picker at the top. The settings below load for that company. (Each company is configured independently; switching the picker reloads its own settings.)
2. In the **SO Approval** card, use the **Enable SO Approval Workflow** switch:
    - **Off** (default) — every sales order confirms immediately, with no approval step. When the switch is off, no other fields are shown.
    - **On** — the threshold fields appear and the approval check becomes active for this company.
3. With the workflow enabled, fill in:
    - **Approval Threshold Amount** — the order value at or above which approval is required. The hint beneath the field reads *"Sales orders above this value require approval."* An order whose gross total is **below** this amount confirms straight through; an order **at or above** it is routed to approval when confirmed.
    - **Currency** — the currency the threshold amount is expressed in (defaults to `TZS`).
4. Click **Save Settings**. A confirmation *"Sales settings saved"* appears.

What happens next. Once enabled, the check runs automatically every time someone confirms a sales order for this company (including orders placed through the counter/POS path). Orders that meet or exceed the threshold are handed to the approvals engine and wait for a decision instead of confirming; orders below it are unaffected. Turning the switch back off removes the check entirely — subsequent orders confirm without approval.

> **A zero or empty threshold catches everything.** If the workflow is enabled and you leave the threshold at `0`, every order (any value) requires approval. The amount must be zero or positive — a negative value is rejected with *"Threshold amount must be zero or positive."*
>
> **The threshold is per company, not per branch.** All branches of the selected company share the same threshold. Configure each company separately.

Blocking sales that would take stock negative

What this control does. Below the SO Approval card, the same screen carries a **Block sales that would take stock negative** switch. It decides what happens when someone tries to sell more of a product than the branch actually has on hand:

- **On (Block)** — finalising an invoice or creating a delivery is **rejected** when there isn't enough stock to cover it. The user sees a friendly message naming the product, how much is available and how much was requested, and telling them to *"enable backorder in Sales Settings"* if the sale should go through anyway. Stock can never be driven below zero this way.
- **Off (Allow / backorder)** — overselling is permitted: the sale completes even when there isn't enough on hand, and stock is allowed to go negative (a backorder). Use this when you routinely sell ahead of receiving goods.

The hint beneath the switch reads *"When on, a sale is blocked if there isn't enough stock. Turn off to allow overselling (backorder)."* The check applies to ordinary stock-tracked products; services and other non-stocked items are never affected.

When it takes effect. The switch sits on its own, independent of the SO Approval workflow above — you can use either, both, or neither. To change it:

1. On the Sales Settings screen, choose the **Company** at the top.
2. Set the **Block sales that would take stock negative** switch on or off.
3. Click **Save Settings**. (The same `SALES.SETTINGS.MANAGE` permission governs this control.)

> **Allow until configured, then block by default.** A brand-new company that has **never opened and saved** Sales Settings is *not* yet guarded — its sales are allowed to oversell until you configure the screen. The moment you save Sales Settings for the first time, the switch defaults to **Block**, so a saved company blocks negative-stock sales unless you deliberately turn the switch off.

Fiscal / EFD Configuration

What this is. In markets that require it (for example Tanzania's TRA), a finalised sales invoice must produce a **fiscal / EFD receipt** — a tax-authority-registered receipt number and verification link. The system has a fiscalisation *seam*: a pluggable provider that talks to the fiscal device or service. Which provider is active is a **deployment-level setting an administrator/IT sets at the server**, not a screen inside the app — but the result of that setting is visible to operators on every finalised invoice, so administrators need to understand it.

Why it is a deployment setting, not an in-app form. A real fiscal device integration needs credentials and a certified adapter that are configured once per environment, not per user. The system deliberately **never fabricates** a fiscal number: until a real device is wired, it reports honestly that none is configured rather than inventing a receipt that would look like a filed tax document but is not.

The provider options. IT selects the active provider (`erp.fiscal.provider`) when deploying:

- `none` (the default) — no device is wired. Every fiscalisation attempt returns the honest status **NOT_CONFIGURED**; nothing is invented.
- `simulated` — a demo provider that produces a clearly-marked fake number, for **development and QA only**. The application **refuses to start in production** with this setting, so a demo receipt can never masquerade as a real one.
- `tra` (future) — the real tax-authority adapter, dropped in when it ships; selecting it is a configuration change, not an app change.

Required permissions (operator side).

- `FISCAL.VIEW` — see the Fiscal Receipt panel on an invoice.
- `FISCAL.MANAGE` — issue or retry a fiscal receipt.

These codes are seeded and granted to **ORG_ADMIN** automatically.

Where operators see fiscal status

Fiscalisation happens on a **finalised** sales invoice (a draft has no invoice number or frozen totals yet; a voided invoice is not fiscalised). Open the invoice at **Sales › Invoices** and scroll to the **Fiscal Receipt** panel (shown only on finalised invoices, and only to holders of `FISCAL.VIEW`).

The panel shows a status tag and, for holders of `FISCAL.MANAGE`, an **Issue EFD receipt** button (labelled **Retry EFD receipt** once an attempt already exists):

- **NOT_CONFIGURED** — no EFD device is configured for this company yet. The panel reads *"No EFD device is configured for this company yet."* This is expected whenever the provider is `none`; the attempt can be retried later once a device is wired.
- **ISSUED** — a real fiscal number and verification link were obtained; both are shown, with the verification URL opening in a new tab. This is terminal — an issued receipt is never re-issued (re-issuing would double-report to the tax authority), so no button is offered.
- **FAILED** — a device was attempted but errored (offline, rejected, timeout); a friendly reason is shown and you can **Retry**.
- **PENDING / VOID** — transitional/reserved states shown as a neutral tag.

> **NOT_CONFIGURED and FAILED are surfaced as errors, not silent successes.** Clicking Issue on a company with no device wired does not produce a receipt — the system tells you so plainly rather than pretending it worked. To actually issue real receipts, IT must configure a fiscal provider for the environment.

Audit Trail

What the audit trail is. The audit trail is a chronological, append-only log of every significant action performed in the system. Each record captures who did it (the actor), what they did (the action code), which record was affected (the target), in which company and branch, and when. It cannot be edited, backdated, or deleted — not even by `rootadmin`.

Why it exists. The audit trail serves several business and legal purposes. It creates accountability: every change to a user account, every permission grant, every password reset has a named, timestamped owner. It supports security investigations: if an account is suspected of misuse, the audit log shows exactly what actions it took and when. It satisfies compliance requirements: many financial regulations require evidence that access controls were applied and that privilege changes were authorised.

When audit records are written. Every create, update, status change (such as enabling or disabling a user), role grant, and role revocation generates an audit record. Login successes, failures, and

lockout events are also recorded. Actions by `rootadmin` that cross company boundaries produce an additional `ROOT.BYPASS` record. Audit records are written in the same database transaction as the action they record — if the action rolls back, the audit record rolls back with it.

What is and is not recorded. Action codes and target identifiers are always recorded. For profile-field edits (email, phone, display name) only the fact of the change is recorded — not the old or new values — to minimise personal data in the audit store. Passwords and token values are never recorded.

Required permission: `AUDIT.VIEW`

Navigate to **Administration › Audit** (`/admin/audit`) in the sidebar.

The audit trail is an append-only log of every significant action performed in the system — who did it, what they did, and when. It cannot be edited or deleted.

What the audit trail records

Every create, update, state change (such as enabling or disabling a user), grant, and revoke generates an audit record. Records include:

- The **action** (for example, `USER.CREATE`, `ROLE.GRANT`, `BRANCH.UNASSIGN`). Action codes are dotted throughout.
- The **actor** — the username who performed the action (shown as *system* when there is no signed-in user).
- The **target** — the type of the affected record (for example `USER`, `user_branch`).
- The **timestamp** (date and time).
- For cross-company actions by `rootadmin`, a special `ROOT.BYPASS` entry is also recorded.

The audit list is a table with columns **When**, **Actor**, **Action**, **Target**, **Scope** (Company or Branch), **IP**, and **Detail** (an expandable *view* link for any extra context). It is paginated.

Reviewing the audit log

1. Navigate to **Administration › Audit** (`/admin/audit`).
2. Use the filter bar at the top to narrow the results:
    - **Action** — a dropdown of known action codes (choose *Any action* for all).
    - **Target type** — a free-text field (for example `USER`).
    - **Actor** — a name picker that resolves to the chosen user.
    - **From** and **To** — a date-and-time range. Click **Filter** to apply, or **Clear filters** to reset.
3. The list shows the most recent events first. Use the pager to browse older records.

Audit records show the actor's username and the action and target *type* — not raw internal identifiers. Sensitive data (such as password hashes) is never included in audit details; for profile-field edits only the fact of the change is recorded, not the old or new values.

Key Concepts Reference

uid vs id — why records have two identifiers

What they are. Every record in the system carries two numeric identifiers. The `id` is an internal database sequence number used only for database-level joins between tables — you never see it in the interface or in URLs. The `uid` is a ULID (Universally Unique Lexicographically Sortable Identifier) — a 26-character string such as `01HY7FKMQ5T3V6NP8A2X4BQERD` — used in every URL and in every place where a record is referenced outside the database.

Why two identifiers. Sequential numeric IDs in URLs expose information about record counts (a competitor could learn how many orders you have by looking at the URL of the latest one). ULIDs are opaque and reveal nothing about volume or sequence. They are also time-sortable, collision-resistant, and URL-safe without encoding. The internal `id` remains for efficient database foreign-key joins, which are performance-sensitive. This duality is a deliberate design choice throughout the system.

What this means for you. You never need to know or type a uid. The system passes uids between pages in URLs automatically. When you pick a user, a branch, or a role by name in a form, the picker stores the uid in the background — the interface always works by name.

Fiscal period

What a fiscal period is. A fiscal period is a defined accounting time window — typically a calendar month — within a financial year. Transactions (invoices, payments, journal entries) are posted to an open period; once a period is closed, historical records in it cannot be altered.

Why fiscal periods matter. They are the foundation of financial reporting: a profit-and-loss statement, a balance sheet, and a cash-flow report all aggregate transactions by period. Closing a period locks the numbers so that prior-period reports are stable and comparable. Without fiscal periods, a transaction entered late could silently change a figure that was already reported.

When you interact with periods. Accountants and finance managers interact with fiscal periods when posting journal entries, running period-end reports, and performing the year-end close. Transactions in operational modules (sales, purchases, inventory) automatically post to the current open period of the active branch's company.

Master Data

Master data is the reference information shared across the system: the parties you trade with, the products you sell or buy, the prices you charge, the currencies you transact in, the taxes you apply,

and the routes your sales team covers. Set this up first; every transaction in Sales, Procurement, Inventory, and Finance depends on it.

All master data screens are under the **Admin** section of the navigation. Your access depends on the permissions assigned to your role — the sections below note which permission is required for each area.

Customers

Navigation: Parties › Customers (`/admin/customers`) | **Permission to view:** `CUSTOMER.VIEW` | **Permission to create / edit:** `CUSTOMER.MANAGE`

A **customer** is any person or organisation that your business sells to. The customer record is the permanent, reusable identity for that buyer: it carries their legal details, contact information, VAT registration, and credit terms, and it is referenced by every sales document you raise against them. Without a customer record you cannot create a quotation, a sales order, or an invoice for that buyer.

Why it exists. Storing buyer details once — rather than re-entering them on every sale — gives you consistent names on documents, a single place to update a phone number or credit limit, an audit trail of all transactions with that party, and the foundation for aged-debtor reporting. The customer record is also the control point for credit: a customer classified as a credit-account holder carries a credit limit the sales process can check.

When it is used. A customer record is created by a sales administrator or master-data manager before (or during) the first sale to that party. It is used every time a quotation, sales order, or invoice is raised, and every time a payment or receipt is applied to that buyer's account.

How it works. A customer is created with status **Active**, assigned a system-generated code (`CUST-0001`, `CUST-0002`, …), and scoped to one company. It is then associated with one or more branches so that those branches can see and select it in sales flows. Archiving a customer makes it unavailable for new transactions but preserves it in historical records. The record can be restored at any time.

Customers are the parties you sell to. Each customer belongs to one company and carries a system-generated code (`CUST-0001`, `CUST-0002`, …). You never enter or see the internal uid — the system uses that behind the scenes.

Customer types

Every customer record has two classification fields set at creation time:

Field	Options	Notes
Party type	Individual, Business	A Business party must have a TIN at creation.
Kind	Cash / Walk-in, Credit Account	Credit-account customers carry a credit limit and payment terms (set on the

		detail page).

Party type distinguishes a private individual from a registered legal entity. For a business, a Tax Identification Number (TIN) — the government-issued taxpayer reference — is required at creation because it must appear on formal tax invoices. Individuals are exempt. On the detail/edit page the TIN is no longer blocked for businesses; the label there reads *(recommended for businesses)*.

Kind describes the trading relationship. A **Cash / Walk-in** customer pays at the point of sale; no ongoing credit account is maintained. A **Credit Account** customer is extended a line of credit: the business ships goods or delivers services now and expects payment within agreed terms (for example, 30 days). Credit-account customers therefore carry a **credit limit** (the maximum outstanding balance the business will allow) and **payment terms** (the number of days before payment is due). These fields are **not** part of the create form — they appear on the customer detail page when the Kind is set to Credit Account, and are hidden for Cash / Walk-in customers.

Once saved, Party type and Kind can be changed on the detail edit form.

How to create a customer

The create form is deliberately **minimal** — it captures only the fields needed to identify the party. Everything else (credit terms, contact details, addresses) is added afterwards on the customer detail page.

1. Navigate to **Parties › Customers** (`/admin/customers`).
2. Click **New Customer**. An inline form appears below the toolbar.
3. Enter the **Display name** (required).
4. Select **Party type** (Individual or Business).
5. Select **Kind** (Cash / Walk-in or Credit Account).
6. If **Party type** is **Business**, an identity row appears with a required **TIN** plus optional **Legal name**, **Business reg. no.**, and a **VAT registered** checkbox. If you tick **VAT registered**, a **VRN** field appears — you cannot enter a VRN unless VAT registered is ticked. (For an Individual, only an optional **TIN** is shown.)
7. Click **Create**.

The system assigns a unique code and sets the status to **Active**. The new row appears in the list immediately.

> **No credit, contact, or address fields at create time.** Selecting **Credit Account** here does **not** reveal credit-limit or payment-terms inputs, and there are no phone, email, or address fields on the create form. To set a credit limit, payment terms, contact details, or addresses, open the new customer's detail page after creating it (see *How to view and edit a customer* below).

How to search for a customer

On the **Parties › Customers** (`/admin/customers`) list:

- Type in the **Search** box (placeholder **Name, code…**). Typing filters the list automatically after a short pause; pressing Enter or clicking the search button applies it immediately. The search matches on name or code and is case-insensitive.
- The list resets to the first page when you start a new search.
- Click **Clear** to return to the full unfiltered list.
- The list is paginated; use the pager at the bottom (First / Previous / page numbers / Next / Last) to move between pages. See *List screens — search and pagination* in **Getting Started › Common UI Patterns**.

How to view and edit a customer

1. Click the **Edit** action on any row in the customer list to open the detail page (`/admin/customers/uid/<uid>`).
2. The URL contains the customer's uid — you do not need to read or type this.
3. The detail page carries the **full** set of customer fields — far more than the create form. In addition to Display name, Legal name, Party type, Kind, TIN, VAT registered and VRN, and Business reg. no., you can set:

    - **Contact:** Phone, Mobile money no., Email.
    - **Address:** Physical address, Postal address, Region, District.

4. If Kind is **Credit Account**, three more fields appear: **Credit limit amount**, **Currency**, and **Payment terms (days)**. If Kind is **Cash / Walk-in**, these are hidden — switch to Credit Account to reveal them.
5. The credit-limit **Currency** is chosen with the **Currency Picker** (a dropdown of the company's enabled currencies, defaulting to the company default) rather than free-typed — see **Getting Started › Common UI Patterns**.
6. Click **Save changes** to apply.

The header status badge, the Kind tag, and the **Archive** / **Restore** controls sit above the form.

How to archive and restore a customer

An archived customer remains in the database for historical reporting but is not available for new transactions.

1. Open the customer detail page (`/admin/customers/uid/<uid>`).
2. Click **Archive**. The status badge changes to **Archived**.
3. To reverse, click **Restore**. The status returns to **Active**.

Archiving and restoring are both immediate and do not require a reason.

Branch associations

A customer can be associated with specific branches of your company. This determines which branches can see the customer in their scoped views.

A **branch association** links a party to a specific operating location within the company. Because your business may have multiple branches (offices, warehouses, or sales points), each transaction is tied

to the branch that raised it. A customer that has not been associated with any branch will not appear in selection lists at any branch, even though the record exists in the system. Associating a customer with a branch makes them available to that branch's sales team.

1. Open the customer detail page (`/admin/customers/uid/<uid>`).
2. Scroll to the **Branch Associations** panel.
3. Select the **Company** from the first dropdown, then select the **Branch** (shown as `code — name`) from the second.
4. Click **Assign**. The branch appears in the association list with the date it was assigned.
5. To remove a branch, click **Remove** on the relevant row.

You need the `PARTY.BRANCH.ASSIGN` permission to assign or remove branches. You can only assign branches that belong to the same company as the customer.

Example — Register a new credit-account business customer

Scenario: Sales admin Fatuma Msongo is on-boarding Karibu Wholesale Ltd, a new B2B buyer on 30-day credit terms.

1. Navigate to **Parties › Customers** (`/admin/customers`). Click **New Customer**.
2. Enter Display name `Karibu Wholesale Ltd`, Party type `Business`, Kind `Credit Account`.
3. In the Business identity row, enter TIN `100-456-789`, Legal name `Karibu Wholesale Limited`. Tick **VAT registered** and enter VRN `40-045678-H`.
4. Click **Create**. The system assigns code `CUST-0012` and status **Active**. (No credit-limit, payment-terms, contact, or address fields appear at this stage.)
5. Click the **Edit** action on the `CUST-0012` row to open `/admin/customers/uid/<uid>`.
6. In the **Details** form, the credit-account fields are now shown. Enter Credit limit amount `5000000`, leave **Currency** at the pre-selected company default (or pick another enabled currency from the **Currency Picker**), and Payment terms (days) `30`. Optionally fill in Phone `+255 22 211 0099`, Email `orders@karibuwholesale.co.tz`, Region `Dar es Salaam`. Click **Save changes**.
7. In the **Branch Associations** panel, select Company `Orbix Trading Co.`, Branch `DSM — Dar es Salaam Branch`. Click **Assign**. The branch association is saved.

Karibu Wholesale Ltd is now available as a customer on all sales flows for the DSM branch.

Suppliers

Navigation: Parties › Suppliers (`/admin/suppliers`) | **Permission to view:** `SUPPLIER.VIEW` | **Permission to create / edit:** `SUPPLIER.MANAGE`

A **supplier** is any person or organisation that your business purchases from. The supplier record is the permanent identity for that vendor: their legal details, tax registration, contact information, and

the kind of goods or services they provide. Without a supplier record you cannot raise a purchase order, record a goods receipt, or register an invoice from that vendor.

Why it exists. Centralising supplier details ensures that purchase orders always go to the right party with the right tax and legal details, that every procurement transaction is traceable back to an approved supplier, and that accounts-payable balances can be correctly allocated. It also enables three-way matching: matching a purchase order to a goods receipt to a supplier invoice — the core control that prevents paying for goods you did not order or receive.

When it is used. A procurement officer or master-data manager creates the supplier record before (or at the time of) the first purchase from that vendor. It is referenced on every purchase order, goods receipt, and supplier invoice.

How it works. Suppliers follow the same lifecycle as customers: created **Active**, assigned a `SUPP-####` code, scoped to one company, associated with branches, and archivable. The key difference from a customer is the **Supplier Kind** field — Goods or Service — which indicates the nature of supply. There are no credit-limit or payment-terms fields on a supplier record; those are managed on the AP (Accounts Payable) side.

Suppliers are the parties you purchase from. The data structure mirrors customers, with one difference: the kind field distinguishes **Goods** suppliers from **Service** suppliers (there are no credit limit or payment terms fields on a supplier record).

Supplier codes are prefixed `SUPP-` (for example, `SUPP-0001`).

How to create a supplier

The supplier create form mirrors the customer one and is equally **minimal**.

1. Navigate to **Parties › Suppliers** (`/admin/suppliers`).
2. Click **New Supplier**.
3. Enter **Display name** (required), **Party type**, and **Kind** (Goods or Service).
4. If Party type is Business, an identity row appears with a required **TIN** plus optional **Legal name**, **Business reg. no.**, and a **VAT registered** checkbox (which reveals a **VRN** field). For an Individual, only an optional **TIN** is shown.
5. Click **Create**.

The same rules apply: TIN required at creation for Business parties, VRN only when **VAT registered** is ticked. Contact details and addresses (where applicable) are added on the supplier detail page after creation, exactly as for customers.

Search, edit, archive, restore, and branch associations

These work exactly as described for Customers above, substituting the **Parties › Suppliers** (`/admin/suppliers`) screen and the detail page at `/admin/suppliers/uid/<uid>`, using the `SUPPLIER.MANAGE` / `PARTY.BRANCH.ASSIGN` permissions.

Example — Register a goods supplier

Scenario: Procurement officer Hassan Kamau adds Tembo Industries Ltd as a VAT-registered goods supplier.

1. Navigate to **Parties › Suppliers** (`/admin/suppliers`). Click **New Supplier**.
2. Enter Display name `Tembo Industries Ltd`, Party type `Business`, Kind `Goods`.
3. In the Business identity row, enter TIN `100-789-321`. Tick **VAT registered** and enter VRN `40-078901-T`.
4. Click **Create**. System assigns code `SUPP-0008` and status **Active**.
5. Open the supplier from its **Edit** action to add contact details (Phone `+255 27 254 4400`, Region `Arusha`) and any branch associations.

Other Parties

Navigation: Parties › Other Parties (`/admin/other-parties`) | **Permission to view:** `OTHERPARTY.VIEW` | **Permission to create / edit:** `OTHERPARTY.MANAGE`

An **other party** is any third party that your business has a financial or operational relationship with but that does not fit neatly into the customer or supplier categories. Common examples include landlords (you pay rent to them), utility providers (you pay electricity or water bills), regulatory bodies (you pay licence fees or levies), and freight or clearing companies (you pay logistics costs). Without an other-party record, these payments would have no addressable counterpart in the system.

Why it exists. The customer and supplier masters are purpose-built for sales and procurement flows. Forcing every conceivable counterpart into those categories would pollute the selection lists that sales and procurement teams use daily. Other Parties is a catch-all master that keeps the core lists clean while still giving every payable a named, traceable counterpart for accounting and audit purposes.

When it is used. A finance administrator or master-data manager creates an other-party record when a new type of expenditure or relationship arises that is not covered by the supplier master — for example, when setting up a monthly rent payment to a landlord for the first time.

How it works. Other parties follow the same lifecycle as customers and suppliers: created **Active**, assigned an `OTHR-####` code, scoped to one company, and archivable. The only structural difference is the **Kind / category** field (shown as **Kind** in the list column), which is free text rather than a fixed list. You can type any descriptive label (for example, `Landlord`, `Utility`, `Freight Forwarder`) to classify the party informally.

Other Parties covers any third party that is not a customer, supplier, or agent — for example, landlords, regulatory bodies, utility providers, or freight companies. Other Party codes are prefixed `OTHR-`.

The key difference from customers and suppliers is the **Kind / category** field (the *Kind* column in the list), which is free text (not a fixed list). You can type any label, such as "Landlord", "Utility", or "Freight Forwarder". The field is optional.

All other behaviour — TIN rule for Business parties, VAT/VRN pairing, archive/restore lifecycle, and branch associations — is identical to Customers and Suppliers. The detail page for an other party is at `/admin/other-parties/uid/<uid>`.

Sales Agents

Navigation: Parties › Sales Agents (`/admin/agents`) | **Permission to view:** `AGENT.VIEW` | **Permission to create / edit:** `AGENT.MANAGE`

A **sales agent** is the person or organisation responsible for bringing in a sale. An agent is credited on sales documents (quotations, orders, invoices) and is the link between a customer and the company's sales team. Agents are referenced by distribution routes, by opportunities in the CRM module, and by sales invoices — where the agent's primary route is automatically carried across to provide a geographic reference for the sale.

Why it exists. Tracking which agent made which sale enables commission reporting, performance management, and territory analysis. The agent is also the connection between the geographic route structure and individual sales staff: assigning an agent to a route as its primary agent means that any sale to a customer on that route is automatically tagged with the correct route on the invoice.

When it is used. A master-data manager or HR administrator creates an agent record when onboarding a new sales representative (internal) or registering a new external reseller or freelance agent (external). The agent is then assigned to routes and used on sales documents.

How it works. The agent has a status lifecycle identical to customers and suppliers (Active → Archived → Active). An important distinction governs the agent's relationship to the system's user accounts:

Sales agents represent the people or organisations that sell on your behalf. Agent codes are prefixed `AGNT-`.

Agent kinds

Kind	Meaning	User link
Internal	An employee who is also an app user	Must be linked to an active user in the same company
External	A third-party agent, not an app user	Must NOT be linked to an app user

An **Internal** agent is a staff member who logs in to the system. Linking the agent record to a user account enables the system to associate that person's sales activity with their login identity — useful

for task lists, permission-gated views, and commission attribution. The linked user must be active and belong to the same company.

An **External** agent is a freelance representative, a distributor, or a third-party reseller who does not have a login to your ERP system. They are tracked as a party for document and reporting purposes only.

How to create an agent

1. Navigate to **Parties › Sales Agents** (`/admin/agents`).
2. Click **New Agent**.
3. Enter **Display name**, **Party type**, and **Agent kind** (the dropdown options read **External** and **Internal (IAM user)**).
4. If Kind is **Internal (IAM user)**, an **App user** selector appears. Choose the user from the list (each option shows the display name and username). The system stores the link internally — you do not type a user id.
5. If Kind is **External**, the App user selector is hidden.
6. Click **Create**.

Switching an agent between Internal and External

On the agent detail page (`/admin/agents/uid/<uid>`), changing Kind from Internal to External clears the user link automatically on save. Changing from External to Internal requires you to select a user before saving.

Search, edit, archive, restore, and branch associations

These work as described for Customers, using the **Parties › Sales Agents** (`/admin/agents`) screen and the `AGENT.MANAGE` and `PARTY.BRANCH.ASSIGN` permissions.

Example — Create an external field agent and assign them to a route

Scenario: Operations manager registers Juma Rashidi as a freelance distribution agent for the Coast route.

1. Navigate to **Parties › Sales Agents** (`/admin/agents`). Click **New Agent**.
2. Enter Display name `Juma Rashidi`, Party type `Individual`, Agent kind `External`.
3. Click **Create**. System assigns code `AGNT-0004`.
4. Open the route at **Parties › Routes** (`/admin/routes`), click the **Edit** (pencil) action on the **Coast Distribution Route** row.
5. In the **Agents** panel, type `Juma` and select `AGNT-0004 — Juma Rashidi`. Tick **Primary**. Click **Assign**.

Products

Navigation: Products › Products (`/admin/products`) | **Permission to view:** `PRODUCT.VIEW` | **Permission to create / edit:** `PRODUCT.MANAGE`

A **product** is any item or service that your business sells, buys, or manufactures. The product record is the central catalogue entry that links a name and code to its cost, its selling prices, its unit of measurement, and — for stocked goods — its inventory tracking. Every sales line, purchase line, and stock movement references a product record.

Why it exists. Without a product catalogue, every transaction would require staff to invent descriptions, prices, and codes on the spot — leading to inconsistency, mispricing, and an inability to report on what was sold or bought. The product master is the single source of truth for what the business trades in: it enforces consistent naming, links prices to agreed price lists, defines the packaging hierarchy (base unit and bulk packs), and controls whether an item appears in sales or procurement flows.

When it is used. A catalogue manager or product administrator creates product records before the first transaction involving those items. Products are used on every sales quotation and order (if sellable), every purchase order and goods receipt (if a goods product), every stock movement (if stockable), and every manufacturing or assembly job (if it has a recipe).

How it works. A product is created **Active** with a `PROD-####` code (or a custom code you supply), scoped to one company, and associated with branches. Its lifecycle is Active → Archived → Active. Once created, you can add barcodes for scanning at the point of sale, define bulk-pack conversions (for example, 50 kg bags per carton), set selling prices on each of your price lists, and define a component recipe for manufactured or bundled items. You can also build all of this — identity, pricing, units of measure, opening stock and barcodes, and branch availability — in a single pass on the **Product Master** screen (the **Full product form** button on the products list) instead of visiting each panel separately; see *The Product Master — one screen for the whole product* below.

Products are the items you sell, buy, or manufacture. Each product belongs to one company and carries a system-generated code (for example, `PROD-0001`) unless you supply your own code at creation time.

Product types

Field	Options	Rules
Type	Goods, Service	Service products cannot be stockable (the Stockable checkbox is forced off).
Stockable	Yes / No	Only Goods products can be stockable.
Sellable	Yes / No	Controls whether the product appears in sales flows.
VAT Status	Standard, Zero-rated, Exempt	Defaults to Standard.

Type determines the fundamental nature of the item. A **Goods** product is a physical item that can be received into stock, transferred between locations, and dispatched to customers. A **Service** product is an intangible deliverable — consulting, installation, transport — that cannot be stocked or counted in a warehouse. This distinction matters because inventory and stock-movement rules apply only to goods.

Stockable controls whether the system maintains an inventory balance for this product. A non-stockable good might be a consumable expensed immediately on purchase; a non-stockable service is an intangible. Only goods can be stockable — the system prevents a service product from being marked stockable because there is nothing physical to count.

Sellable controls whether the product appears on sales quotations and orders. An internal intermediate product used only in manufacturing recipes would typically not be sellable.

VAT Status determines how value-added tax is calculated on sales lines for this product. **Standard** applies the current standard VAT rate (18%). **Zero-rated** applies 0% — the line is technically within the VAT system but taxed at nil (common for basic food items in some jurisdictions). **Exempt** items are outside the VAT system entirely and produce no VAT entry. These statuses drive the tax lines on invoices and the VAT return.

Age restriction

A product can carry an **age-restriction classification** that marks it as something which may only be sold to buyers above a certain age — for example, alcohol or tobacco. There are three settings: **None**, **18+**, and **21+**. Every product is **None** by default, so existing products are unaffected and nothing changes until you deliberately mark an item as restricted.

The classification is purely a label on the product record; on its own it does not block anything. Its effect is felt at the point of sale: when a cashier rings up a product marked **18+** or **21+**, the till prompts the cashier to confirm the buyer's age before the sale can complete (see **Point of Sale** for how this works at the till). Setting a product back to **None** removes the prompt for that item.

Weighed goods (sell by weight)

Some products are not sold by the piece but **by weight** — loose rice, sugar, produce, or deli goods scooped and weighed on a counter scale. For these, the price is quoted per weight unit (for example, TZS 2,000 per kg) and the sale line carries the **actual weighed amount** (0.813 kg), not a whole-number count. Marking a product as *sold by weight* makes that intent explicit so it is billed as a weight, never rung up as "1 unit".

The prerequisite: a weight base unit. A weighed product's **base unit must be a weight unit** — kilograms or grams — because the whole point is to price and record a weight. The seeded units of measure `KG` and `GRAM` are weight units; `EA`/`PCS` and volume units (litre, millilitre) are not. If the product's base unit is not a weight unit, the **Sold by weight** switch is disabled and a hint reads *"Choose a weight base unit (e.g. kg) to sell this by weight."* Pick a weight base unit when you create the product (or change it before turning weighing on).

Where you set it. On the product detail page (`/admin/products/uid/<uid>`) there is a **Weighed Goods** card; the same controls also appear on the **Product Master** one-screen form. It has one switch and three optional fields:

Field	What it does
Sold by weight	Turns weighing on for the product. When on, the sale path requires a weight unit on every line for this product and rejects a by-count unit with a clear *"…is sold by weight — choose a weight unit such as kilograms"* message.
Tare weight	The container/packaging weight, in the base unit (e.g. `0.050` kg for a tub). This is **configuration for the weighing client** to deduct before it submits the net weight — the server does **not** auto-deduct it, so a net scale label is never double-tared. Optional.
Scale step	The scale's division — the smallest weight increment it reports, e.g. `0.005` kg. When set, the sold weight is **rounded to the nearest multiple** of this step (see *How rounding shows on the sale* below). Leave blank for no rounding.
Max sale weight	A safety ceiling, in the base unit (e.g. `25` kg). A weighed line above this weight is **rejected** — it catches a kg/gram fat-finger such as keying `813` (meaning 813 g) against a per-kg product and ringing 813 kg. Optional.

Fill in the fields you want and click **Save weighing profile**. All three numeric fields are optional — turning **Sold by weight** on with all three blank simply enforces the weight-unit rule with no rounding and no ceiling.

How rounding shows on the sale. When a scale step is set, entering a weight that is not an exact multiple rounds it to the nearest division — 0.813 kg at a 0.005 kg step bills as 0.815 kg. So the customer can see the adjustment was deliberate, the sale line records both figures and displays *"entered 0.813 → billed 0.815"* (see **Sales and Point of Sale** for the seller's view). A weight so small it would round down to zero is rejected with *"that weight is too small to sell. The smallest amount is 0.005 Kilogram."* (the unit shown in these messages is the product's unit **name** — `Kilogram` for the seeded `KG` unit — not its short code).

Example — set up loose rice sold by the kilogram. A shop sells rice scooped from a sack, priced at TZS 2,000/kg, on a scale that reads to 5 g. Create the product `Loose Rice`, Type `Goods`, Base unit `KG — Kilogram`, Sellable + Stockable, and set a `RETAIL` price of 2,000. Open its detail page, and in the **Weighed Goods** card turn on **Sold by weight**, set Scale step `0.005`, Max sale weight `50`, and (if the scoop tub weighs 10 g) Tare weight `0.010`. Click **Save weighing profile**. From now on a sales line for this product must use kg, its weight rounds to the nearest 5 g, and any line above 50 kg is refused.

How to create a product

1. Navigate to **Products › Products** (`/admin/products`).
2. Click **New Product**.
3. Optionally enter a **Code**. If you leave it blank the system assigns `PROD-####` (the hint reads *Blank = PROD-####*). If you type a code it is trimmed of spaces and converted to upper case.
4. Enter the **Name** (required).
5. Select **Type** (Goods or Service). If you select Service, the **Stockable** checkbox becomes unavailable (shown as *N/A for service*).
6. Select the **Base unit** from the dropdown by its code and name (for example, `EA — Each`). Only active units of measure are offered. (If no units exist yet, a *Create units first* link appears.)
7. Tick **Sellable** and/or **Stockable** as required.
8. Optionally enter a **Description**, then set the **VAT Status**.
9. Enter the **Cost amount**. The **Currency** beside it is the **Currency Picker** — a dropdown of the company's enabled currencies, pre-set to the company default — not a free-text code (see **Getting Started › Common UI Patterns**).
10. Click **Create**.

The Product Master — one screen for the whole product

What it is. Next to **New Product** on the products list toolbar, a **Full product form** button opens the **Product Master** (`/admin/products/master`, permission `PRODUCT.MANAGE`) — one screen, organised into five tabs, that captures the whole product record in a single save: identity, pricing, supplier and units of measure, opening stock and barcodes, and branch availability.

Why it exists. The quick create form above, followed by the separate Barcodes, Bulk Packs, Product prices, and Branch Associations panels on the product detail page, gets you there, but takes several round trips even for a straightforward new item. The Product Master orchestrates the same underlying steps from one screen: it creates the product first, then submits each section you filled in, in turn, against the new product's uid — reporting exactly which parts saved and which need attention if a step fails partway through.

How it works. The five tabs are **General**, **Pricing**, **Supplier & UoM**, **Stock & Barcodes**, and **Branches**. Switching tabs does not save anything by itself — nothing is written until you click the save button at the bottom of the screen.

1. Navigate to **Products › Products** (`/admin/products`) and click **Full product form**.
2. If your organisation has more than one company, select the **Company** first (the picker only appears when there is more than one; it is fixed once the product is created).
3. On the **General** tab, enter the identity fields: optionally a **Code** (blank assigns `PROD-####`; not editable once created), the required **Name**, **Type** (Goods or Service — Service forces **Stockable** off), **VAT Status**, **Description**, **Department / Category** (free text, not linked to HR departments), **Brand / Trade name**, **Manufacturer**, **HS Code**, **Image URL**, the **Sellable** / **Stockable** / **Purchasable** flags, the **Lot tracked** / **Serial tracked** / **Expiry tracked** flags (disabled when **Type** is Service), and **Internal Notes**.
4. On the **Pricing** tab, enter the **Cost (buying) price** — an **Amount** and a **Currency** (the **Currency Picker**) — then click **Add price list** for each **Selling price** row and set its **Price list**, **Amount**,

Currency, and optional **Effective from** date. A first selling-price row is pre-added automatically, pre-selected to the company's default price list where the system can resolve one (the list flagged as default, a list coded `DEFAULT`/`STANDARD`, or the only list that exists); otherwise it is left for you to choose.

5. On the **Supplier & UoM** tab (headed *Supplier & Unit of Measure*), select the required **Base unit of measure**, optionally add **Pack / bulk units** — a **Unit** and a **Factor to base**, then click **Add** — the same bulk-pack conversions described under *Bulk packs* below, optionally search for and select a **Preferred supplier**, and set the **Stock planning defaults** (**Reorder level**, **Reorder qty**, **Safety stock**, **Min stock**, **Max stock** — disabled when **Type** is Service, since a Service product cannot be stockable).
6. On the **Stock & Barcodes** tab, add any **Barcodes / Article numbers** — a **Barcode value**, optional **Type** and **Unit**, and a **Primary** checkbox (the first barcode you add is marked primary automatically even if you don't tick it) — and, if the product is stockable, an **Opening stock Quantity** and **Note**, seeded into your current active branch.
7. On the **Branches** tab, leave **Make available in all branches** ticked (the default) to activate the product everywhere, or untick it and set each branch's own **Active** switch, **Reorder level** (stockable products only), and **Branch price** (blank inherits the price-list price).
8. Click **Create product**.

Validation. Name and **Base unit of measure** are required — leaving either blank shows an error and switches you to the tab that needs it. If you add any barcodes, exactly one must be marked **Primary**.

The save result. If the product itself fails to save, the error is surfaced as a toast (for example a duplicate code) or, for an unexpected failure, the "Something went wrong" dialog — the form shows no inline message and the result panel below does not appear until the product has actually been created. Correct the issue and click **Create product** again. Once the product itself has saved, the screen always shows a result panel listing every section — **Product**, **Selling prices**, **Barcodes**, **Pack units**, **Branch availability**, **Opening stock** — each with a status: a check for a section that saved, a cross with the error message for one that failed, or a dash for one you left empty. Click **Retry** next to a failed section (other than **Product** itself) to resubmit just that part without repeating the whole form. From here, click **Open product** to go to the product's detail page, **Back to list** to return to the product list, or — if any section failed — **Continue editing** to go back to the form.

Editing. The product list's **Edit** action opens the classic product detail page described in the sections below (Barcodes, Bulk packs, Product prices, Product components, Branch Associations), not the Product Master screen.

How to set a custom code

Type the code in the **Code** field. The system converts it to upper case (so `sku-001` becomes `SKU-001`). Codes must be unique within the company — if you enter a duplicate you will see an error after you submit.

How to edit a product

1. Click the **Edit** (pencil) action on any product row to open the detail page (`/admin/products/uid/<uid>`).

2. Modify fields as needed. The **Code** field is read-only on the detail page.
3. Click **Save changes**.

If you change Type from Goods to Service, the Stockable checkbox is forced off automatically.

How to archive and restore a product

Open the product detail page (`/admin/products/uid/<uid>`) and click **Archive** (to make it unavailable) or **Restore** (to make it active again). Archived products are excluded from order lines and component pickers.

How to search for a product

The Products list toolbar has two lookups:

- A **Search** box (placeholder **Name, code…**) that filters the list by name or code as you type, with **Search** and **Clear** buttons.
- A **Barcode** lookup with the icon and placeholder **Scan or enter barcode…**. Scan or type a barcode and click the barcode button: a match opens an info banner showing the product code, name, and type with a **View** link; if nothing matches you see *No product found for that barcode*.

The list is paginated — use the pager at the bottom to move between pages.

Branch associations

Works exactly as described for Customers. The permission required is `PRODUCT.BRANCH.ASSIGN`.

Barcodes

A **barcode** is a scannable value printed on product packaging — EAN-13, UPC, QR code, or a supplier's own code. Registering barcodes against a product enables point-of-sale staff to scan an item and have the system identify it instantly, rather than searching by name or code. One barcode is designated **primary** — it is the default identifier used on documents and the one that scanning resolves to first.

In the **Barcodes** panel on the product detail page:

1. Type the **Barcode value**.
2. Tick **Set as primary** if this is the product's primary barcode.
3. Click **Add Barcode**.
4. To remove a barcode, click **Remove** on the relevant row.

Each barcode row shows a **Primary** or **Secondary** tag.

Scale labels (weight and price barcodes)

Supermarket scales print their own labels for loose goods sold by weight — for example, a label on a tray of meat that carries the item plus the weighed amount or the calculated price inside the barcode itself. The system can read these labels: when such a label is scanned in the product barcode lookup or at the till, it identifies the product and reads the embedded weight or price out of the barcode automatically, so the cashier does not have to key in the amount.

How a particular store's labels are laid out is set up once by an administrator as a set of **barcode symbology rules** for the company (this requires the `PRODUCT.SYMBOLOGY.MANAGE` permission). These rules are configured in the back office rather than on a screen in the main application; once they are in place, the product **Barcode** lookup and the till accept scale labels with no further setup. If your business does not sell weighed goods, you can ignore this — ordinary EAN-13 and UPC barcodes work without any rules.

Bulk packs

A **bulk pack** defines how a product is packaged for storage or sale in larger quantities than its base unit. For example, if the base unit is `EA` (Each), a carton might contain 24 units. Bulk packs are used in procurement (ordering by the carton), in warehousing (counting by pallet or crate), and in wholesale sales (pricing by the case). The **factor** is the number of base units in one pack — the conversion ratio that lets the system translate between units.

Bulk packs define how many base units fit into a larger packaging unit (for example, 24 `EA` in a `CTN — Carton`).

1. In the **Bulk Packs** panel, select the **Unit** (the larger packaging unit) from the dropdown by code and name.
2. Enter the **Factor to base** — the number of base units in one pack (must be greater than zero).
3. Click **Add Bulk Pack**.
4. To remove a bulk pack, click **Remove**.

A product's allowed units of measure. A product's base unit (set when it is created) plus any bulk packs added here together form the **complete set of units this product can be transacted in**. Everywhere a line item lets you pick a unit — purchase orders, sales orders, sales invoices, sales quotations, RFQs, blanket and standing orders, purchase requisitions, CRM opportunities, and Point of Sale — the **Unit** field on that line loads only this product's configured units once you select the product, defaults to the base unit, and stays disabled until a product is chosen. You cannot pick a unit that isn't this product's base unit or one of its active bulk packs; the system rejects any other unit rather than silently mis-converting the quantity.

Product prices

A **product price** is the selling price of this product on a specific price list, for a specific **unit**. A price must be set on a price list before the product can be sold at that list's rate. You can maintain different prices on different lists — for example, a higher retail price and a lower wholesale price for the same product — and, on each list, a separate price for each pack the product is sold in.

The **Prices** table has three columns: **Price list**, **Unit**, and **Price**. The **Unit** column shows **Base** for a price set against the product's base unit, or the pack's unit name (for example, `Carton`) for a per-pack price.

You can set a selling price for this product on each of your price lists.

1. In the **Prices** panel, under **Set Price**, select the **Price list** by its code and name.

2. Choose the **Unit**. This dropdown defaults to **Base unit** and also lists every **bulk pack** you have configured for the product (see *Bulk packs* above). Pick **Base unit** to price the base unit, or a pack unit to set that pack's own price.
3. Enter the **Amount**. The **Currency** beside it is the **Currency Picker** (the company's enabled currencies, defaulting to the company default) — you pick from the list rather than typing a code (see **Getting Started › Common UI Patterns**).
4. Click **Set Price**.

You can only price a pack unit that is already a configured bulk pack. If the unit you want is not in the dropdown, add it in the **Bulk Packs** panel first (with its factor to base), then return here. Trying to price a unit that is not the base unit and not a configured pack is rejected.

How the base row and pack prices work together. Setting a **Base unit** price is the normal case and is all most products need: when the product is sold in a pack, the system multiplies the base-unit price by that pack's **factor to base**. So a product priced at 5 per base unit, sold in a `Carton` of factor 24, is offered at 120 for the carton automatically — you do not have to enter it.

Non-linear pack pricing. When a pack should *not* simply be the base price times the factor — for example a 1,000 g pack that you sell for less than 1,000 times the per-gram price as a bulk incentive — set a price directly against that pack's unit. That explicit per-unit price is then used as-is for the pack and overrides the factor calculation; the base-unit price still governs every other unit. This lets you price a big pack independently of the per-unit rate.

Each price list holds **one price per unit** for the product: setting a price for a price-list-and-unit combination that already has one overwrites it. Setting a **Base unit** price does not touch any pack price, and vice versa. To remove a price, click **Remove** on its row — this removes only that price list's price for that specific unit (Base or the named pack).

Product components (recipe)

A **product component** (also called a recipe or bill of materials) records the constituent parts of a composed product — for example, the raw materials needed to assemble a finished good, or the individual items bundled together in a gift set. In the current version, the recipe records the structure only: it does not automatically trigger stock movements or cost calculations. That behaviour is reserved for the manufacturing module.

Components define the ingredients or sub-products that make up this product — used in manufacturing or bundled sales.

1. In the **Components / Recipe** panel, start typing a product name in the search box.
2. Select the component product from the results (shown as `code – name`). The product itself and archived products are excluded from the list.
3. Enter the **Quantity** (must be greater than zero).
4. Click **Add Component**.
5. To remove a component, click **Remove** on the row.

Example — Set up Sugar (1 kg) with a retail price, a carton bulk pack, and a barcode

Scenario: Catalogue manager sets up a new FMCG line before the first purchase order.

1. Navigate to **Products › Products** (`/admin/products`). Click **New Product**.
2. Leave Code blank. Enter Name `Sugar 1kg`, Type `Goods`, Base unit `KG — Kilogram`, tick **Sellable** and **Stockable**, VAT Status `Standard`, Cost amount `1800`. Leave **Currency** at the pre-selected company default. Click **Create**. System assigns `PROD-0034`.
3. Click the **Edit** action on `PROD-0034` to open `/admin/products/uid/<uid>`.
4. **Barcodes panel:** Enter `6009876543210`, tick **Set as primary**, click **Add Barcode**.
5. **Bulk Packs panel:** Select Unit `CTN — Carton`, Factor to base `50`. Click **Add Bulk Pack**. (50 kg bags per carton.)
6. **Prices panel:** Select Price list `RETAIL — Retail Price List`, leave **Unit** at **Base unit**, Amount `2500`, leave **Currency** at the default. Click **Set Price**.
7. **Prices panel:** Select Price list `WHOLESALE — Wholesale Price List`, **Unit** `Base unit`, Amount `2200`, leave **Currency** at the default. Click **Set Price**.
8. **Prices panel (non-linear carton price):** Select Price list `WHOLESALE — Wholesale Price List`, **Unit** `Carton`, Amount `105000`, leave **Currency** at the default. Click **Set Price**. (Fifty 1 kg bags would otherwise resolve to 50 × 2,200 = 110,000; pricing the carton directly at 105,000 gives buyers a bulk discount independent of the per-kilogram wholesale rate.)

The product `PROD-0034 — Sugar 1kg` is now available for sale at the correct retail price, at a discounted wholesale carton price, and will appear in stock movements tracked in kilograms. The Prices table shows three rows: `RETAIL / Base`, `WHOLESALE / Base`, and `WHOLESALE / Carton`.

Example — Set up a product in one pass with the Product Master

Scenario: Catalogue manager sets up Cooking Oil 5L from scratch using the one-screen template instead of the classic multi-panel flow.

1. Navigate to **Products › Products** (`/admin/products`). Click **Full product form**.
2. **General tab:** leave Code blank, enter Name `Cooking Oil 5L`, Type `Goods`, VAT Status `Standard`, tick **Sellable** and **Stockable**.
3. **Pricing tab:** Cost (buying) price Amount `12000`, Currency left at the company default. On the pre-added Selling price row, leave the Price list at the resolved default and enter Amount `15500`.
4. **Supplier & UoM tab:** Base unit of measure `LTR — Litre`. Under Pack / bulk units, select Unit `CTN — Carton`, Factor to base `4`, click **Add**.
5. **Stock & Barcodes tab:** enter Barcode value `6009876500001` (kept as primary automatically, being the first row). Opening stock Quantity `200`.
6. **Branches tab:** leave **Make available in all branches** ticked.

7. Click **Create product**.

The save-result panel shows **Product**, **Selling prices**, **Barcodes**, **Pack units**, **Branch availability**, and **Opening stock** all marked done. Clicking **Open product** opens the new record on the classic detail page, already carrying its retail price, its carton bulk pack, its barcode, and 200 litres of opening stock — all set up from the one screen.

Units of Measure

Navigation: Products › Units of Measure (`/admin/units`) | **Permission to view:** `UOM.VIEW` | **Permission to create / edit:** `UOM.MANAGE`

A **unit of measure (UoM)** is the label attached to a quantity: it defines what one "unit" of a product means. Examples include `EA` (Each), `KG` (Kilogram), `LTR` (Litre), and `CTN` (Carton). Every product must be assigned a base unit, and every order line, stock movement, and bulk pack references a unit.

Why it exists. Without defined units, quantities on documents are ambiguous — does "10" mean ten individual items, ten kilograms, or ten cartons? Consistent units ensure that stock balances are measured correctly, that picking and packing instructions are unambiguous, and that unit conversions (via bulk packs) are mathematically reliable. Centralising units in a master also provides a single pick-list that avoids the "pcs vs piece vs pieces" label drift that arises when staff type units freehand.

When it is used. A master-data manager creates units before creating products, because every product requires a base unit. Units are also referenced when defining bulk packs (the larger packaging unit) and on order lines where a specific packaging unit is selected.

How it works. Each unit has a short **Code** (used as the label on documents) and a **Name** (the display name). Units can be archived to remove them from selection dropdowns; archived units are excluded from product creation but remain on existing records for historical accuracy.

Behind the scenes each unit also carries a **physical dimension** — *count* (each, piece, carton), *weight* (kilogram, gram), or *volume* (litre, millilitre). The seeded default units are already classified correctly, so you do not set this on screen. The dimension is what lets the system tell a weight unit from a count unit: a product marked *sold by weight* (see *Products › Weighed goods*) must use a **weight** base unit such as `KG`, and the sale path uses the dimension to keep weighed products off by-count units.

Units of measure (UoM) are the quantity labels used on products, bulk packs, and order lines — for example, `EA` (Each), `KG` (Kilogram), `CTN` (Carton).

How to create a unit

1. Navigate to **Products › Units of Measure** (`/admin/units`).
2. Click **New Unit**.
3. Enter the **Code** (for example, `CTN`) and the **Name** (for example, `Carton`). Both are required and the code must be unique within the company.

4. Click **Create**.

How to edit a unit

Click **Edit** on a row, change the **Name** (the Code is read-only after creation), and click **Save**.

Archive and restore

Click **Archive** to deactivate a unit. Archived units are removed from product and bulk-pack dropdowns — only active units are selectable. Click **Restore** to make the unit active again.

Price Lists

Navigation: Products › Price Lists (`/admin/price-lists`) | **Permission to view:** `PRICELIST.VIEW` | **Permission to create / edit:** `PRICELIST.MANAGE`

A **price list** is a named set of selling prices. Rather than storing a single price on each product, the system lets you maintain multiple lists — for example, a Retail list, a Wholesale list, and a Distributor list — each with different prices for the same product. When a sales document is created, the system looks up the product's price from the price list assigned to that customer or order, ensuring that different categories of buyer are automatically charged at their agreed rates.

Why it exists. Different customer segments — retail walk-ins, wholesale buyers, key distributors — typically receive different pricing. Without named price lists, a business would have to manually enter prices on every order line and hope for consistency. Price lists enforce pricing discipline: the price is looked up, not typed, so discrepancies and pricing errors are structurally prevented.

When it is used. A pricing manager or catalogue administrator creates price lists once, then sets prices on each product for each list (in the Product detail page). Price lists are assigned to customers or selected on individual orders at sale time.

How it works. A price list has a short **Code** (such as `RETAIL`) and a **Name** (such as `Retail Price List`). Both are fixed at creation; the code is unique within the company. The list can be archived to prevent it from being selected on new orders; archiving does not remove prices already set on products.

Price lists group selling prices. You might have a Retail list (`RETAIL`), a Wholesale list (`WHOLESALE`), and a Distributor list. Customers and orders are assigned a price list, and the system looks up the price from there.

How to create a price list

1. Navigate to **Products › Price Lists** (`/admin/price-lists`).
2. Click **New Price List**.
3. Enter a **Code** (for example, `RETAIL`) and a **Name** (for example, `Retail Price List`). Both are required and the code must be unique within the company.
4. Decide whether the **Prices include VAT** toggle should be on or off (see *VAT-inclusive vs exclusive pricing* below). For a new list it is **pre-checked** — the help text reads *The customer-facing price*

already includes VAT. New price lists default to this on. Leave it on for a retail/shelf list where the price you type is the price the customer sees; clear it for a wholesale list quoted as a net price plus VAT.

5. Click **Create**.

VAT-inclusive vs exclusive pricing

Each price list carries a single VAT stance that decides how the prices in it are read at invoicing. You set it with the **Prices include VAT** toggle on the create form and can change it later by editing the list. A **VAT** column on the list shows the current stance at a glance with an **Incl. VAT** or **Excl. VAT** badge.

- **Prices include VAT (inclusive).** The price you enter is the **gross** shelf price — the amount the customer actually pays. When the sale is invoiced, the system splits the VAT back out of that figure so the net and VAT add up to exactly the price you typed, with no rounding drift. This is the natural choice for retail and counter sales where prices are displayed VAT-inclusive on the shelf. New price lists default to this stance.
- **Prices exclude VAT (exclusive).** The price you enter is the **net** amount and VAT is added on top when the sale is invoiced, so the customer pays the price plus VAT. This suits wholesale and B2B lists that are quoted as "price + VAT". This is the long-standing behaviour; existing price lists keep it unless you turn the toggle on.

The stance applies per list, so you can run a retail list (inclusive) and a wholesale list (exclusive) side by side for the same products. VAT-exempt and zero-rated products are unaffected either way — there is no VAT to add or split out.

Edit, archive, restore

Click **Edit** on a row to change the name and the **Incl. VAT** setting (code is read-only after creation). Archive and restore work as on all master records.

Pricing Rules

Navigation: Sales › Pricing Rules (`/admin/pricing-rules`) | **Permission to view:** `SALES.PRICING.RULE.VIEW` | **Permission to create / deactivate:** `SALES.PRICING.RULE.MANAGE`

The standard price list gives every buyer one price per product. **Pricing Rules** lets you go further in two common situations: rewarding bigger orders with a lower unit price, and giving a particular customer their own negotiated price. Both live on a single screen with two tabs — **Price Tiers** and **Customer Prices** — and both feed the price the system proposes when a sales document is raised.

Why it exists. Wholesale and distribution businesses rarely charge one flat price. A buyer who takes ten cartons expects a better rate than one who buys a single unit, and a key account may have a contract price agreed for the year. Capturing these rules as data — rather than relying on staff to remember and key them in by hand — keeps pricing consistent and auditable.

How it works. You first pick the **Company** at the top of the screen (the picker only appears when you have more than one). Each rule is then created against a product (and, for tiers, a price list) or a customer. Rules are never deleted: instead you **deactivate** a rule you no longer want, which preserves the history while removing it from future pricing.

Price tiers (quantity breaks)

A **price tier** sets a special unit price that applies once the order quantity reaches a minimum. For example, you might price a product at its normal rate for one to nine units, but drop the unit price for ten or more. Each tier is recorded against a specific product on a specific price list, so the same product can have different break points on your Retail and Wholesale lists.

To view existing tiers, on the **Price Tiers** tab select a **Product** and a **Price List** from their pickers, then click **Load Tiers**. The table lists each tier's minimum quantity, unit price, currency, and status.

To add a tier (you need the manage permission):

1. Click **Add Price Tier**. A **New Price Tier** form appears.
2. Choose the **Product** and **Price List** from their pickers.
3. Enter the **Min Quantity** (the order size at which this price starts to apply) and the **Unit Price**.
4. The **Currency** is chosen with the **Currency Picker** (the company's enabled currencies, defaulting to the company default).
5. Click **Save Tier**.

Each product/price-list combination can have only one **active** tier at a given minimum quantity. If you previously deactivated a tier at that quantity, you are free to create a fresh one at the same quantity — the limit applies only to tiers that are currently active. To retire a tier, click the deactivate (slash-circle) button on its row; its status changes and it no longer affects pricing.

Customer prices (contract prices)

A **customer price** is a fixed unit price for one product agreed with one specific customer — a contract or negotiated rate that overrides the ordinary price list for that buyer. You can optionally bound it with an **Effective From** and **Effective To** date, so a seasonal or promotional rate switches itself on and off automatically.

To view a customer's prices, switch to the **Customer Prices** tab, select the **Customer** from the picker, and click **Load Prices**. The table shows each product's agreed unit price, currency, the effective-date window (a dash means open-ended), and status.

To add one (you need the manage permission):

1. Click **Add Customer Price**. A **New Customer Price** form appears.
2. Choose the **Customer** and the **Product** from their pickers.
3. Enter the **Unit Price** and choose the **Currency** with the **Currency Picker**.
4. Optionally set **Effective From** and **Effective To** dates (leave them blank for a price with no time limit).
5. Click **Save Price**.

As with tiers, a customer price is deactivated rather than deleted — click the deactivate button on its row to stop it applying.

Currencies and FX Rates

Navigation: FX / Currency › Exchange Rates (`/admin/fx/rates`) | **Permission to view:** `CURRENCY.VIEW` | **Permission to add rates:** `CURRENCY.MANAGE`

A **currency** is a monetary unit of account — Tanzanian Shillings (TZS), US Dollars (USD), Euros (EUR), Kenyan Shillings (KES), and so on. Every monetary amount in this system is recorded as a pair: a number and a currency code. This means the system is currency-aware from the start, so transactions in foreign currencies are recorded correctly alongside local-currency ones.

Why currencies are always explicit. Storing a bare number without a currency — for example, "1,000" with an implied TZS — is a source of silent errors: import prices in USD would be compared directly with local costs in TZS, and reports would add unlike amounts. Every price, cost, credit limit, and invoice total in this system therefore carries its currency code alongside the number.

The enabled-currency allow-list and default document currency. Each company has a **base currency** (seeded as **TZS**) and an admin-configured **allow-list of enabled currencies**, optionally refined per branch, together with a **default document currency**. Anywhere a form asks for a currency, you choose from a filtered **Currency Picker** that offers only the company's enabled currencies and is pre-set to the resolved default — you no longer type a free-text three-letter code. This is the same picker used for the customer credit-limit currency, product cost and price currencies, and every other currency field across Sales, Procurement, and Finance; it is documented once in **Getting Started › Common UI Patterns**. The enabled list and default are maintained by an administrator with the `CURRENCY.MANAGE` permission. The base currency itself **cannot be changed once journal entries exist**.

An **FX rate** (foreign exchange rate) is the conversion factor between two currencies on a given date. When you receive a supplier invoice in USD, or raise a customer invoice in USD, the system needs to know how many TZS equal one USD on that particular day in order to record the correct local-currency equivalent in the general ledger and for reporting.

Why FX rates exist. Without exchange rates, foreign-currency transactions cannot be translated into the company's reporting currency. The rate on the day of the transaction is the authoritative rate for that transaction; a rate entered later cannot retroactively fix a document. Recording each day's rate as an immutable append-only row gives a permanent audit trail that regulators and auditors can verify.

When they are used. The finance officer or treasury administrator enters FX rates each day (or each time a foreign-currency transaction is expected). The system uses the most recent effective-dated rate for each currency pair when converting amounts.

How it works. Currencies are global reference data — you cannot create or delete them; an administrator instead enables a subset per company (the allow-list above). FX rates are **append-only**: you add a new row for each rate change; you never edit a past rate. If you discover an error, you add a corrected row with the right date and value. The list is sorted newest-first. A rate between two currencies is selected by finding the row with the latest effective date on or before the transaction date.

The seeded base currency is **TZS**. You can enable additional currencies (USD, EUR, KES, and others) for a company and record foreign exchange rates to support transactions in them.

Currency list

Currencies are global reference data — you cannot create or delete them. The system-seeded currencies (TZS, USD, EUR, KES, and others) appear in the **From** / **To** dropdowns on this screen. Unlike the filtered **Currency Picker** used elsewhere, these two dropdowns list **all** active currencies (the full global reference set), **not** only the company's enabled allow-list — so you may record a rate for any currency pair. (If the currency list fails to load, each dropdown falls back to a free-text three-letter ISO-code input.) Which currencies a company may use *on documents* — and the default — is still controlled by the admin-managed enabled-currency allow-list described above.

How to add an FX rate

The on-screen page heading is **Currency Exchange Rates** (subtitle *Effective-dated rates used for multi-currency transactions*).

1. Navigate to **FX / Currency › Exchange Rates** (`/admin/fx/rates`).
2. Click **New Rate**.
3. Select the **From Currency** and the **To Currency** from the dropdowns. They must be different.
4. Enter the **Rate** (must be greater than zero). The hint reads *Units of To-currency per 1 unit of From-currency*.
5. Set the **Effective Date** (required; use the date picker).
6. Optionally choose a **Rate Type** from the dropdown (— none —, Spot, Forward, or Official) and a free-text **Source** (for example, `Central Bank`).
7. Click **Save Rate**.

FX rates are **append-only**: you cannot edit a rate in place. To correct a rate, add a new row with the corrected value and the correct effective date. The system uses the latest effective-dated rate for each currency pair when converting amounts.

The rates list is sorted newest-first and is paginated.

Example — Record today's USD buying rate

Scenario: Finance officer records the Bank of Tanzania mid-rate on 14 June 2026 for USD invoices received from an overseas supplier.

1. Navigate to **FX / Currency › Exchange Rates** (`/admin/fx/rates`). Click **New Rate**.
2. From Currency `USD`, To Currency `TZS`, Rate `2542.50`, Effective Date `2026-06-14`, Rate Type `Spot`, Source `Central Bank`.
3. Click **Save Rate**. The row `USD → TZS @ 2,542.50 (2026-06-14)` appears at the top of the list.

Tomorrow, if the rate changes to `2,548.00`, simply click **New Rate** again and submit the new row — the old record is preserved for historical reporting.

Tax Rates

Navigation: Sales › Tax Rates (`/admin/tax-rates`) | **Permission to view:** `TAXRATE.VIEW` | **Permission to create / edit:** `TAXRATE.MANAGE`

A **tax rate** is the percentage applied to a sale line to calculate value-added tax (VAT). VAT is a consumption tax collected by the business on behalf of the tax authority: the business charges the customer a price plus VAT, then remits the VAT element to the government. Getting the rate right on every transaction is a legal obligation, not an option.

Why tax rates exist as a configurable master. The VAT rate in Tanzania (and in many countries) is set by law and can change. Hardcoding 18% into the software would require a code change every time the rate changed. Instead, the system maintains three configurable VAT bands per company — Standard, Zero-rated, and Exempt — each with an editable rate. When the government adjusts the rate, the finance manager updates the single master record and all future transactions use the new rate automatically.

The three bands explained:

- **Standard** — the normal VAT rate, currently 18% in Tanzania. Applied to most goods and services. The tax amount on a sale line is the net price multiplied by this rate.
- **Zero-rated** — technically within the VAT system but taxed at 0%. Businesses selling zero-rated goods can still reclaim input VAT on their purchases. Common for staple food items in many jurisdictions.
- **Exempt** — outside the VAT system entirely. No VAT is charged and no VAT can be reclaimed on inputs. Different from zero-rated because exempt status completely removes the item from the VAT computation.

When it is used. A finance manager or system administrator reviews and (if required) adjusts the rates when the tax authority changes them. The rates are applied automatically to every sales and purchase line based on the product's VAT status (set on the product record).

How it works. The three bands are normally seeded automatically when a company is created, and you can only edit the rate of each one. The updated rate applies to all future transactions that reference that band; past transactions retain the rate that was in effect when they were created. If a company's seeding was skipped or only partially completed — so one or more classifications are

missing — you can create the missing band(s) yourself from this screen instead of waiting on a seeder run; see *How to add a tax rate* below. There is no archive or delete on tax rates: once a band exists for a company it is permanent, and you can only ever have one row per classification.

Three VAT bands are seeded per company:

Band	Default rate
Standard	18% (0.18)
Zero-rated	0% (0.00)
Exempt	0% (0.00)

How to edit a tax rate

1. Navigate to **Sales › Tax Rates** (`/admin/tax-rates`).
2. Click **Edit** on the relevant band row.
3. Enter the new **Rate** as a percentage (for example, `18` for 18%). The value must be between 0 and 99.99.
4. Click **Save**.

The rate applies to all future transactions that reference this VAT band on a product.

How to add a tax rate

If a company is missing one or more of the three VAT classifications, an **Add tax rate** section appears below the table.

1. Navigate to **Sales › Tax Rates** (`/admin/tax-rates`).
2. Under **Add tax rate**, select the **VAT classification** — the dropdown offers only classifications not yet configured for this company (Standard, Zero Rated, Exempt).
3. Enter the **Rate (%)** as a percentage (for example, `18` for 18%). The value must be between 0 and 99.99.
4. Click **Add**.

The new band appears in the table immediately. Once all three classifications are configured, the **Add tax rate** section is replaced by the message *All VAT classifications are configured.* Submitting a rate for a classification that already exists is rejected with *A rate for this classification already exists.* You need the `TAXRATE.MANAGE` permission to add a rate, same as to edit one.

Distribution Routes

Navigation: Parties › Routes (`/admin/routes`) | **Permission to view:** `ROUTE.VIEW` | **Permission to create / edit / assign branches:** `ROUTE.MANAGE` | **Permission to assign customers and agents:** `ROUTE.ASSIGN`

A **distribution route** (or simply a route) is a named geographic or logical territory that groups a set of customers and assigns the sales agent or agents responsible for serving them. Routes answer the

question "which customers does this agent visit, and on which road or region?" They provide an organising layer above individual customers and are the bridge between the customer master, the agent master, and the sales invoice.

Why routes exist. In distribution-heavy businesses — FMCG, wholesale, van-sales — a sales team covers fixed territories. Without a route structure, there is no way to know which agent is responsible for which customers, to plan delivery runs efficiently, or to report sales performance by territory. Routes solve these problems by grouping customers under a named area and assigning one or more agents to that area, with a **primary** agent designated as the default for invoices raised against customers on that route.

When they are used. A distribution or operations manager creates routes when setting up the company's sales territories, then assigns customers and agents to those routes. Once set up, routes are largely static — they are updated when territory boundaries change, when customers are transferred between routes, or when agents change. On every sales invoice, the system automatically carries across the selling agent's primary route, so invoices are tagged geographically without any manual entry by the sales team.

How it works. A route is created with a name and an optional free-text location identifier (describing the geography informally). It is then associated with branches (so branch-level users can see it), with customers (so those customers appear in the route's list for run-planning), and with agents (so the agent is responsible for that route). Only **External** agents can be assigned to routes — internal agents work within the application and do not need a field territory assignment. One agent can be marked **Primary** on a route; this agent's route is the default on invoices, making geographic reporting automatic. The route lifecycle follows the same Active/Archived pattern as other masters.

Routes represent geographic or logical delivery areas used to group customers and assign agents. Each route has a system-generated code, a name, and an optional location identifier.

How to create a route

1. Navigate to **Parties › Routes** (`/admin/routes`).
2. Click **New Route**.
3. Enter the **Name** (required) and optionally a **Location Identifier**.
4. Click **Save**.

The system assigns a code. Status defaults to Active.

How to edit a route

1. Click the **Edit** (pencil) action on any route row to open the detail page (`/admin/routes/uid/<uid>`).
2. Change the name or location identifier (code and company are read-only).
3. Click **Save**.

Archive and restore

Click **Archive** on the route detail page (`/admin/routes/uid/<uid>`) to deactivate it. Click **Restore** to reactivate.

Assigning customers to a route

1. Open the route detail page (`/admin/routes/uid/<uid>`).
2. In the **Customers** panel, start typing a customer name in the search box.
3. Select the customer from the results (shown as `code — displayName`).
4. Click **Assign**.
5. To remove a customer from the route, click **Remove** on the row.

Only active customers from the same company appear in the picker. You need the `ROUTE.ASSIGN` permission.

Assigning agents to a route

1. In the **Agents** panel, start typing an agent name.
2. Select the agent from the results. Only **External** agents are available — internal agents cannot be assigned to a route.
3. Tick **Primary** if this agent is the primary agent for this route.
4. Click **Assign**.
5. To remove an agent, click **Remove**.

You need the `ROUTE.ASSIGN` permission.

Assigning branches to a route

1. In the **Branches** panel, select the **Company** from the first dropdown, then select the **Branch** (shown as `code — name`).
2. Click **Assign**.
3. To remove a branch, click **Remove**.

You need the `ROUTE.MANAGE` permission (not `ROUTE.ASSIGN`) to manage branch assignments on a route.

Example — Set up the Northern Route with customers and an agent

Scenario: Operations manager creates the Arusha / Moshi distribution route before the first delivery run.

1. Navigate to **Parties › Routes** (`/admin/routes`). Click **New Route**.
2. Enter Name `Northern Route`, Location Identifier `Arusha–Moshi Corridor`. Click **Save**. System assigns code `RTE-0003`.
3. Click the **Edit** (pencil) action on the `RTE-0003` row to open `/admin/routes/uid/<uid>`.
4. **Branches panel:** Company `Orbix Trading Co.`, Branch `ARU — Arusha Branch`. Click **Assign**.
5. **Customers panel:** type `Kilimanjaro`, select `CUST-0007 — Kilimanjaro Stores Ltd`. Click **Assign**. Repeat for `CUST-0011 — Moshi Distributors`.
6. **Agents panel:** type `Baraka`, select `AGNT-0004 — Baraka Hamisi` (External). Tick **Primary**. Click **Assign**.

The Northern Route is now ready. The delivery team can filter orders and customers by route, and the agent Baraka Hamisi appears as the primary contact on route-based reports.

Bulk Import — mass create & update from Excel

When you have many products, customers, suppliers or prices to enter or change at once, the **Bulk Import** tool loads them from a spreadsheet instead of one form at a time. It is a guided four-step wizard: pick what to import, download a spreadsheet, fill it in, then upload it to validate and commit.

Where: Master Data › **Bulk Import** (`/admin/bulk-import`).

Who can use it: the menu item and screen appear only for users whose role grants an import permission — `PRODUCT.IMPORT` (products), `CUSTOMER.IMPORT` (customers), `SUPPLIER.IMPORT` (suppliers) or `PRICE.MASS_UPDATE` (prices). You only see the entity types you are allowed to import.

What you can import:

Entity type	Creates / updates
Products	The product catalogue — code, name, type, sellable/stockable, base unit, unit cost, VAT status, reorder levels, barcode …
Customers	Customer master — name, TIN, VAT registration, contacts, credit limit, payment terms, segment …
Suppliers	Supplier master — name, TIN, contacts, payment terms, lead time, minimum order value …
Product prices & cost	The selling price on a chosen price list **and** the unit cost, in one sheet

The four steps

1. **Choose what to import.** Pick the entity type. The screen then shows the download and upload panels for that type.
2. **Download a spreadsheet.** Two buttons:
    - **Download Template** — a blank sheet with just the column headers, for entering brand-new records.
    - **Export current data** — the same sheet **pre-filled** with the records already in the system, so you can edit what is there and upload it back. (For **Product prices & cost**, first pick the **company** and the **price list** to export — exporting without choosing a list falls back to the company default, which is often empty.)
3. **Fill it in (in Excel).** The sheet is self-describing:
    - **Required** columns are marked with * in the header; a blank required cell is rejected.
    - Columns with a fixed set of values (e.g. *Party Type*, *VAT Status*, *Yes/No*) show a **drop-down** so you pick a valid value.

- **Number** columns — Cost Amount, Selling Price, Credit Limit, quantities, terms/lead-time days — are written as **real numeric cells**, so you can use `SUM`, formulas and arithmetic directly (no "number stored as text"). A separate **Instructions** sheet lists every column with a note and its allowed values.
- **Blue "reference" columns** (for example the product name beside a price) are **read-only context** — shown to help you find the right row, but never imported. Edit the white cells; the blue ones are ignored.
- On an **update** (a row whose code matches an existing record), a **blank optional cell keeps the current value** — you only fill the cells you want to change.

4. **Upload & validate, then commit.** Choose the filled-in `.xlsx` file. The system checks every row and shows a results table with one row per spreadsheet line and an outcome tag:

   - **Create** — a new record will be added.
   - **Update** — an existing record (matched by code) will be changed.
   - **Skip** — nothing to do (e.g. an all-blank optional row).
   - **Error** — the row has a problem; the **Message** column explains it in plain language.

Commit Import stays disabled until *every* row is clean, so a bad row can never post a half-import. Fix the flagged rows in the spreadsheet, upload again, and when the error count is `0`, click **Commit Import**. The bulk import runs through exactly the same validation, scope, code-generation and audit as manual entry — nothing bypasses the rules.

> **Round-trip editing.** *Export current data → edit in Excel → upload* is the fastest way to change many records at once: the upload bulk-**updates** the rows the sheet contains (matched by their code) and leaves everything else untouched.
>
> **Cost on the price sheet.** Changing the **Cost Amount** on the *Product prices & cost* sheet needs product-management permission (`PRODUCT.MANAGE` or `PRODUCT.IMPORT`); it is only applied when the cost actually differs from the current value, so a price-only user can safely re-upload a pre-filled sheet without being blocked on the cost column.

Example — raise every RETAIL price and correct two costs

1. Go to **Master Data › Bulk Import**, choose **Product prices & cost**.
2. Under *Download a spreadsheet*, pick the **RETAIL** price list and click **Export current data**.
3. In Excel, edit the **Selling Price** column (the figures are real numbers, so a quick formula works), and correct the **Cost Amount** on the two products whose cost changed. Leave every other cell as exported.
4. Save, return to the wizard, and choose the file under **Upload & validate**. Review the results — each edited row shows **Update**; untouched rows show **Skip**.
5. When **Errors: 0**, click **Commit Import**. The new prices and costs are live immediately.

Sales and Point of Sale

This chapter covers everything from quoting a customer through to collecting payment, including recurring and blanket agreements, advanced pricing, and the Point of Sale cashier workflow.

Overview

The sales module follows the order-to-cash (O2C) path:

```
Quotation → Sales Order → Delivery → Sales Invoice → Payment
```

Walk-in cash sales skip the first three steps and begin directly with a Sales Invoice or a POS sale.

What "order-to-cash" means. Order-to-cash is the end-to-end business process that starts the moment a customer expresses intent to buy and ends when the business has received and accounted for the money. Each step in the chain creates a document that serves as a control point: stock is only committed when an order is confirmed, goods only leave the warehouse when a delivery is recorded, and revenue is only recognised when an invoice is finalised. Without this chain, businesses would have no audit trail, no way to match what was promised to what was shipped, and no reliable basis for the accounts receivable ledger.

Required permissions — the navigation menu only shows items your role includes. Key permission groups:

Activity	Permission codes required
Quotations	SALES.QUOTE.VIEW, SALES.QUOTE.CREATE, SALES.QUOTE.SEND, SALES.QUOTE.ACCEPT
Sales Orders	SALES.ORDER.VIEW, SALES.ORDER.CREATE, SALES.ORDER.CONFIRM, SALES.ORDER.CANCEL
Deliveries	SALES.DELIVERY.VIEW, SALES.DELIVERY.CREATE
Sales Invoices	SALES.INVOICE.VIEW, SALES.INVOICE.CREATE, SALES.INVOICE.SETTLE, SALES.INVOICE.VOID
Sales Returns	SALES.RETURN.VIEW, SALES.RETURN.CREATE
Blanket Orders	SALES.BLANKET.VIEW, SALES.BLANKET.CREATE
Standing Orders	SALES.STANDING.VIEW, SALES.STANDING.CREATE
Pricing Rules	SALES.PRICING.RULE.VIEW, SALES.PRICING.RULE.MANAGE
Sales Settings (SO approval threshold)	SALES.SETTINGS.MANAGE
Fiscal (EFD) receipts	FISCAL.VIEW, FISCAL.MANAGE
POS (tills)	POS.TILL.VIEW, POS.TILL.MANAGE
POS (cashier)	POS.SESSION.OPEN, POS.SALE.CREATE, POS.SESSION.VIEW
POS (reverse / age override)	POS.SALE.VOID, POS.SALE.AGE_OVERRIDE
POS (close/reconcile)	POS.SESSION.CLOSE, POS.SESSION.RECONCILE

Contact your administrator if an expected menu item is missing.

Concurrent edits. If two people act on the same document at the same time (for example, both try to confirm the same order), the second action is rejected with a conflict message asking you to reload and retry, rather than failing silently or corrupting the record. Reload the screen to see the current state and act again.

1. Quotations

Navigate to **Sales › Quotations** (`/admin/quotations`).

What a quotation is. A quotation (also called a quote or a sales proposal) is a formal written offer that the business sends to a customer. It states the products, quantities, unit prices, any discounts, and a validity period — that is, the date by which the customer must respond if the offered price is to be honoured.

Why quotations exist. Without a quotation, pricing agreements between a salesperson and a customer exist only verbally. A quotation creates a timestamped, auditable record of what was offered at what price, protects the business from disputes, and gives management visibility of the sales pipeline (how many offers are outstanding, what value, and when they expire). It also means that once a customer accepts, the system can convert the offer into a Sales Order automatically, carrying the agreed prices across without any re-entry.

When a quotation is used. A quotation is raised when a customer asks "what will it cost me?" before committing to buy — typically by a salesperson or sales assistant. It sits at the very beginning of the O2C chain: nothing is reserved from stock and no financial entry is made; the quotation is a promise, not a transaction.

How a quotation flows. A quotation begins as a `DRAFT` (editable, no number yet). When it is sent to the customer the status moves to `SENT` and a `QUOTE-####` number is assigned. If the customer accepts, the quotation moves to `ACCEPTED` and a Sales Order is created automatically with the same lines and agreed prices. If the customer declines it is `REJECTED`; if the validity date passes without a response the system marks it `EXPIRED` and acceptance is blocked.

1.1 Create a quotation

1. Navigate to **Sales › Quotations** (`/admin/quotations`).
2. Click **New Quotation**.
3. In the **Customer** field, type part of the customer name or code and select the correct entry from the list. Do not type or paste a raw ID.
4. Choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — only the company's enabled currencies are offered, defaulting to the company default).

5. Set **Quote Date** (today by default) and **Valid Until** (the date the offer expires; it must be on or after the Quote Date).
6. Click **Create Quotation**. The quotation is saved in **DRAFT** status. A quote number is assigned later when you send it.

Required fields: Customer, Currency, Quote Date, Valid Until.

1.2 Add lines to a quotation

1. Open the draft quotation (navigate to **Sales › Quotations** then click the **Open** action on the quotation row, or go to `/admin/quotations/uid/{uid}`).
2. In the **Lines** section, search for the product by name or code and select it.
3. Choose a **Unit**, enter **Quantity**, optionally enter a **Price Override** (otherwise the list price is used), and optionally enter a **Disc %** (line discount as a percentage — the quotation and Sales Order line forms only offer a percentage discount, not a fixed amount).
4. Click the **+** (Add line) button. The system calculates net amount, VAT, and gross from the configured price list.

Unit choices are product-scoped. The **Unit** dropdown is disabled until a product is selected. Once a product is chosen, it lists only that product's configured units — its base unit plus any active bulk-pack units (e.g. CARTON) — never the full company unit list, and it defaults to the base unit. This applies to every line form in this chapter (quotations, sales orders, invoices, blanket orders, standing orders, and POS sales): you can no longer select a unit that is not configured on the product, which previously could silently mis-record the quantity.

Prices are per the selected unit. Each product can carry a distinct price for each of its units — a price per PCS and a separate price per CARTON, for example. When you choose the **Unit**, the price the system resolves (and the amount shown in the line's **Unit Price** column) is the price for **that** unit, not the base-unit price multiplied out. So a line of 5 CARTON is priced at the carton price × 5, not the piece price × 5. If you type a **Price Override**, enter it as the price for the unit you selected. Switching the unit re-prices the line, so pick the unit first, then confirm or override the price.

Repeat for each product. You can also add **Service** products; these are priced the same way but do not affect stock.

To remove a line, click the delete icon on the line row. Lines can only be changed while the quotation is in DRAFT.

1.3 Send a quotation

When the quotation is ready to share with the customer:

1. Open the draft quotation.
2. Click **Send to Customer**.
3. The status changes to **SENT** and a quote number (QUOTE-####) is assigned.

Prerequisites: The quotation must have at least one line, and the Valid Until date must be today or in the future.

1.4 Accept or reject a quotation

When the customer responds:

- **Accept** — click **Accept & Convert to Order** on the sent quotation. A **Sales Order** is created automatically with the same lines and discounts. The quotation status changes to **ACCEPTED**. A success message shows the new order number and provides a link to it.
- **Reject** — click **Reject**. The quotation status changes to **REJECTED**.

Both actions require the `SALES.QUOTE.ACCEPT` permission. If the Valid Until date has already passed, the system prevents acceptance and marks the quotation **EXPIRED**.

1.5 Quotation statuses

Status	Meaning
DRAFT	Being prepared; lines can be added or removed
SENT	Sent to customer; awaiting response
ACCEPTED	Customer accepted; Sales Order created
REJECTED	Customer declined
EXPIRED	Valid Until date passed before acceptance

Example — Quotation for Karibu Supermarkets:

Salesperson Ali opens **Sales › Quotations** (`/admin/quotations`) and clicks **New Quotation**. He types "Karibu" in the Customer field and selects **Karibu Supermarkets Ltd**. He sets Quote Date to **2026-06-14** and Valid Until to **2026-07-14**, then saves. The quotation is created in DRAFT with no number yet.

Ali adds two lines:

- Product **Unga wa Ngano 2kg**, Unit **CARTON (12 pcs)**, Qty **50**, Line Discount **0%** — system prices at TZS 18,000 per carton = TZS 900,000 net.
- Product **Mafuta ya Kupikia 1L**, Unit **CARTON (12 pcs)**, Qty **30**, Line Discount **5%** — list price TZS 22,000; after 5% = TZS 20,900 per carton = TZS 627,000 net.

VAT at 18% is added by the system: total gross = TZS 1,535,400 + VAT. Ali clicks **Send to Customer** — status becomes SENT and the number **QUOTE-0047** is assigned.

Karibu calls back and accepts. Ali clicks **Accept & Convert to Order**. The system creates **Sales Order SO-0112** from the same lines and shows a link. Quotation status is now ACCEPTED.

2. Sales Orders

Navigate to **Sales › Sales Orders** (`/admin/sales-orders`).

What a Sales Order is. A Sales Order (SO) is the internal document that records a customer's confirmed purchase intent. It lists the products, quantities, agreed prices, and any discounts. Unlike a quotation (which is an offer), a Sales Order is a commitment: the business has agreed to supply, and the customer has agreed to buy.

Why Sales Orders exist. The Sales Order is the control centre of the fulfilment process. Two things happen that do not happen at the quotation stage: first, confirming the order **reserves stock** so those goods cannot be sold to someone else; second, the order creates the traceability link between the customer's request, the delivery that ships the goods, and the invoice that bills them. Without Sales Orders, a warehouse would not know what to pick, finance would have no basis for revenue recognition, and there would be no way to track partial deliveries or backorders systematically.

When a Sales Order is used. A Sales Order is created either automatically (when a customer accepts a quotation) or directly (when a salesperson or order-desk clerk enters it fresh — for example, a telephone order that was never quoted). It is used any time a customer is buying goods that need to be fulfilled from stock and billed after delivery, as opposed to a walk-in cash purchase which goes straight to an invoice.

How a Sales Order flows. An SO begins as `DRAFT` (a `SO-####` number is assigned immediately at creation, even in draft). When the user confirms it, the status moves to `CONFIRMED` and stock is soft-reserved. As deliveries are made against the order the status tracks fulfilment progress (`PARTIALLY_FULFILLED` → `FULFILLED`). As invoices are raised from those deliveries it tracks invoicing progress (`PARTIALLY_INVOICED` → `CLOSED`). Cancellation at any point releases the reservations.

Stock reservation explained. When you confirm a Sales Order, the system writes a "soft reservation" against each product in the warehouse. The reserved quantity is not physically moved — the goods stay on the shelf — but they are marked as committed. This means the available-to-promise figure (what can still be sold to other customers) is reduced immediately. A reservation prevents double-selling: two salespeople cannot independently confirm orders for the same last 10 units. When a delivery is made, the reservation for the delivered quantity is released (because the goods have actually left) and the on-hand balance is reduced instead.

2.1 Create a standalone Sales Order

1. Navigate to **Sales › Sales Orders** (`/admin/sales-orders`).
2. Click **New Sales Order**.
3. Pick the **Customer** by name.
4. Choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — the list is limited to the company's enabled currencies and defaults to the company default).
5. Set **Order Date**. The **Agent** field is pre-filled with **your own agent** (if your user is linked to a sales agent) — you can leave it, clear it, or type another agent's name to change it. Optionally add **Notes**.
6. Click **Create Order**. The order is created in **DRAFT**.

> **Agent pre-fill.** The Agent box shows *your* linked agent as soon as you open the form so a route agent does not have to search for themselves every time. It is only a convenience

> default — the field stays fully editable, and if your user has no linked agent (for example a root/admin login) the box simply opens empty.

2.2 Add lines to a Sales Order

The same process as adding quotation lines, including the **Unit** dropdown being scoped to the selected product's configured units (see section 1.2). Lines can only be added, edited, or removed while the order is in DRAFT.

2.3 Confirm an order

Confirming an order reserves stock for every GOODS line.

1. Open the draft order at **Sales › Sales Orders** then click the **Open** action on the order row (or navigate to `/admin/sales-orders/uid/{uid}`).
2. The order must have at least one line.
3. Click **Confirm Order**. A confirmation dialog appears explaining that confirming will attempt to reserve stock for all order lines, and that **if insufficient stock is available the order will confirm with a backorder**.
4. Click **Yes, Confirm**.
5. The status changes to **CONFIRMED** and each line shows its reserved quantity. Where stock was short, the line keeps an **Open (backorder)** quantity that you fulfil with a later delivery.

This requires the `SALES.ORDER.CONFIRM` permission. A user who can create orders but not confirm them will not see this button.

Approval threshold (large orders). Your company can require manager approval for high-value orders. When the SO-approval workflow is switched on (see *Sales Settings* below) and the order's gross total is **at or above the configured threshold**, clicking **Confirm Order** does not confirm it. Instead the system automatically **submits the order for approval** and refuses the confirmation with *"This order exceeds the approval threshold and has been submitted for approval. It can be confirmed once it is approved."* The order stays in **DRAFT**, but it is now **awaiting approval**: its lines are frozen (you cannot add or remove lines while it is pending) and the **Confirm Order** button is disabled. An authorised approver acts on it from **Approvals › Approval Inbox**. Once it is **approved**, come back and click **Confirm Order** again to reserve stock as normal. If the approval is **rejected**, the order can no longer be confirmed — cancel it and raise a new one. Orders below the threshold (or when the workflow is off) confirm immediately with no approval step.

> You can also submit a draft order for approval yourself, before trying to confirm it: an order that has never been submitted shows a **Submit for Approval** button on its detail page. This routes it through the same inbox.

Configuring the threshold (Sales Settings). A user with `SALES.SETTINGS.MANAGE` sets this up under **Sales › Sales Settings** (`/admin/sales-settings`). Pick the **Company**, tick **Enable SO Approval Workflow**, enter the **Approval Threshold Amount** (sales orders above this value require approval) and its **Currency**, then click **Save Settings**. Leaving the workflow disabled means no order is ever auto-held at confirm.

Credit-control hard block. When the customer is a **Credit Account** customer, confirming the order runs a credit-control check. Confirmation is blocked (the order stays in DRAFT and the system returns a clear conflict message) if **any one** of these three independent conditions is true:

- the customer's **credit status** is `ON_HOLD` or `STOPPED`;
- the customer is on a **manual hold** (a credit-control staff override — the hold reason, if recorded, is shown in the message); or
- the order's gross total, added to the customer's current outstanding balance, would **exceed their credit limit**.

The block is overridable only by a user holding the `SALES.CREDIT.OVERRIDE` permission; every override is recorded in the audit trail. **Cash / walk-in customers are exempt** — this check never applies to them.

Credit limit and other currencies. The limit is checked in your base currency. The customer's balance counts each foreign-currency item at the rate it was booked at; older foreign-currency items that have no reliable booked rate, and an order or invoice that is itself in a foreign currency, are converted at **today's** exchange rate (the latest rate on or before today). If a currency involved has **no exchange rate at all**, the system cannot work out the exposure, so it treats the limit as exceeded: the order or invoice is blocked with "The credit limit could not be checked because there is no exchange rate for USD. Add a USD rate under Currency rates and try again." A user with `SALES.CREDIT.OVERRIDE` can still proceed, and the audit record notes which rate was missing. The same rule applies to the credit-limit check at invoice finalisation.

> A separate, advisory credit warning may also appear without blocking confirmation; it is informational only and the order still confirms.

2.4 Cancel an order

Cancelling an order releases any stock reservations.

1. Open the order.
2. Click **Cancel Order**, enter an optional reason, and click **Confirm Cancel**.

Cancellation is allowed from any status except **CANCELLED** and **CLOSED**.

2.5 Set or change the agent

What the order's agent is. Every sales order carries a sales agent — the salesperson or route agent credited with the sale — which the invoicing flow depends on. An order can be created without one (Agent is optional in section 2.1), but an invoice generated from an agentless order cannot be finalised.

> **New orders pre-fill your own agent.** When you create a fresh Sales Order the Agent field already shows your linked agent (section 2.1), so a route agent's own orders normally arrive with the correct agent set. Use **Assign Agent / Change Agent** below when an order was created without one, was created by someone with no linked agent (e.g. root), or was credited to the wrong agent.

Why this action exists. Before this action, an order created with no agent (or the wrong one) had no way to be corrected in place — every attempt to invoice it would keep failing. **Assign Agent** / **Change Agent** lets an authorised user fix the agent on an existing order without recreating it.

1. Open the sales order (navigate to `/admin/sales-orders/uid/{uid}`).
2. Click **Assign Agent** if the order has no agent, or **Change Agent** if it already has one — both open the same form.
3. In the **Agent** field, type part of the agent's name or code and select the correct entry from the suggestions list.
4. Click **Save Agent**.

The order's agent is updated immediately and a confirmation is shown.

When the action is available. The button is shown only while the order is in a pre-invoice status — **DRAFT**, **CONFIRMED**, **PARTIALLY_FULFILLED**, or **FULFILLED**. Once the order has been invoiced (**PARTIALLY_INVOICED**, **CLOSED**) or is **CANCELLED**, the agent can no longer be changed this way.

No-agent banner. An order with no agent shows a warning banner on its detail page: *"No agent assigned. This order cannot be invoiced until a sales agent is assigned."*

Required permission: `SALES.ORDER.CREATE` (the same permission used to create and edit draft orders — there is no separate set-agent permission).

2.6 Order status lifecycle

Status	Meaning
DRAFT	Being prepared; no stock reserved
CONFIRMED	Stock reserved
PARTIALLY_FULFILLED	At least one delivery made; not all lines delivered
FULFILLED	All lines delivered
PARTIALLY_INVOICED	Some deliveries invoiced
CLOSED	Fully fulfilled and fully invoiced
CANCELLED	Cancelled; reservations released

Approval status is separate from order status. When the approval workflow holds an order (see section 2.3), the order stays in **DRAFT** but its detail page shows an **awaiting approval** state; it is not a distinct row in the table above. The approval outcome (pending, approved, rejected) governs whether **Confirm Order** is available, while the statuses above track fulfilment and invoicing once the order is confirmed.

3. Deliveries

Navigate to **Sales › Deliveries** (`/admin/deliveries`).

What a Delivery is. A Delivery is the document that records goods physically leaving the warehouse and being shipped or handed to the customer. It references the Sales Order it fulfils and specifies the exact quantities dispatched on that date. It is also sometimes called a "dispatch note" or "delivery note."

Why Deliveries exist. Without a delivery document there is no system record of when goods actually left — only what was ordered. The Delivery is the trigger for two critical events: it reduces the physical stock balance (goods have left), and it becomes the source document for the customer's invoice (you invoice what you delivered, not what was ordered — because partial deliveries are common). The delivery is also the moment that the cost of those goods is posted to the Profit and Loss account as Cost of Goods Sold (COGS), matching the cost to the revenue period in which the goods are billed.

When a Delivery is used. A delivery is created by a warehouse or logistics clerk once goods are ready to ship, always against a confirmed Sales Order. Multiple deliveries can be made against a single order (backorders), and each delivery generates its own invoice.

How a Delivery flows. A delivery can only be created against a `CONFIRMED` or `PARTIALLY_FULFILLED` Sales Order. When created it is immediately `CONFIRMED` (there is no separate pick/confirm step in the current version). The delivery is immutable — once confirmed it cannot be edited; corrections are handled through a Sales Return. Stock is reduced at the branch and the SO line counters are updated. An invoice is generated from the delivery as a separate action.

Full delivery vs partial delivery (backorder). If you deliver less than the full ordered quantity on any line, the system creates a partial delivery and the order moves to `PARTIALLY_FULFILLED`. The remaining undelivered quantity is the **backorder**. You create a second delivery later for the remaining quantity. Each delivery is independent and can be invoiced separately.

3.1 Create a delivery

Deliveries are always created from a Sales Order. The **Deliveries** list (`/admin/deliveries`) is view-only — it has no "New Delivery" button (its subtitle reads *"Create deliveries from the Sales Order detail screen."*).

1. Open a **CONFIRMED** (or **PARTIALLY_FULFILLED**) Sales Order and click its **Create Delivery** action. This opens the delivery create form at `/admin/deliveries/create` for that order; there is no Sales Order picker — the order is carried through from the button.
2. The form's **Lines to Deliver** table lists all open (undelivered) lines with the remaining (Open Qty) quantity pre-filled in **Deliver Qty**.
3. Untick a line's **Include** checkbox to leave it out, or lower its **Deliver Qty** if you are making a **partial delivery** (backorder). The quantity you enter cannot exceed the open balance.
4. Set **Delivery Date** (required) and optionally enter **Notes**.
5. Click **Create Delivery**.

Deliveries are created immediately in **CONFIRMED** status and cannot be undone. Each delivery is assigned a DELIVERY-#### number.

Not enough stock (negative-stock block). Creating a delivery issues stock from the branch, so if your company has turned on **Block sales that would take stock negative** (in *Sales Settings*), **Create Delivery** is refused when a delivered line would take that product's available stock below zero. You get a plain message naming the product with how much is available versus requested, and no delivery is created. Lower the **Deliver Qty** (which leaves the rest as a backorder — section 3.2), receive or transfer more stock in first, or have a user with `SALES.SETTINGS.MANAGE` turn the setting off to allow the shortfall. This is the same block described under section 4.4; when the setting is off (or Sales Settings has never been saved) deliveries are never blocked this way.

3.2 Partial delivery (backorder)

Enter a quantity less than the open balance on any line to create a partial delivery. The Sales Order status moves to **PARTIALLY_FULFILLED**. Create another delivery later for the remaining quantity.

3.3 Generate an invoice from a delivery

Once goods are delivered, you can invoice the customer for that delivery:

1. Open the delivery (navigate to **Sales › Deliveries** and click the **Open** action on the delivery row, or go to `/admin/deliveries/uid/{uid}`).
2. Click **Invoice this Delivery**.
3. A draft **Sales Invoice** is created automatically with the delivered lines. The doc discount from the source order is pro-rated to the delivered quantity.

Prerequisite: the source order must have an agent. Invoice this Delivery reads the agent from the underlying Sales Order. If that order has no agent, the action is refused with *"The sales order has no agent assigned."* Open the Sales Order and use **Assign Agent** (section 2.5) to set one, then retry.

Proceed to section 4 to finalise the invoice.

4. Sales Invoices

Navigate to **Sales › Invoices** (`/admin/sales-invoices`).

What a Sales Invoice is. A Sales Invoice is the formal billing document sent to the customer. It is the legal record of the sale: it states what was sold, at what price, the VAT due, and the amount the customer owes. Once finalised, a sales invoice is immutable — it cannot be edited, only voided (which raises a reversing credit note).

Why Sales Invoices exist. The invoice is the document that creates the customer's obligation to pay. In accounting terms, finalising an invoice posts the revenue to the General Ledger (DR Accounts Receivable or Cash / CR Sales Revenue and VAT Payable). For credit customers it opens an AR (Accounts Receivable) item — a record of the amount owed — which is then tracked and aged until payment is received. Without invoices, the business has no formal claim on the customer and no basis for its revenue figures or tax filings.

Direct invoices vs SO-sourced invoices. There are two origins for a sales invoice:

- `DIRECT` **(walk-in):** created manually for a cash customer who is buying on the spot with no prior order. Stock is issued and revenue posted at the moment of finalisation.
- `SALES_ORDER`**-sourced:** created from a Delivery (section 3.3). These invoices post revenue only — stock was already issued when the delivery was confirmed. This distinction prevents the same goods from being costed twice.

Why the origin matters. If a `SALES_ORDER`-sourced invoice also issued stock, the Cost of Goods Sold would be posted twice: once at delivery and once at invoicing. The system prevents this by tracking the origin on every invoice and skipping the stock-issue step for SO-sourced invoices. A walk-in invoice (DIRECT) has no prior delivery, so it must issue stock at finalisation — that is the only point at which goods leave.

The VAT calculation. All prices are entered tax-exclusive (net). The system calculates VAT per line using each product's VAT status (Standard 18%, Zero-Rated 0%, or Exempt 0%). The VAT rate is snapshotted onto the line at sale time so a later rate change cannot silently alter a historical invoice. The invoice prints a VAT analysis breaking down the tax by rate band.

Price snapshots. When you add a product line to an invoice (or any sales document), the system reads the current price from the price list and records it permanently on that line. If the price list is updated tomorrow, the historical invoice is unaffected — it retains the price that applied at sale time. This is called a "price snapshot" and is mandatory for any document that is legally an audit record.

4.1 Create a direct (walk-in) invoice

1. Navigate to **Sales › Invoices** (`/admin/sales-invoices`).
2. Click **New Invoice**.
3. Pick the **Customer** by name.
4. Choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — enabled currencies only, defaulting to the company default).
5. The **Agent** field is pre-filled with **your own agent** (if your user is linked to one) — leave it, clear it, or type another agent to change it. Optionally pick a **Route**. If you clear the Agent and leave it blank, the system still falls back to your linked agent and that agent's primary route on save; the pre-fill just makes the default visible up front. (Root/admin logins with no linked agent open with the box empty.)
6. Click **Create Invoice**. A draft invoice is created.

4.2 Add lines to an invoice

Same process as adding lines to a quotation or order, including the product-scoped **Unit** dropdown (see section 1.2). Lines can only be added, edited, or removed while the invoice is in DRAFT.

"incl. VAT" vs "excl. VAT" on a line. Each line shows a small tag beside its **Unit Price** reading either **incl. VAT** or **excl. VAT**. It tells you how the unit price relates to VAT, and it simply reflects the VAT stance of the price list the line's price came from (set once per list — see *Price Lists › VAT-inclusive vs exclusive pricing* in chapter 02, Master Data):

- **incl. VAT** — the unit price shown is the customer-facing **gross** price, with VAT already inside it. The system splits the net and the VAT back out of that figure (shown in the line's **Net** and **VAT Amt** columns) so the customer pays exactly the price shown, to the shilling. This is the natural fit for retail/shelf pricing.
- **excl. VAT** — the unit price shown is the **net** price, and VAT is added on top, so the customer pays the price plus VAT. This is the usual wholesale/B2B "price + VAT" convention.

The tag is a read-only indicator — you never set it on the line, and it does not change which product or price you picked, only how the VAT is derived from that price. The same tag appears on quotation and sales-order lines. Zero-rated and exempt products still show a tag but attract no VAT either way.

Weighed-goods lines (sell by weight)

When a product is configured as **sold by weight** (loose rice, sugar, produce — see *Master Data › Products › Weighed goods*), the line behaves the same as any other, with three differences you will notice:

- **You must use a weight unit.** The **Unit** dropdown for a weighed product offers only its weight units (kilograms, grams). If a by-count unit is somehow submitted, the line is refused with *"<Product> is sold by weight — choose a weight unit such as kilograms, not Pieces."* Enter the weighed amount as the **Quantity** — for example `0.813` for 813 grams on a per-kg product. (In these messages the trailing unit is the unit's full **name** — `Pieces`, `Kilogram` — not its short code; the exact by-count name shown is whichever unit was attempted.)
- **The weight is rounded to the scale step.** If the product declares a scale division (say 0.005 kg), the quantity you enter is rounded to the nearest division when the line is saved. Entering `0.813` kg records a **billed** quantity of `0.815` kg. The line shows both so the rounding is transparent: under the quantity it reads **entered 0.813 → billed 0.815**. The amount is always computed from the billed (rounded) weight × the per-unit price.
- **Guards catch scale mistakes.** A weight so small it would round down to nothing is refused (*"that weight is too small to sell. The smallest amount is 0.005 Kilogram"*), and a weight above the product's **max sale weight** — the classic kg/gram fat-finger, keying `813` instead of `0.813` — is refused with *"813 Kilogram is above the maximum sale weight of 50 Kilogram. Please check the weight."* Nothing is saved; correct the quantity and re-add the line.

Everything else — VAT, discounts, the incl./excl. VAT tag, price snapshots — works exactly as for a by-count line.

Override a line's unit price

Sometimes the price the system resolved from the price list is not the price you want to bill — a one-off negotiated rate, a price-match, or a correction. On a **DRAFT** invoice you can override any line's unit price in place (this needs the `SALES.INVOICE.OVERRIDE` permission — users without it do not see the control):

1. In the **Lines** table, click the **pencil** (Override price) icon next to the line's unit price. A small input replaces the price.

2. Type the new unit price and click the **save** (check) button. Click the ✕ to cancel and keep the original price.

The override is applied to that line only, and the line's net, VAT, and gross recalculate immediately. Two rules apply:

- **The price must be greater than zero.** A zero or negative override is refused with *"Enter a valid unit price greater than zero."* — you cannot give the goods away or drive the line total negative through this field. (The server enforces the same rule with its own message, *"The override price must be greater than zero."*, for direct API callers.)
- **A wildly-off price asks for confirmation (soft guard).** If the price you enter is more than about **10×** or less than about **0.1×** the line's list price, a confirmation pops up — *"This is about 12× the list price (2,000). Apply anyway?"* — before it is applied. This is a fat-finger guard, **not** a hard block: any positive price is allowed once you confirm. Confirm to apply, or cancel to keep editing. When the list price is unknown or the entered price is within the normal range, no confirmation appears.

4.3 Record a payment

Payments can be recorded on a draft invoice before it is finalised.

1. In the **Record Payment** panel (shown on a DRAFT invoice).
2. Choose the **Tender type**: Cash or Mobile Money.
3. Enter the **Amount**, and optionally a **Reference** (for example, the M-Pesa transaction reference for Mobile Money).
4. Click **Record**.

Recording payments requires the `SALES.INVOICE.SETTLE` permission (separate from the permission to create lines).

For **cash / walk-in customers** the total payments must equal the invoice gross before you can finalise. Credit customers may have a balance that becomes an open AR item.

4.4 Finalise an invoice

1. Open the draft invoice.
2. Click **Finalise**.
3. The status changes to **FINALISED** and an invoice number is assigned.

After finalisation:

- For DIRECT invoices, stock is issued from the branch.
- For invoices from a delivery, revenue is posted (stock was already issued at delivery).
- For credit customers, an AR open item is created for any unpaid balance.

Paid-in-full rule: walk-in (cash) customers must be fully paid before finalisation is allowed.

Rounding follows the invoice's currency. Line amounts, VAT and totals are rounded to the decimal places of the invoice's own currency: whole shillings for a TZS invoice, cents for a USD, EUR or KES

invoice (18% VAT on USD 12.00 is USD 2.16). The same applies to quotations, sales orders and POS sales. The accounting entry is always posted in the base currency, each amount converted at the invoice's exchange rate.

Credit limit: if a credit customer's outstanding balance plus this invoice would exceed their credit limit, finalisation is blocked unless you hold the `SALES.CREDIT.OVERRIDE` permission. (This is a credit-limit check at finalisation. For SO-sourced sales, the broader credit-control hard block — covering credit status, manual hold, and the limit — already runs earlier, at Sales Order confirm; see section 2.3.)

Not enough stock (negative-stock block). If your company has turned on **Block sales that would take stock negative** (in *Sales Settings*), finalising a walk-in **DIRECT** or **POS** invoice is refused when a line would take that product's available stock below zero. The refusal is a plain message naming the product with how much is available versus requested — for example *"Not enough stock of Sukari 1kg to complete this sale — 3 available, 5 requested. To allow this, enable backorder in Sales Settings."* Nothing is posted; the invoice stays in DRAFT. To clear it you can lower the quantity, receive or transfer more stock in first, or — if selling ahead of stock is deliberate — have a user with `SALES.SETTINGS.MANAGE` turn the setting off, which lets the sale go through and stock go negative (backorder). When the setting is **off**, or the company has never saved any Sales Settings, sales are never blocked this way and stock is allowed to go negative. Invoices raised from a delivery are not checked here (their stock was issued and checked at the delivery — see below), and **the same block applies when you create a Delivery** against a sales order (section 3.1).

4.5 Void an invoice

What voiding means. Voiding a finalised invoice reverses its financial effect: the revenue is reversed, the AR item is cancelled, and VAT is adjusted. The original invoice number is retained on the record (voiding is not deletion — the document remains as evidence that the transaction happened and was corrected). A reversing credit note is raised automatically. Use voiding only when an invoice was issued in error; for goods returned by the customer use a Sales Return (section 5) instead.

A finalised invoice can be voided if it was issued in error:

1. Open the finalised invoice (navigate to **Sales › Invoices**, open it from its row action, or go to `/admin/sales-invoices/uid/{uid}`).
2. Click **Void Invoice**, enter a mandatory reason, and confirm.
3. The invoice status changes to **VOID** and a reversing credit note is posted.

The original invoice number is retained on the voided record. Voiding is not the same as deletion.

4.6 Issue a fiscal (EFD) receipt

What a fiscal receipt is. In Tanzania (and similar tax regimes) a taxable sale must be recorded on an approved fiscal device — an EFD (Electronic Fiscal Device) or VFD (Virtual Fiscal Device) — which returns a government **fiscal receipt number** and a **verification URL** the customer can use to confirm the receipt with the revenue authority. This ERP sends each finalised invoice to the company's configured fiscal provider and stores the result on the invoice.

Where it appears. A **Fiscal Receipt** panel is shown on the invoice detail page (`/admin/sales-invoices/uid/{uid}`) **only once the invoice is FINALISED** and only to users with the `FISCAL.VIEW` permission. It is the same panel for every finalised invoice regardless of origin — a walk-in DIRECT invoice, an invoice raised from a delivery, or a POS counter sale (open the sale's invoice via **View Invoice** — see section 9.4).

To issue (or retry) the receipt:

1. Open the **FINALISED** invoice.
2. In the **Fiscal Receipt** panel, read the current status tag (see the table below). If no receipt exists yet it reads **Not issued**.
3. Click **Issue EFD receipt** (the button reads **Retry EFD receipt** if a previous attempt exists). This needs the `FISCAL.MANAGE` permission — view-only users see the status but no button.
4. On success the status becomes **ISSUED** and the panel shows the **Fiscal number** and, where the provider returns one, a **Verification** link (opens the authority's verification page in a new tab). Hand or print this to the customer.

Receipt states:

Status	Meaning / what to do
Not issued	No attempt yet — click **Issue EFD receipt**.
PENDING	Submitted to the device; awaiting its response.
ISSUED	Success — fiscal number and verification URL are shown. This is final; a receipt is never re-issued.
FAILED	The device rejected or could not be reached; the reason is shown. Fix the cause and click **Retry EFD receipt**.
NOT_CONFIGURED	No EFD/VFD device is set up for this company yet — the panel says so. Ask your administrator to configure the fiscal provider, then retry.

Gotchas.

- The receipt can only be issued **after** the invoice is finalised — there is no fiscal button on a DRAFT invoice.
- **ISSUED is permanent** and a **VOID** invoice cannot be fiscalised, so no button is offered in either case.
- Issuing is safe to retry: repeated clicks will not produce two fiscal numbers for the same invoice.

4.7 Invoice status reference

Status	Meaning
DRAFT	Being prepared; lines and payments editable
FINALISED	Issued; posted to AR and GL; immutable
VOID	Reversed; credit note posted; number retained

Example — Full O2C: Karibu Supermarkets (credit account):

Continuing from section 1's example, Sales Order **SO-0112** was created from the accepted quotation. The warehouse confirms goods are ready.

1. **Confirm SO:** Ali opens **Sales › Sales Orders** (`/admin/sales-orders`), clicks the **Open** action on the SO-0112 row, clicks **Confirm Order**, and clicks **Yes, Confirm** in the dialog. Karibu is within its credit limit and not on hold, so the credit-control check passes. Status becomes CONFIRMED; stock reserved — 50 cartons Unga + 30 cartons Mafuta.
2. **Deliver:** From the confirmed **SO-0112** detail page, Ali clicks **Create Delivery**. The delivery form opens with both lines pre-filled at their full open quantity. He keeps both lines included (50 + 30 cartons), sets Delivery Date 2026-06-15, and clicks **Create Delivery**. Delivery **DELIVERY-0089** is created; SO status → FULFILLED.
3. **Invoice from delivery:** Ali opens DELIVERY-0089 at `/admin/deliveries/uid/{uid}` and clicks **Invoice this Delivery**. A DRAFT invoice is created. Since Karibu is a CREDIT_ACCOUNT customer, Ali clicks **Finalise** without adding a payment — the unpaid balance of TZS 1,535,400 (plus 18% VAT = TZS 1,811,772 gross) becomes an open AR item. Invoice number **INV-0203** is assigned.

Example — Walk-in direct invoice (cash customer):

Cashier Fatuma opens **Sales › Invoices** (`/admin/sales-invoices`) and clicks **New Invoice**. She picks customer **Amina Hassan (walk-in)**. She adds one line: **Sukari 1kg**, Unit **KG**, Qty **5**, price TZS 2,200/kg = TZS 11,000 net; VAT 18% = TZS 1,980; gross = TZS 12,980. In the **Record Payment** panel she sets Tender type **Cash**, Amount **TZS 12,980**, and clicks **Record**. She clicks **Finalise** — status becomes FINALISED, invoice number **INV-0204** is assigned, stock is issued, and the cash is recorded.

5. Sales Returns (RMA)

Navigate to **Sales › Sales Returns** (`/admin/sales-returns`).

What a Sales Return is. A Sales Return (also called an RMA — Return Merchandise Authorisation) is the document that records goods coming back from the customer. It is always tied to a specific delivery so the system knows exactly which shipment is being reversed.

Why Sales Returns exist. When a customer returns goods — because they are damaged, wrong, or surplus — several things need to happen simultaneously: the stock must come back into the warehouse, the customer's account must be credited (so they do not owe money for goods they no longer have), the revenue must be reversed, and the cost of those goods must be put back. Doing these four things as separate manual steps would be error-prone and would leave the accounts temporarily out of balance. A Sales Return handles all four atomically: on creation, stock is returned

to the branch, a credit note is raised automatically, revenue and VAT are reversed, and (for a credit customer) the AR open item is reduced.

When a Sales Return is used. A Sales Return is created by a warehouse clerk or sales supervisor when goods arrive back from the customer. It can only reference a previous delivery — you cannot return more than was delivered on that delivery, and returns against the same delivery can be processed in multiple batches up to the full delivered quantity.

How a Sales Return flows. A Sales Return is created and immediately `CONFIRMED` in a single step. There is no draft stage. The return number (`RET-####`) is assigned at creation. A credit note is raised in the same transaction.

What a credit note is. A credit note is the financial document that reduces what the customer owes. If an invoice says "you owe us TZS 100,000," a credit note for TZS 20,000 on the same account means the customer's balance is reduced to TZS 80,000. Credit notes are raised automatically by the system on a Sales Return (for the returned goods) and on a void (for a fully reversed invoice); they cannot be raised manually through the sales return screen.

5.1 Create a return

The **Sales Returns** list (`/admin/sales-returns`) is view-only — it has no "New Return" button (its subtitle reads *"Create a return from the Delivery detail screen."*). There are two ways to reach the create form:

- Open the relevant CONFIRMED delivery and click **Create Return** (the delivery is pre-loaded), or
- Go directly to `/admin/sales-returns/create` and select the **Delivery** from the picker, then click **Load** to pull in its lines.

1. With the delivery loaded, the form shows the delivered lines with **Delivered**, **Already Returned**, and **Returnable** columns.
2. Enter the **Return Qty** for each line being returned (cannot exceed the **Returnable** balance — delivered minus what has already been returned).
3. Set the **Return Date** and optionally enter a **Reason**.
4. Click **Confirm Return**.

Returns are created directly in **CONFIRMED** status. Stock is returned to the branch. A credit note is raised automatically (pro-rated to the returned quantity).

How the credit is worked out. The credit note gives back the same share of the order line's net and VAT as the share of goods returned — returning 2 of 5 credits 2/5 of what the customer was charged for that line, discounts and VAT-inclusive prices included. Amounts are rounded in the order's currency (whole shillings for TZS, cents for USD).

5.2 Returnable quantity

Each return reduces the returnable balance for that delivery line. You can process multiple returns against the same delivery line until the full delivered quantity has been returned.

Example — Partial sales return (Karibu Supermarkets):

Two days after delivery, Karibu reports 5 cartons of Mafuta ya Kupikia arrived leaking. The stock controller opens delivery **DELIVERY-0089** and clicks **Create Return** (which pre-loads that delivery). She enters **Return Qty = 5** on the Mafuta line, sets return date **2026-06-17**, reason **"Damaged packaging — leaking oil"**, and clicks **Confirm Return**. Return **RET-0031** is created in CONFIRMED status. Five cartons of Mafuta stock are returned to the warehouse and a credit note for TZS 104,500 (5 × TZS 20,900) plus VAT is automatically raised against INV-0203.

6. Blanket Orders

Navigate to **Sales › Blanket Orders** (`/admin/blanket-orders`).

What a Blanket Order is. A Blanket Order is a framework supply agreement with a customer that fixes the unit price for a product and commits to a total quantity over a defined validity window. Instead of raising a new Sales Order with price negotiations each time the customer buys, both parties agree upfront: "you will buy up to 1,000 bags at TZS 6,500 each over the next six months." Each actual purchase draws down against this agreement — these draws are called **releases** or **call-offs**.

Why Blanket Orders exist. Regular customers who buy in predictable volumes benefit from negotiated prices locked in for a period, while the business gains revenue predictability and avoids repeated pricing discussions. Without a blanket order, each purchase is independent — a busy sales desk might accidentally apply inconsistent prices to the same customer, or forget what was agreed. The blanket order is the single source of truth for the agreed terms. It also automatically prevents over-delivery: the system tracks how much has been called off against the committed quantity and refuses to draw more than the total commitment.

When a Blanket Order is used. A blanket order is created by a sales manager when a long-term supply contract is signed with a customer. Once active, the sales team creates call-off Sales Orders against it whenever the customer exercises part of their commitment.

How a Blanket Order flows. A blanket order is `ACTIVE` from creation. Call-offs (draw-downs) produce ordinary Sales Orders that flow through the normal O2C chain (confirm → deliver → invoice). The blanket itself tracks the remaining committed quantity on each product line. When all quantities are fully drawn, the validity window expires, or a manager closes it manually, the blanket becomes `CANCELLED` (no further releases). The blanket document itself posts no stock and no GL entries — only the resulting Sales Orders do.

6.1 Create a blanket order

1. Navigate to **Sales › Blanket Orders** (`/admin/blanket-orders`) and click **New Blanket Order**, or go directly to `/admin/blanket-orders/create`.
2. Select the **Company** and **Branch**.
3. Pick the **Customer** by name.
4. Choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — enabled currencies only, defaulting to the company default), then set **Valid From** and **Valid To** dates.
5. Add one or more **Lines**: for each, pick the product by name, choose a unit (limited to that product's configured units — see section 1.2), and enter the committed quantity and unit price.
6. Optionally add notes (up to 500 characters).
7. Click **Create Blanket Order**.

The blanket order is created with status **ACTIVE** and assigned an order number.

6.2 Create a release (draw-down)

When the customer calls off part of their commitment:

1. Open the blanket order (navigate to `/admin/blanket-orders/uid/{uid}`).
2. Click **Draw Release** (visible only when the blanket is ACTIVE and you hold the manage permission).
3. Enter the **Branch ID** for the delivery branch.
4. Pick the **Agent** by name.
5. For each line you want to include, tick it and enter the **Draw Quantity** (cannot exceed the remaining committed quantity).
6. Click **Create Release**.

A **Sales Order** is created and linked to the blanket. The committed remaining quantity on each drawn line decreases.

6.3 Cancel a blanket order

Open the blanket order and click **Cancel** then confirm. The status changes to **CANCELLED**. Previously generated Sales Orders from this blanket are unaffected.

6.4 Blanket order statuses

Status	Meaning
ACTIVE	Agreement in force; releases can be drawn
CANCELLED	Cancelled; no further releases

Example — Blanket supply agreement:

Duka Kuu Ltd signs a 6-month supply deal for 1,000 bags of Mchele wa Zambia at TZS 6,500/bag. The sales manager opens **Sales › Blanket Orders** (`/admin/blanket-orders`), creates a new blanket for

Duka Kuu Ltd, Valid From **2026-07-01** to **2026-12-31**, adds one line: **Mchele wa Zambia 10kg**, Unit **BAG**, Qty **1,000**, Unit Price **6,500**. Order is saved as ACTIVE.

In July, Duka Kuu calls off 200 bags. The sales manager opens the blanket, clicks **Draw Release**, draws 200 bags → Sales Order **SO-0145** is created. Remaining committed quantity on the blanket is now 800 bags.

7. Standing Orders (Recurring)

Navigate to **Sales › Standing Orders** (`/admin/standing-orders`).

What a Standing Order is. A Standing Order (also called a recurring order or repeat order) is a template that tells the system to generate a new Sales Order automatically on a regular schedule — weekly, bi-weekly, or monthly. It holds the customer, the products, the quantities, and the prices for a typical delivery cycle.

Why Standing Orders exist. Some customers receive the same goods on the same schedule every week or month — a hotel that takes 50 loaves of bread every Monday, or a distributor that replenishes the same five products on the first of each month. Without standing orders, the sales desk must manually create the same Sales Order repeatedly, risking forgetting, using the wrong quantities, or applying the wrong prices. A standing order removes the repetitive work and ensures consistent, timely order creation without manual intervention.

When a Standing Order is used. Standing orders are set up by a sales manager or sales administrator for customers with regular, predictable buying patterns. Once active, the cashier or sales desk does not need to do anything — orders appear automatically. The standing order can be paused if supply is interrupted and resumed when normal service resumes.

How a Standing Order flows. A standing order is `ACTIVE` from creation. The system runs a nightly check and generates a new Sales Order (in `DRAFT` status — a human must confirm it deliberately) for every active standing order whose next run date is today or earlier. After generation, the next run date advances by one period. A standing order can be `PAUSED` (no generation while paused) and `RESUMED`, or permanently `CANCELLED`. The generated Sales Orders flow through the normal O2C chain.

7.1 Create a standing order

1. Navigate to **Sales › Standing Orders** (`/admin/standing-orders`) and click **New Standing Order**, or go directly to `/admin/standing-orders/create`.
2. Pick the **Branch** and **Customer**, then choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — enabled currencies only, defaulting to the company default).
3. Choose a **Frequency**: Daily, Weekly, Bi-Weekly, or Monthly.
4. Set a **Start Date**. Optionally set an **End Date**; leave it blank for open-ended.

5. Add lines: pick each product by name; the unit choices are limited to that product's configured units (see section 1.2); enter quantity and unit price.
6. Click **Create Standing Order**.

The standing order is created with status **ACTIVE** and the first `Next Run Date` is set.

7.2 Pause and resume

- **Pause** — open the standing order (navigate to `/admin/standing-orders/uid/{uid}`) and click **Pause**. No Sales Orders are generated while the order is paused.
- **Resume** — click **Resume** to make it active again.

7.3 Trigger a run manually

Click **Trigger Now** to generate a Sales Order immediately (without waiting for the scheduled run). The next run date advances by the configured frequency.

7.4 Cancel a standing order

Click **Cancel** to stop the standing order permanently. The status changes to **CANCELLED** and no further Sales Orders are generated.

7.5 Automatic generation

The system checks every night at midnight and generates Sales Orders for all ACTIVE standing orders whose next run date is today or earlier. Each generated order advances the next run date by one period.

7.6 Standing order statuses

Status	Meaning
ACTIVE	Generating on schedule
PAUSED	Temporarily stopped; can be resumed
CANCELLED	Permanently stopped

Example — Weekly bread delivery for Hoteli ya Pwani:

Hoteli ya Pwani orders 50 loaves of bread every Monday. The sales rep opens **Sales › Standing Orders** (`/admin/standing-orders`), creates a new standing order for **Hoteli ya Pwani**, Frequency **Weekly**, Start Date **2026-06-16**, no end date. Line: **Mkate Mzima**, Unit **PCS**, Qty **50**, Unit Price **TZS 800**. The system auto-generates **Sales Order SO-0151** on Monday 16 June, then **SO-0158** on 23 June, and so on every week without manual action.

8. Pricing Rules

Navigate to **Sales › Pricing Rules** (`/admin/pricing-rules`).

What pricing rules are. Pricing rules are pre-configured exceptions to the standard price list. Without any rules, every customer is charged the standard list price for a product. Rules let the business offer lower prices automatically under specific conditions — for example, a lower price per bag when a customer orders more than 100 bags at once (a quantity break), or a privately negotiated price that applies only to one specific customer.

Why pricing rules exist. Manual price overrides by sales staff are error-prone and untraceable. A salesperson might give a loyal customer a discount one day and forget it the next, or apply the wrong discount tier. Pricing rules encode the business's commercial agreements in the system so that the correct price is applied automatically and consistently every time, without needing the salesperson to remember or calculate. They also create an audit trail: when a line is priced, the system records which rule was applied (standard list, a tier, or a customer-specific price) as a diagnostic field on the line.

How pricing rules resolve. When a product line is added to any sales document (quotation, order, invoice, or POS sale), the system runs a single price-resolution check in the following priority order, applying the first rule that matches:

1. **Customer price** (highest priority) — a contract price for this exact customer and product
2. **Active promotion** — a time-limited offer matching the product or product category
3. **Price tier** — a volume-break price if the ordered quantity meets the tier's minimum
4. **Standard list price** — the product's price on the customer's assigned price list
5. **No price found** — the line is rejected; the product cannot be sold without a price

Once the price is resolved, the standard totals calculation (net, VAT, gross) runs unchanged — pricing rules only affect the unit price input.

VAT stance travels with the resolved price. Whichever source wins above, the resolved price also carries the VAT-inclusive or -exclusive stance of the price list it came from. That stance is what shows as the **incl. VAT** / **excl. VAT** tag on the sales line (section 4.2): it decides whether VAT is split out of the price or added on top, but it does not change which price wins the resolution. See *Price Lists › VAT-inclusive vs exclusive pricing* in chapter 02 for how the stance is set on a list.

8.1 Price tiers (quantity breaks)

What a price tier is. A price tier is a volume-break discount: if a customer orders at least a minimum quantity of a specific product, they receive a lower unit price than the standard list price. For example, the standard price for a 50 kg bag of cement is TZS 15,200, but any order of 100 or more bags is priced at TZS 14,500 per bag.

Why tiers exist. Volume pricing rewards large orders and encourages customers to consolidate purchases. Without tiers, a salesperson would have to manually override the price and justify the discount each time — an inconsistent and unaudited process. Tiers make the volume price automatic, consistent, and visible on the price list.

A price tier gives a lower unit price when a customer orders at least a minimum quantity of a product on a given price list.

To create a tier:

1. Open **Sales › Pricing Rules** (`/admin/pricing-rules`) and go to the **Price Tiers** tab.
2. Click **Add Price Tier**.
3. Pick the **Product** and **Price List** by name.
4. Enter **Min Quantity** and **Unit Price**, and choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — enabled currencies only, defaulting to the company default).
5. Click **Save Tier**.

The tier status is **ACTIVE**. To deactivate a tier, click the **Deactivate** button on the row; the tier is soft-deactivated and no longer applied to new transactions.

You cannot have two active tiers for the same product, price list, and minimum quantity combination.

8.2 Customer prices (contract prices)

What a customer price is. A customer price (also called a contract price or a customer-specific price) is a fixed unit price agreed between the business and one specific customer for one specific product. It overrides every other pricing rule — including tiers and promotions — and applies regardless of quantity, as long as it is active and within its effective date window.

Why customer prices exist. Key accounts and long-term customers often negotiate individualised prices as part of a supply agreement — prices that are lower than the standard list but not published generally. Storing these as customer prices means the correct price is applied automatically on every transaction for that customer, with no risk of the wrong price being used by a different salesperson who does not know the agreement.

A customer price sets a fixed unit price for a specific product for a specific customer, overriding the standard price list.

To create a customer price:

1. Open **Sales › Pricing Rules** (`/admin/pricing-rules`) and go to the **Customer Prices** tab.
2. Click **Add Customer Price**.
3. Pick the **Customer** and **Product** by name.
4. Enter the **Unit Price** and choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — enabled currencies only, defaulting to the company default).
5. Optionally set **Effective From** and **Effective To** dates for a time-limited contract.
6. Click **Save Price**.

Only one customer price record can exist per customer-and-product pair. Deactivate the existing record before creating a new one is not possible (the unique constraint is status-agnostic); raise a support request to change an existing contract price.

8.3 Pricing resolution order

When a sale line is priced the system applies the first matching rule in this priority:

1. Customer price (highest priority)
2. Active promotion (managed via the API; no UI currently)
3. Price tier
4. Standard list price
5. No price configured (the line is rejected)

Example — Volume tier for cement:

The sales manager opens **Sales › Pricing Rules** (`/admin/pricing-rules`), goes to **Price Tiers**, and clicks **Add Price Tier**. He picks product **Saruji 50kg**, Price List **Wholesale TZS**, Min Quantity **100**, Unit Price **TZS 14,500**, Currency **TZS**. Any order for 100+ bags on the Wholesale price list will now use TZS 14,500 instead of the standard TZS 15,200.

Example — Contract price for Karibu Supermarkets:

Under the **Customer Prices** tab the manager creates: Customer **Karibu Supermarkets Ltd**, Product **Unga wa Ngano 2kg**, Unit Price **TZS 17,500** (negotiated). From the next sale, whenever a sales line is added for this customer and product, TZS 17,500 is applied — regardless of the price list.

9. Point of Sale

Navigate to the **Point of Sale** group in the sidebar.

What the Point of Sale module is. Point of Sale (POS) is the in-store face-to-face retail workflow. It provides a cashier-facing checkout screen to ring up products, accept cash, and issue receipts. Everything processed through POS is ultimately a sales invoice — POS wraps the invoice channel with till management and session-level drawer accountability.

Why POS exists as a distinct module. A back-office sales invoice is fine for credit-account customers who receive goods on account and pay later. Counter retail is different: a cashier is processing many small transactions rapidly, cash is flowing in and out of a physical drawer, and at end of day the business needs to verify that the cash in the drawer matches what the system says was collected. The POS module adds the `till` and `session` layer on top of the invoice to manage this accountability — without it, cash sales would have no way to reconcile the physical drawer to the books.

What a till is. A till is a physical cash register position at a branch (for example, "Counter 1" or "Counter 2"). In the system a till is a named record tied to a branch and to a bank/cash account that represents the drawer. Multiple tills can operate at the same branch simultaneously. A till must be `ACTIVE` before a session can be opened on it.

What a session is. A session is the till's working period — typically one business day or one shift. Before a cashier can ring sales, they open a session by declaring the opening float (the cash placed in the drawer to make change). During the session every POS sale, refund, and payout is tracked against that session. At end of day the cashier or manager closes the session by counting the cash in the drawer, and then a manager reconciles the session to post any variance to the General Ledger.

What a POS sale is. A POS sale is a cash counter transaction. It produces a finalised sales invoice with origin `POS` (it is stamped with the originating POS session), the cash payment is recorded automatically for the full amount, and revenue is posted on finalisation. The invoice number (`INV-####`) is assigned on the spot. No quotation, sales order, or delivery step is involved — POS is designed for speed at the counter.

> **Note on stock.** A `POS`-origin invoice issues stock at finalisation exactly like a walk-in `DIRECT` invoice: finalising the invoice decrements on-hand stock for each line (the stock-issue step runs for both `DIRECT` and `POS` origins). A POS sale completed at the till therefore both posts revenue **and** reduces inventory.

POS is used for face-to-face retail transactions. A **till** is a physical cash register position. Each till must be opened in a **session** before sales can be processed. The session is closed and reconciled at end of day.

9.1 Roles

Role	Typical permissions
Cashier	Open session, ring sales, view sessions
Manager	All cashier permissions plus create/deactivate tills, close sessions, reconcile

Your administrator assigns the appropriate POS permissions to your role. Contact them if POS is not visible in your menu.

9.2 Set up a till

This is a one-time setup task done by a manager.

1. Navigate to **Point of Sale › POS Tills** (`/admin/pos/tills`).
2. Click **New Till**.
3. Enter a **Till Name** (e.g. "Counter 1").
4. Pick the **Branch** by name.
5. Click **Create Till**.

The till is created with status **ACTIVE**. To deactivate a till, click **Deactivate** on its row.

9.3 Open a session (start of day)

What opening a session means. Opening a session declares the start of a cashier's working period on a specific till. The opening float is the starting cash in the drawer (coins and notes placed there before the first sale so the cashier can make change). The system records this amount and uses it as

the baseline for the end-of-day cash reconciliation. Only one session can be open on a till at a time — you cannot accidentally open a second session on the same counter without closing the first.

The **POS Sessions** list shows every session with its number, status (OPEN, CLOSED, RECONCILED), opening float, and expected cash; use the **Open Session** button to start a new one and the **View** action on a row to open a session's detail.

1. Navigate to **Point of Sale › POS Sessions** (`/admin/pos/sessions`).
2. Click **Open Session**.
3. Pick the **Till** by name (only ACTIVE tills are listed).
4. Enter the **Opening Float** — the cash amount placed in the drawer at the start of the day.
5. Click **Open Session**.

A new session is created with status **OPEN**. Only one session can be open on a till at a time.

9.4 Ring a sale

What "ringing a sale" means. This is the cashier's checkout step: entering the products and quantities the customer is buying, taking the cash the customer hands over, and completing the transaction. The system calculates the total, computes the change due, and — on completion — records the cash payment, finalises the sales invoice, posts the revenue, and issues the receipt. (On completion a `POS`-origin invoice issues stock just like a `DIRECT` walk-in invoice — see the stock note above.)

What the tendered amount is. The tendered amount is the cash the customer physically hands to the cashier — often a round number larger than the total. If the total is TZS 13,000 and the customer hands over TZS 20,000, the tendered amount is TZS 20,000 and the change is TZS 7,000. The system calculates the change and the cashier returns it. A sale cannot be submitted if the tendered amount is less than the total.

1. Navigate to **Point of Sale › Point of Sale** (`/admin/pos/sell`) — this is the checkout screen.
2. If your organisation has more than one company, select the correct company.
3. Pick the **Session** — only OPEN sessions are listed. Each option is labelled *"Session <…> (Till <till id>)"* (a short fragment of the session's UID plus its till id), not the `POS-####` session number.
4. Pick the **Customer** (type in the search box above the picker to filter the list, then select).
5. Pick the **Agent** (search then select) — required; leaving Agent blank will cause the sale to be rejected.
6. Choose the **Currency** from the Currency Picker (see *Common UI Patterns* in chapter 00 — enabled currencies only, defaulting to the company default).
7. Click **Add Line**. Pick the **Product**; the **Unit** field is disabled until a product is picked and then lists only that product's configured units (defaulting to its base unit — see section 1.2), so confirm it or choose another from the list; enter **Quantity** and **Unit Price**, and optionally a line **Discount** (entered as an amount).
8. Add further lines as needed. The **Total** updates in the footer.
9. Enter the **Tendered Amount** (the cash handed over by the customer). The **Change** is calculated immediately. The sale cannot be submitted if the tendered amount is less than the total.

10. Click **Complete Sale**.

A success receipt is displayed showing the invoice number and total. Click **View Invoice** to open the full invoice, or **New Sale** to start the next transaction.

Fiscal (EFD) receipt for a counter sale. A completed POS sale produces a normal FINALISED invoice, so the government fiscal receipt is issued from that invoice — not from the checkout screen. Click **View Invoice** on the success panel (or open it later from **Sales › Invoices**) and use the **Fiscal Receipt** panel to issue the EFD receipt and read its fiscal number and verification URL. See section 4.6 for the full steps and the receipt states; it needs the `FISCAL.VIEW` / `FISCAL.MANAGE` permissions.

Notes:

- On this checkout screen the sale is settled in cash — you enter a single **Tendered Amount** and the payment is recorded as Cash automatically. (The POS sale itself can also accept several tenders together; see *Splitting payment across tenders* below.)
- The agent field is mandatory on the backend; leaving it blank will cause the sale to be rejected.
- If the chosen session has been closed in the meantime, the sale is rejected with **"This POS session is not OPEN."** so you know to re-open or re-select an OPEN session.
- If a **Complete Sale** click is interrupted (network drop, slow response) and the cashier retries, the system recognises the repeat and returns the original sale instead of ringing it twice — a sale is never double-posted, so it is safe to retry.

Splitting payment across tenders

A POS sale does not have to be settled with a single cash amount. It can be split across **several tenders** at once — for example part **cash** and part **card**, or cash plus **mobile money** — as long as the tenders together cover the sale total. Each tender is recorded as its own payment on the resulting invoice (cash, card, mobile money, or cheque), so the receipt and the books show exactly how the customer paid. The standard checkout screen above records a single cash tender; mixed-tender sales are taken on a connected POS terminal or device that offers the tender breakdown.

Age-restricted items

If any product on the sale is **age-restricted** (for example an 18+ or 21+ line — see *Products and Catalog* in chapter 02), the sale is **blocked** until age has been dealt with. The cashier must either confirm that the customer's age has been verified (the prompt to confirm appears when an age-restricted line is present) or hold the `POS.SALE.AGE_OVERRIDE` permission. Without one or the other, completing the sale is refused so restricted goods cannot be sold without an age check.

Scale labels (embedded weight or price barcodes)

Deli, butchery, and produce items are often weighed at a counter scale that prints a special **scale label** — a barcode that carries the item plus its weight or its price inside the code. When such a label is scanned at the till, the system recognises the format, identifies the product, and works out the **quantity** (or the line price) automatically from the embedded value, so the cashier does not type the weight by hand. Ordinary fixed-price barcodes are read as usual.

Reversing (voiding) a POS sale

A completed POS sale that was rung in error can be **reversed** at the till. Reversing a sale undoes everything the sale did: it reverses the revenue and VAT, refunds the cash out of the drawer, and returns the goods to stock — the opposite of the original transaction, recorded as evidence rather than deleted.

A reversal is only allowed while the **till session is still OPEN**, so that the cash refund comes out of the same drawer that took the money. Once the session has been closed or reconciled, a mis-rung sale is corrected through a back-office invoice void (section 4.5) instead. Reversing a sale requires the `POS.SALE.VOID` permission; you enter a reason, which is recorded on the void and in the audit trail.

9.5 Record a payout

What a payout is. A payout is any cash that leaves the drawer during the session that is not change given to a customer. The two types are:

- **Paid Out:** a safe drop (moving excess cash from the drawer to the safe mid-shift) or a petty-cash payment made from the drawer.
- **Refund:** cash paid back to a customer as a refund.

Both types reduce the expected closing cash and are recorded so the end-of-day reconciliation remains accurate. Without recording payouts, the drawer would appear short at close-of-day even though the cash was accounted for.

A payout records cash leaving the drawer during the session — for example, a drop to the safe or a petty-cash refund.

1. Open the session detail (**Point of Sale › POS Sessions** (`/admin/pos/sessions`), click **View** on the OPEN session, or navigate to `/admin/pos/sessions/uid/{uid}`).
2. Click **Record Payout**.
3. Select the **Type**: Paid Out (cash removed from the drawer) or Refund (customer cash refund).
4. Enter the **Amount** and a **Reason**.
5. Click **Record**.

Both payout types reduce the expected closing cash. The live X-read total updates automatically.

9.6 X-Read (live totals during the day)

What an X-Read is. An X-Read (from the retail term "X-reading the register") is a snapshot of running totals for the current session without closing or resetting it. Cashiers and managers use it to verify the session is on track during the day — for example, after a safe drop, to confirm the expected cash figure has decreased correctly. Unlike a Z-Read (see section 9.8), an X-Read does not close anything.

The **X-Read** card on the session detail page shows running totals without closing the session:

Field	Meaning
Sales Total	Sum of all POS sale totals in this session
Payouts	Sum of all payouts (PAID_OUT + REFUND)
Expected Cash	Opening Float + Sales Total − Payouts

Invoice Count	Number of sales processed

Click the refresh icon to reload the X-read at any time.

9.7 Close a session (end of day)

What closing a session means. Closing a session is the end-of-shift step where the cashier physically counts the cash in the drawer and enters the counted amount. The system compares this to the expected cash (computed from the opening float plus all sales minus all payouts) and calculates the variance. A zero variance means the drawer balances perfectly. A positive variance (more cash than expected) is a till surplus. A negative variance (less cash than expected) is a till shortage. The session moves to `CLOSED` but the variance is not yet posted to the General Ledger — that happens at reconciliation.

Closing records the physical cash count.

1. Open the session detail (navigate to `/admin/pos/sessions/uid/{uid}`).
2. Click **Close Session**.
3. Enter the **Counted Cash** — the amount physically in the drawer.
4. Optionally add closing notes.
5. Click **Close Session**.

The session status changes to **CLOSED** and a **variance** is computed:

```
Variance = Counted Cash − Expected Cash
```

- **Positive variance** (over): more cash in the drawer than expected.
- **Negative variance** (short): less cash than expected.
- **Zero variance**: drawer balances exactly.

9.8 Reconcile a session (Z-Read)

What reconciliation is. Reconciliation is the final accounting step for a session. A manager reviews the closed session, confirms the figures are correct, and posts the cash variance — if any — to the General Ledger. After reconciliation the session is permanently locked and no further changes are possible. The result is called the **Z-Read** (again from retail terminology: the Z-read "zeroes" the register for the next session).

What the GL posting means. If the drawer is over (more cash than expected), the excess is income — the business has more cash than it should, which is a gain. The system debits the Cash account and credits a Till Surplus income account. If the drawer is short, the shortfall is an expense — the business is missing cash. The system debits a Till Shortage expense account and credits Cash. A zero variance produces no journal entry.

Reconciliation posts the variance to the general ledger and produces the final Z-Read report.

1. Open a **CLOSED** session (navigate to `/admin/pos/sessions/uid/{uid}`).
2. Click **Reconcile**.

3. Optionally add notes.
4. Click **Reconcile**.

The session status changes to **RECONCILED**. The **Z-Read** card shows all session figures plus the variance and (if non-zero) the journal reference:

- **Over variance** — debit Cash, credit income account 4900 (Till Surplus).
- **Short variance** — debit expense account 5170 (Till Shortage), credit Cash.
- **Zero variance** — no journal posted.

After reconciliation the session is read-only and no further sales or payouts can be recorded.

9.9 Cash variance explained

What cash variance is. Cash variance is the difference between the cash that should be in the drawer (the expected cash, calculated by the system) and the cash that is actually in the drawer (the counted cash, declared by the cashier). Every business aims for zero variance — a perfectly balanced drawer — but small discrepancies occur in practice due to rounding on change, counting errors, or occasional till errors.

The formula is:

```
Expected Cash = Opening Float + Sum of all cash sales in the session – Sum of all payouts
Variance = Counted Cash – Expected Cash
```

A variance greater than zero means there is more cash in the drawer than the sales records account for (a surplus — perhaps the cashier made change errors that favoured the business). A variance less than zero means there is less cash than expected (a shortage — perhaps an error or a discrepancy). Both are posted to the GL at reconciliation so the books always reflect the actual cash held.

9.10 Session lifecycle

Status	Meaning
OPEN	Sales and payouts can be recorded
CLOSED	Session counted; reconciliation pending
RECONCILED	Final Z-read produced; GL posted; session closed

Transitions are one-way: OPEN → CLOSED → RECONCILED. A session cannot be re-opened.

9.11 Daily workflow summary

1. **Open** a session on your till with the day's opening float.
2. **Ring sales** as customers arrive.
3. **Record payouts** for any cash removed from the drawer.
4. Check the **X-Read** at any time for running totals.
5. At end of day, **count** the cash in the drawer.
6. **Close** the session by entering the counted amount.

7. A manager **reconciles** the closed session; the system posts any variance to the GL.

Example — Walk-in cash sale (full POS day):

Cashier Jane starts her shift at Duka Moja. She navigates to **Point of Sale › POS Sessions** (`/admin/pos/sessions`) and clicks **Open Session**. She picks till **Counter 1** (Branch: Dar es Salaam Main) and enters Opening Float **TZS 100,000**. Session **POS-0041** opens with status OPEN.

During the morning Jane processes three customers at **Point of Sale › Point of Sale** (`/admin/pos/sell`):

1. She picks her open session from the Session picker (shown as *"Session … (Till …)"* — the picker does not display the `POS-0041` number), customer **Mteja wa Kawaida**, agent **Omar Salim**, currency TZS. She adds: **Sukari 1kg** × 2 @ TZS 2,500 = TZS 5,000; **Mafuta ya Kupikia 1L** × 1 @ TZS 8,000 = TZS 8,000. Total TZS 13,000. Customer hands over TZS 20,000 — Change shown as TZS 7,000. Jane clicks **Complete Sale** — Invoice **INV-0211** issued.
2. Second sale: **Unga wa Ngano 2kg** × 3 @ TZS 3,200 = TZS 9,600. Tendered TZS 10,000, change TZS 400. Invoice INV-0212 issued.
3. Third sale: **Chumvi 500g** × 5 @ TZS 500 = TZS 2,500. Tendered exact. Invoice INV-0213 issued.

At midday Jane does a safe drop: she opens session detail (`/admin/pos/sessions/uid/{uid}`), clicks **Record Payout**, Type **Paid Out**, Amount **TZS 20,000**, Reason "Midday safe drop". Expected cash now: TZS 100,000 + TZS 25,100 − TZS 20,000 = **TZS 105,100**.

Jane checks the X-Read: Sales Total TZS 25,100, Payouts TZS 20,000, Expected Cash TZS 105,100, Invoice Count 3. Correct.

At end of day Jane counts the drawer: TZS 105,200 (TZS 100 over). She clicks **Close Session**, enters Counted Cash **TZS 105,200** — Variance is **+TZS 100.00** (over).

Manager Rehema opens the session detail, clicks **Reconcile**. Status → RECONCILED. Z-Read confirms the +TZS 100 variance and shows Journal **JNL-0519**: DR Cash 100 / CR Till Surplus (4900) 100.

10. Sales analysis reports

10.1 Sales Summary (sales by customer, agent, route, branch, day or cashier)

Navigate to **Sales > Sales Summary** (`/admin/reports/sales-summary`). Requires `SALES.INVOICE.VIEW` — the same permission as the Sales Report. In the search palette (Ctrl+K) it is also found as "sales by customer", "agent performance", "sales by route", "daily sales" or "cashier sales".

Pick a **From** and **To** date, a **Group by** option and, optionally, a **Branch**, then click **Run report**. Each row is one customer, sales agent, route, branch, day or cashier, showing: number of invoices, quantity (in base units, so a pack of 12 counts as 12), gross sales, discount, VAT, net sales, cost of sales, margin and margin %. A TOTAL row sums every group.

- Only **finalised** invoices count; drafts and voided invoices do not. Returns and credit notes are **not** deducted — the same as the Sales Report.
- **All amounts are in the company's base currency** (e.g. TZS). An invoice raised in another currency (e.g. USD) is converted at the exchange rate stamped on that invoice when it was finalised — the same rate its accounting entry used — so a USD 14.16 sale at 2,500 shows as TZS 35,400, not 14.16. Each line is converted and rounded on its own, so every grouping adds up to the same total as the Sales Report and the Profitability Report. Margin is the converted net less the cost of sales (which is always recorded in base currency). The export notes how many invoices were converted.
- **Cashier** means the user who created the invoice (at a till, the cashier who rang the sale). **Day** uses the company's time zone.
- Cost of sales is what the goods cost at the moment of sale. When an item was sold before its stock had ever been costed, its group's cost, margin and margin % show a dash (unknown — not zero), the group is left out of the Cost and Margin totals, and a yellow banner says how many groups and items are affected.
- Exports (PDF, Excel, CSV) additionally require `REPORT.EXPORT`.

10.2 Payment Summary (daily cash-up)

Navigate to **Sales > Payment Summary (Cash-up)** (`/admin/reports/payment-summary`). Requires `POS.CASHUP.VIEW` — a managers' permission (Sales Manager, Branch Manager, Accountant, Finance Director). Cashiers do not hold it: the till's X-read and Z-read show their own session, while this report shows every cashier's takings.

The screen opens on today. Optionally pick a date range, a **Branch** and a **Cashier** (the cashier list shows everyone who took a payment in the period). Each line is one day, cashier and currency, with columns for **Cash**, **Mobile money**, **Card**, **Cheque**, **Total** and the number of payments. There is one TOTAL row per currency — amounts in different currencies are never added together.

- Amounts are what the business kept: tendered amount less change given back.
- Payments are counted on the day they were taken, by the person who took them, for sales rung at a till or settled at the counter. A voided sale's payment is not counted.
- Not included: payments received later against credit invoices (Receivables receipts), till payouts and expenses, and the opening float. To balance one drawer, use the session's X-read / Z-read.
- Exports additionally require `REPORT.EXPORT`.

10.3 Restaurant Dashboard and Flash Report

Navigate to **Point of Sale > Restaurant Dashboard** (`/admin/restaurant/dashboard`). Requires `SALES.INVOICE.VIEW` — the same permission as the Sales Report. In the search palette (Ctrl+K) it is also found as "flash report", "daily summary" or "busy hours".

The screen opens on today. Pick a **From date** and **To date**, or tap **Today**, **Yesterday**, **Last 7 days** or **This month**; optionally choose a **Branch** (left empty, it covers every branch you may see), then click **Refresh**. A range can be at most one year.

- **Key figures** — Sales, Receipts, Average receipt, Table orders paid, Guests, Spend per guest, Discounts, Complimentary, Refunds, Receipts voided, Orders cancelled and Items voided. Each shows how it changed against the same number of days just before (for **Today**, against yesterday), marked green when the change is good and red when it is not — for discounts, refunds and voids, *down* is good. No change is shown when the earlier figure was zero.
- **Right now** — tables open, bills printed but not paid, and the money waiting to be paid. This part always shows the present moment, whatever dates are chosen.
- **Sales by day** (for a range of two days or more), **Busy hours**, **Days of the week** (the average per day, so a month with five Mondays compares fairly), **Departments** (the product's category), **Best sellers**, **Waiters** and **How bills were paid**.
- **Complimentary** — every bill settled as complimentary, with its receipt, time and who it was given to.

Links under the cards open the matching report in **Restaurant Reports** (10.4).

Flash Report. Flash report PDF and **Flash report Excel** download the dashboard as a one-page daily summary: every key figure next to the earlier period and the change, then right now, payments, departments, best sellers, waiters and complimentary bills. Downloading requires `REPORT.EXPORT` as well.

How the figures are counted:

- **Sales** are finalised receipts at their full amount including VAT, after discounts, in the company's base currency, counted at the time they were paid. A receipt voided afterwards is left out of sales and counted under **Receipts voided**.
- **Table orders**, **Guests** and **Waiters** come from restaurant orders paid in the period. Sales rung up without a table order appear in the waiter list as **Counter sales (no table order)**, so the waiter list adds up to total sales.
- **Spend per guest** is what the tables that gave a guest count spent, divided by those guests.
- **Discounts** are everything taken off the bills, including VAT. **Complimentary** is what was settled with the Complimentary payment type. **Refunds** are cash paid back at a till as a refund payout.
- The waiter is whoever holds the table: whoever opened it, or the waiter it was last moved to.
- Not shown: staff (labour) cost, meal periods (breakfast, lunch, dinner) and menu profitability.

10.4 Restaurant Reports

Navigate to **Point of Sale > Restaurant Reports** (`/admin/restaurant/reports`). Requires `SALES.INVOICE.VIEW`; exports (PDF, Excel, CSV) additionally require `REPORT.EXPORT`. Choose a report along the top, set the filters, then click **Run report**. Filters: **From date** and **To date**; **Waiter** (shown to users who also manage restaurant orders or setup); **Branch** (optional — empty means every branch you may see).

The first three reports have a **Summary** / **Detail** switch:

Report	Summary	Detail
Sales by waiter	Per waiter: bills, guests, average bill and amount	The items each waiter sold — item, unit, quantity and amount — with a subtotal per waiter
Bills paid	Per waiter: bills, guests and the amount split by payment type (Cash, Mobile money, Card, Cheque, Complimentary)	Every bill: when paid, order, table, waiter, cashier, receipt, how it was paid and amount
Pending bills	Per waiter: orders open, bills printed, items, the amount on the printed bills and the oldest order	Every order not yet paid, with its status and items

Pending bills has no dates: it shows what is unpaid now. An open order has no amount until its bill is printed. In **Bills paid**, a voided receipt is listed in the detail (marked *Void*) but left out of every total.

Sales by hour & day — sales for each hour of the day, then for each day of the week, each split by department, with the number of receipts and the average per day. Use it to see the busy hours and the busy days when planning staff and offers.

Payments & completion — for the period:

- **Payments**: each payment type, the number of payments, the amount and its share of the total.
- **Order completion**: of the orders opened in the period, how many were paid, cancelled or are still waiting, and the completion rate (paid ÷ opened).
- **Refunds** (cash paid back at a till), **Voided receipts**, **Cancelled orders** and **Voided items**, each with when, who and the reason given.

Sales by hour & day and *Payments & completion* have no waiter filter. Amounts in every restaurant report are in the company's base currency.

Procurement (Procure-to-Pay)

This chapter covers the full procure-to-pay (P2P) chain from raising a purchase request through to settling the supplier's invoice, including goods receipt, landed costs, purchase returns, and purchase settings.

Overview

What the P2P chain is and why it exists. Every business that buys goods or services needs a structured buying process. Without it, anyone could commit the business to purchases without authorisation, prices would go unverified, goods might be received without a matching order, and the business would have no audit trail when a supplier dispute arose. The Procure-to-Pay chain is the end-to-end control framework for buying: it starts with an internal request, works through supplier selection, raises a formal commitment to buy, records what actually arrived, validates the supplier's invoice against what was ordered and received, and ends with a payment that clears the liability. Each step is a gate — the next step cannot start until the previous one is completed and, where required, approved. This is how the system enforces budget control, prevents fraud, and supports accurate financial reporting.

The P2P chain follows this path:

```
Purchase Requisition → RFQ → Supplier Quotes → Award → Purchase Order
    → Goods Receipt → [Landed Cost] → Supplier Bill → 3-Way Match → AP Payment
```

For direct purchases without a sourcing process, the chain can start at the Purchase Order (created from a converted requisition or from the RFQ award).

Required permissions — the navigation menu only shows items your role includes:

Activity	Permission codes
Requisitions	`PURCHASE.REQUISITION.VIEW`, `PURCHASE.REQUISITION.CREATE`, `PURCHASE.REQUISITION.APPROVE`
RFQ / Supplier Quotes	`PURCHASE.RFQ.VIEW`, `PURCHASE.RFQ.MANAGE`
Purchase Orders	`PURCHASE.ORDER.VIEW`, `PURCHASE.ORDER.CREATE`, `PURCHASE.ORDER.VOID`, `PURCHASE.ORDER.APPROVE`
Goods Receipt	`PURCHASE.GOODS_RECEIPT.VIEW`, `PURCHASE.RECEIVE`
Landed Cost	`PURCHASE.LANDEDCOST.VIEW`, `PURCHASE.LANDEDCOST.MANAGE`
Supplier Bills / AP	`AP.VIEW`, `AP.BILL.ENTER`, `AP.BILL.MATCH`
Purchase Returns	`PURCHASE.RETURN.VIEW`, `PURCHASE.RETURN.CREATE`
Purchase Settings	`PURCHASE.SETTINGS.MANAGE`

Contact your administrator if an expected menu item is missing.

1. Purchase Requisitions

Navigate to **Purchasing › Purchase Requisitions** (`/admin/purchase-requisitions`).

The list shows every requisition with its number, status, required-by date, cost centre, line count, and creation date. Use **+ New Requisition** to raise one, and the **Open** button on a row to view or act on it.

What a purchase requisition is. A purchase requisition (also called a "purchase request" or PR) is a formal internal document raised by a member of staff to request that the business buys goods or services. It is not sent to a supplier — it is an internal request that must be reviewed and approved before any external commitment is made. Think of it as a "permission to buy" request.

Why it exists. Without a requisition step, any employee could initiate a purchase directly, bypassing budget checks, management oversight, and cost-centre accountability. The requisition creates a written record of what is needed, when it is needed, and at what estimated cost. This allows management to prioritise spending, check that the purchase fits the budget, and maintain an audit trail from the first idea to the final payment.

When it is used. A requisition is raised whenever a department or individual needs to buy something and does not have pre-authorised standing orders in place. Common triggers are low stock (detected by the Low Stock flag in Inventory), a project requirement, or routine scheduled re-ordering. The person raising the requisition is typically a storekeeper, department head, or anyone with the `PURCHASE.REQUISITION.CREATE` permission.

How it flows. A requisition starts as a DRAFT (being prepared) and must be submitted before it enters the approval queue (SUBMITTED). An authorised approver then approves or rejects it. An APPROVED requisition can be converted — either directly into a Purchase Order if the supplier is already known, or into an RFQ if prices need to be gathered from multiple suppliers first. Once converted, the requisition status becomes CONVERTED and no further action is possible on it; the work continues on the PO or RFQ that was created.

1.1 Create a requisition

1. Navigate to **Purchasing › Purchase Requisitions** (`/admin/purchase-requisitions`).
2. Click **New Requisition**, or go directly to `/admin/purchase-requisitions/create`.
3. Set the **Required By** date and optionally a cost centre and notes.
4. Add lines: for each item, pick the **Product** by name, choose a **Unit**, and enter the **Requested Quantity** and an **Estimated Unit Cost**. The **Unit** field is disabled until a product is picked; once picked, it lists only that product's configured units (its base unit and any active bulk-pack units) — not every unit in the system.
5. Click **Create Requisition**. The requisition is saved in **DRAFT**.

1.2 Submit a requisition

When the requisition is complete and ready for approval:

1. Open the draft requisition (navigate to `/admin/purchase-requisitions/uid/{uid}`).
2. Click **Submit for Approval**.
3. The status changes to **SUBMITTED** and the requisition is routed for approval.

1.3 Approve or reject a requisition

An approver (a user with `PURCHASE.REQUISITION.APPROVE`) reviews submitted requisitions at **Purchasing › Purchase Requisitions** (`/admin/purchase-requisitions`).

- **Approve** — open the submitted requisition and click **Approve**. Status → **APPROVED**. The Convert action becomes available.
- **Reject** — click **Reject**, enter a mandatory reason, and confirm. Status → **REJECTED**. The requisitioner is notified via the audit trail.

1.4 Convert a requisition

Converting an approved requisition creates the target document — a Purchase Order or an RFQ — in one step, carrying over the requisition lines. You choose the target type **and** the supplier(s) up front; the requisition then locks to CONVERTED and a link opens the document that was created.

1. Open the approved requisition (navigate to `/admin/purchase-requisitions/uid/{uid}`).
2. Click **Convert**. An inline **Convert Requisition** form opens.
3. In **Convert to**, choose the target type. The rest of the form changes to suit it:
    - **Purchase Order** — pick a single **Supplier** (required; only active suppliers of the requisition's company are listed) and optionally a **Currency** (a 3-letter code such as `USD` — leave blank to use the company default). A DRAFT PO is created at this supplier for the requisition lines.
    - **Request for Quotation (RFQ)** — build the invite list: choose a supplier in **Add a supplier** and click the **+** button to add it, repeating for each supplier you want to quote (remove one with the ✕ on its row). At least one supplier is required. A DRAFT RFQ is created inviting those suppliers.
4. Click **Confirm Convert**. This button stays disabled until you have chosen the required supplier(s) — a PO needs one supplier, an RFQ needs at least one invitee.
5. The requisition status changes to **CONVERTED**. A **Converted to …** line appears in the summary card with a **View RFQ** / **View Purchase Order** link that opens the document that was just created — continue there (section 2 for an RFQ, section 3 for a PO).

1.5 Cancel a requisition

A requisition can be cancelled while it is still DRAFT or SUBMITTED:

1. Open the requisition.
2. Click **Cancel**, enter an optional reason, and confirm. Status → **CANCELLED**.

1.6 Requisition status reference

Status	Meaning
DRAFT	Being prepared
SUBMITTED	Awaiting approval
APPROVED	Approved; ready to convert
REJECTED	Rejected by approver
CONVERTED	Converted to PO or RFQ
CANCELLED	Cancelled

Example — Requisition for office stationery:

Store clerk Amani opens **Purchasing › Purchase Requisitions** (`/admin/purchase-requisitions`) and clicks **New Requisition**. He sets Required By **2026-06-20**, notes "Monthly stationery re-order", and adds two lines:

- Product **Karatasi A4 (Ream)**, Unit **REAM**, Qty **20**, Estimated Cost **TZS 8,500** each.
- Product **Kalamu Nyeusi**, Unit **BOX**, Qty **5**, Estimated Cost **TZS 3,200** each.

He clicks **Create Requisition** — requisition **REQ-0072** is created in DRAFT. He opens it and clicks **Submit for Approval** — status → SUBMITTED.

Purchasing manager Neema opens the requisition and clicks **Approve** — status → APPROVED, estimated total TZS 186,000. She clicks **Convert**, sets **Convert to** to **RFQ**, adds **Ofisi Supplies Ltd** and **Karatasi Traders** to the invite list, and clicks **Confirm Convert** — RFQ **RFQ-0031** is created in DRAFT and the **View RFQ** link opens it.

2. RFQ (Request for Quotation)

Navigate to **Purchasing › RFQs / Sourcing** (`/admin/rfqs`).

The list shows each RFQ with its number, status, how many suppliers were invited, the response-due date, and the creation date. Use **+ New RFQ** to start one, and **Open** to send it, capture quotes, or award it.

What an RFQ is. An RFQ (Request for Quotation) is a document sent to one or more suppliers asking them to submit their prices and delivery terms for a specified list of goods or services. It is not a commitment to buy — it is a competitive enquiry. The business collects the responses (supplier quotes), compares them, and chooses the best offer.

Why it exists. Without a sourcing step, the business might always buy from the same supplier at whatever price they name, with no mechanism to check whether better value is available elsewhere. An RFQ enforces competitive sourcing: multiple suppliers are asked the same question at the same time, their responses are recorded in the system, and the selection is documented — protecting the business from claims of favouritism and ensuring value for money.

When it is used. An RFQ is used when the buying price is not already fixed by contract or catalogue and at least one competitive comparison is warranted. It is typically triggered by an approved purchase requisition (the Convert → RFQ path) or raised directly by a purchasing officer when restocking at scale. The person creating and sending the RFQ, capturing quotes, and awarding it

holds the `PURCHASE.RFQ.MANAGE` permission (the same permission covers all three actions); viewing an RFQ requires `PURCHASE.RFQ.VIEW`.

How it flows. An RFQ is created in DRAFT with the product lines and the invited suppliers. When sent (SENT), suppliers are notified to respond. As each supplier responds with a price, a **Supplier Quote** is captured against the RFQ (QUOTES_RECEIVED). The purchasing officer then compares the quotes and awards the RFQ to the preferred supplier (AWARDED). Awarding automatically creates a Purchase Order in DRAFT at the winning quote's prices — the sourcing stage is complete and the buying stage begins.

2.1 Create an RFQ

An RFQ can be created directly or by converting an approved requisition (see section 1.4).

To create directly:

1. Navigate to **Purchasing › RFQs / Sourcing** (`/admin/rfqs`) and click **New RFQ**, or go to `/admin/rfqs/create`.
2. Set the **Response Due Date** and optionally add notes.
3. In the **Invite Suppliers** section, choose a supplier in the **Add a supplier** picker and click the **+** button to add it to the invite list. Repeat for each supplier; invite at least one. Each added supplier is shown by name and code — both in this list and later on the RFQ detail screen's **Invited Suppliers** panel — never as a raw reference number.
4. Add lines: pick each product by name, choose a unit, and enter the required quantity. The unit dropdown is disabled until a product is picked, and then lists only that product's configured units.
5. Click **Create RFQ**. The RFQ is created in **DRAFT**.

2.2 Send an RFQ to suppliers

1. Open the DRAFT RFQ (navigate to `/admin/rfqs/uid/{uid}`).
2. Click **Send to Suppliers**. Status → **SENT**. Suppliers are notified that they should submit a quote.

2.3 Capture supplier quotes

What a supplier quote is. A supplier quote (also called a quotation or bid) is the formal price response a supplier submits in reply to the RFQ. It states the price per unit, any lead time, and any validity period. The system captures these responses electronically so they can be compared side-by-side.

When a supplier responds with a price:

1. Open the SENT RFQ (navigate to `/admin/rfqs/uid/{uid}`).
2. Click **Capture Supplier Quote**. The capture form opens.
3. Pick the **Supplier** by name (only invited suppliers are listed).
4. Optionally set a valid-until date, lead time in days, and notes.
5. For each RFQ line, enter the **Quoted Quantity** and **Unit Price**.
6. Click **Save Quote**. The quote is recorded with status **RECEIVED** and the RFQ status moves to **QUOTES_RECEIVED**.

Repeat for each responding supplier. You can compare their prices side-by-side in the quotes panel on the RFQ detail page.

2.4 Award the RFQ

To select the winning supplier and create a Purchase Order:

1. In the quotes panel on the RFQ detail page (`/admin/rfqs/uid/{uid}`), identify the preferred quote (usually the lowest compliant price).
2. Click **Award** on that quote row.
3. The winning quote status changes to **AWARDED** and all other quotes become **NOT_AWARDED**. The RFQ status changes to **AWARDED**.
4. A **Purchase Order** is created in DRAFT from the awarded quote lines and prices. A link to the PO is shown.

2.5 Cancel an RFQ

Open the RFQ (navigate to `/admin/rfqs/uid/{uid}`) and click **Cancel RFQ**. Status → **CANCELLED**. An awarded RFQ cannot be cancelled.

2.6 RFQ status reference

Status	Meaning
DRAFT	Being prepared
SENT	Sent to suppliers; awaiting responses
QUOTES_RECEIVED	At least one supplier quote captured
AWARDED	Winning supplier selected; PO created
CANCELLED	Cancelled

Example — RFQ for cement (continuing from requisition example, fresh scenario):

A warehouse requisition for 500 bags of **Saruji 50kg** has been approved and converted to RFQ **RFQ-0031**. Purchasing officer Zawadi opens **Purchasing › RFQs / Sourcing** (`/admin/rfqs`), opens RFQ-0031, and adds two invited suppliers: **Tanzania Cement Distributors** and **Simba Cement Ltd**. Response Due Date is set to **2026-06-17**. She clicks **Send to Suppliers** — RFQ goes to SENT.

Both suppliers respond. Zawadi captures two quotes:

- **Tanzania Cement Distributors**: 500 bags @ TZS 14,800 each = TZS 7,400,000 (lead time 3 days).
- **Simba Cement Ltd**: 500 bags @ TZS 14,500 each = TZS 7,250,000 (lead time 5 days).

After review, Zawadi awards the RFQ to **Simba Cement Ltd** (cheaper price, acceptable lead time). Purchase Order **PO-0088** is created in DRAFT at TZS 14,500/bag.

3. Purchase Orders

Navigate to **Purchasing › Purchase Orders** (`/admin/purchase-orders`).

The list shows each PO with its order number, supplier, status, currency, total, and creation date. A search box and status filter narrow the list. Use **+ New Order** for a direct PO, and **Open** to add lines, place, close, or void one.

What a Purchase Order is. A Purchase Order (PO) is the formal, legally binding document that a business sends to a supplier to commit to buying specific goods or services at agreed prices and quantities. It defines what is being ordered, how many units, at what price, and by when. Once placed, it is the reference document for everything that follows — the goods receipt checks deliveries against it, the supplier invoice is matched against it, and the payment settles it.

Why companies use Purchase Orders. Without a PO, the business has no formal record of what it committed to buy. The supplier could deliver the wrong quantity or charge a different price, and there would be no agreed baseline to dispute it. POs provide commitment control (approvals before spending), a budget anchor (the ordered amount is known), an audit trail (who ordered what, when, at what price), and the document foundation for both the goods receipt (what was ordered versus what arrived) and the 3-way match (ordered, received, billed — all three must agree). They also protect the business legally: a supplier cannot claim an order was placed if no PO exists.

When a PO is raised. A PO is raised after a purchase has been authorised. There are three ways a PO can originate:

- By converting an approved requisition directly into a PO (see section 1.4).
- By awarding an RFQ, which creates the PO automatically at the winning supplier's quoted prices (see section 2.4).
- By creating one directly on the Purchase Orders list using the inline **New Order** form (see section 3.1), without a requisition or RFQ — useful for direct purchases where the supplier and prices are already known.

How a PO flows. A PO starts as a DRAFT (lines can be edited freely). If a PO approval threshold is enabled in Purchase Settings and this order's total is at or above the configured amount, the DRAFT must be submitted for approval and approved before it can be placed (section 3.3) — the system refuses to place an over-threshold PO that has not yet been approved. When the lines are finalised (and approved, if required), the PO is placed (ORDERED), which sends it to the supplier, locks the lines, and assigns the PO number. Goods arrive and are recorded against the PO via Goods Receipts — the PO tracks how many units remain outstanding and moves through PARTIALLY_RECEIVED to RECEIVED as deliveries arrive. Once fully received (or if the business accepts a shortfall), the PO can be closed (CLOSED). If the PO is no longer needed before all goods are received, it can be voided (VOID).

3.1 Create a Purchase Order directly

The Purchase Orders list (`/admin/purchase-orders`) has an inline create form for raising a PO without a requisition or RFQ (requires `PURCHASE.ORDER.CREATE`).

1. Navigate to **Purchasing › Purchase Orders** (`/admin/purchase-orders`).
2. Click **New Order**. The **New Purchase Order** form opens above the list.
3. Pick the **Supplier** (search by name or code; only active suppliers are listed).
4. Choose the **Currency** from the Currency Picker — the list is limited to the company's enabled currencies and defaults to the company default (see "Common UI Patterns" in the Getting Started chapter).
5. Optionally set an **Expected Date** and **Notes**.
6. Click **Create Order**. A DRAFT PO is created (with no lines yet) and a success notification appears. Open it to add lines (section 3.2).

3.2 View and manage a DRAFT Purchase Order

1. Navigate to **Purchasing › Purchase Orders** (`/admin/purchase-orders`).
2. Open the DRAFT PO (navigate to `/admin/purchase-orders/uid/{uid}`).
3. While the PO is in DRAFT you can:

    - **Add a line** — pick the product by name, choose a unit, enter the ordered quantity and unit cost. The **Unit** field is disabled until a product is picked; once picked, it lists only that product's configured units (its base unit and any active bulk-pack units).
    - **Remove a line** — click the delete icon on the line row. (Lines cannot be edited in place; to change a line, remove it and add it again.)

3.3 Submit a Purchase Order for approval

If your administrator has enabled a PO approval threshold in Purchase Settings (section 8) and this order's total is at or above the configured amount, it must clear approval before it can be placed. Once submitted, the order shows an **Awaiting approval** / **Approved** / **Approval rejected** status tag next to its status.

1. Open the DRAFT PO (navigate to `/admin/purchase-orders/uid/{uid}`) — it must have at least one line.
2. Click **Submit for Approval**. This button appears only when the order's total requires approval and it has not already been submitted or approved.
3. An **Awaiting approval** banner appears, with a link to **Go to Approvals inbox**, and the **Place Order** button is removed from the screen. The order is routed to an approver as an approval request (see chapter 11, Approvals).

Requires `PURCHASE.ORDER.CREATE` (the same permission used to create, add lines to, and place a PO).

After a decision is made. When the approver approves or rejects the request in the Approvals inbox (chapter 11), **reopen or refresh the Purchase Order** to pick up the outcome: the banner updates to **Approved** — and **Place Order** reappears so you can place the order — or to **Approval rejected**. The screen reconciles the decision from the approvals engine every time it is opened, so a manual refresh after the approver acts is all that is needed (there is no live auto-refresh yet). An administrator may also record the decision directly via the `PURCHASE.ORDER.APPROVE` action. Orders whose total is *below* the approval threshold never enter this flow and place normally.

3.4 Place a Purchase Order

Placing the PO sends it to the supplier and locks the lines. If the order requires approval (section 3.3) but has not yet been submitted, **Place Order** is shown but disabled (with a *Submit for approval before placing* tooltip). While an order is **Awaiting approval** the control is hidden; once it has been **Approved** — reopen or refresh the order to pick up the decision (section 3.3) — **Place Order** reappears and the order can be placed.

1. Open the DRAFT PO (it must have at least one line, and be approved if approval is required).
2. Click **Place Order**.
3. Status → **ORDERED** and a PO number (PO-####) is assigned.

3.5 Close a Purchase Order

Closing finalises the PO without receiving all goods (for example, if a partial shipment is accepted as complete).

1. Open the PO (navigate to `/admin/purchase-orders/uid/{uid}`) — status ORDERED, PARTIALLY_RECEIVED, or RECEIVED.
2. Click **Close Order**.
3. Status → **CLOSED**. The PO is read-only.

3.6 Void a Purchase Order

Voiding cancels the PO if goods have not all been received.

1. Open the PO (status DRAFT, ORDERED, or PARTIALLY_RECEIVED).
2. Click **Void Order**. An inline reason form opens — enter a mandatory reason and click **Confirm Void**.
3. Status → **VOID**.

3.7 PO status reference

Status	Meaning
DRAFT	Being prepared; lines editable
ORDERED	Placed; sent to supplier; lines locked
PARTIALLY_RECEIVED	Some lines received; outstanding qty remains
RECEIVED	All lines fully received
CLOSED	Manually closed
VOID	Cancelled before full receipt

Example — Placing the cement PO:

Zawadi opens **Purchasing › Purchase Orders** (`/admin/purchase-orders`), finds PO-0088 (DRAFT, 500 bags @ TZS 14,500), reviews the line, and clicks **Place Order**. Status → ORDERED. The formal PO number is confirmed and the document is locked for editing. A PDF can be generated and sent to Simba Cement Ltd.

4. Goods Receipt

Navigate to **Purchasing › Goods Receipts** (`/admin/goods-receipts`).

The list shows each receipt with its GRN number, status, a **View PO** link to the originating Purchase Order, when it was received, and any notes. Use **+ New Receipt** to record an arrival, and **Open** to view a receipt.

What a Goods Receipt is. A Goods Receipt (GR), sometimes called a Goods Received Note (GRN), is the document that records the physical arrival of goods from a supplier. It is raised by the storekeeper or receiving officer at the moment goods are checked in, linking the delivery to the Purchase Order that authorised it. The GR is the point at which inventory increases: the quantities received are added to stock on-hand at the branch.

Why it exists. A Goods Receipt serves three critical purposes. First, it records what actually arrived — not what was ordered, not what was billed, but what the storekeeper physically counted and accepted. Second, it updates the stock ledger immediately so the business knows what it holds (an important distinction: ordering goods does not increase stock; receiving them does). Third, it forms the third document in the 3-way match: the supplier's invoice can only be paid once the system confirms that the goods billed were both ordered (PO) and received (GR). Without a GR, the business could pay for goods it never received.

When it is used. A GR is created by the storekeeper or receiving officer each time a supplier delivers goods against an outstanding Purchase Order. If a supplier delivers in multiple shipments, a separate GR is created for each delivery. The permission required is `PURCHASE.RECEIVE`. Only placed Purchase Orders (ORDERED or PARTIALLY_RECEIVED) can have a GR raised against them.

How it flows. The storekeeper picks the PO and the system shows all outstanding (unreceived) lines pre-filled with the remaining quantities. The storekeeper adjusts the quantities if the delivery is partial (and unchecks any lines not included in this delivery), optionally records a lot/batch number, manufacture date, expiry date, or serial numbers per line, adds notes, and records the receipt. The GR is created with status RECEIVED, a GRN number is assigned, stock increases at the branch, and the PO's outstanding quantities are updated. Any batch or serial details captured at receipt feed the read-only Stock Batches and Stock Serials screens (see the Inventory & Manufacturing chapter, sections 6–7). The PO moves to PARTIALLY_RECEIVED or RECEIVED depending on whether all lines are now complete. A GR cannot be edited after submission; errors are corrected by voiding the GR (section 4.3) or by raising a Purchase Return (section 7).

4.1 Receive goods

1. Navigate to **Purchasing › Goods Receipts** (`/admin/goods-receipts`) and click **New Receipt**, or go directly to `/admin/goods-receipts/create`.
2. Pick the **Purchase Order** by its PO number.
3. The form lists all open (unreceived) lines, each with a tick box (included by default) and the outstanding quantity pre-filled in the **Receive Qty** field.

4. Adjust individual quantities if you are receiving a **partial shipment**, and untick any lines not in this delivery. By default the quantity cannot exceed the outstanding balance on each line — unless your company has set an **over-receipt tolerance** (see below), in which case each line may go up to that percent over, and the field shows a **max** hint of the amount allowed.
5. For any line — typically a lot-tracked or serialised product — click **Batch** to expand its batch/serial details, and optionally enter the **Lot / Batch number**, **Manufacture date**, **Expiry date**, and **Serial / IMEI numbers** (one per line). The **Batch** toggle appears on every receipt line regardless of the product's tracking settings; all of these fields are optional at receipt time.
6. Optionally add **Notes**.
7. Click **Record Receipt**.

The goods receipt is created with status **RECEIVED** and assigned a GRN-#### number. Stock is added to the branch. The PO status updates:

- Partial receipt → PO status **PARTIALLY_RECEIVED**
- Full receipt → PO status **RECEIVED**

Over-receipt tolerance (receiving slightly more than ordered). By default the system is strict: you cannot receive more than a line's outstanding quantity, and a line entered above it is rejected. But commodities delivered by weight — rice, produce, cement — rarely match the order to the exact bag or kilogram, and a delivery can legitimately arrive a little over. A company can therefore set an **over-receipt tolerance** (a percent) in Purchase Settings (section 8) so a receipt may exceed the outstanding quantity by up to that margin. When it is set:

- An information banner at the top of the receipt form reads *"Over-receipt tolerance: 5% — each line may be received up to 5% over its outstanding quantity."*
- Each **Receive Qty** field shows a **max** hint of the ceiling for that line (outstanding × (1 + tolerance), for example `max 105` on a line of 100 outstanding at 5%).
- Entering more than the ceiling is refused with a plain message naming the line (product) and stating that the quantity received exceeds what is outstanding — reduce it and try again; nothing is posted. (The allowed maximum is shown up front as the **max** hint on the field, not repeated in the rejection message.)

When no tolerance is configured (the default), receiving stays strict and the banner does not appear. See section 8.2 for how to set it.

4.2 Partial receipts (multiple deliveries)

If the supplier delivers in stages, create a separate goods receipt for each delivery. Each GRN records the quantity received on that date. The PO tracks the cumulative received and outstanding quantities across all GRNs.

4.3 Void a goods receipt

If a receipt was recorded in error, void it to fully reverse it. Voiding is available from the goods receipt detail page and requires the `PURCHASE.VOID` permission.

1. Open the receipt (navigate to `/admin/goods-receipts/uid/{uid}`) — it must be in status RECEIVED.
2. Click **Void Receipt**. An inline form opens.
3. Enter a mandatory **Reason** and click **Confirm Void**.
4. The receipt status changes to **VOID** and the void reason is shown on the receipt.

What the void reverses. Voiding is a complete unwind of the receipt, not just a stock adjustment:

- **Quantity** — the received quantity is removed from stock on-hand at the branch, and the PO's outstanding quantities are restored so the lines can be received again.
- **General ledger** — the goods-received accounting entry is reversed.
- **Batch (lot) and serial/IMEI tracking** — any lot/batch quantities and serial or IMEI numbers that were captured on the original receipt (see section 4.1, step 5) are backed out too: batch balances are decremented and the serials recorded by that receipt are removed from stock. This keeps the Stock Batches and Stock Serials screens (Inventory & Manufacturing chapter, sections 6–7) consistent with the reversal.

A serial that has already moved on (for example, one that has since been issued) is left untouched; the rest of the reversal still completes.

4.4 Goods receipt status reference

Status	Meaning
RECEIVED	Active receipt; stock increased
VOID	Voided (reversed); stock, GL, and batch/serial tracking all backed out

Example — Receiving cement:

Simba Cement delivers 500 bags on 2026-06-22. Storekeeper John opens **Purchasing › Goods Receipts** (`/admin/goods-receipts`), clicks **New Receipt**, and picks PO **PO-0088**. The form shows 500 bags Saruji 50kg outstanding. John keeps all 500 bags and clicks **Record Receipt** — GRN **GRN-0061** is created (status RECEIVED), 500 bags added to stock at the branch, PO-0088 status → RECEIVED.

Partial receipt scenario: If Simba had delivered only 300 bags on day 1, John would receive 300 bags (GRN-0061), PO → PARTIALLY_RECEIVED, outstanding = 200 bags. When the remaining 200 arrive, John creates GRN-0062 for 200 bags, PO → RECEIVED.

5. Landed Costs

Navigate to **Purchasing › Landed Costs** (`/admin/landed-costs`).

The list shows each landed-cost document with its LC number, status, allocation basis (By Value or By Quantity), currency, total charge, and creation date. Use **+ New Landed Cost** to create one, and **Open** to review and confirm it.

What landed costs are. Landed cost is the total cost of getting an imported or shipped product to your warehouse — not just the purchase price, but all the additional charges incurred along the way: freight, customs duty, port clearing fees, insurance, and other incidentals. The "landed cost" is what the goods actually cost you once they are physically in your possession.

Why they are captured. If only the purchase price is recorded as the inventory cost, the business undervalues its stock and understates the true cost of goods sold (COGS). For example, cement bought at TZS 14,500/bag but with TZS 2,900/bag in freight and clearing costs actually costs TZS 17,400/bag to hold. Selling it at any price below TZS 17,400 is a loss — but a business recording only TZS 14,500 would not see that loss until the end of the period. Capitalising landed costs into inventory value ensures the stock is valued at its true cost, the cost-of-goods-sold figure is accurate, and the balance sheet reflects the real investment in inventory.

When it is used. A landed cost is entered after the goods have been received (a GRN exists) and the incidental charges are known — either at the time of receipt or when the freight/clearing invoice arrives. The accountant or purchasing officer enters the charges against the relevant GRN(s) and confirms the document. The permission required is `PURCHASE.LANDEDCOST.MANAGE` (covers both creating and confirming); viewing requires `PURCHASE.LANDEDCOST.VIEW`.

How it flows. A landed cost document is created (DRAFT) with the allocation basis (By Value or By Quantity), linked to one or more GRNs, and the charge lines (Freight, Duty, Clearing, Insurance, Other) are entered. On confirmation (CONFIRMED), the system allocates each charge proportionally to the GR lines and capitalises the allocated amount into the inventory value of each product — raising the moving-average cost and posting the GL entry. The accounting entry at confirmation is: **DR Inventory (1300) / CR Landed Cost Clearing (2160)**. When the freight or duty invoice later arrives from the supplier and is bill-matched, the clearing account is debited back: **DR Landed Cost Clearing / CR Accounts Payable** — leaving a zero balance in the clearing account. A confirmed landed cost is immutable.

5.1 Create a landed cost

1. Navigate to **Purchasing › Landed Costs** (`/admin/landed-costs`) and click **New Landed Cost**, or go directly to `/admin/landed-costs/create`.
2. Select the **Allocation Basis**:
    - **By Value** — charges are spread proportionally to the value of each GR line.
    - **By Quantity** — charges are spread proportionally to the quantity received on each GR line.
3. Pick the **Goods Receipt(s)** by GRN number. You can include multiple GRNs in one landed cost document.
4. Add one or more **Charges**: select the charge type (Freight, Duty, Clearing, Insurance, or Other) and enter the amount.
5. Click **Create Landed Cost**. The landed cost is created in **DRAFT**.

5.2 Confirm a landed cost

Confirming allocates the charges to the GR lines and posts the cost adjustment to the GL.

1. Open the DRAFT landed cost (navigate to `/admin/landed-costs/uid/{uid}`).
2. Click **Confirm & Allocate**.
3. Status → **CONFIRMED**. The allocation per GR line is shown in the detail.

A confirmed landed cost cannot be edited. If there is an error, contact your administrator.

5.3 Landed cost status reference

Status	Meaning
DRAFT	Being prepared; charges editable
CONFIRMED	Charges allocated; GL posted; immutable

Example — Landed cost for imported cement:

The cement shipment also incurred TZS 850,000 in port clearing fees and TZS 600,000 in freight. Accountant Sarah opens **Purchasing › Landed Costs** (`/admin/landed-costs`), clicks **New Landed Cost**, selects Allocation Basis **By Quantity**, and picks GRN **GRN-0061** (500 bags). She adds two charges:

- Type **Clearing**, Amount TZS 850,000.
- Type **Freight**, Amount TZS 600,000.

Total landed cost TZS 1,450,000. She clicks **Create Landed Cost** (DRAFT), reviews the per-bag allocation (TZS 2,900/bag), and clicks **Confirm & Allocate**. Status → CONFIRMED. The moving-average cost for Saruji 50kg increases by TZS 2,900/bag, and the GL is posted accordingly.

6. Supplier Bills and 3-Way Bill Match

Navigate to **Accounting › Payables** (`/admin/ap/supplier-bills`).

What a supplier bill is. A supplier bill (also called a purchase invoice or vendor invoice) is the invoice the supplier sends requesting payment for goods delivered or services rendered. It is the supplier's claim against the business. In the system, it is entered as a formal financial document that creates an accounts payable liability — the business now owes the supplier money.

Why a supplier bill must be matched before payment. A supplier could, accidentally or deliberately, send an invoice for more units than were delivered, at a higher price than agreed, or for items never ordered at all. Paying it without verification means the business overpays. The 3-way match is the systematic check that prevents this: it compares the bill to both the Purchase Order (what was

agreed) and the Goods Receipt (what was actually received). All three must align within an acceptable tolerance before payment is authorised. This process is called "3-way matching" because it matches three documents: the bill, the PO, and the GR.

When it is used. A supplier bill is entered when the supplier's invoice arrives, after the goods have been received and (optionally) landed costs applied. It is entered by an accounts payable clerk with the `AP.BILL.ENTER` permission. Matching is triggered automatically when the bill is entered (if a PO is linked) or can be run from the bills list.

How the 3-way match works. The system compares each bill line against the corresponding PO line (agreed price and quantity) and GR line (received quantity). If the billed price and billed quantity are within the configured tolerance of the ordered and received values, the line is MATCHED. If either is outside tolerance, the line is HELD (HELD_PRICE_VARIANCE or HELD_QTY_VARIANCE). A held bill cannot be approved for payment until each held line is either corrected or manually accepted (VARIANCE_ACCEPTED) by a user with `AP.BILL.MATCH`. The GL entry posted at bill-match for goods lines is: **DR GRNI (2150) / CR Accounts Payable (2100)** — this clears the GRNI bridge set up at the goods receipt. Service bill lines (no GR) post: **DR Purchases (5150) / CR Accounts Payable**.

6.1 Enter a supplier bill

1. Navigate to **Accounting › Enter Bill** (`/admin/ap/supplier-bills/enter`).
2. Pick the **Supplier** by name or code (search-as-you-type).
3. Enter the supplier's own **Supplier Invoice No.**, **Bill Date**, and **Due Date**.
4. Optionally enter the **VAT Amount** (leave 0 if none).
5. Choose the **Currency** from the Currency Picker — the list is limited to the company's enabled currencies and defaults to the company default (see "Common UI Patterns" in the Getting Started chapter).
6. Optionally pick the **Purchase Order** from the picker. Linking the PO enables the 3-way match (see section 6.2). For service bills with no PO, leave this blank.
7. Add **Bill Lines** with **Add Line**. Each line is a free-text **Description**, a **Billed Qty**, and a **Unit Cost**; the **Line Net** is computed for you. To match a line to the order, choose the corresponding **PO Line** in the optional picker on that row. (There is no product picker on the bill line — the description is free text.)
8. Click **Enter Bill & Match**. The bill is created and the 3-way match runs automatically; the per-line match result panel appears.

6.2 Understanding the 3-way match

The 3-way match compares three documents for each bill line:

```
Supplier Bill line  ↔  Purchase Order line  ↔  Goods Receipt line
    (billed qty/price)       (ordered qty/price)      (received qty/price)
```

The match result for each line is one of:

Match Status	Meaning
MATCHED	Quantities and prices are within tolerance; line approved

HELD_PRICE_VARIANCE	Bill unit price is outside the acceptable tolerance versus the PO price
HELD_QTY_VARIANCE	Billed quantity is outside the acceptable tolerance versus the received quantity
VARIANCE_ACCEPTED	A held variance was reviewed and manually accepted

The overall bill status depends on its lines:

Bill Status	Meaning
MATCHED	All lines matched; bill ready for payment approval
HELD	One or more lines have an unresolved variance

6.3 Accept a variance

If a bill line is HELD due to a price or quantity variance:

1. On the per-line match result panel shown immediately after **Enter Bill & Match** (section 6.1), review the variance amount and percentage shown on each held line.
2. If the variance is acceptable, click **Accept Variance** on that line.
3. When all held lines are resolved, the bill status moves to **MATCHED**.

Accepting variances requires the `AP.BILL.MATCH` permission. The **Accept Variance** control appears only on the match-result panel that follows entering the bill — the read-only bill detail page does not offer it.

6.4 Reviewing a matched or held bill

The 3-way match runs once, automatically, when the bill is entered (see section 6.1); there is no UI to re-run a match after entry. On the bills list at **Accounting › Payables** (`/admin/ap/supplier-bills`), a **Match** action appears for DRAFT and HELD bills — it is a link that opens the read-only bill detail page (`/admin/ap/supplier-bills/uid/{uid}`) where you can review the bill header and lines. Variances are accepted on the match-result panel at entry time (section 6.3), not from this page.

6.5 Service bills (no PO)

For invoices from service suppliers where there is no corresponding PO or GR:

- Leave the Purchase Order field blank when entering the bill.
- No 3-way match is run.
- The bill is entered for manual review and approval.

6.6 Bill status reference

Status	Meaning
DRAFT	Entered but not yet matched
MATCHED	All lines matched; ready for payment

HELD	One or more lines have an open variance
APPROVED	Approved for payment by the AP team
PARTIALLY_PAID	Payment partially applied
PAID	Fully paid

6.7 Record an AP payment

What an AP payment is. An Accounts Payable (AP) payment is the settlement of a supplier bill — the act of transferring funds to the supplier to clear the liability created when the bill was entered. Recording the payment in the system updates the bill status and reduces the AP balance, completing the P2P cycle.

Payments against supplier bills are managed in the Accounts Payable module. Navigate to **Accounting › Record Payment** (`/admin/ap/payments/record`) to record a payment. See the Finance chapter for details on recording and reconciling AP payments.

Example — Supplier bill for Simba Cement (clean 3-way match):

Simba Cement sends Invoice **SIM/2026/1041**, Bill Date 2026-06-22, Due Date 2026-07-22, for 500 bags @ TZS 14,500 each = TZS 7,250,000 net.

Sarah opens **Accounting › Enter Bill** (`/admin/ap/supplier-bills/enter`), picks supplier **Simba Cement Ltd**, enters Supplier Invoice No **SIM/2026/1041**, Bill Date 2026-06-22, Due Date 2026-07-22, and selects Currency **TZS** from the Currency Picker. She links PO **PO-0088** and adds one bill line: Description **Saruji 50kg**, Billed Qty 500, Unit Cost TZS 14,500, and selects the matching **PO Line** on that row. She clicks **Enter Bill & Match**.

The system runs the 3-way match:

- Bill line: 500 bags @ 14,500
- PO line: 500 bags @ 14,500 ✓
- GRN line: 500 bags received ✓

All lines → **MATCHED**. Bill status → MATCHED. Bill **BILL-0051** is ready for payment.

Example — Bill with price variance (held):

A different shipment arrives and the supplier bills at TZS 14,900/bag (TZS 400 over the PO price). When the AP manager clicks **Enter Bill & Match**, the match-result panel shows the bill line as **HELD_PRICE_VARIANCE** with variance TZS 200,000. She reviews the variance, decides it is within business tolerance, and clicks **Accept Variance** on that line. Line moves to VARIANCE_ACCEPTED; bill → MATCHED.

7. Purchase Returns

Navigate to **Purchasing › Purchase Returns** (`/admin/purchase-returns`).

The list shows each return with its number, supplier, status, currency, gross value, and creation date. Use **+ New Return** to raise one against a goods receipt, and **Open** to review and confirm it.

What a purchase return is. A purchase return is the formal process of sending goods back to the supplier — typically because the goods arrived damaged, were incorrect, failed quality inspection, or are surplus to requirements. It is the reverse of a goods receipt: where a GR increases stock, a confirmed purchase return decreases stock and triggers the AP module to raise a debit note against the supplier.

Why it exists. Without a formal return process, the business would need to adjust stock manually (which lacks a clear link to the supplier transaction) and would have no systematic way to claim money back from the supplier. A purchase return document creates an auditable record of what was returned, why, and at what value — forming the basis for the AP debit note that reduces the amount owed to the supplier. It also keeps inventory accurate: goods sent back should not remain in the stock count.

When it is used. A purchase return is raised after a goods receipt has been confirmed (RECEIVED) and the goods in question have been identified for return — for example, after inspection reveals damage, or after a quality failure is reported. The storekeeper or purchasing manager raises the return against the specific GRN, and a purchasing manager or authorised user confirms it. The permission required is `PURCHASE.RETURN.CREATE` (it covers both creating and confirming a return); viewing requires `PURCHASE.RETURN.VIEW`.

How it flows. A purchase return starts as a DRAFT referencing the original GRN and specifying the quantities being returned (which cannot exceed what was received on that GRN). A mandatory reason must be entered. When confirmed (CONFIRMED), two things happen simultaneously: stock decreases by the returned quantity (a reversal of the original goods receipt movement at the original cost), and the AP module records a debit note against the supplier — a document that reduces the business's payable to the supplier by the value of the returned goods. A confirmed return cannot be edited.

7.1 Create a purchase return

1. Navigate to **Purchasing › Purchase Returns** (`/admin/purchase-returns`) and click **New Return**, or go directly to `/admin/purchase-returns/create`.
2. Pick the **Goods Receipt** by GRN number (the GR must have status RECEIVED).
3. Enter a mandatory **Reason**.
4. For each line being returned, enter the **Returned Quantity** (cannot exceed the quantity originally received on that GR line).
5. Click **Create Return**. The return is created in **DRAFT**.

7.2 Confirm a purchase return

Confirming the return physically ships the goods back and adjusts stock.

1. Open the DRAFT purchase return (navigate to `/admin/purchase-returns/uid/{uid}`).
2. Click **Confirm Return**.
3. Status → **CONFIRMED**. Stock is removed from the branch and a purchase return event is posted.

7.3 Purchase return status reference

Status	Meaning
DRAFT	Being prepared; quantities editable
CONFIRMED	Return confirmed; stock decremented; supplier debit notified

Example — Purchase return for damaged cement:

After receiving GRN-0061, the storekeeper discovers 20 bags of cement arrived wet and unusable. He opens **Purchasing › Purchase Returns** (`/admin/purchase-returns`), clicks **New Return**, picks GRN **GRN-0061**, enters Reason **"20 bags arrived wet — product damaged"**, and sets Returned Quantity **20** on the Saruji 50kg line. He clicks **Create Return** — return **PRET-0018** is created in DRAFT.

The purchasing manager reviews and clicks **Confirm Return** — status → CONFIRMED. Stock decreases by 20 bags (480 bags remain). The AP module raises a supplier debit note for 20 × TZS 14,500 = TZS 290,000 against Simba Cement.

8. Purchase Settings

Navigate to **Purchasing › Purchase Settings** (`/admin/purchase-settings`).

What purchase settings are. Purchase settings are the company-level configuration controls that govern how the procurement workflow operates — specifically, whether Purchase Orders above a certain value require a second-level approval before they can be placed, and how much over the ordered quantity a goods receipt may accept.

Why a PO approval threshold exists. For low-value purchases, requiring a manager to approve every PO would create unnecessary bottlenecks. For high-value purchases, however, committing the business without a second review is a financial control risk. The approval threshold is the balance: below the threshold, POs flow through automatically; above it, they pause for authorisation. This is a common internal control required by auditors and risk frameworks.

Purchase settings control the PO approval workflow and the goods-receipt over-receipt tolerance.

8.1 PO approval threshold

Setting	Description
Enable PO Approval Workflow	When turned on, Purchase Orders above the threshold amount require approval before they can be placed
Approval Threshold Amount	The minimum order total that triggers the approval requirement (shown only when the workflow is enabled)
Currency	The currency of the threshold amount, chosen from the Currency Picker (limited to the company's enabled currencies, defaulting to the company default — see "Common UI Patterns" in the Getting Started chapter)

To change these settings, navigate to **Purchasing › Purchase Settings** (`/admin/purchase-settings`), pick the **Company**, update the values, and click **Save Settings**.

When PO approval is enabled, an order that exceeds the threshold is submitted from its detail screen (section 3.3) and decided by an approver in the Approvals inbox (requires `APPROVALS.DECIDE`; see chapter 11, Approvals). `PURCHASE.ORDER.APPROVE` gates a separate, administrative approve/reject action on the order itself, available only via the API — see the Known limitation note in section 3.3.

8.2 Over-receipt tolerance

Setting	Description
Goods-receipt over-receipt tolerance (%)	How much over a purchase-order line's outstanding quantity a goods receipt may accept, as a percent (e.g. `5` = 5%). A receipt line may be received up to outstanding × (1 + tolerance ÷ 100). Leave **blank** (or `0`) for strict receiving — no over-receipt, the default.

This governs the goods-receipt behaviour described in section 4.1. It exists because goods delivered by weight or in bulk (rice, produce, cement) seldom match the order exactly, and a delivery a fraction over the ordered amount is normal in trade — blocking it outright would force artificial short-receipts and re-orders. Setting a small tolerance (commonly 2–5%) lets the storekeeper book the actual delivered quantity in one receipt, while the ceiling still stops a grossly wrong quantity being received against the wrong PO.

To set it, navigate to **Purchasing › Purchase Settings** (`/admin/purchase-settings`), pick the **Company**, enter the percent in **Goods-receipt over-receipt tolerance (%)**, and click **Save Settings**. The permission required is `PURCHASE.SETTINGS.MANAGE`.

Example — Enabling PO approval:

The CFO wants all purchase orders above TZS 5,000,000 to require a second-level approval. She opens **Purchasing › Purchase Settings** (`/admin/purchase-settings`), turns on **Enable PO Approval Workflow**, sets **Approval Threshold Amount** to **5,000,000**, selects **Currency TZS** from the Currency Picker, and clicks **Save Settings**. From now on any DRAFT PO with a total above TZS 5,000,000 must be approved before it can be placed; the system refuses to place such a PO until an authorised approver has approved it.

Example — Allowing a 5% over-receipt on commodities:

The procurement manager knows rice deliveries never match the order to the exact kilogram. She opens **Purchasing › Purchase Settings** (`/admin/purchase-settings`), enters **Goods-receipt over-receipt tolerance (%)** = **5**, and clicks **Save Settings**. Now when the storekeeper receives against a PO line of 100 bags outstanding, the receipt form shows a **max 105** hint and accepts up to 105 bags in a single GRN; a keyed 120 is still refused.

9. End-to-end procure-to-pay example

The following steps illustrate a complete P2P cycle for a stock purchase with real sample values.

Scenario: Warehouse restocking — 500 bags of Saruji 50kg from Simba Cement Ltd

Step 1 — Raise a Requisition

Storekeeper John opens **Purchasing › Purchase Requisitions** (`/admin/purchase-requisitions`), clicks **New Requisition**, sets Required By **2026-06-18**, notes "Stock replenishment — cement for construction projects". He adds one line: **Saruji 50kg**, Unit **BAG**, Qty **500**, Estimated Cost **TZS 14,800**. He clicks **Create Requisition** (REQ-0080 = DRAFT), then opens it and clicks **Submit for Approval** (status → SUBMITTED).

Step 2 — Approve

Purchasing manager Neema opens REQ-0080 and clicks **Approve** (status → APPROVED).

Step 3 — Convert to RFQ

Neema clicks **Convert**, sets **Convert to** to **RFQ**, adds the two suppliers she wants to quote (**Tanzania Cement Distributors** and **Simba Cement Ltd**) to the invite list, and clicks **Confirm Convert** — RFQ-0031 is created in DRAFT and the **View RFQ** link opens it.

Step 4 — Invite suppliers and send

Neema opens **Purchasing › RFQs / Sourcing** (`/admin/rfqs`), opens RFQ-0031, adds invited suppliers **Tanzania Cement Distributors** and **Simba Cement Ltd**, sets Response Due Date **2026-06-17**, and clicks **Send to Suppliers** (status → SENT).

Step 5 — Capture supplier quotes

Two suppliers respond:

- Tanzania Cement Distributors: 500 bags @ TZS 14,800 = TZS 7,400,000.
- Simba Cement Ltd: 500 bags @ TZS 14,500 = TZS 7,250,000 (lead time 5 days).

Purchasing officer Zawadi captures both quotes on RFQ-0031. RFQ status → QUOTES_RECEIVED.

Step 6 — Award the RFQ

Zawadi clicks **Award** on the Simba Cement quote (lower price). RFQ status → AWARDED. Purchase Order **PO-0088** (DRAFT, 500 bags @ TZS 14,500) is created automatically.

Step 7 — Place the PO

Zawadi opens **Purchasing › Purchase Orders** (`/admin/purchase-orders`), finds PO-0088, reviews the line, and clicks **Place Order** (status → ORDERED, total TZS 7,250,000).

Step 8 — Receive goods

On 2026-06-22, 500 bags arrive. Storekeeper John opens **Purchasing › Goods Receipts** (`/admin/goods-receipts`), clicks **New Receipt**, picks PO-0088, keeps 500 bags, and clicks **Record Receipt**. GRN-0061 created (RECEIVED); PO-0088 status → RECEIVED; 500 bags added to stock.

Step 9 — Allocate landed costs

Port clearing TZS 850,000 + freight TZS 600,000 are entered as a landed cost against GRN-0061 (Basis: By Quantity). Accountant Sarah opens **Purchasing › Landed Costs** (`/admin/landed-costs`), creates the landed cost, and clicks **Confirm & Allocate** — TZS 2,900/bag added to the moving-average cost.

Step 10 — Enter the supplier bill and run 3-way match

Simba Cement's invoice arrives: SIM/2026/1041, 500 bags @ TZS 14,500. Sarah opens **Accounting › Enter Bill** (`/admin/ap/supplier-bills/enter`), picks the supplier, selects Currency **TZS** from the Currency Picker, links PO-0088, adds a bill line (Description **Saruji 50kg**, Billed Qty 500, Unit Cost TZS 14,500) and selects the matching **PO Line**, then clicks **Enter Bill & Match**. All lines → MATCHED. BILL-0051 is ready for payment.

Step 11 — Record AP payment

Finance officer David opens **Accounting › Record Payment** (`/admin/ap/payments/record`), picks BILL-0051 (TZS 7,250,000 due 2026-07-22), records a bank transfer payment on 2026-07-20. The bill status moves to PAID and the AP balance for Simba Cement is cleared.

Step 12 — Purchase return (if needed)

If 20 bags arrived damaged, John opens **Purchasing › Purchase Returns** (`/admin/purchase-returns`), creates a return against GRN-0061 for 20 bags, and the manager confirms it — stock decreases by 20 bags and the AP module raises a TZS 290,000 debit note against Simba Cement.

10. Purchase Reports

Four reports under **Purchasing** answer the questions a buyer, storekeeper or owner asks about what was bought. Each one can be printed or downloaded as **PDF**, **Excel** or **CSV** by a user who also holds **REPORT.EXPORT** (the buttons are hidden otherwise). The **Branch** filter offers only the branches you are assigned to; choosing a branch you are not assigned to is refused with a message saying so — clear the filter to see the whole company. Figures in a foreign currency are listed in their own currency and left out of the totals, and the screen tells you how many.

10.1 Goods Received Register

Purchasing › Goods Received Register (`/admin/reports/purchases/goods-received`) — permission **PURCHASE.GOODS_RECEIPT.VIEW**.

Every item received from suppliers over a period, one line per goods-receipt line: date, GRN number, order number (or **Direct** for stock taken in with **Receive Without Order**), supplier, branch, item, quantity, unit cost and value, with the total at the foot.

- **Filters:** From / To date (required), Branch, Supplier, Product (all optional; type in the supplier or product box to search).
- **Voided receipts:** the receipt stays on the day it was received, and appears again in red, as a **negative** line marked *Voided*, on the day it was voided — the same way the Stock Movement report shows it. A month's register therefore never changes after the month is over.
- **Values exclude VAT.** A goods receipt does not hold VAT (VAT comes from the supplier's bill), and landed costs added later are not included.
- Long registers are paged; the total at the foot always covers **all** pages.

10.2 Purchases by Supplier

Purchasing › Purchases by Supplier (`/admin/reports/purchases/by-supplier`) — permission **PURCHASE.GOODS_RECEIPT.VIEW**.

One line per supplier for a period, largest first: number of receipts, value **received** (less receipts voided in the period — it agrees with the Goods Received Register total for the same period and branch), **returns** (confirmed purchase returns at receipt cost), **net purchases** (received − returns), **billed** (supplier bills dated in the period, excluding VAT) and **unpaid** (what is still owed today on those bills, including VAT).

- **Filters:** From / To date, Branch.

- **Returns / Net purchases** are shown only to users who can see purchase returns (**PURCHASE.RETURN.VIEW**); **Billed / Unpaid** only to users who can see supplier bills (**AP.VIEW**). Hidden columns are left off the report, never shown as zero, and the screen says which are hidden and why.
- Billing happens separately from receiving, so Billed is not expected to equal Received in any one period.

10.3 Open Purchase Orders

Purchasing › Open Purchase Orders (`/admin/reports/purchases/open-orders`) — permission **PURCHASE.ORDER.VIEW**.

Goods ordered and still to come: each placed order line with quantity outstanding — order number, order date, expected date (marked **Late** once it has passed), supplier, item, ordered / received / outstanding quantity in the unit ordered, unit price, outstanding value (excluding VAT) and age in days. Opens straight away, as at today.

- **Filters:** As at date (blank = today), Branch, Supplier.
- Only **placed** orders appear — an order above the approval threshold cannot be placed until it is approved, so drafts and orders awaiting approval are never listed. Closed and voided orders drop off.
- **As at** really looks back: run it for an earlier date and receipts, voids and closings after that date are treated as not having happened yet.
- Orders created automatically behind a **Receive Without Order** receipt are never listed — nobody is going to deliver against them.

10.4 Purchase Price Variance

Purchasing › Purchase Price Variance (`/admin/reports/purchases/price-variance`) — permissions **PURCHASE.ORDER.VIEW** *and* **PURCHASE.GOODS_RECEIPT.VIEW**.

Receipts in a period where you paid a different price from the purchase order: order price, receipt cost and receipt variance; and — for users with **AP.VIEW** — the price on the supplier bill(s) matched to that receipt line, the variance per unit, in total and as a percentage. A **positive** variance means you paid more than the order price.

- **Filters:** From / To date, Branch, Supplier.
- A receipt takes its cost from its order, so the receipt variance is normally zero; the bill is where price changes show. Two bills against one receipt line are combined at their quantity-weighted average price. Draft bills and voided receipts are ignored.
- Without **AP.VIEW** the bill columns are hidden and only receipts whose cost differs from the order are listed.

Inventory and Manufacturing

This chapter covers all inventory screens and the manufacturing (Bill of Materials and Work Orders) screens. All screens are available from the **Inventory** and **Manufacturing** groups in the left-hand navigation.

1. Permissions quick reference

Before starting, confirm that the required permission codes have been granted to your role.

Task	Permission code required
View stock on-hand	`STOCK.VIEW`
Record adjustments / set reorder levels	`STOCK.ADJUST`
Record opening balances	`STOCK.OPENING`
View stock locations	`STOCK.LOCATION.VIEW`
Manage stock locations	`STOCK.LOCATION.MANAGE`
View stock transfers	`STOCK.TRANSFER.VIEW`
Create / dispatch / cancel transfers	`STOCK.TRANSFER.CREATE`
Receive a transfer	`STOCK.TRANSFER.RECEIVE`
View stock counts	`STOCK.COUNT.VIEW`
Create / enter / cancel stock counts	`STOCK.COUNT.CREATE`
Post a stock count	`STOCK.COUNT.POST`
View van-stock reconciliations	`STOCK.VAN_RECON.VIEW`
Create / enter / reconcile / cancel a van reconciliation	`STOCK.VAN_RECON.MANAGE`
View stock batches (by-location / detail)	`STOCK.VIEW`
View stock serials (by-location / by-product / lookup)	`STOCK.VIEW`
View expiring batches	`INVENTORY.EXPIRY.VIEW`
View inventory valuation report	`INVENTORY.VALUATION.VIEW`
Set opening valuation	`INVENTORY.OPENING.SET`
View / manage Bills of Materials	`BOM.VIEW` / `BOM.MANAGE`
View Work Orders / cost report / WIP	`MANUFACTURING.VIEW`
Create / edit / cancel Work Orders	`WORKORDER.MANAGE`
Release a Work Order	`WORKORDER.RELEASE`
Complete / close a Work Order	`WORKORDER.CLOSE`

Navigation items are hidden when the corresponding permission is absent. Attempting to access a route directly without the permission shows a **Forbidden** message.

2. Stock on-hand

What "stock on-hand" means. Stock on-hand is the quantity of a product that is physically present and available at a branch right now. It is the central fact the business needs to answer questions like "how many bags of cement do we have?", "can we fulfil this order?", and "are we running low on cooking oil?" The system maintains this number in real time: every goods receipt adds to it, every sale or delivery deducts from it, and every adjustment, transfer, or stock count correction changes it. The on-hand figure is in the product's base unit (e.g. kilogrammes, pieces, bags) and is accurate to three decimal places.

Why it is maintained as a running balance, not derived from history. The system stores both a maintained on-hand balance and an append-only movement ledger. The maintained balance gives an instant O(1) answer to "what do we have right now" — crucial for fast sales processing and re-order decisions. The ledger provides the full history and lets the balance be independently verified (on-hand should always equal the sum of all movements). Both are always in sync: every movement updates both the ledger and the balance in the same database transaction, so they can never diverge.

2.1 Viewing the on-hand list

Navigate to **Inventory > Stock On-Hand** (`/admin/stock`).

The table shows every stockable product that has had at least one movement at the active branch. The columns are **Product** (code and name shown together), **Quantity** (on-hand, to three decimal places), **Reorder Level**, **Flags**, and an actions column. There is no separate unit-of-measure column. Two derived flags can appear in the **Flags** column:

- **Negative** — the quantity has gone below zero (an overselling indicator; the system does not hard-block it).
- **Low** — the quantity is at or below the reorder level. This flag is blank when no reorder level has been set.

Filtering and pagination. Use the search box to filter by product name or code (the list refreshes after a short pause). Use the paginator controls (First, Previous, page numbers, Next, Last) to move between pages. The paginator hides itself when there is only one page.

Switching views. The list offers three view modes via the tabs at the top:

- **On-Hand** (default) — one row per product, summed across all locations.
- **By Location** — one row per product-location combination. A branch must be selected.
- **By Product** — pick a product from the search picker to see its quantity broken down by every location holding it.

2.2 Recording a manual adjustment

What a stock adjustment is. A stock adjustment is a direct correction to the on-hand quantity of a specific product when the physical stock and the system quantity do not agree for a reason other than a formal stock count. Adjustments are used for damage, spoilage, theft, unexplained shrinkage,

or errors discovered after the fact. Every adjustment is permanent, carries a mandatory reason, and is reflected in the movement ledger immediately.

Why adjustments exist separately from stock counts. An adjustment is a single-product, immediate correction — useful for fixing a known discrepancy right away without pausing all other operations. A stock count (section 5) is a systematic, multi-product reconciliation exercise at a location that freezes a snapshot and allows bulk entry across multiple sessions before committing. Use adjustments for one-off corrections; use stock counts for periodic reconciliation.

Use an adjustment to correct a stock quantity that is wrong for any reason other than a physical count (which has its own workflow — see section 5).

1. On the on-hand list, find the product row and click **Adjust**.
2. The product is pre-selected. Choose the **Reason** from the dropdown:

Reason code	When to use
`COUNT_CORRECTION`	Correcting after an informal count
`DAMAGE`	Goods physically damaged
`SHRINKAGE`	Unexplained loss
`EXPIRY`	Goods past their expiry date
`RECEIPT_CORRECTION`	Correcting a goods-receipt error
`OTHER`	Any other reason — add a note

3. Enter the **Quantity** — positive to increase, negative to decrease.
4. Optionally add a free-text note.
5. Click **Save**.

The system creates a new stock movement (`ADJUSTMENT`) and reloads the on-hand list. Adjustments are permanent records; they cannot be deleted or edited after posting.

Validation. Quantity must be non-zero. Reason is required. A role with only `STOCK.VIEW` cannot see the Adjust button; a direct API call returns 403.

2.3 Recording an opening balance

What an opening balance is. An opening balance is the initial stock quantity entered for a product at a branch that has no prior movement history. It is the "starting point" for that product at that location — the quantity that existed before the system began tracking it. This is a one-time operation; once a product has any movement at a branch, its on-hand can only be changed by the normal transaction flows (receipts, sales, adjustments, etc.).

When to use it. Opening balances are entered at go-live (when migrating from a previous system or manual records), when a new branch is opened, or when a new product is added and stock already exists that needs to be brought onto the books.

An opening balance sets the initial quantity for a product that has never had any movement at this branch. Use this task at go-live or when adding a new branch or product.

1. On the on-hand list, click **Opening Balance**.
2. Pick the product from the picker (search by name or code).
3. Enter the quantity. Must be greater than zero.
4. Optionally add a note such as `go-live`.
5. Click **Record**.

Important. The system rejects an opening balance if the product already has any prior movement at the active branch. A second opening balance on the same product at the same branch is not permitted. To adjust existing stock, use the Adjust flow (section 2.2).

2.4 Setting a reorder level

What a reorder level is. A reorder level (also called a reorder point or minimum stock level) is the quantity at which a product should be reordered. When on-hand falls to or below the reorder level, the system flags the product row with the **Low Stock** indicator. This is a monitoring tool — the flag is a signal to the purchasing team to raise a requisition; it does not automatically place an order.

A reorder level triggers the Low Stock flag when the on-hand quantity reaches or falls below it.

1. On the on-hand list, click the inline edit icon in the **Reorder Level** column.
2. Enter a positive number and save. To remove the reorder level, clear the field and save.

The Low Stock flag recalculates immediately after saving.

2.5 Viewing movement history

What the movement ledger is. The movement ledger is the append-only record of every quantity change for a product at a branch — every goods receipt, every sale issue, every adjustment, every transfer in or out, every opening balance, every production issue and receipt. It is the audit trail for on-hand. Because movements are append-only (never edited or deleted), the ledger is tamper-evident: the on-hand balance can always be recomputed by summing all movements.

Click **Ledger** on a product row to open the movement ledger drawer (its header reads **Ledger — <product>**). Movements are displayed in chronological order with:

- Movement type (Goods Receipt, Sale Issue, Adjustment, Opening Balance, Transfer In/Out, etc.)
- Direction (IN or OUT)
- Signed quantity
- Date and time
- Reason code or note where applicable

The drawer has its own paginator. Movements are append-only records; there is no edit or delete.

3. Stock locations

Navigate to **Inventory > Stock Locations** (`/admin/stock/locations`).

What a stock location is. A stock location is a named physical area within a branch where stock is stored and counted. Locations let a business track stock at a finer level than the branch — for example, distinguishing between the main warehouse, the shop floor, a quarantine area for goods awaiting inspection, and a van for a mobile sales team. Every stock movement and stock count is associated with a specific location, so the system can answer not just "how many bags of cement does the Dar es Salaam branch have?" but "how many are in the Warehouse versus the Store?"

Why locations exist. Without locations, the business knows only how much stock is at a branch in aggregate. With locations, it can see where exactly the stock is, which is essential for efficient warehousing, picking, physical counting, and segregating goods that should not be issued until inspected. Locations are also the boundary for stock counts — a count covers one location at a time.

Finding locations across branches. The list shows locations for one branch at a time, chosen from the **Branch** filter above the table. It defaults to your active branch (the one set by the branch switcher), and the filter lists **every** branch in the company — not only the branches you are assigned to. If you pick a branch you are not assigned to, the list responds with a **Forbidden** message (the same `STOCK.LOCATION.VIEW` scope that guards the screen). For a branch you *are* assigned to, a location created for it is no longer invisible — it simply appears once you filter to that branch. After you create a new location, the filter switches automatically to the branch it was created for, so the new row is visible immediately without an extra step.

Location types

Type	Typical use
`WAREHOUSE`	Main storage area
`STORE`	Shop floor / retail
`VAN`	Mobile / vehicle storage
`QUARANTINE`	Held goods pending inspection
`OTHER`	Any other purpose

3.1 Creating a location

1. Click **New Location** (the button toggles to **Cancel** while the form is open).
2. Enter a short **Code** (up to 30 characters, unique within the branch) and a **Name** (up to 120 characters).
3. Choose the **Type**.
4. Pick the **Branch** from the picker.
5. If you chose the **VAN** type, an **Agent** picker appears. Optionally select the route agent who runs this van — the hint reads *"Assigns this van to the route agent who runs it."* The assignment is company-scoped and carries onto that van's day-end reconciliation worksheets (section 6). The Agent field is shown only for VAN locations; switching the type away from VAN clears it.
6. Tick **Set as default** if this should be the primary location for the branch. There can be only one default location per branch — making a new location the default automatically clears the prior one.
7. Click **Create Location**.

New locations are created in **Active** status.

3.2 Editing a location

Click the edit icon on a row. You can change the name and location type. The code is not editable after creation. For a **VAN** location, the row also exposes an **Agent** picker so you can assign, reassign, or clear (choose "No agent") the route agent — the current agent is shown in the list's **Agent** column.

3.3 Marking as default

In the row's Actions column, click the star icon button (its accessible label is "Set <code> as default") on any active, non-default location. The previous default is cleared automatically.

3.4 Deactivating and reactivating

In the row's Actions column, click the pause-circle icon button (accessible label "Deactivate <code>") to set the location to **Inactive**. It no longer appears in pickers used by transfers and counts. Click the play-circle icon button (accessible label "Reactivate <code>") to restore it to Active.

Locations are never hard-deleted. The list always shows every location, both Active and Inactive — there is no status filter, so an Inactive location and its history simply remain visible in the list.

4. Stock transfers

Navigate to **Inventory > Stock Transfers** (`/admin/stock-transfers`).

What a stock transfer is. A stock transfer is a document that moves stock from one physical location to another — either between two locations within the same branch (for example, from the Warehouse to the Store) or between two different branches (for example, from the Arusha branch to the Dar es Salaam branch). A transfer records a physical movement of goods without buying or selling them; it re-attributes stock from one place to another.

Why transfers do not affect the income statement. A transfer does not create a sale (no revenue, no COGS) and is not a purchase (no supplier, no invoice). It is a value-preserving internal movement: the cost of the goods is the same before and after the transfer — it has simply moved to a different location. Because the moving-average cost is maintained per product at the company level (not per location), the transfer does not change the financial value of inventory — it only re-attributes it between locations. No GL journal entry is posted for a standard same-cost-grain transfer.

When transfers are used. Transfers are used when stock needs to be redistributed — to replenish a retail store from a warehouse, to move goods to a van for a field sales team, or to consolidate slow-moving inventory at one location. The `STOCK.TRANSFER.CREATE` permission is required to initiate and dispatch; `STOCK.TRANSFER.RECEIVE` is required to receive.

How transfers flow — two modes. An **Instant** transfer is for moves between two locations within the same branch; it completes in a single step with no in-transit period. An **In-transit** transfer is for cross-branch moves: when dispatched, the source location's stock decreases immediately (goods are "in the truck") and the destination location's stock only increases when the destination operator

confirms receipt. Between dispatch and receipt, the goods are "in transit" — not counted at either location. This two-step model prevents double-counting and gives each branch an accurate view of what it physically holds.

A stock transfer moves stock from one location to another. Two modes are available:

- **Instant** — for transfers between two locations within the same branch. Stock moves in a single step.
- **In-transit** — for transfers that cross branches. The transfer follows a Dispatch → Receive workflow; stock is in transit between the two events.

Transfer status lifecycle

```
DRAFT
  |-- (Instant mode)  --> COMPLETED
  |-- (In-transit mode) -> DISPATCHED --> RECEIVED
  \-- (any mode, before dispatch) --> CANCELLED
```

4.1 Creating a transfer

1. Navigate to **Stock Transfers > Create** (`/admin/stock-transfers/create`).
2. Pick the **Source Branch** and **Source Location**.
3. Pick the **Destination Branch** and **Destination Location**. Source and destination must be different locations.
4. Set the **Transfer Date**.
5. Choose the **Transfer Mode**: Instant (same-branch) or In-transit (cross-branch).
6. Click **Add line** for each product to transfer. Pick the product by name and enter the quantity.
7. Click **Create Transfer**.

The transfer is created with status **Draft** and a system-generated transfer number. The screen navigates to the transfer detail.

Validation. Source and destination locations must differ. At least one line is required. Quantity per line must be positive. Transfer date is required.

4.2 Dispatching an in-transit transfer

On a Draft, In-transit transfer, click **Dispatch**. The status changes to **Dispatched** and the source location's stock decreases immediately.

The Dispatch button is only available when the transfer is in Draft status and the mode is In-transit. Dispatching requires the `STOCK.TRANSFER.CREATE` permission.

Insufficient source stock. If the source location does not allow negative stock, dispatch is rejected (409 Conflict) when the available quantity at the source is less than the quantity being transferred — the message reports the available and requested quantities and states that the source location does not allow negative stock (it does not name the product, so check the transfer's lines to see which one is short). The transfer stays in Draft so you can correct the lines or top up the source. (A source location whose `allowNegative` flag is set will let the dispatch proceed and the source on-hand can go negative.)

4.3 Receiving an in-transit transfer

On a Dispatched transfer, the destination operator clicks **Receive**. The status changes to **Received** and the destination location's stock increases.

Receiving requires the `STOCK.TRANSFER.RECEIVE` permission. This allows organisations to separate the dispatcher and receiver roles.

4.4 Completing an instant transfer

On a Draft, Instant transfer, click **Complete (Instant)**. The transfer completes in a single step; both locations update simultaneously. As with dispatch, completion is rejected (409 Conflict) if the source location does not allow negative stock and the available quantity is less than the quantity being transferred.

4.5 Cancelling a transfer

On a Draft transfer, click **Cancel**. The status changes to **Cancelled** and no stock movement is recorded. Only Draft transfers can be cancelled.

4.6 Viewing the transfer list and detail

The list shows transfer number, transfer date, mode, status, source location, and destination location. To open the detail view, click the **View** action (eye icon) in the row's Actions column — there is no row-click. The transfer is referenced by its human transfer number throughout the UI; the internal identifier appears only in the browser address bar.

Example — In-transit dispatch from Arusha Warehouse to DSM Store:

Storekeeper Grace Mwenda at Arusha branch needs to send 200 bags of Pembe Flour and 50 cartons of Cooking Oil to the Dar es Salaam main store.

1. Navigate to **Inventory › Stock Transfers › Create** (`/admin/stock-transfers/create`).
2. Source Branch: `Arusha Branch`; Source Location: `Arusha Warehouse`.
3. Destination Branch: `DSM Branch`; Destination Location: `DSM Main Store`.
4. Transfer Date: `2026-06-10`; Transfer Mode: **In-transit**.
5. Add lines:
    - Product: `Pembe Flour 2kg (FLR-002)`, Qty: `200`.
    - Product: `Cooking Oil 3L (OIL-003)`, Qty: `50`.
6. Click **Create Transfer**. Transfer `TRF-0042` is created with status **Draft**.
7. Grace reviews the lines and clicks **Dispatch**. Status becomes **Dispatched**. The Arusha Warehouse stock for both items decreases immediately (200 bags and 50 cartons deducted).
8. The following day, DSM storekeeper Omari Njau opens **Inventory › Stock Transfers** (`/admin/stock-transfers`), finds `TRF-0042` with status Dispatched, and clicks the **View** action (eye icon) in its Actions column to open the detail.

9. Omari counts the physical delivery — both lines match — and clicks **Receive**. Status becomes **Received**. DSM Main Store stock increases by 200 bags and 50 cartons.
10. Both storekeepers can now see `TRF-0042` with status **Received** in the transfer list. No cancellation is possible at this stage.

5. Stock counts

Navigate to **Inventory > Stock Counts** (`/admin/stock-counts`).

What a stock count is. A stock count (also called a physical inventory or stocktake) is a scheduled exercise where a team physically counts the items held at a location and compares the counted quantities to the quantities the system believes are there. Any discrepancy (a "variance") is recorded and — after review — posted as an adjustment to bring the system records into alignment with physical reality.

Why stock counts are necessary. Even in a well-run warehouse, small discrepancies accumulate over time: items get damaged and not immediately reported, products are picked without being scanned, counting errors occur during receipts, or theft occurs. Without periodic counts, these discrepancies compound silently and the business makes decisions (ordering, sales commitments, valuations) based on wrong numbers. A stock count is the mechanism to catch and correct these discrepancies systematically. Unlike ad-hoc adjustments, a count involves a controlled snapshot of the entire location's stock at a point in time, multi-session entry, and formal posting — producing an auditable record of what was found versus what was expected.

How variances affect the books. When a count is posted, each line with a variance is converted into a stock adjustment movement at the current moving-average cost. The value difference is posted to the `STOCK_ADJUSTMENT` expense account (5160) against the `INVENTORY` account (1300). A negative variance (less found than expected) is an expense; a positive variance is a credit to the adjustment account. This ensures the inventory balance on the balance sheet and the GL always stay in sync.

When it is used. Stock counts are run periodically — monthly, quarterly, or annually depending on the business's risk appetite and the volatility of the products. **Full** counts cover all products at a location. **Cycle** counts cover a rotating subset of products (for example, high-value or fast-moving lines), allowing more frequent reconciliation without counting everything at once. The `STOCK.COUNT.CREATE` permission is needed to create and enter counts; `STOCK.COUNT.POST` (typically held by an accountant or supervisor) is needed to post the variances.

A stock count records what is physically present at a location and reconciles it against the system quantity. Any variance is posted as a stock adjustment and a GL entry.

Count status lifecycle

```
COUNTING --> POSTED
     \-----> CANCELLED
```

When a count is created the system immediately freezes the on-hand quantities (the snapshot) and moves the document to **Counting** status.

5.1 Creating a stock count

1. Navigate to **Stock Counts > Create** (`/admin/stock-counts/create`).
2. Select the **Company** and **Branch**. (The Company selector appears only when you have access to more than one company, and the Branch selector only when the chosen company has more than one branch; otherwise the single company/branch is used automatically and no selector is shown.)
3. Pick the **Stock Location** from the picker.
4. Set the **Count Date** (defaults to today).
5. Choose the **Count Type**:

    - **Full** — all products held at the location are included.
    - **Cycle** — recorded as a cycle count for reporting/classification. The on-screen hint reads *"FULL = entire location. CYCLE = selected products."*

    > **Current limitation — cycle scoping.** The create screen has **no product picker**. It collects only Location, Count Date, Count Type, and Notes. Whether you choose **Full** or **Cycle**, the count is currently snapshotted over **all** products held at the location — there is no UI control to restrict a cycle count to a chosen subset of products. The **Count Type** value is stored and shown on the document, but it does not change which lines are created. To count only a few products, either run a full count and enter quantities for just those lines, or use a single-product Adjustment (section 2.2) instead.

6. Click **Create Count**.

The count is created with status **Counting** and a system-generated count number. The system records the current on-hand quantity for each product as the **System Qty** snapshot. This snapshot is frozen and cannot change.

5.2 Entering counted quantities

Open the count detail. For each product line:

1. Enter the physically counted quantity in the **Counted Qty** column.
2. Optionally type a free-text reason in the **Reason** box on lines that have a variance.
3. Click **Save Counted Qtys**.

After posting, the **Variance Qty** column shows `Counted Qty − System Qty`. A positive variance means more stock was found than expected; a negative variance means less was found.

The document stays in Counting status after saving. You can enter counts in multiple sessions.

5.3 Posting a count

Posting creates stock adjustment movements for every line with a variance and generates a single GL variance journal.

1. Open a count in Counting status.
2. Click **Post Count** to reveal the posting form.
3. Enter the **Posting Date**.
4. Click **Confirm Post**.

Posting requires the `STOCK.COUNT.POST` permission (typically held by an accountant or supervisor). After posting, the document is read-only.

5.4 Cancelling a count

Open a Counting count and click **Cancel Count**. No stock movements or GL entries are created. A Posted count cannot be cancelled. If corrections are needed after posting, create a new count.

Example — Cycle count of sugar and rice with a variance posted:

Accountant supervisor Boniface Kessy wants to reconcile two fast-moving products at the DSM Main Store. Because the create screen has no product picker (see the limitation note in section 5.1), the count snapshots every product at the location; Boniface simply leaves the other lines un-entered and enters counts only for the two products he is interested in.

1. Navigate to **Inventory › Stock Counts › Create** (`/admin/stock-counts/create`).
2. Company: `Kijenge Trading Ltd`; Branch: `DSM Branch`; Location: `DSM Main Store`; Count Date: `2026-06-12`; Count Type: **Cycle**.
3. Click **Create Count**.
4. Count `CNT-0009` is created with status **Counting**. The system records snapshot quantities for **every** product at DSM Main Store, including Sugar = 850 bags and Rice = 240 bags.
5. Storekeeper Omari Njau physically counts the two shelves he is responsible for. He opens `CNT-0009` and, leaving every other line blank, enters:

    - Sugar counted: `843` (variance: −7 bags).
    - Rice counted: `245` (variance: +5 bags).
    - In the **Reason** box on the Sugar line he types `SHRINKAGE`. For Rice no reason is needed (positive variance — unrecorded receipt correction). Click **Save Counted Qtys**.

6. Boniface reviews the variances and clicks **Post Count**. Posting Date: `2026-06-12`. Confirms. Lines with no counted quantity entered are treated as no-variance and post nothing.
7. The system posts two stock adjustment movements:

    - Sugar: −7 bags (ADJUSTMENT, reason SHRINKAGE).
    - Rice: +5 bags (ADJUSTMENT). A single GL variance journal posts: DR Inventory Variance / CR Inventory for the sugar loss (valued at moving-average cost); the rice surplus reverses this direction.

8. The count document is now read-only with status **Posted**. On-hand quantities at DSM Main Store are now: Sugar = 843, Rice = 245.

6. Van-stock reconciliation

Navigate to **Inventory > Van-Stock Reconciliations** (`/admin/van-reconciliations`).

What a van reconciliation is. A van-stock reconciliation is the day-end route worksheet for a mobile (van) sales team. At the start of the day a van is loaded with stock (via stock transfers into the van's location); the driver-salesperson sells along the route and returns any unsold stock at the end of the day. The reconciliation is the sheet that answers, product by product, the question: *did the stock that left the van get accounted for by sales plus returns?* For each product it lays out **Loaded − Sold − Returned = Expected** on the van, then compares that Expected figure against the **Physical** count of what is actually still on the van. Any gap is the **Variance**.

Why it is record-only. The reconciliation is a control and monitoring document, not an accounting transaction. It does **not** post any stock movement or GL entry — the underlying stock has already moved through the normal flows (transfers loaded the van, sales issued stock out, transfer-returns brought stock back). Reconciling simply freezes the worksheet as the day's agreed record and flags any variance for a supervisor to investigate (a shortage may mean under-reported cash sales, theft, or a miscount). If a variance needs to correct the books, that is done separately through a stock adjustment (section 2.2) or a stock count (section 5) — the reconciliation itself never touches on-hand.

The van ↔ agent link. A VAN-type stock location can be assigned a **route agent** on the Stock Locations screen (section 3) — the person who runs that van. When a reconciliation is created for a van location, the assigned agent's name is carried onto the worksheet and shown in the list and header, so each day's sheet is attributable to the person responsible for the van. Assigning an agent is optional and applies only to VAN-type locations.

Reconciliation status lifecycle

```
OPEN --> RECONCILED
     \-> CANCELLED
```

A reconciliation is created **Open** and stays editable until it is either **Reconciled** (frozen as the final record) or **Cancelled**. Reconciled and Cancelled are both terminal — a frozen sheet cannot be reopened; create a new one if corrections are needed.

6.1 Creating a reconciliation

1. On the Van-Stock Reconciliations list, click **New Reconciliation** (`/admin/van-reconciliations/new`).
2. Select the **Company** and **Branch**. (The Company selector appears only when you have access to more than one company, and the Branch selector only when the chosen company has more than one branch; otherwise the single company/branch is used automatically.)
3. Pick the **Van Location** from the picker. Only **VAN**-type locations for the branch are listed; each option shows the van's code and, where set, its assigned agent. If the branch has no VAN

locations, a "No VAN-type locations found for this branch" hint is shown — create a VAN location first (section 3.1) and, ideally, assign its agent.
4. Set the **Business Date** (defaults to today) — the route day being reconciled.
5. Optionally add **Notes**.
6. Click **Create Reconciliation**.

The reconciliation is created with status **Open** and a system-generated reconciliation number (shown as **Recon #**). The screen opens the worksheet directly.

How the lines are pre-filled. On creation the system reads the van's transfer activity for that business date and pre-fills one line per product that moved through the van, with:

- **Loaded** — the total stock transferred *into* the van location that day (the sum of Transfer-In movements).
- **Returned** — the total stock transferred *out of* the van that day (the returns).

Both figures are pre-filled but remain **editable** on the worksheet, so you can correct them if the physical reality differs from the recorded transfers.

One per van per day. Only one reconciliation can exist for a given van on a given business date. Attempting to create a second returns "A van reconciliation for this van on this business date already exists" — open the existing one instead.

6.2 Entering sold and physical counts

Open an Open reconciliation. The lines grid has one row per product with seven columns: **Product**, **Loaded**, **Sold**, **Returned**, **Expected**, **Physical**, and **Variance**. While the sheet is Open (and you have manage permission), the Loaded, Sold, Returned, and Physical cells are editable; Expected and Variance are computed live and cannot be typed.

1. For each product, enter the **Sold** quantity — the total sold off the van that day. This is the one figure the system cannot derive for you (it is not in the stock ledger), so it must be entered.
2. Enter the **Physical** count — what is actually still on the van at day-end. Leave it blank on any line you have not yet counted.
3. Adjust the pre-filled **Loaded** or **Returned** figures on any line where the recorded transfers do not match reality.
4. Click **Save Lines**.

As you type, the worksheet recomputes each row instantly:

- **Expected = Loaded − Sold − Returned** — how much *should* still be on the van.
- **Variance = Physical − Expected** — the discrepancy. It is only computed once a Physical count is entered; until then the row shows **Not counted**.

The Variance cell is tagged for quick scanning:

Tag	Meaning
Not counted	No physical count entered yet — variance not

	computed.
Balanced	Physical matches Expected (within rounding) — nothing to explain.
Over	Physical exceeds Expected — a surplus on the van (e.g. sales were under-reported). Shown in green.
Short	Physical is less than Expected — a shortage. Shown in red.

Where a snapshot unit cost is available, the row also shows the variance valued in money (variance quantity × unit cost). This is an indicative figure only — it is never posted to the ledger.

A row of **KPI tiles** above the grid totals Loaded, Sold, Returned, Expected, Physical, and Variance across all lines, giving the day's overall picture at a glance.

Saving is a patch. Save Lines records only the rows you actually edited this session; untouched rows keep their existing values. You can save partway through and come back — the sheet stays Open across sessions. You must enter a Sold or Physical value on at least one line before saving.

6.3 Reconciling

When the counts are complete and the variances are understood, click **Reconcile**. The status changes to **Reconciled**, the worksheet is frozen (the cells become read-only), and a green confirmation banner notes the reconcile time and restates that this is **record-only — no stock/GL posting**. A Reconciled sheet is the day's final agreed record for that van.

Reconciling requires the `STOCK.VAN_RECON.MANAGE` permission and is only available while the sheet is Open.

6.4 Cancelling

To discard a worksheet that was created in error or is no longer needed, click **Cancel** on an Open reconciliation. The status changes to **Cancelled** and the sheet becomes read-only. As with reconcile, no stock or GL entries are ever involved. A Cancelled sheet cannot be reopened — create a new reconciliation if one is still needed for that van and day.

6.5 Viewing the reconciliation list

The list shows every reconciliation for your active branch (switch the branch selector to see another branch's vans), with columns **Recon #**, **Van**, **Agent**, **Business Date**, and **Status**. Click the **View** action (eye icon) in a row to open its worksheet. Viewing requires `STOCK.VAN_RECON.VIEW`; the **New Reconciliation** button and all editing actions require `STOCK.VAN_RECON.MANAGE`.

7. Batches and lot tracking

Navigate to **Inventory > Stock Batches** (`/admin/stock/batches`). Requires the `STOCK.VIEW` permission.

What a batch (lot) is. A batch — also called a lot — is a group of units of the same product that were manufactured or received together and share the same identity attributes, most importantly an expiry date and a manufacture date. For example, a batch of medicines all manufactured on the same day with the same expiry date is one lot. Batch tracking allows the business to know exactly which physical batch a unit came from — critical for food, pharmaceutical, and chemical products where recall or expiry management is required.

Why batch tracking matters. Without lot tracking, if a product recall is announced (for example, a contaminated batch of cooking oil), the business cannot identify which specific units on its shelves belong to the recalled batch. With batch tracking, the system can identify every unit of that batch, where it is held, and how much remains — enabling targeted removal without wasting uninvolved stock. Batch tracking is also necessary for FEFO (First Expired, First Out) stock rotation: the system ensures that stock with the earliest expiry date is issued first, minimising spoilage and waste.

How batches are created. Batches are created automatically by the purchasing flow when lot-tracked products are received via a Goods Receipt — the system assigns a lot number and records the manufacture and expiry dates at that point. You cannot create batches manually on this screen.

What happens when a Goods Receipt is voided. Voiding a Goods Receipt now backs out the batch quantities it created. When a receipt is voided, the system reverses the receipt movement and, for each lot-tracked line, reduces the batch's on-hand by the received quantity at the location it was received into — so a mistaken or reversed receipt no longer leaves a phantom batch on the books. The back-out is deliberately gentle: if the batch has already been partly consumed or the data cannot be matched cleanly, that line is skipped and logged rather than blocking the void. Voiding a receipt for a product with a blank lot number is still reversed (received-with-no-lot is tracked under an "untracked" lot placeholder). See chapter 4 (Purchasing) for how to void a Goods Receipt.

Batches (lots) are created automatically when lot-tracked products are received, and are backed out when that receipt is voided. This screen provides a read-only view; you cannot create or edit batches directly.

7.1 Viewing batches by location and product

1. Pick a **Location** from the picker.
2. Pick a **Product** from the picker.
3. The table lists all batches at that location for that product, showing lot number, manufacture date, expiry date, quantity on-hand, and an expiry flag.

7.2 Expiring batches report

Click the **Expiring Soon** tab. Set a horizon date (default: 30 days from today). The report lists all batches whose expiry date falls on or before the horizon and whose quantity is greater than zero.

- Batches already past their expiry date are flagged in red.
- Batches expiring before the horizon are flagged as a warning.

The expiring batches tab requires the `INVENTORY.EXPIRY.VIEW` permission.

8. Serial number tracking

Navigate to **Inventory > Stock Serials** (`/admin/stock/serials`). Requires the `STOCK.VIEW` permission.

What serial number tracking is. Serial number tracking assigns a unique identifier to each individual unit of a product — for example, every laptop, refrigerator, or generator has its own serial number. Unlike batches (which group many units of the same type), a serial identifies one specific physical item. The system records where each serial is, whether it is in stock, has been issued to a customer, or has been returned, giving full unit-level traceability.

Why it is used. Serial tracking is valuable for high-value items, warranty management, and theft prevention. When a laptop is sold, the system records which serial number left the warehouse and to which customer. If a customer returns a laptop claiming it is faulty, the system confirms whether that serial was genuinely sold to them. Serial numbers also help with insurance claims (proving what was held) and regulatory compliance.

How serials are managed. Serial numbers are created and updated automatically by the purchasing (goods receipt), sales (delivery), and transfer flows. This screen is a read-only view and lookup tool; you cannot create or modify serials directly here.

What happens when a Goods Receipt is voided. Voiding a Goods Receipt now removes the serial numbers it brought into stock. For each serialised line on the voided receipt, the system deletes the serial records it created — but only while they are still **In-Stock**. A serial that has since been **Issued** (sold/delivered) or **Returned** is left untouched, so voiding a receipt can never wipe out the history of a unit that has already moved on. As with batches, any serial that cannot be cleanly matched is skipped and logged rather than blocking the void.

Serial numbers are assigned to individual units of serialised products. This screen is read-only; serials are created and updated by the purchasing, sales, and transfer flows, and In-Stock serials are removed when their originating receipt is voided.

The screen has two view-mode tabs: **By Location** and **Product History**.

8.1 Viewing serials by location

1. Select the **By Location** tab.
2. Pick a **Location** and a **Product**.
3. Optionally filter by **Status**: All, IN_STOCK, ISSUED, RETURNED.
4. The table shows serial number, current status, and the related documents.

8.2 Viewing serial history by product

Switch to the **Product History** tab. Pick a product to see all of its serials across all statuses and locations.

8.3 Looking up a serial number

There is no separate lookup tab. In the **By Location** tab, once a product is selected, a **Lookup by serial #** panel appears above the table. Type the serial number and click **Lookup**. The system returns the current status and related documents, or shows a "Serial number not found for this product" message if the serial does not exist for that product.

9. Inventory valuation

What inventory valuation is. Inventory valuation is the process of assigning a monetary value to the goods held in stock. The business needs to know not just how many units it has but what those units are worth — for the balance sheet (Inventory is an asset), for the cost of goods sold when items are sold (COGS reduces profit), and for management decisions (is this product profitable to sell?). The system uses the **moving-average cost method**: the average unit cost is recalculated each time stock is received, blending the new purchase cost with the existing average. This means all units of a product at a branch carry the same average cost, regardless of when they were purchased.

How the moving average is maintained. When a goods receipt is posted, the system computes the new average as: `(existing stock value + new receipt value) / (existing quantity + received quantity)`. This weighted average is then applied to all units held. When goods are sold, the COGS is the quantity sold multiplied by the current average cost at the moment of the sale. When stock is adjusted, the adjustment value is computed at the current average. This means the Inventory account on the balance sheet always equals the sum of (on-hand quantity × average cost) across all products — a relationship the valuation report verifies.

9.1 Valuation report

Navigate to **Inventory > Stock Valuation** (`/admin/stock/valuation`). Requires the `INVENTORY.VALUATION.VIEW` permission.

The report is not loaded automatically — the initial screen shows an empty state ("Click Refresh to load the current valuation"). Click **Refresh** to build the report. It then shows every stockable product with its average cost, quantity, and calculated inventory value. A reconciliation bar at the top compares the sum of on-hand values (the stock ledger) against the GL inventory account balance:

- **Reconciled to GL** (green) — the stock ledger and GL agree.
- A red **Finance-grade alarm — Stock ledger and GL are out of sync** banner — there is a discrepancy. The stock total, the GL 1300 balance, and the difference amount are shown, and the GL Reconciliation card's status tag reads **Out of balance**. Finance review is required.

9.2 Setting an opening valuation

What opening valuation is. Opening valuation is the one-time act of assigning an initial monetary cost to stock that already has a quantity on-hand but no established cost. This occurs at system go-live (when stock was loaded via opening balances before the cost data was entered) or when a new product is added and given an opening balance. Until an average cost is established, the system

cannot post COGS for sales of that product — it will issue the stock but leave the cost leg blank, flagging the anomaly.

Navigate to **Inventory > Opening Valuation** (`/admin/stock/valuation/opening`). Requires the `INVENTORY.OPENING.SET` permission.

Use this screen to assign an initial cost to products that have a quantity on-hand but no established average cost. It is a single form, not a per-row entry table.

1. Pick a product from the **Product (unvalued on-hand rows)** dropdown — only products with on-hand quantity but no cost appear in it.
2. Enter the **Opening Unit Cost** for that product.
3. Click **Set Opening Valuation**.

The system posts a GL entry (DR Inventory / CR Opening Balance Equity) and the product's average cost is established. Opening valuation is a one-time operation per on-hand row. Once a row has been valued it no longer appears on this screen.

9.3 Reorder Report

Navigate to **Inventory > Reorder Report** (`/admin/reports/reorder`). Requires `STOCK.VIEW`.

Lists every stock line at or below its reorder level (set per item and location on the Stock On-Hand screen — see 2.4). Optional filters: **Branch** and **Preferred supplier** (type to search); filtering by supplier gives that supplier's order list. Columns: code, description, branch / location, on hand, reorder level, shortfall, **suggested quantity** and preferred supplier.

- The suggested quantity fills the line back up to its maximum level when one is set; otherwise it uses the product's standard reorder quantity; otherwise it is just the shortfall. It is never less than the shortfall.
- Users who also hold `INVENTORY.VALUATION.VIEW` see two more columns — **Last cost** (the unit cost of the latest goods receipt of the item) and **Order value** — plus an estimated order total. Without that permission the cost columns are not shown at all.
- Only active products are listed. The reorder level on the product master is not used; the level on the stock line is (the same one the low-stock flag and notification use).
- Exports additionally require `REPORT.EXPORT`.

9.4 Stock Ageing

Navigate to **Inventory > Stock Ageing** (`/admin/reports/stock-ageing`). Requires `INVENTORY.VALUATION.VIEW` (it shows stock value).

Shows, per item, the quantity on hand split into five age bands — **0–30**, **31–60**, **61–90**, **91–180** and **over 180 days** — with the value, and the number of days since the item last sold ("Never" if it has not). Optional filters: **As of** date (defaults to today) and **Branch**. Tick **Only items not sold in 90 days or more** to see slow and dead stock; the totals still cover every item.

How the age is worked out. Stock is not labelled with the day it arrived, so the report assumes **first in, first out**: the stock on hand is taken to be the most recently received units. Receipts, opening

balances, production, positive adjustments (including count gains and bulk stock imports) and stock transferred in from another branch count as arrivals; moves between your own locations do not change an item's age, and reversals (a voided sale putting stock back) are not new arrivals. Stock is costed at moving average, so each band is valued at the item's average cost — ageing is a view of quantities, and the band values add up exactly to the stock value.

- When the recorded arrivals do not explain all of an item's stock (stock older than the system's records), the unexplained part is put in the **over 180 days** band and marked with an asterisk.
- Items with no cost show a dash for value and are left out of the value totals; items with negative stock have no age and are left out of the list. A banner counts both.
- For a past **As of** date, quantities are rebuilt as of that date but valued at today's average cost.
- Exports additionally require `REPORT.EXPORT`; the PDF prints the first-in, first-out note in its header.

10. Bills of Materials

Navigate to **Manufacturing > Bills of Materials** (`/admin/boms`).

What a Bill of Materials is. A Bill of Materials (BOM) is the formal recipe or formula that defines exactly what components — and in what quantities — are needed to produce one run of a finished product. For example, a BOM for "Ugali Pack 1kg" might specify 1.05 kg of maize flour (the extra 5% is scrap allowance), or a BOM for a piece of furniture might specify 2 pieces of timber, 4 bolts, 1 m² of fabric, and 200 g of adhesive. The BOM is the production blueprint; without it, a work order cannot know what to consume.

Why BOMs are versioned. Products change: a recipe might be reformulated, a component supplier might change, or the manufacturing process might be improved. Each change requires a new BOM version. The system keeps all historical BOM versions so that an old production run can be reproduced exactly as it was originally planned — using the BOM that was active at the time the work order was released — even if the current BOM is different.

When a BOM is used. A BOM is created and maintained by the production or engineering team. It becomes active when activated (with an effective-from date), at which point it can be referenced by Work Orders. Only one BOM can be active at a time per finished product — activating a new version automatically archives the previous one. A BOM must be set up before any Work Order for that finished product can be released.

How it connects to manufacturing. When a Work Order is released, the system looks up the active BOM for the finished product, explodes it to its leaf components (recursively resolving any sub-assemblies), and materialises the planned component lines on the work order. The BOM is then pinned to that work order — subsequent changes to the BOM do not affect work orders that are already in progress.

A Bill of Materials (BOM) defines what components and quantities are needed to produce a given quantity of a finished product. One BOM can be active at a time for each finished product; creating and activating a new BOM version automatically archives the previous one.

BOM status lifecycle

```
DRAFT --> (activate) --> ACTIVE --> (archive) --> ARCHIVED
```

Only a DRAFT BOM can be activated. ARCHIVED is a permanent terminal state.

10.1 Creating a BOM

1. On the BOM list, click **New BOM**.
2. Pick the **Parent Product** from the picker (its placeholder reads "Select finished product"; search by name or code). The product must be a GOODS type and must be active.
3. Enter the **Output Quantity** (how many units the BOM produces per run) and optionally the **Yield %** (default 100%).
4. Optionally add notes.
5. Click **Create BOM**.

The BOM is created in **Draft** status with the next version number for that finished product (v1 for the first BOM, v2 for the next, etc.).

Validation. Output quantity must be positive. Yield must be between 0.0001% and 100%.

10.2 Adding components

What a BOM component is. A BOM component is one ingredient or raw material in the recipe. Each component line specifies the product to consume, the quantity required per one run of the BOM output, and an optional scrap percentage (an allowance for material that is consumed but does not make it into the finished good — for example, offcuts when cutting fabric). A component is classified as either **MAKE** (the component is itself manufactured — the system will look for its own BOM) or **BUY** (the component is purchased from a supplier and is a raw material).

Open a Draft BOM detail and click **Add Component**.

1. Pick the **Component Product** from the picker.
2. Enter **Qty Per** (quantity of the component per one run of the BOM output).
3. Optionally enter **Scrap %** (allowance for waste, 0–99.9999%).
4. Optionally enter a **Reference** (for engineering cross-reference).
5. Choose **Sourcing**:

    - **Auto (derive)** — the system determines whether the component is made internally (MAKE) or purchased (BUY) based on whether it has an Active BOM.
    - **MAKE** — the component is itself manufactured.
    - **BUY** — the component is purchased from a supplier.

6. Click **Add**.

Components can be added, edited, or removed only while the BOM is in Draft status.

10.3 Editing a BOM header

There is no Edit button. The **BOM Header** form is always shown inline on the BOM detail. On a Draft BOM its Output Qty, Yield %, and Notes fields are editable; on an Active BOM only Notes can be changed (the structural fields are disabled). Make your changes and click **Save Header**.

10.4 Activating a BOM

A BOM must have at least one component before it can be activated.

1. Open a Draft BOM.
2. Click **Activate**.
3. Enter an **Effective From** date.
4. Confirm.

Activating a BOM automatically archives the current Active BOM (if any) for the same finished product. Only one BOM can be Active per product at a time.

Validation. Effective From date is required. The BOM must have at least one component. A circular BOM (where a component's BOM ultimately references this product back) is rejected.

10.5 Archiving a BOM

On an Active BOM, click **Archive**. The **Archive** button is shown only while the BOM is Active — it does not appear on a Draft BOM. The BOM moves to Archived status permanently, and header and component editing controls disappear. (A Draft BOM that is never needed is simply left in Draft; archiving applies to Active BOMs, consistent with the DRAFT → ACTIVE → ARCHIVED lifecycle above.)

11. Work Orders

Navigate to **Manufacturing > Work Orders** (`/admin/work-orders`).

What a Work Order is. A Work Order (WO) is the production document that authorises the manufacture of a specified quantity of a finished product. It is to manufacturing what a Purchase Order is to procurement: a formal instruction to produce. The work order drives the full production accounting cycle — it specifies what to make, what materials to consume, and how much labour and overhead to apply, and it records the cost of everything consumed in producing the finished goods.

Why Work Orders are used. Without a Work Order, there is no formal record of what was produced, what materials were consumed, or what the finished goods cost. The business could not track whether production is efficient (planned versus actual component usage), could not correctly value the finished goods entering inventory, and could not identify variances between the standard (budgeted) cost and the actual cost. Work Orders also create the link between the BOM (the recipe) and the actual production run — allowing the system to issue the right components out of stock and receive the finished goods back into stock at their true cost.

How the Work-in-Progress (WIP) account works. During production, costs accumulate in a temporary balance sheet account called **Work-in-Progress** (WIP). When components are issued from

the storeroom to the production floor, their value moves: DR WIP / CR Inventory (components leave the warehouse, enter the production area). When labour and overhead costs are applied to the order, they also accumulate in WIP: DR WIP / CR the relevant cost account. When production is complete and the finished goods are received back into the warehouse, the accumulated WIP is relieved: DR Inventory (finished goods) / CR WIP. Any residual WIP at close (due to rounding or variance) is cleared to a Manufacturing Variance account. The net effect: raw materials enter, finished goods come out, and WIP returns to zero for a closed order.

What COGS means in manufacturing context. When a manufactured finished good is later sold, the Cost of Goods Sold (COGS) posted by the sale is the moving-average cost of the finished good — which was set when the work order was completed (WIP divided by the good quantity produced). The COGS therefore reflects the actual cost of production, not just the purchase price of raw materials.

When it is used. A Work Order is created by the production planner or manufacturing supervisor when a production run is scheduled. It is released (which locks the BOM and generates the component plan), components are issued from stock, labour and overhead are applied, the finished goods are completed and received into stock, and the order is closed. The lifecycle covers five states: PLANNED → RELEASED → IN_PROGRESS → COMPLETED → CLOSED.

A Work Order authorises the production of a specified quantity of a finished product and tracks the cost of materials, labour, and overhead consumed.

Work Order status lifecycle

```
PLANNED --> (release) --> RELEASED --> (first issue) --> IN_PROGRESS
        --> (cancel)  --> CANCELLED
                                  \--> (complete) --> COMPLETED
                                                  \--> (close) --> CLOSED
                                  \--> (cancel)   --> CANCELLED
```

CANCELLED, COMPLETED (after close), and CLOSED are terminal. A COMPLETED Work Order must be closed before any other action.

11.1 Creating a Work Order

1. On the Work Orders list, click **New Work Order**.
2. Pick the **Finished Product** from the picker.
3. Pick the **Branch** from the picker.
4. Enter the **Planned Qty**.
5. Optionally pin a specific BOM by typing its UID into the **BOM UID (optional)** field — this is a plain text input (placeholder "pin specific BOM…"), not a picker. If blank, the system uses the product's current Active BOM at release time.
6. Optionally enter a **Planned Date** and **Notes**.
7. Click **Create Work Order**.

The Work Order is created in **Planned** status with a generated Work Order number.

11.2 Editing a Work Order

A Work Order can only be edited while in Planned status. There is no Edit button — while the order is Planned, the **Edit Work Order** form is shown inline on the detail page. You can change the Planned Qty, **BOM UID (override)**, Branch, Planned Date, and Notes, then click **Save**.

11.3 Adding and removing operations

What operations are. Operations are the discrete steps in the production process — for example, Cutting, Mixing, Assembly, Finishing. Each operation can carry an estimated and actual labour cost and overhead cost, giving the business a breakdown of where production costs are incurred within the work order. Operations are optional; a work order can be costed with a single bulk labour/overhead application if step-level detail is not needed.

Operations represent discrete production steps (e.g. Cutting, Assembly) with associated labour and overhead cost estimates. They can be added to a Work Order at any status before it is Closed or Cancelled.

- **Add operation**: In the **Add Operation** form, enter Seq, Description, Work Centre, and optional Labour Amt / Overhead Amt. Click **Add**.
- **Remove operation**: Click the trash-icon button in the operation row's Actions column. An operation that has already had costs applied to it cannot be removed.

11.4 Releasing a Work Order

What releasing means. Releasing a Work Order is the act of committing to produce. At this point the system resolves and locks the BOM (so the recipe is frozen for this production run), explodes it to all leaf-level raw material components, and generates the planned component lines on the work order — the list of what will need to be issued from stock. No stock movement or GL posting happens at release; it is a planning step. Once released, the work order is ready for component issue.

Releasing a Work Order locks the BOM and generates the component plan.

1. Open a Planned Work Order.
2. In the **Release Work Order** section, optionally override the BOM by typing its UID into the **BOM UID (optional override)** field — this is a plain text input (placeholder "leave blank for active BOM"), not a picker.
3. Click **Release**.

Status changes to **Released**. The system emits a production event. No stock movements or GL entries are posted yet.

Validation. The finished product must have an Active BOM (or a BOM must be pinned). Releasing requires the `WORKORDER.RELEASE` permission.

11.5 Issuing components

What component issue means. Issuing components is the physical act of taking raw materials from the stock location and bringing them to the production area. In the system, this deducts the components from inventory and charges them to the Work-in-Progress account. The GL posting is:

DR WIP / CR Inventory for each component at its current moving-average cost. If any component has no established average cost (it has never been received or opened), the quantity deduction still posts but the WIP cost leg is skipped and the incomplete-cost flag is set on the work order — the production team should investigate and correct the missing cost.

Issuing deducts the component materials from stock and accumulates costs in the Work-in-Progress (WIP) account.

1. Open a Released or In-Progress Work Order.
2. Enter the **Posting Date**.
3. Click **Issue All Components**.

The system issues all un-issued component lines simultaneously (full issue). Status moves to **In-Progress** on the first issue.

Stock movements of type `PRODUCTION_ISSUE` are posted for each component. GL entries: DR WIP / CR Inventory.

Validation. Posting date is required. If a component's average cost is not yet established, that component is cost-skipped (the quantity still moves but no GL leg is posted). An incomplete-cost indicator appears on the Work Order header when any component was cost-skipped.

11.6 Applying labour and overhead costs

What labour and overhead costs are. Labour costs are the wages and salaries paid to the workers who produce the goods. Overhead costs are the indirect production costs that cannot be assigned to a single unit but are incurred as part of running the factory — energy, depreciation of machinery, supervision, etc. Both are debited to WIP when applied to a Work Order: **DR WIP / CR the relevant cost account**. Applying these costs ensures that the finished good's cost reflects all the inputs that went into making it, not just the raw materials.

1. Open a Released or In-Progress Work Order.
2. In the **Apply Labour / Overhead Cost** section, enter a **Labour Amount** and/or an **Overhead Amount** and a **Posting Date**.
3. Optionally link the cost to a specific operation via the Operation picker.
4. Click **Apply Cost**.

GL entries: DR WIP / CR the relevant cost account. An operation can only have costs applied to it once; a second attempt is rejected.

11.7 Completing a Work Order

What completion does. Completing a Work Order records that production has finished and the finished goods are ready to move from the production area back into the finished goods warehouse. The system computes the unit cost of the finished good as: total WIP debited divided by the good quantity produced. This computed unit cost is passed to the moving-average recompute for the finished product — so the finished good acquires its average cost through the same engine that handles purchase receipts. The GL posting is: **DR Inventory (finished goods) / CR WIP** for the value

relieved. Scrap (units produced but rejected) is recorded informationally; only good quantity enters inventory.

Completing records the finished goods receipt and calculates the unit cost.

1. Open an In-Progress Work Order.
2. In the **Complete Work Order** section, enter **Good Quantity** produced, **Scrap Quantity** (if any), and a **Posting Date**.
3. If the combined good and scrap quantities exceed the planned quantity, tick **Allow overrun**.
4. Click **Complete**.

Status changes to **Completed**. A `PRODUCTION_RECEIPT` stock movement is posted for the finished goods. The computed unit cost is the total WIP debit divided by the good quantity. GL entries: DR Finished Goods / CR WIP.

Validation. Good quantity must be positive. If good + scrap exceeds planned quantity and Allow overrun is not ticked, the submission is rejected.

11.8 Closing a Work Order

What closing does. Closing a Work Order is the final step that clears any remaining WIP balance. After completion, there may be a small residual WIP balance due to rounding or small variances between the planned and actual costs. Closing posts this residual to the **Manufacturing Variance** account — a P&L account that captures the difference between what production was expected to cost (based on the BOM and standard costs) and what it actually cost. After closing, the WIP balance for this order is zero and the order is read-only.

Closing clears any residual WIP balance (rounding or variance) and marks the order as final.

1. Open a Completed Work Order.
2. In the **Close Work Order** section, enter a **Posting Date**.
3. Click **Close**.

Status changes to **Closed**. Any residual WIP is posted to the Manufacturing Variance account. GL entries: DR or CR Manufacturing Variance / CR or DR WIP (depending on sign).

11.9 Cancelling a Work Order

A Work Order can be cancelled from Planned, Released, or In-Progress status.

1. Open the Work Order.
2. Click **Cancel** and enter an optional reason.
3. Confirm.

If components have already been issued, the system reverses all issue movements and GL entries automatically (`PRODUCTION_ISSUE_REVERSAL`). Applied labour and overhead costs are also reversed. No reversal is needed for Planned Work Orders (nothing has moved).

Why reversals are at the original issue cost. The system reverses each component issue at the exact cost it was issued at (read from the original movement record), not at the current average. This ensures the cancellation is symmetric — the books return to their exact pre-issue state with no phantom gain or loss introduced.

A Completed or Closed Work Order cannot be cancelled.

11.10 Work Order cost report

From the Work Order detail, click **Cost Report** or navigate directly to `/admin/work-orders/uid/:uid/cost-report`.

The report shows:

Section	Contents
Components	Planned vs actual component lines, quantity, unit cost, total value
Labour	Applied labour costs
Overhead	Applied overhead costs
WIP summary	Total WIP debits, total WIP credits, net WIP balance
Unit cost	Computed unit cost (WIP debit / good qty)
Variance	Residual variance (cleared at close)

An incomplete-cost indicator appears when any component was cost-skipped.

12. WIP reconciliation

Navigate to **Manufacturing > WIP Reconciliation** (`/admin/manufacturing/wip-reconciliation`). Requires the `MANUFACTURING.VIEW` permission.

What WIP reconciliation is. The WIP reconciliation report is the manufacturing equivalent of the inventory valuation report's GL reconciliation bar. It compares the total WIP balance accumulated across all open Work Orders (RELEASED, IN_PROGRESS, and COMPLETED orders that have not yet been closed) against the WIP account balance (account 1320) in the General Ledger. They must agree at all times — if they do not, it means a posting was made to the WIP account that was not recorded on a Work Order, or vice versa, which indicates a data integrity problem requiring investigation.

1. Select the **Company**.
2. The report is not loaded automatically — click **Refresh** to run it. It compares the sum of open Work Order WIP balances (the manufacturing ledger) against the WIP Inventory GL account balance (account 1320).

When the two totals agree, a green **WIP balances reconcile — computed equals expected** banner is shown and the difference row carries a **Reconciled** status tag. When they do not, a red **Finance-grade defect: WIP per work orders does not match the WIP Inventory GL balance** alert is shown and the difference row carries a **Defect** status tag — a finance review is required.

13. Frequently asked questions

Can I adjust stock below zero? Yes. The system records negative on-hand and flags the row with the Negative indicator, but it does not block the transaction. The overselling indicator is a monitoring tool; you should investigate and correct the root cause.

What is the difference between an adjustment and a stock count? An adjustment corrects a single product's quantity immediately. A stock count covers all products at a location, freezes the system quantities as a snapshot, lets you enter physical counts across multiple sessions, and only posts variances when you explicitly post the count.

Does reconciling a van change my stock or accounts? No. A van-stock reconciliation (section 6) is record-only: neither creating, saving, nor reconciling it posts any stock movement or GL entry. It is a control worksheet that documents Loaded − Sold − Returned = Expected versus the physical count on the van, and flags variances for follow-up. If a variance must correct the books, do that separately with a stock adjustment (section 2.2) or a stock count (section 5).

Where do the Loaded and Returned figures on a van reconciliation come from? They are pre-filled from the van's stock transfers for that business date — Loaded is the total transferred into the van, Returned is the total transferred back out. Both remain editable, so you can correct them if the physical reality differs. Sold is the only figure you must enter yourself; the system cannot derive it.

Can I have more than one active BOM for a product? No. Only one BOM can be active at a time per product. Activating a new version automatically archives the previous one. Historical archived versions remain visible.

Can I cancel a Work Order after it is completed? No. Once a Work Order reaches Completed status it can only be Closed. Use the Close action to clear any remaining WIP balance.

Why does the average cost change when I receive goods? The system uses a moving-average cost method. Each time goods are received, the new receipt cost is blended with the existing inventory value to produce a new weighted average: `(old value + receipt value) / (old quantity + received quantity)`. This means all units of a product always carry the same average cost, which changes with each new receipt.

What happens to COGS if a product has no average cost? If a product has never been received and has no established average cost, the system will still issue it out of stock (the quantity deducts) but it will skip the COGS GL leg and flag the anomaly. You should use the Opening Valuation screen to establish the cost before selling costed goods.

Fixed Assets

What is the Fixed Assets module? A fixed asset is a tangible item a business buys and uses over multiple years — machinery, vehicles, computers, office furniture. Unlike stock, which is sold and replaced constantly, a fixed asset sits on the company's balance sheet as long as it is in use. Because the asset is consumed gradually over its useful life, its cost is spread across accounting periods as **depreciation**: a periodic charge that reduces the asset's book value and recognises the consumption on the profit and loss account. Without a formal asset register, capital purchases get mis-coded as expenses (overstating costs and understating the balance sheet), depreciation goes unrecorded, and the financial statements do not reflect the real value of the business. The Fixed Assets module (ADR-0030) provides the register, the depreciation engine, and the GL integration that keeps the balance sheet and the profit and loss account accurate.

This chapter covers registering and managing fixed assets, running depreciation, transferring assets between branches, and disposing of or writing off assets. All screens are available from the **Finance / Fixed Assets** navigation group in the sidebar.

1. Permissions quick reference

Task	Permission code required
View asset categories	`FA.CATEGORY.VIEW`
Create / edit / archive asset categories	`FA.CATEGORY.MANAGE`
View the asset register, schedule, revaluations, depreciation runs, reconciliation	`FA.VIEW`
Register, edit, place in service, transfer assets	`FA.REGISTER.MANAGE`
Dispose, write off, or revalue an asset	`FA.DISPOSE`
Preview and post depreciation runs	`FA.DEPRECIATE`

Navigation items are hidden when the corresponding permission is absent.

2. Asset categories

Navigate to **Finance / Fixed Assets > Asset Categories** (`/admin/asset-categories`).

What is an asset category, and why does it exist? An asset category is a classification template that groups assets of the same type together — for example "Motor Vehicles", "Machinery", or "Office Furniture". It is used because assets of the same type typically depreciate at the same rate, have the same useful life, and should post to the same General Ledger (GL) accounts. Rather than setting the

depreciation method, useful life, and three GL account codes on every individual asset, you set them once on the category and every asset in that category inherits them. This ensures consistency, reduces data-entry errors, and means a change in accounting policy (such as adjusting the useful life for a class of machinery) can be applied at the category level without re-editing each asset. Before any asset can be registered the relevant category must exist.

An asset category defines the depreciation method, useful life, and GL accounts used for assets of a particular type (e.g. Machinery, Vehicles, Furniture). Categories must be set up before any asset can be registered.

The list shows each category's code, name, depreciation method, life (in periods), and status. Use **+ New Category** (top right) to add one, or **Open** on a row to view and edit it.

2.1 Creating a category

1. Click **New Category**.
2. Enter a unique **Code** (e.g. `MACH`) and a **Name** (e.g. Machinery).
3. Choose the **Depreciation Method**:

    - **Straight Line** — equal charge each period.
    - **Reducing Balance** — percentage of the remaining book value each period. Requires a **Rate % per month**, entered as a percentage (`2.5` means 2.5%). The rate is charged **every month** — it is not a yearly rate. If your policy or tax schedule gives a yearly rate, divide it by 12: 25% a year is entered as `2.0833`. (Entering `25` charges 25% of the book value every month — a 48,000,000 van would lose 12,000,000 in its first month.)

4. Enter the **Default Life Periods** — the standard useful life in accounting periods.
5. Enter the three **GL Account IDs** by their numeric identifier:

    - Asset Account (the balance-sheet asset account, e.g. 1510)
    - Accumulated Depreciation Account (the contra account, e.g. 1515)
    - Depreciation Expense Account (the P&L charge account — on a newly provisioned company this is **5600 Depreciation Expense**, the account mapped to the `DEPRECIATION_EXPENSE` GL configuration key)

6. Click **Create Category** (the button shows **Saving…** while the request is in flight).

New categories are created with status **ACTIVE** (the status is shown throughout the UI as the raw uppercase value, `ACTIVE` or `INACTIVE`).

Validation. Code must be unique within the company. Reducing Balance requires a Reducing Rate. All three GL account IDs are required.

2.2 Editing a category

Open the category detail (from the list, click **Open** on the category's row). If you have the `FA.CATEGORY.MANAGE` permission the detail screen shows the edit form directly (under the heading **Edit Category**) — there is no separate "Edit" button to click. Change the name, method, life, or

account IDs, then click **Save** (the button shows **Saving…** while in flight). The code is not editable after creation.

2.3 Archiving a category

Open the category detail and click **Archive** (this button appears, in the heading of the Edit Category card, only while the category is still active; it shows **Archiving…** while in flight). The status changes to **INACTIVE** (the UI has no separate "Archived" label — an archived category simply shows status `INACTIVE`). Inactive categories are hidden from the category dropdown on the asset-registration form. An archived category is not deleted; its history and associated assets remain.

3. Asset register

Navigate to **Finance / Fixed Assets > Fixed Assets** (`/admin/fixed-assets`).

What is the asset register? The asset register is the master list of every fixed asset the company owns. It is the single source of truth for capital investment: it records the original cost of each asset, the depreciation accumulated against it so far, and the resulting **net book value (NBV)** — the carrying value shown on the balance sheet. Every purchase of a capital item must be entered here (not coded to expense) so that the balance sheet correctly shows the asset, the profit and loss account receives only the proportionate depreciation charge each period, and the year-end accounts accurately reflect the company's capital base. The register is used by the finance team and reviewed by auditors to verify that assets exist, are in service, and are depreciated appropriately. The system keeps the register in step with the GL: every capitalisation, depreciation run, revaluation, and disposal posts a matching GL entry, and the FA-to-GL reconciliation screen (section 9) confirms the two agree.

The register lists all fixed assets for the selected company. Use the status filter to show assets by state: Draft, In Service, Disposed, or Written Off. The list is paginated; use the pager controls (first / previous / page numbers / next / last) beneath the table to move through large registers.

Each row shows the asset number, name, status, depreciation method, acquisition cost, current NBV, and acquisition date. Use **+ Register Asset** (top right) to add an asset, or **Open** on a row to view its detail.

Asset status lifecycle

```
DRAFT
  |-- (place in service) --> IN_SERVICE
                               |-- (dispose/SALE)     --> DISPOSED
                               |-- (write-off)        --> WRITTEN_OFF
                               \-- (transfer)         --> IN_SERVICE (branch changes)
```

DISPOSED and WRITTEN_OFF are terminal states.

3.1 Registering an asset

1. Navigate to **Finance / Fixed Assets > Fixed Assets** and click **Register Asset** (`/admin/fixed-assets/create`).
2. Select the **Company**.
3. Choose the **Category** from the dropdown (only Active categories are listed).
4. Pick the **Branch** from the picker (search by name).
5. Enter the **Asset Name** (e.g. Lathe #1).
6. Enter the **Acquisition Cost** (the purchase price, excluding VAT).
7. Enter the **Salvage Value** (the estimated residual value at the end of useful life; enter 0 if none).
8. Choose the **Depreciation Method** (defaults from the category, can be overridden).
9. For Reducing Balance, enter the **Rate % per month** (a monthly rate — divide a yearly rate by 12; see section 2).
10. Enter the **Life Periods** (can be overridden from the category default).
11. Enter the **Acquisition Date** and **Depreciation Start Date** (ISO format yyyy-MM-dd).
12. Optionally enter a **Location**, **Asset Tag**, and **Cost Centre ID**.
13. Click **Register Asset**.

The asset is created with status **Draft** and a system-generated asset number (e.g. `AST-0001`). No GL posting occurs at this stage.

Validation. All required fields must be present. Reducing Balance requires a Reducing Rate. Life Periods must be at least 1.

> **Acquiring an asset from a supplier bill.** When a capital item arrives through procurement, an asset can be created directly from a matched AP supplier bill line rather than re-keying the figures. This takes the bill line's net amount as the acquisition cost, registers the asset against the same company, and posts its own capitalisation journal. An asset created this way carries a **Source Bill** link on its detail screen (see section 3.3) so the audit trail back to the original purchase is preserved. This requires the `FA.REGISTER.MANAGE` permission.

3.2 Editing asset details

Non-financial fields (name, location, asset tag, cost centre) can only be edited while the asset is in **Draft** status. Open the asset detail; while the asset is in Draft (and you have `FA.REGISTER.MANAGE`) the **Edit Asset** form is shown directly on the detail screen — there is no separate "Edit" button to click. Make your changes, then click **Save Changes** (the button shows **Saving…** while in flight).

Financial fields (acquisition cost, method, life, dates) cannot be changed after the asset is registered. To correct these, you must dispose of or write off the asset and register a new one.

3.3 Viewing the asset detail

Open any asset from the list. The detail screen shows:

- Header: asset number, name, and status badge.
- A **key-metrics row** of four figures: **Acquisition Cost**, **Carrying Cost**, **Accumulated Depreciation**, and **NBV (Net Book Value)**. Carrying Cost equals the acquisition cost until the asset is revalued,

after which it diverges to reflect the revised carrying value; NBV is the carrying cost less accumulated depreciation.

- An **Asset Details** panel listing category, branch, depreciation method, life periods, dates, and (where set) salvage value, the revaluation reserve balance, location, and asset tag. If the asset was capitalised from an AP supplier bill, this panel also shows a **Source Bill** link (**View Source Bill**) to the originating bill.
- **Depreciation Schedule** (shown when In Service, Disposed or Written Off — any asset that has a schedule) — a line for each period showing the planned charge, accumulated depreciation after, NBV after, and a posted flag. Holders of `FA.VIEW` **and** `REPORT.EXPORT` see **Export PDF / Export Excel / Export CSV** buttons beside the heading; the file lists every schedule version (a revaluation starts a new version), the posted-to-date total, and the asset's current accumulated depreciation and NBV.
- **Revaluation History** (shown when revaluations exist) — every revaluation in date order with its direction, delta, carrying-before and carrying-after values, and reason.

The asset number is the human identifier shown throughout the UI. The internal identifier appears only in the browser address bar.

4. Placing an asset in service

What does "placing in service" mean? A Draft asset has been registered but not yet capitalised: it exists in the register but has no effect on the books. Placing an asset in service is the act of formally recognising it on the balance sheet — the moment the business acknowledges it owns an asset with an economic value. When you place an asset in service, two things happen simultaneously: (1) a GL journal entry is posted that moves the cost onto the Fixed Assets account (the balance-sheet impact), and (2) the full depreciation schedule is generated for the asset's entire useful life, so the system knows exactly how much to charge in each future period. The posting date must fall within an open fiscal period because it is a real accounting event. Until an asset is placed in service, it produces no depreciation and appears nowhere on the financial statements.

Placing an asset in service capitalises it: the system posts a GL entry and generates the depreciation schedule.

1. Open a **Draft** asset.
2. Click **Place in Service**.
3. Enter the **Posting Date** (must fall in an open fiscal period).
4. Confirm.

Status changes to **In Service**. A capitalisation GL entry is posted: **DR Asset Account / CR Fixed Asset Clearing account** (the dedicated clearing account configured for the company, default code 1650 — the place-in-service form labels this "CR Clearing account"). The depreciation schedule is generated for the full useful life.

Validation. Posting date is required. The fiscal period containing the posting date must be open. The action is available only on Draft assets.

5. Transferring an asset

What is an asset transfer? A transfer is a purely administrative change that moves an asset from one branch or cost centre to another — for example, when a vehicle is reassigned from the Dar es Salaam branch to the Arusha branch. It has no accounting effect: the asset's cost, accumulated depreciation, and NBV remain unchanged, and no GL entry is posted. The purpose is to keep the register accurate so that each branch's asset list reflects what is physically present there, which matters for insurance, physical verification, and cost-centre reporting.

A transfer changes the branch or cost centre of an asset without affecting its financial values. No GL entry is posted.

1. Open a **Draft** or **In Service** asset.
2. Click **Transfer**.
3. Enter the target **Branch ID** and optionally a new **Location** and **Cost Centre ID**.
4. Confirm.

The asset's branch and location are updated immediately. Disposed and Written-Off assets cannot be transferred.

> **Note.** The transfer form accepts branch and cost centre as typed numeric IDs rather than pickers. Check with your system administrator for the correct numeric IDs if you do not know them.

6. Depreciation

What is depreciation, and why is it run periodically? Depreciation is the systematic allocation of an asset's cost over its useful life. A delivery van costing TZS 24,000,000 that is expected to last 4 years does not cost the business TZS 24,000,000 in year one — it costs roughly TZS 6,000,000 per year (on the straight-line method). Recording that annual charge on the profit and loss account gives a realistic view of operating costs and ensures the balance sheet shows the asset at its current economic value, not its original price. Without running depreciation, the P&L understates costs, profits are overstated, and the balance sheet carries assets at inflated values. The system enforces one depreciation run per fiscal period per company: once a period's charges are posted, they cannot be doubled-up.

6.1 Supported methods

Method	Behaviour
Straight Line	Equal charge each period: (Acquisition Cost −

	Salvage Value) / Life Periods
Reducing Balance	Percentage of the opening book value each month: NBV × Rate % per month (the rate is monthly, not yearly)

Straight Line is simpler and produces equal charges — appropriate for assets that provide roughly equal benefit in each period (office furniture, computers). **Reducing Balance** produces a higher charge early and a lower charge later — appropriate for assets that lose value quickly in the first years of use (vehicles, plant). In both cases the final period's charge is a residual plug that ensures the asset reaches exactly its salvage value: there is no rounding drift over the asset's life.

6.2 Previewing a depreciation run

What is a depreciation run preview? A preview is a read-only simulation: it shows you exactly which assets would be charged and what amount each would attract if you were to post the run right now. No journal is posted and no data is changed. This is the recommended step before posting, because once a run is posted for a period it cannot be reversed or re-run. Reviewing the preview lets you catch anomalies — an unexpected zero charge, a newly capitalised asset you forgot to check — before they reach the books.

Before posting, preview the run to see what charges will be created.

1. Navigate to **Finance / Fixed Assets > Run Depreciation** (`/admin/depreciation-runs/post`).
2. Select the **Company**.
3. Enter the **Fiscal Period UID** for the period you want to depreciate.
4. Click **Preview**.

The preview table lists each eligible asset with its planned charge for the period, plus a total. Nothing is posted.

6.3 Posting a depreciation run

What happens when you post a depreciation run? Posting a depreciation run does three things at once: (1) it creates a `DEPR-####` run record that acts as the audit trail for the period; (2) it posts one GL journal **per branch** — each asset's depreciation is booked to the branch the asset belongs to, so each branch's profit and loss shows exactly the depreciation of that branch's assets; within each journal there is one Debit to Depreciation Expense and one Credit to Accumulated Depreciation per asset category; and (3) it marks each asset's schedule line for the period as posted and increases each asset's accumulated depreciation balance. Only one run is permitted per company per fiscal period: if a run already exists for that company and period, a second attempt is **hard-rejected** with the message *"Depreciation run already posted … Duplicate runs are not allowed"* (HTTP 409). The run is rejected, not silently returned — so always confirm a period has not already been run before posting, and use the preview step first.

After reviewing the preview:

1. Enter the **Posting Date** (must fall within the selected open fiscal period).

2. Click **Post Run**.

The system creates a depreciation run with status **Posted** and a run number (e.g. `DEPR-0001`). One GL entry is posted for each branch that has eligible assets (the run detail lists a **View GL Entry** link for each). Each asset's accumulated depreciation balance increases. The schedule lines for the period are marked as posted.

Validation. Only one depreciation run is allowed per company per fiscal period. Attempting a second run for the same period is rejected with a 409 conflict ("Duplicate runs are not allowed"); it is not a safe no-op. The fiscal period containing the posting date must also be open.

6.4 Viewing depreciation runs

Navigate to **Finance / Fixed Assets > Depreciation Runs** (`/admin/depreciation-runs`). The list shows all posted runs in reverse date order. Each row shows the run number, fiscal period, posting date, status, total charge, the number of assets covered, and when the run was executed. Click **Open** on a run to see the detail, which includes per-asset lines showing the charge amount, accumulated depreciation after the run, and NBV after the run. Use **Run Depreciation** (top right) to preview and post a new run (section 6.2–6.3).

7. Revaluing an asset

What is an asset revaluation, and when is it needed? An asset revaluation adjusts the carrying value of an asset to reflect its current fair market value, typically when an independent appraisal shows that the asset is worth significantly more or less than its book value. An upward revaluation increases the asset's carrying value on the balance sheet and creates a credit to a **Revaluation Reserve** (an equity account): the company is wealthier on paper, but the gain is deferred in equity rather than taken to income. A downward revaluation reduces the carrying value and is charged to the profit and loss account (a loss). In both cases the remaining depreciation schedule is regenerated from the new carrying value over the remaining useful life, so future depreciation charges reflect the revised base. Revaluation is done by the finance team when an appraisal indicates the book value is materially different from market value — typically at year-end or when preparing the accounts for a transaction such as a disposal or a valuation exercise.

Revaluation adjusts the carrying cost of an In Service asset to its current fair value. The depreciation schedule is regenerated after a revaluation.

1. Open an **In Service** asset.
2. Click **Revalue**.
3. Choose the **Direction**: Up or Down.
4. Enter the **Delta Amount** (the change in carrying cost, always a positive number).
5. Enter the **Revaluation Date** (the fiscal period containing this date must be open, as the revaluation posts a GL entry).
6. Enter a **Reason** (e.g. market appraisal).
7. Confirm.

For an **Up** revaluation: carrying cost increases by the delta; the revaluation reserve increases.

For a **Down** revaluation: carrying cost decreases by the delta. The delta must not reduce the carrying cost below the accumulated depreciation balance (the NBV cannot go below zero due to a revaluation).

The revaluation is recorded in the Revaluations tab. The depreciation schedule is regenerated from the new carrying cost over the remaining useful life.

8. Disposing of an asset

What is an asset disposal? A disposal is the formal removal of an asset from the register when it is sold or scrapped. When an asset leaves the business, its gross cost must be removed from the Fixed Assets account, its accumulated depreciation must be cleared from the contra account, and any difference between the proceeds received and the asset's net book value at that date is recognised as a **gain or loss on disposal** on the profit and loss account. Failing to record a disposal leaves "ghost" assets on the balance sheet — assets the company no longer owns, overstating the balance sheet and inflating accumulated depreciation. The disposal also posts any outstanding scheduled depreciation up to the disposal date, ensuring the NBV used to calculate the gain or loss is accurate.

8.1 Disposal by sale

Use this option when the asset is sold.

1. Open an **In Service** asset.
2. Click **Dispose**.
3. Enter the **Disposal Date** (must fall in an open fiscal period).
4. Enter the **Proceeds Amount** (the sale price; enter `0` if the asset is given away for nothing).
5. Enter an optional reason.
6. Confirm.

The system first posts any depreciation charges that are scheduled up to the disposal date but have not yet been posted. It then calculates the net book value at the disposal date and computes the gain or loss:

```
Gain / (Loss) = Proceeds − NBV at disposal
```

Status changes to **Disposed**. A disposal GL entry is posted. An asset can only be disposed of once.

8.2 Write-off

What is a write-off? A write-off is used when an asset is scrapped, lost, stolen, or so impaired that it has no recoverable value — so no sale proceeds are received. It is identical to a disposal by sale except the proceeds are forced to zero, meaning the entire remaining NBV becomes a loss on the profit and loss account. Common examples include equipment damaged beyond repair, assets destroyed in a fire, or obsolete technology with zero resale value.

Use this option when the asset is scrapped, lost, or fully impaired and no proceeds are received.

1. Open an **In Service** asset.
2. Click **Write Off**.
3. Enter the **Write-Off Date** and an optional reason.
4. Confirm.

The loss equals the full NBV at the write-off date (proceeds are forced to zero). The same final-period depreciation logic applies. Status changes to **Written Off**.

9. FA to GL reconciliation

What is the FA-to-GL reconciliation, and why does it matter? The reconciliation screen confirms that the asset register and the General Ledger agree. Because every capitalisation, depreciation run, revaluation, and disposal in this module posts a matching GL journal, the carrying cost of the in-service assets in the register should always equal the debit balance on the Fixed Assets GL account, and the accumulated depreciation of the in-service assets in the register should always equal the balance on the Accumulated Depreciation GL account. A discrepancy means someone has posted a manual journal directly to one of those GL accounts, bypassing the register — a data-integrity problem that must be investigated. A green **TIED** badge confirms the books are clean; a red **MISMATCH** badge is a flag for the finance team to investigate before month-end or year-end close.

Navigate to **Finance / Fixed Assets > FA Reconciliation** (`/admin/fixed-assets/reconciliation`). Requires the `FA.VIEW` permission.

1. Select the **Company**.
2. Click **Refresh** to load (or reload) the figures.
3. The report shows two cards, **Asset Cost** and **Accumulated Depreciation**, each comparing a register figure against the matching GL balance:

    - **Asset Cost** card:
        - **Register: Σ carrying_cost (IN_SERVICE)** — the sum of the *carrying cost* of all **in-service** assets in the register. This is the post-revaluation carrying value, not the original acquisition cost, and it excludes Draft, Disposed, and Written-Off assets.
        - **GL: Fixed Assets debit balance** — the debit balance on the Fixed Assets GL account.
        - **Difference** — register figure minus GL figure.
    - **Accumulated Depreciation** card:
        - **Register: Σ accumulated_depreciation (IN_SERVICE)** — the sum of accumulated depreciation across all **in-service** assets in the register.
        - **GL: Accum Dep balance (negated for positive presentation)** — the Accumulated Depreciation GL account balance, sign-flipped so it shows as a positive figure.
        - **Difference** — register figure minus GL figure.

Each card shows a green **TIED** badge when its register and GL figures agree, or a red **MISMATCH** badge when they do not. A mismatch typically indicates a manual GL journal was posted directly to an asset account, which bypasses the register.

9.1 Fixed Asset Register report

Navigate to **Finance / Fixed Assets > Fixed Asset Register** (`/admin/fixed-assets/register`). Requires `FA.VIEW`; exporting also requires `REPORT.EXPORT`.

What it shows. Every asset acquired on or before the **As at** date — asset number, name, category, branch, location, acquisition date, **cost**, **accumulated depreciation**, **NBV** and **status** — grouped by category with a **subtotal** per category and a **grand total**.

Filters. As at (required, defaults to today), and optionally **Category**, **Status**, **Branch**, **Location** (matches any part of the location text) and **Cost centre**. Click **Run report**.

How the "as at" figures are worked out.

- **Cost** is the carrying cost on that date — the acquisition cost, adjusted by any revaluation dated on or before it.
- **Accumulated depreciation** counts only the depreciation charges posted for periods on or before that date.
- **Status** is the status the asset had on that date: an asset sold in March shows as *In service* in a February register; an asset capitalised after the date shows as *Draft*.
- **NBV** is cost less accumulated depreciation for assets in service on the date. Draft, disposed and written-off assets show **no book value** (not 0.00) and are **left out of the totals** — a note under the table says how many.
- **Branch and location** are where each asset is *now* (transfers keep no history).

Agreement with the reconciliation. Run as at today (or any date after the last posting), the grand-total cost and accumulated depreciation equal the *Register* figures on the FA Reconciliation screen.

Export. Export PDF / Excel / CSV download the same rows with category subtotal lines, the grand total and the company letterhead.

If you filter by a branch you are not assigned to, the report says so: choose a branch you work in, or clear the branch filter to see the whole company.

10. Frequently asked questions

When does a GL entry get posted for a new asset? No GL entry is posted when the asset is registered (Draft). The capitalisation entry is posted when you click Place in Service.

Can I change the depreciation method after placing an asset in service? No. Method and financial parameters are fixed at registration time. If a correction is needed, dispose of the asset and register a new one.

What happens to scheduled depreciation at the time of disposal? The system automatically posts any depreciation that is scheduled and not yet posted, up to the disposal date. This ensures NBV is accurate before the gain/loss is calculated.

Can I run depreciation more than once for the same period? No. The system enforces exactly one run per company per fiscal period. A repeat attempt for a period that has already been run is **hard-rejected** with a 409 conflict ("Duplicate runs are not allowed") — it is *not* a safe idempotent no-op, so retrying after a posted run will not silently return the existing run. Use the preview function first to confirm the charges before you post.

Does a branch transfer post a GL entry? No. A transfer is a location update only and has no accounting effect.

Projects

What is the Projects module? A project (also called a job) is a discrete unit of work undertaken for a customer or for internal purposes, with its own budget and a defined scope. The Projects module gives the business a **job-costing lens**: it tags costs (materials issued, supplier bills, labour timesheets) and revenues (sales invoices) with a project identifier so that the profit or loss on each individual job can be tracked — not just the company's overall profit. Without job costing, a company knows it made a profit last month but cannot tell which jobs were profitable and which were loss-making. This module does not create a separate cost ledger; instead it adds an analytical tag on the same journal lines the financial modules already post, and the Project P&L is a filtered view of the General Ledger grouped by project. This design guarantees that the project figures always agree with the company's financial statements (ADR-0033).

This chapter covers creating and managing projects, adding tasks, recording time, issuing materials to a project, and viewing the project P&L and the cross-project WIP report. All screens are available from the **Projects** navigation group.

1. Permissions quick reference

Task	Permission code required
View the project list and project detail	`PROJECTS.PROJECT.VIEW`
Create a project	`PROJECTS.PROJECT.CREATE`
Edit a project and manage its lifecycle	`PROJECTS.PROJECT.MANAGE`
View project tasks	`PROJECTS.TASK.VIEW`
Create, edit, and deactivate tasks	`PROJECTS.TASK.MANAGE`
View timesheets for a project	`PROJECTS.TIMESHEET.VIEW`
Record a timesheet entry	`PROJECTS.TIMESHEET.RECORD`
Issue materials to a project	`PROJECTS.ISSUE.CREATE`

View the project P&L and WIP report	PROJECTS.COSTING.VIEW

Navigation items are hidden when the corresponding permission is absent. A user who has `PROJECTS.COSTING.VIEW` but not `PROJECTS.PROJECT.VIEW` sees only the WIP Report link.

2. Project lifecycle

Why does a project have a lifecycle? A project lifecycle controls what actions can be taken at each stage. A Draft project is being set up; costs and timesheets cannot yet be recorded against it. An Active project is open for work. On Hold pauses activity while still allowing ad-hoc material issues if needed. Completed and Cancelled are terminal: once a job is done or abandoned, no more costs can be added (which would distort the final profitability figure). The lifecycle exists to prevent accidental cost postings to the wrong job state and to create a clear audit trail showing when a project was open for charges.

A project passes through a defined set of statuses. The allowed transitions are:

```
DRAFT
  |-- (activate)  --> ACTIVE
  |                     |-- (hold)     --> ON_HOLD
  |                     |                     |-- (resume) --> ACTIVE
  |                     |-- (complete) --> COMPLETED (terminal)
  \-- (cancel)    --> CANCELLED (terminal)
                 (cancel also from ACTIVE, ON_HOLD)
```

- Only **Active** and **On Hold** projects accept material issues and timesheets.
- **Completed** and **Cancelled** are terminal; no further lifecycle transitions are possible.
- A project cannot be moved back to Draft from any other status.

3. Creating a project

Navigate to **Projects > Projects** (`/admin/projects`) and click **New Project**. An inline create form opens.

1. Enter the **Name** (required, up to 160 characters).
2. Optionally enter a **Budget Amount**, a **Start Date** and **End Date**, and **Notes**.
3. Click **Create Project**.

The create form has only these five fields — there is no customer, manager, or currency input at create time. The budget amount is recorded in the company's base currency.

The project is created with status **Draft** and a system-generated project number (e.g. `PRJ-0001`). A success alert shows the project number.

To set the customer, open the project detail after creation (see section 4.2). (The project manager cannot currently be set from the UI — see section 4.2.)

Validation. Project name is required. Name must not exceed 160 characters.

4. Project detail

Click the **Open** button on any project row to navigate to the project detail screen (`/admin/projects/uid/:uid`). The detail screen is divided into panels:

- **Header** — project number, name, and two status tags (the lifecycle status and the master record status), with the lifecycle action buttons.
- **Project Details** — the always-visible inline edit form (see section 4.2).
- **Tasks** — the list of tasks assigned to this project.
- **Timesheets** — paged list of time entries.
- **Issue Materials to Job** — material and cost issue panel.
- **Project P&L Report** — project profit and loss panel (requires `PROJECTS.COSTING.VIEW`).

The project number is the human identifier shown throughout the UI. The internal identifier appears only in the browser address bar.

4.1 Lifecycle actions

The lifecycle-transition buttons shown depend on the current status:

Current status	Lifecycle-transition buttons available
Draft	Activate, Cancel
Active	Hold, Complete, Cancel
On Hold	Resume, Complete, Cancel
Completed	(none — terminal)
Cancelled	(none — terminal)

Click the relevant button to apply the transition immediately. A reason is not required for any transition. The lifecycle buttons appear only when the user holds `PROJECTS.PROJECT.MANAGE`.

The table above lists only the **lifecycle-transition** buttons. The header also shows an **Archive** button independently of the lifecycle status: it appears for any project whose master status is not already **Archived** — including terminal (Completed and Cancelled) projects — whenever the user holds `PROJECTS.PROJECT.MANAGE`. See section 4.3.

4.2 Editing the project

There is no separate Edit button. The detail screen always shows an inline **Project Details** card. When the user holds `PROJECTS.PROJECT.MANAGE`, the fields are editable and a **Save Changes** button appears; otherwise the fields are read-only (disabled) and no Save button is shown.

The card contains the following fields:

- **Name**
- **Budget Amount** (in base currency)
- **Start Date** and **End Date**
- **Notes**
- **Customer (optional)** — chosen via the customer picker.
- **Manager (optional)** — chosen via the user picker (hint shows username).

Click **Save Changes** to apply.

> **Known limitation — the Manager picker does not work.** Although the **Manager** picker is shown on the form, the project manager is **never saved**. The backend ignores the submitted manager when creating or updating a project, so picking a manager has no effect and no project manager can be set from the UI today.
>
> **Known limitation — the Customer and Manager pickers do not pre-fill on load.** When you open a project, the Project Details card loads the Name, Budget, dates, and Notes, but the **Customer** and **Manager** pickers always start blank — even when the project already has a customer linked. Because saving the form sends whatever the picker currently shows, **clicking Save Changes without re-selecting the customer will clear the existing customer link.** If you only need to change another field, re-pick the customer first, or make the edit through a workflow that does not require Save. (Setting the Customer to a value and saving does persist that customer.)

4.3 Archiving a project

Click **Archive** (in the header action buttons, available when the user has `PROJECTS.PROJECT.MANAGE`) to move the project to **Archived** master status. Archived projects are hidden from the project list.

Archiving does not change the lifecycle status (a DRAFT project stays DRAFT; it is simply hidden from the list).

> **Note.** The project list has no status filter. It always shows the company's **Active**-master-status projects, so once a project is archived there is currently no way to view it again from the project list UI. (The status tag in the list reflects the lifecycle status — Draft/Active/On Hold/Completed/Cancelled — not the master status.)

5. Project tasks

What is a project task? A task is a sub-division of a project — a discrete work package within the job. Tasks allow costs and time to be recorded at a more granular level than the project as a whole. For example, a construction project might have tasks for "Foundation Works", "Structural Frame", and "Electrical Installation". When materials are issued or timesheets are recorded, they can be linked to a specific task, which lets the project manager see which parts of the job are over budget or behind

schedule. Tasks are optional: if a project is simple enough, all costs and time can be recorded against the project without specifying a task.

Tasks are managed within the **Tasks** panel on the project detail screen. There is no standalone task list screen.

5.1 Creating a task

1. In the Tasks panel, click **Add Task**.
2. Enter a **Code** (the task code, up to 30 characters, unique within the project) and a **Name** (up to 160 characters).
3. Enter optional **Planned Hours**.
4. Tick **Billable** if time spent on this task is billable to the customer.
5. Click **Create Task**.

The task is created with **Active** status.

5.2 Editing a task

Click the pencil (edit) icon on a task row. The task form reopens pre-filled. You can change the code, name, planned hours, and billable flag. Click **Update Task**.

5.3 Deactivating a task

Click the **deactivate** (x) icon on an Active task row. The task moves to **Inactive** status and disappears from the default (Active) task list. Inactive tasks are not deleted and can be viewed by filtering for Inactive tasks via the API. Deactivation is a soft operation. (The x icon is shown only on rows whose status is Active.)

6. Timesheets

What is a timesheet entry? A timesheet entry records the hours a person worked on a project on a given day. In this module, timesheet entries are **informational** rather than financial: they are stored and shown on the project but they do not post a labour cost to the General Ledger (in v1, actual labour cost reaches the project P&L through payroll journals tagged to the project, not through timesheet entries directly). Timesheets are used to track planned vs actual hours, monitor workforce utilisation, and support billing for time-and-materials projects. Entries are permanent once recorded: they cannot be edited or deleted.

Timesheets record hours worked against a project (and optionally a specific task). They are managed within the **Timesheets** panel on the project detail screen.

6.1 Recording a timesheet entry

1. In the Timesheets panel, click **Record Time**.
2. Enter the **User ID** (the numeric user identifier — ask your administrator if you do not know it).
3. Enter the **Work Date** (yyyy-MM-dd).
4. Enter the **Hours** (decimal; minimum 0.01).

5. Optionally enter a **Rate (optional)** — an informational planned rate; it is stored but does not post any GL entry.
6. Optionally pick a **Task (optional)** from the picker to link the entry to a specific task.
7. Optionally enter **Notes**.
8. Tick **Billable** if the time is billable.
9. Click **Record**.

The timesheet entry is appended to the panel list. Time entries are permanent; they cannot be edited or deleted after recording.

Validation. User ID, Work Date, and Hours are required. Hours must be greater than zero.

6.2 Viewing timesheets

The Timesheets panel lists entries with columns **Date**, **User** (shown as the user's display name), **Hours**, **Billable**, and **Notes**. Entries are shown in pages of 20; use the paginator (First, Previous, page numbers, Next, Last) to move between pages.

7. Issuing materials to a project

What is a material issue to a job? Issuing materials to a project is the act of transferring stock items from the warehouse to a specific job. When you issue materials, three things happen simultaneously: (1) the stock quantity is reduced at the current branch; (2) the stock value (based on the product's current moving-average cost) is transferred from the Inventory balance sheet account to Cost of Sales on the profit and loss account; and (3) the GL entry is tagged with the project identifier, so the cost appears in the project P&L under the "Material" cost type. This is how the cost of physical materials consumed on a job is tracked. Without issuing materials, materials pulled from the store for a job would remain as stock on the balance sheet even though they have been consumed, overstating inventory and understating job costs.

The **Issue Materials to Job** panel on the project detail screen records the issue of stock items to the project. The issue deducts stock and posts a COGS entry tagged to the project.

Materials can only be issued to **Active** or **On Hold** projects. The panel is hidden for Draft, Completed, and Cancelled projects (and also requires `PROJECTS.ISSUE.CREATE`).

7.1 Recording an issue

1. Open the project detail (status must be Active or On Hold).
2. In the Issue Materials to Job panel, click **Issue Materials**.
3. For each item line:

    - Pick the **Product** from the picker (hint shows the product code). Only GOODS (stockable) products are valid.
    - Enter the **Qty**.
    - Click **Add Line** to add another line.

4. Optionally enter an **Issue Date (optional)** and a **Reason (optional)**.
5. Click **Issue to Job**.

The system generates an issue number (e.g. `PJI-0001`). A success banner then appears in the panel showing the issue number, the **total value** issued (with its currency), and a **View COGS Entry** link that opens the posted GL journal entry.

For each line, the system:

1. Deducts the quantity from stock at the current branch.
2. Values the issue at the product's current moving-average cost.
3. Posts a GL entry: DR Cost of Sales / CR Inventory.
4. Tags the stock movement and GL entry with the project (and task if specified) dimension.

Cost-skipped lines. If a product has no established average cost, the quantity is still deducted from stock but no GL cost entry is posted. An anomaly is logged. The line appears in the issue with a value of zero.

Validation. At least one line is required. Product and quantity are required per line. Quantity is treated as a positive magnitude regardless of sign.

Example — Issue materials to a construction job and verify stock deduction:

Project manager Salma Abdallah is running project `PRJ-0007` (Kariakoo Office Fit-Out), status **Active**. The site team needs electrical cables and paint for the first week.

1. Navigate to **Projects › Projects** (`/admin/projects`), click **Open** on `PRJ-0007` to open the detail at `/admin/projects/uid/:uid`.
2. Scroll to the **Issue Materials to Job** panel. Click **Issue Materials**.
3. Add lines:
    - Product: `Electrical Cable 2.5mm (ELC-025)`, Qty: `150` (metres).
    - Product: `Interior Paint 20L (PNT-INT)`, Qty: `8` (tins).
4. Issue Date: `2026-06-10`; Reason: `Week 1 site works`. Click **Issue to Job**.
5. System generates issue `PJI-0014`. For each line:
    - 150 metres of cable deducted from stock at DSM Branch at the cable's current moving-average cost (TZS 4,200/m = TZS 630,000).
    - 8 tins of paint deducted at TZS 38,500/tin = TZS 308,000.
    - GL entries posted: DR Cost of Sales / CR Inventory, each tagged to project `PRJ-0007`.
6. Total materials issued: TZS 938,000. The project's P&L now reflects this cost under the **Material** cost type.

8. Project P&L

What is the Project P&L, and what does it show? The Project P&L (Profit and Loss) is a filtered view of the General Ledger that shows only the income and costs tagged to a single project. Revenue is the total of sales invoices tagged to the project; cost is broken down by type — Material (stock issues and goods purchases), Labour (payroll entries tagged to the project), Subcontract (service supplier bills), Overhead (other expense bills), and Other. The margin is the difference between revenue and total cost. The **WIP (Work in Progress)** figure shows how much cost has been incurred that has not yet been matched by billing: it represents work done but not yet invoiced, which sits as an asset on the balance sheet until the customer is billed. The budget variance shows whether the job is tracking above or below its planned cost. A Reconciliation bar confirms that the P&L figures are consistent with the underlying GL postings.

The **Project P&L Report** panel is shown when the user holds `PROJECTS.COSTING.VIEW`. Click **Load P&L** in the panel to run the report. It then displays as a set of KPI tiles plus a cost-by-type table and a reconciliation bar:

KPI / section	Contents
Revenue	Total income tagged to this project from GL
Total Cost	Sum of all cost lines
Margin	Revenue − Total Cost, with the margin % shown beneath (blank if no revenue)
WIP (unbooked)	max(0, Total Cost − Revenue) — unbilled cost
Budget	The planned budget set on the project
Budget Variance	Budget − Total Cost
Cost by Type	A table of subtotals per cost type (Material, Labour, Overhead, Subcontract, Other)
Reconciliation	Computed totals from the project ledger vs GL account totals

The reconciliation bar shows **Reconciliation OK** when the computed figures match the GL figures. If they do not, it shows **Reconciliation MISMATCH** with the computed-vs-GL revenue and cost figures — this indicates a system defect requiring finance/support review.

Example — View the project P&L mid-project and check the WIP balance:

Three weeks into project `PRJ-0007` (Kariakoo Office Fit-Out), Salma Abdallah wants to check profitability before the final billing.

1. Open the project detail at `/admin/projects/uid/:uid` for `PRJ-0007`.

2. In the Project P&L Report panel, click **Load P&L** (requires `PROJECTS.COSTING.VIEW`). The panel loads:

Section	Amount (TZS)
Revenue	3,500,000
Cost — Material	2,175,000
Cost — Labour	840,000
Cost — Overhead	120,000
Total Cost	3,135,000
Gross Margin	365,000
Margin %	10.4%
Budget	4,200,000
Budget Variance	+1,065,000 (cost below budget)
WIP	0 (Revenue > Cost)

The reconciliation bar shows **Reconciliation OK** — the project ledger ties to the GL account totals. Revenue of TZS 3.5M was posted via a sales invoice tagged to this project; costs include the two material issues (TZS 938,000 from week 1 + TZS 1,237,000 from week 2) plus labour timesheets. Since revenue exceeds total cost, WIP is zero. Salma notes the healthy margin and continues to the next billing milestone.

9. Cross-project WIP report

What is the WIP report, and who uses it? The WIP (Work in Progress) report is a company-wide summary that shows, for every project, how much cost has been incurred versus how much has been billed. WIP represents costs that have been spent but not yet recovered from the customer — it is an asset (money owed back to the company through future billing) and it appears on the balance sheet. Finance managers and project directors use the WIP report at month-end to understand the total unbilled exposure across all jobs, to flag jobs that are heavily over-cost relative to billing, and to support the preparation of interim billing or progress claims. A project with high WIP and low revenue may indicate that billing is overdue.

Navigate to **Projects > WIP Report** (`/admin/projects/wip-report`); the screen is titled **Cross-Project WIP Report**. Requires `PROJECTS.COSTING.VIEW`.

1. Select the **Company** (the selector is shown only when you belong to more than one company).
2. Click **Run Report**.

The report lists the company's active projects that have WIP (cost incurred greater than billed), showing:

Column	Contents
Project #	Human project number

Name	Project name
Cost Incurred	Total cost posted to the project
Billed	Total revenue or billings tagged to the project
WIP	max(0, Cost Incurred − Billed)
(Actions)	An **Open** button (eye icon) that navigates to the project detail

A footer row shows the totals (Cost Incurred, Billed, WIP) across all listed projects. When no project has WIP, the panel shows "No active projects with WIP for the selected company."

The WIP report is not paginated. All projects are shown in a single list.

10. Frequently asked questions

Can I issue materials to a project that is On Hold? Yes. Both Active and On Hold projects accept material issues and timesheet entries. Issue is blocked only for Draft, Completed, and Cancelled projects.

What happens if I complete a project with open WIP? Completing a project changes its status to Completed but does not post any accounting entries. Open WIP remains on the balance sheet until it is cleared by a billing entry or a journal.

Can I record timesheets against a Completed project? No. Timesheet recording (and material issue) requires the project to be Active or On Hold.

Why does my issue have zero value for some lines? The product had no established moving-average cost at the time of issue. The quantity was deducted from stock, but the COGS entry was skipped. Set an opening valuation for the product (see the Inventory chapter) and then record the cost via a manual GL journal for this issue.

What does the WIP balance on the P&L represent? WIP (Work in Progress) is the cost incurred on the project that has not yet been matched by revenue or billing. It represents an asset on the balance sheet — costs that are recoverable but not yet recognised as expense. Once the project is billed and revenue is recognised, the WIP reduces to zero.

Can I reopen a Completed or Cancelled project? No. These are terminal statuses. If you need to continue work on the project, create a new project and reference the original project number in the notes.

Finance & Accounting

This chapter covers every module available under the **Accounting** navigation group: General Ledger, Accounts Receivable, Accounts Payable, Cash & Bank, Tax, and Foreign Exchange (FX). The chapter is written for finance staff — accountants, AP/AR clerks, and treasury officers — who use the system day-to-day.

> **Currency fields.** Most currency fields in this chapter (receipts, opening balances, supplier bills, credit notes, and so on) use the filtered **Currency Picker** — a dropdown limited to the company's enabled currencies and pre-set to the company default, where you do not type a 3-letter currency code. This is documented once in Chapter 0 (Getting Started) → *Common UI Patterns*; this chapter only points to it. Two screens are exceptions: the AP **debit-note** modal has no currency field at all (it follows the bill's currency), and the **FX Exchange Rate** form offers the full seeded currency list (not the enabled-only allow-list) and falls back to typing a 3-letter ISO code if that list fails to load — see *Maintaining Currencies and Rates*.
>
> **Concurrency and error handling.** If two users edit the same record at once, the second save is rejected with a `409 Conflict` and a retryable "please retry" message — reload and try again. Bad input and data-integrity problems surface as clean `400`/`409`/`415` alerts, not raw server errors. These responses apply across all finance screens and are referenced throughout this chapter.

General Ledger

The **General Ledger (GL)** is the central book of record for your company's finances. Think of it as the master filing system into which every financial event — a sale, a payment, a bank transfer, a year-end adjustment — is eventually recorded as a pair of entries. Every other finance module in this system (AR, AP, Cash & Bank, VAT) feeds its financial effect into the GL. If you want to know "where does the company stand financially right now?", you read the GL; if you want to understand what produced that position, you trace back through the documents that posted to it.

The GL works on the principle of **double-entry bookkeeping**, explained below. Two prerequisites must exist before any posting can happen: a **Chart of Accounts** (the master list of ledger accounts) and at least one open **Fiscal Period** (the calendar gate that controls which dates accept entries).

Chart of Accounts

What it is. The Chart of Accounts (CoA) is the master list of all GL accounts for your company. Every financial event in the system is expressed as movements between two or more of these accounts. An account is simply a named bucket that collects amounts of a particular kind — "Cash", "Accounts Receivable", "Sales Revenue", "VAT Payable", etc.

Why it exists. Without a structured account list, the books would be an unclassified mass of transactions with no way to produce a trial balance, profit-and-loss statement, or balance sheet. The CoA is the taxonomy that turns a log of transactions into a set of readable financial statements. Each account is assigned one of five **types** (ASSET, LIABILITY, EQUITY, INCOME, EXPENSE), which determines where it appears on financial reports and what its **normal balance** is.

Understanding account types and normal balances. The five types map to the two sides of the balance sheet and the profit-and-loss statement:

Type	What it represents	Normal Balance
ASSET	Things the company owns or is owed (cash, receivables, inventory)	DEBIT
LIABILITY	Amounts the company owes to others (payables, VAT due)	CREDIT
EQUITY	The owners' stake in the business (capital, retained earnings)	CREDIT
INCOME	Revenue earned	CREDIT
EXPENSE	Costs incurred	DEBIT

A "normal balance" tells you which side — debit or credit — makes the account go up. An ASSET account increases with a debit and decreases with a credit; a LIABILITY account increases with a credit and decreases with a debit. The system derives and stores the normal balance automatically from the account type, so you never need to set it manually.

Double-entry bookkeeping in plain language. Every financial event is recorded as at least two entries — one account is debited (left side) and another is credited (right side) — and the total debits across all lines must always equal the total credits. This is the fundamental rule: **debits = credits in every transaction**. The system enforces this; the Post button is disabled until the entry balances. Why? Because a debit to one account must come from somewhere, and a credit to another must go somewhere. Money does not appear or disappear — it moves. A sale, for example, debits Accounts Receivable (the customer owes more) and credits Sales Revenue (income goes up) and VAT Payable (the tax liability goes up). The two sides always balance because they are two perspectives on the same event.

When it is used. The CoA is set up before any other finance work begins and is maintained by a user with the `GL.MANAGE` permission whenever a new account category is needed. Once created, accounts are available immediately for posting.

Navigate to **Accounting > Chart of Accounts** (`/admin/gl/accounts`).

The table shows:

Column	Meaning
Code	Unique account code assigned at creation

Name	Human-readable account name
Type	One of: **ASSET, LIABILITY, EQUITY, INCOME, EXPENSE**
Normal Balance	Derived from type — ASSET and EXPENSE accounts carry a **DEBIT** normal balance; LIABILITY, EQUITY, and INCOME accounts carry a **CREDIT** normal balance. Not user-editable.
Status	ACTIVE or INACTIVE

Control accounts and manual-posting protection. Some accounts are owned by a sub-ledger and must never be touched by a hand-entered journal. Each account therefore carries two additional, related properties:

- A **control-type classification** — one of `AR`, `AP`, `BANK`, `CASH`, `INVENTORY`, `TAX`, `PAYROLL_CLEARING`, `FX_CLEARING`, or none (an ordinary account). A non-null control type marks the account as owned by a specific sub-ledger (for example, the AR control account is the GL mirror of the receivables sub-ledger).
- An **allow manual posting** flag. When this is off, a manual journal (or a direct cash/bank entry) that targets the account is rejected. Genuine sub-ledger controls — AR, AP, INVENTORY, TAX, PAYROLL_CLEARING, FX_CLEARING — are locked this way so a stray manual entry cannot silently break the sub-ledger reconciliation. **CASH and BANK control accounts are deliberately left postable**, because bank charges, interest, and corrections legitimately need direct cash/bank entries (see Direct Cash/Bank Entries).

The practical effect is that a manual journal line (and a direct cash-entry counter account) targeting a locked control account is refused with a `409 Conflict` and a message naming the account and its control type — see *Posting a Manual Journal* below.

Required dimensions per account. An account may also be flagged to require a **cost-centre**, **department**, or **project** dimension on every *manual* journal line that posts to it. A manual line missing a required dimension is rejected, naming the account and the missing slot. System and event-driven postings (sales, AP/AR settlement, inventory, payroll, depreciation, FX, year-end close) are exempt — they do not carry operator dimension context. This per-account control is independent of the company-wide mandatory-dimension setting (see *Cost-Centre Dimensions*).

> The control-type classification, the allow-manual-posting flag, and the per-account required-dimension flags are administered on the account record. The Chart of Accounts list itself shows Code, Name, Type, Normal Balance, and Active.

To create an account (requires permission `GL.MANAGE`):

1. Click **New Account**.
2. Enter a unique **Code** and **Name**.
3. Choose the account **Type**. The system derives the normal balance automatically.

4. Click **Create**. The new account is immediately available for journal posting and GL config mapping.

> **Note:** The Chart of Accounts list does not expose an edit screen for an existing account's name or type — the only row actions are **Deactivate** and **Reactivate** (below). Editing the name or type of an account already in use is not available in the current UI.

To deactivate an account (requires `GL.MANAGE`):

1. Click **Deactivate** on the row.
2. The account becomes inactive and disappears from all posting pickers.
3. An inactive account can be reactivated by clicking **Reactivate** on its row.

> **Note:** Deactivation is soft — the account record is never deleted. No posting can be made to an inactive account (business rule BR-GL-04).

Posting a Manual Journal

What it is. A journal entry is the fundamental unit of posting: a dated, described set of two or more lines that debit and credit specific accounts in balanced amounts. A **manual journal** is one that you compose directly, as opposed to a journal that the system creates automatically (for example, when a sale is finalised or a receipt is recorded). Manual journals are used for accounting adjustments, accruals (recording an expense before the invoice arrives), prepayment amortisation, and error corrections.

Why it exists. The automated posting paths cover the main transaction types, but accountants always need a mechanism to make entries the system cannot anticipate — month-end accruals, depreciation write-downs, inter-account reclassifications, and period-end corrections. Manual journals provide this escape valve under controlled, permission-gated conditions.

When it is used. Typically at month-end by an accountant or finance manager who holds the `GL.POST` permission. Common triggers include: preparing for period close, recording a provision, or correcting a misposting discovered in review.

How it works. A manual journal posts directly (there is no draft state). You compose the lines, verify that debits equal credits, and click Post. The system validates the balance, checks that each account is active, enforces the control-account and required-dimension guards (below), and checks that the posting date falls inside an open fiscal period. If everything passes, a batch number (`JB-####`) is assigned, the entry is written to the ledger, and it is immediately immutable. Corrections are made by reversal (see below), never by editing. The journal is then visible in the journal list and feeds the trial balance.

Whatever you do here is always recorded with source type **MANUAL**. This endpoint cannot post a system source type (SALES, AR_RECEIPT, YEAR_END_CLOSE, etc.); those are produced only by their

originating modules. Because the entry is MANUAL, the control-account and per-account required-dimension guards always apply to every line:

- **Control-account guard.** A line may not target a locked sub-ledger control account — AR, AP, INVENTORY, TAX, PAYROLL_CLEARING, or FX_CLEARING. Such a line is rejected with a `409 Conflict` naming the account and its control type. (CASH and BANK control accounts remain postable.) The same guard applies whether the account is locked via its **allow manual posting** flag or via its **control-type** classification.
- **Required-dimension guard.** If a target account is flagged to require a cost-centre, department, or project dimension, a manual line missing that dimension is rejected, naming the account and the missing slot.

Navigate to **Accounting > Journals** (`/admin/gl/journals`) and click **Post journal** (`/admin/gl/journals/post`).

The Journal Entries list shows every posted batch — its batch number, posting date, description, source (MANUAL or a system source such as STOCK_RECEIPT or SALES), reference, and total debits — with a **View** action on each row and a **Post Manual Journal** button at the top right. SALES and other system entries are auto-posted and read-only; only MANUAL entries you compose here can later be reversed.

Requirements before posting (requires permission `GL.POST`):

- At least two active accounts must exist.
- The posting date must fall inside an **OPEN** fiscal period.
- Total debits must equal total credits (the entry must be balanced — business rule BR-GL-01).

Steps:

1. Set the **Posting Date** (defaults to today). Verify it falls within an open period.
2. Enter a **Description** summarising the purpose of the entry. Optionally add a **Source Reference** (e.g. a supporting document number).

    - Optionally pick a **Branch**. The journal then shows in that branch's financial statements (the statements run with that branch selected) instead of under *Company-level entries (no branch)*. Leave it at **Company level (no branch)** for entries that belong to the whole company — accruals, year-end adjustments, owner's capital. The picker lists only the branches you are assigned to; posting to any other branch is refused with *"You are not assigned to that branch, so you cannot post to it…"*. The branch you are currently switched into is **not** applied automatically — a journal is company level unless you choose a branch.

3. Each line requires exactly one of a debit or credit amount (not both — business rule BR-GL-08).

    - Use the **Account** dropdown on each line to select an account (shown as `code – name`). Only active accounts are listed.
    - Enter the **Debit** or **Credit** amount for that line, and an optional line **Memo**.

4. The form shows running **Debits**, **Credits**, and **Difference** totals and a **Balanced** indicator. The **Post Journal** button remains disabled until the difference is exactly zero.
5. Click **Post Journal**. A success message shows the generated batch number (`JB-####`). You are redirected to the journal detail page.

Adding and removing lines:

- Click **Add Line** to insert another line.
- Click the remove (trash) icon on a line to delete it. The minimum is two lines.

Validation errors surfaced by the server:

- Unbalanced entry (BR-GL-01) — total debits do not equal total credits.
- Too few lines (BR-GL-01) — a journal needs at least two lines.
- Line not one-sided (BR-GL-08) — a line carries both a debit and a credit, or neither, or a negative amount.
- Inactive account (BR-GL-04) — choose an active account.
- Wrong company account (BR-GL-05) — the account belongs to a different company.
- Closed period (BR-GL-03) — the posting date is in a closed or missing fiscal period.
- Control account rejected (`409 Conflict`) — the line targets a locked control account (AR, AP, INVENTORY, TAX, PAYROLL_CLEARING, or FX_CLEARING) or an account whose **allow manual posting** flag is off. Choose a non-control account. (CASH/BANK control accounts are exempt.)
- Missing required dimension (`409 Conflict`) — the target account requires a cost-centre, department, or project dimension and the line did not supply it.
- Wrong-base-currency line (BR-GL-06) — journal lines post in the company base currency only.
- Concurrent edit (`409 Conflict`) — another change to the same data landed first; retry the action (see *Concurrency and error handling*).

Reversing a Manual Journal

What it is. A reversal is a new journal entry that mirrors an existing one exactly but with every debit and credit swapped. The result is that the two entries cancel each other out on every account, leaving the books as if the original entry had never been made.

Why it exists. The GL is **append-only**: once a journal is posted, it cannot be edited or deleted. This is not a limitation — it is a deliberate design principle that protects the integrity of the audit trail. Any change to a posted entry would make it impossible to reconstruct what the books showed at a prior date. Reversal solves the problem by adding a counteracting entry, so the historical record shows both the original entry and the correction, and investigators can see exactly what happened.

When it is used. When you discover that a manual journal was posted to the wrong account, with the wrong amount, or in error. The reversal is initiated by the same user who posted (or any user with `GL.POST`), typically at month-end during review.

How it works. A reversal defaults to today's date (the reversal date — an explicit date can be supplied), references the original entry via a `Reversal Of` link, and posts with source type MANUAL. When a reason is supplied it is preserved in the reversal entry's description (`Reversal of entry <uid> — <reason>`). Because it is the exact swap of a balanced entry, the reversal is balanced by construction. It lands in its own open fiscal period. The original and the reversal coexist permanently in the ledger. An entry can be reversed only once: a second attempt to reverse the same entry, or an attempt to reverse a reversal entry, is refused with a `409 Conflict` (BR-GL-11).

Corrections to a posted journal are always made by **reversal** — a new entry with every line's debit and credit swapped. The ledger is append-only; the original entry is never modified.

To reverse a journal (requires `GL.POST`):

1. Open the journal detail from **Accounting > Journals**.
2. If the entry has `Source Type = MANUAL` and is not itself a reversal, the **Reverse Entry** button is visible.
3. Click **Reverse Entry**. A new journal is created immediately (defaulting to today as the reversal date) with all amounts swapped. The reversal entry links back to the original and is flagged as a **Reversal entry** on its detail page.

> System-posted entries (source types such as SALES, OPENING_BALANCE, YEAR_END_CLOSE) cannot be reversed here — the **Reverse Entry** button is shown only for MANUAL entries that are not themselves reversals. Correct system-posted entries through their originating module. Already-reversed entries and reversal entries cannot be reversed again (`409 Conflict`, BR-GL-11).

Fiscal Periods (Open/Close)

What they are. The fiscal calendar divides the financial year into monthly accounting periods. Each period has a start date, an end date, and a status (OPEN or CLOSED). A **fiscal year** groups twelve such periods.

Why they exist. Without period gates, journals could be posted with any date — including dates months or years in the past — which would silently change already-reported figures. Closing a period locks it: no new posting can land in a closed period, so the financial statements for that period are frozen once it closes. This is essential for accurate monthly reporting, auditing, and regulatory filing.

When they are used. The finance manager opens a new fiscal year once before it begins (or at system setup). Periods are closed at month-end by a user with the `GL.PERIOD.CLOSE` permission, usually after reconciliations are complete and the month's reports have been approved.

How they work. Each fiscal period covers one calendar month. A posting is accepted only when its posting date falls inside an OPEN period. The system derives which period a date belongs to

automatically. Closing a period is reversible (a period can be reopened if a late adjustment is needed); closing the entire fiscal year is a separate, more final operation (see Year-End Close below).

Navigate to **Accounting > Fiscal Periods** (`/admin/gl/periods`).

The screen shows two panels:

- **Fiscal Years** — each year with its code and current status (OPEN or CLOSED).
- **Fiscal Periods** — the twelve monthly periods within the selected year, each showing period number, date range, and status.

Opening a new fiscal year (requires `GL.MANAGE`):

1. Click **Open fiscal year**.
2. Enter a unique year code (e.g. `FY2027`), the start month (1 = January), and the calendar year.
3. Click **Open Year**. Twelve monthly periods are created, all in OPEN status.

Closing a fiscal period (requires `GL.PERIOD.CLOSE`):

1. On a period row, click **Close**.
2. The period status changes to CLOSED. No further journal postings can be made into a closed period.

Reopening a period (requires `GL.PERIOD.CLOSE`):

1. On a CLOSED period row, click **Reopen**.
2. The period returns to OPEN and accepts journal postings again.

> Closing a period is reversible. Closing a fiscal **year** is a separate, more permanent action (see Year-End Close below).

Trial Balance

What it is. The Trial Balance is a summary report that lists every GL account with its total debits, total credits, and net balance for a selected period. It is the most direct proof that double-entry has been maintained: if the system's books are correct, the grand total of all debit balances must equal the grand total of all credit balances.

Why it exists. The trial balance is the starting point for preparing financial statements (profit-and-loss, balance sheet) and for period-end review. It lets an accountant see every account's movement in one view, spot unexpected balances, and confirm that no unbalanced entries have slipped through.

When it is used. Typically at month-end review and before period close, by an accountant or finance manager holding the `GL.VIEW` permission. It can also be run at any time for a diagnostic check.

How it works. The system aggregates all journal line amounts by account, grouping them by the account type in canonical order (ASSET, LIABILITY, EQUITY, INCOME, EXPENSE). A balanced set of books shows total debits = total credits in the footer. A non-zero difference is a finance-grade defect requiring investigation.

Navigate to **Accounting > Trial Balance** (`/admin/gl/trial-balance`).

- Select your company (if multi-company).
- Optionally select a specific **fiscal period** to view only that period's movements.
- The table groups accounts by type in canonical order (ASSET, LIABILITY, EQUITY, INCOME, EXPENSE) and shows each account's code, name, total debit, total credit, and net balance.
- The footer shows total debits, total credits, and a **Balanced** indicator. A balanced set of books shows equal debits and credits.

Permission required: `GL.VIEW`.

GL Posting-Account Config

What it is. The GL Config is a mapping table that tells the system which specific account in your Chart of Accounts to use when it needs to post automatically. For example, when a sales invoice is finalised, the system needs to know which account is your "Accounts Receivable" control account and which is your "Sales Revenue" account. GL Config provides those answers.

Why it exists. Hardcoding account codes into the system software would force every business to use identical account numbers, which is impractical. GL Config externalises that mapping, letting you point each posting role to whatever account you have created in your CoA. A missing mapping fails the posting loudly (an error is raised) rather than posting to a wrong or null account.

When it is used. Set up once during initial configuration by a user with `GL.MANAGE` permission. Revisited when account restructuring changes the CoA.

How it works. Each config key represents a posting role. The system resolves the relevant key at the moment it needs to post, reads the mapped active account, and uses it as the debit or credit leg of the automatic journal. If the mapped account is inactive or the mapping is missing, the posting fails and the operator is notified to fix the mapping.

Navigate to **Accounting > GL Config** (`/admin/gl/config`). The page is headed **Posting Accounts**.

Permission required: `GL.MANAGE`.

The table shows each configuration key and the currently mapped account. The keys relevant to the core modules include:

- `ACCOUNTS_RECEIVABLE` — the AR control account
- `SALES_REVENUE` — the revenue account for sales auto-posting

- `VAT_PAYABLE` — the output VAT control account
- `CASH` — the default cash posting account
- `RETAINED_EARNINGS` — required for the year-end close

To set or change a posting account:

1. Click **Edit** on the key row (or **Add Posting Account** to map a new key). The inline form opens, titled **Set Posting Account —** followed by the config key.
2. Pick the account from the **Account** dropdown (shown as `code — name (type)`). Only active accounts are listed.
3. Click **Save**. The mapping takes effect immediately.

> All four sales keys (`ACCOUNTS_RECEIVABLE`, `SALES_REVENUE`, `VAT_PAYABLE`, `CASH`) must be configured before sales invoices can be auto-posted to the GL.

Cost-Centre Dimensions

What they are. Dimensions (also called **cost centres** or **department codes**) are analysis tags that can be attached to journal lines. They do not change which account a posting hits — the account, amount, and double-entry balance are completely unaffected. Instead, they let you slice the books by a management category: "Which department incurred this expense?", "Which cost centre drove this revenue?".

Why they exist. The main GL accounts give a company-level view of the books, but management typically needs to see performance broken down by department, branch, project, or profit centre. Dimensions provide that without multiplying the number of GL accounts (one account per department would make the CoA unmanageable). They are the analytical layer on top of the financial layer.

When they are used. Dimension values can be tagged on manual journal lines (per line) through the API, and are inherited automatically from source documents (sales invoices, supplier bills, stock adjustments). Finance or operations staff with `COSTING.MANAGE` permission maintain the dimension value master. Reporting users with `COSTING.VIEW` and `GL.VIEW` run the dimension-sliced trial balance.

> **UI limitation.** The **Post Journal** screen does not expose a cost-centre, department, or project picker on its lines — each line carries only an account, a debit or credit amount, and a memo. Per-line dimension tagging (and therefore posting to an account that requires a dimension) is currently an API/integration capability only; a manual post from the screen to a require-dimension account is rejected with no UI way to supply the value.

How they work. The system seeds two built-in dimension types: **Cost Centre** and **Department**. Alongside these, every company has two further, initially-unused dimension **slots** ("Dimension 3" and "Dimension 4") that a user with `COSTING.MANAGE` can manually claim for a custom dimension —

see *Adding a Custom Dimension* below. Whichever type a dimension slot holds, you create the actual values under it (e.g. "Sales Dept", "Nairobi Branch"). A dimension type can be made **mandatory** on manual journal entries, in which case every manually posted line must carry a value for that slot — system-automated postings (sales, year-end, etc.) are exempt. The dimension-sliced trial balance groups account balances by dimension value, giving a department-level or cost-centre-level P&L.

Navigate to **Accounting > Cost Centre > Dimensions** (`/admin/cost-centre/dimensions`).

Dimension types are pre-seeded per company: **Cost Centre** and **Department** are **built-in** (shown with a lock icon in the **Built-in** column) and can never be created, renamed, or deleted. A company also has two spare custom slots — while a slot is free, a user with `COSTING.MANAGE` can claim it with a custom dimension type of their own naming (see below). Every dimension type, built-in or custom, can only have its **mandatory** flag toggled afterwards — there is no rename or delete. Navigate to **Accounting > Cost Centre > Values** (`/admin/cost-centre/values`) to manage the actual dimension values.

Adding a custom dimension (requires `COSTING.MANAGE`):

What it is. A custom dimension is a company-defined dimension type — for example "Project" or "Region" — that claims one of the two spare slots (`DIMENSION_3`, then `DIMENSION_4`) behind the built-in Cost Centre and Department types. A company can have at most two custom dimensions.

1. On the Dimensions screen, while at least one custom slot is free, an **Add Dimension** form is shown above the dimension-types table.
2. Enter a unique **Code** and **Name** for the new dimension type, and an optional **Description**.
3. Click **Add Dimension**. The system assigns the next free slot automatically — you do not choose the slot — and the new type appears in the table, listed with its assigned slot (`DIMENSION_3` or `DIMENSION_4`) in the **Slot** column.

Once both custom slots are in use, the form is replaced by the message "Both custom dimension slots are in use. You can have at most 2 custom dimensions." If another administrator claims the last slot first (a race), submitting from an already-open form is instead rejected with a shorter inline error under the form fields: "You can have at most 2 custom dimensions."

To create a dimension value (requires `COSTING.MANAGE`):

1. Select the dimension type from the type picker.
2. Click **Add value**.
3. Enter a unique code and name. Optionally select a parent value to build a hierarchy.
4. Save.

Mandatory enforcement: if a dimension is set to mandatory, every manually posted journal line must include that dimension slot. System-posted entries (sales, year-end, etc.) are exempt.

Per-account required dimensions: independently of the company-wide mandatory setting, an individual Chart-of-Accounts account can be flagged to require a **cost-centre**, **department**, or **project** dimension (see *Chart of Accounts*). A manual journal line posting to such an account is rejected if it

omits the required dimension, naming the account and the missing slot. As with the company-wide rule, system and event-driven postings are exempt. This lets you enforce dimension tagging on a specific expense account without making the dimension mandatory across the whole company. Because the Post Journal screen has no line-level dimension picker (see the *UI limitation* note above), such a post can only be supplied through the API.

Viewing the dimension-sliced trial balance: Navigate to **Accounting > Cost Centre > Report** (`/admin/cost-centre/report`). Requires both `COSTING.VIEW` and `GL.VIEW`. Select a **Dimension slot** (Cost Centre, Department, Dimension 3, or Dimension 4 — the last two are offered whether or not the company has claimed them as a custom dimension), optionally filter to a specific value, toggle **Roll up** to include descendants, and click **Run**.

Year-End Close

What it is. The year-end close is an accounting operation performed once at the end of each fiscal year. It posts a special journal entry that transfers the net profit or loss for the year into the **Retained Earnings** account on the balance sheet, and simultaneously zeros out all income and expense accounts so they start the new year at zero. The fiscal year is then locked (CLOSED).

Why it exists. Income and expense accounts accumulate balances over the course of a year. At year-end, those balances need to be moved to equity (retained earnings) so that the new year starts fresh. Without this close, the income and expense accounts would carry over prior-year totals and the P&L for the new year would be polluted by prior-year figures. The year-end close is also the event that legally "locks the books" for the year, preventing backdated adjustments to a period whose financial statements have been approved and filed.

When it is used. Once per year, after all period 12 journals have been posted and reviewed, by a user with the `GL.YEAR.CLOSE` permission. All fiscal periods within the year must have been closed first, and prior fiscal years must already be closed (you cannot close year N if year N-1 is still open).

How it works. The system reads the net balance of every INCOME and EXPENSE account for the year, builds one balanced closing journal (source type `YEAR_END_CLOSE`), and posts it. Each income account is debited to zero and each expense account is credited to zero; the net difference (profit or loss) is posted to the Retained Earnings equity account as either a credit (profit) or a debit (loss). All periods in the year are then closed and the year status becomes CLOSED. The closing journal is visible in the journal list and is permanently linked to the fiscal year record. If the close needs to be undone, a reopen operation (available on the most-recently-closed year only) reverses the closing journal as a new append-only entry and reopens all periods.

Navigate to **Accounting > Year-End Close** (`/admin/gl/year-end`).

Permission required: `GL.YEAR.CLOSE`.

Prerequisites:

- The `RETAINED_EARNINGS` GL config key must be mapped to an active EQUITY account.
- All prior fiscal years must already be CLOSED (you cannot close year N if year N-1 is still open — business rule BR-CLOSE-04).
- The year must have at least one fiscal period.

To close a fiscal year:

1. On the year row showing OPEN, click **Close**.
2. Review the confirmation panel, which describes the retained-earnings posting that will be made.
3. Confirm. The system:
    - Posts a `YEAR_END_CLOSE` journal zeroing every INCOME and EXPENSE account.
    - Credits (net profit) or debits (net loss) the Retained Earnings account.
    - Closes all periods within the year.
    - Sets the year status to CLOSED.
4. A success message appears referencing the closing journal number.

To reopen a fiscal year (requires `GL.YEAR.CLOSE`):

Only the most-recently-closed year may be reopened. Click **Reopen** on the CLOSED year row. The system reverses the closing journal (as a new append-only entry) and reopens all periods.

Accounts Receivable

What it is. Accounts Receivable (AR) is the module that tracks money owed to your company by customers. When a credit-sale invoice is finalised, the system creates an **AR open item** — a record of the amount the customer owes. Every subsequent receipt, credit note, or write-off against that invoice is tracked here. Together these records form the **AR sub-ledger**: the customer-level detail behind a single GL control account (account 1200 Accounts Receivable).

Why it exists. The GL control account tells you the total amount owed to the company, but it does not tell you which customer owes what, how long the balance has been outstanding, or which specific invoice is unpaid. The AR sub-ledger provides that customer-level detail. It also enforces the reconciliation invariant: the sum of all open AR balances in the sub-ledger always equals the balance on account 1200 in the GL. If these two figures disagree, there is a posting error that must be investigated.

When it is used. Every time a credit sale is finalised (automatically), or whenever an AR clerk records a receipt, issues a credit note, writes off a bad debt, or loads an opening balance brought forward from a prior system.

How it works. Each open item carries an original amount (the full invoice value) and a current outstanding amount (reduced every time a receipt is allocated, a credit note is applied, or a write-off is made). The status (OPEN, PARTIAL, PAID, WRITTEN_OFF) is derived automatically from the

outstanding balance. Receipts are posted synchronously to both the AR sub-ledger and the GL in a single operation, so the two are always in agreement.

AR tracks amounts owed to your company by customers. Open items (invoices) are created automatically when a sales invoice is finalised, or manually via the opening-balance screen.

AR Invoices (Open Items)

What they are. An AR invoice (also called an **open item**) is the sub-ledger record of a specific amount a customer owes. For credit sales, open items are created automatically when the sales invoice is finalised. Opening-balance invoices can also be loaded manually to represent debts brought forward from a prior system.

Why they exist. The open item is the unit the AR module tracks through its lifecycle — from creation (OPEN) through partial payment (PARTIAL) to full settlement (PAID) or write-off (WRITTEN_OFF). All receipt allocations, credit notes, and write-offs reference the open item and reduce its outstanding balance. Without this per-invoice tracking, you could not determine which specific debts are unpaid, how old they are, or what the ageing exposure looks like.

When they are used. Created automatically on credit-sale finalisation, or manually loaded as opening balances. Viewed and managed by AR clerks and finance staff with `AR.VIEW` permission.

Navigate to **Accounting > Receivables** (`/admin/ar/invoices`).

The list shows all AR open items for the company: document number, customer name, original amount, outstanding amount, currency, invoice date, due date, and status. Each OPEN or PARTIAL row carries a **Write off** and a **Credit** action (visible to users who hold the relevant permission).

Invoice statuses:

Status	Meaning
OPEN	Unpaid; full outstanding balance remains
PARTIAL	A receipt has been applied but a balance remains
PAID	Fully settled
WRITTEN_OFF	Outstanding amount written off as uncollectable

Filtering: use the customer picker (search by name) and the status dropdown to narrow the list. The customer is selected by name — no uid is typed.

Recording a Receipt

What it is. A receipt records money received from a customer. It consists of two parts: the **cash leg** (which GL account the money went into) and the **allocation** (which open invoice or invoices the money is applied against).

Why it exists. Receiving money from a customer is a separate event from issuing the invoice, and the two must be matched (allocated) to reduce the outstanding balance. A receipt that is recorded but not allocated to any invoice is held **on account** — the customer has a credit balance but no specific invoice is settled. An automatic oldest-first allocation distributes the receipt across the customer's oldest unpaid invoices first, which is standard practice.

When it is used. By an AR clerk when a customer makes a payment — by cash, bank transfer, mobile money, or cheque. Requires the `AR.RECEIPT.RECORD` permission. The receipt triggers a GL posting immediately (DR Cash / CR Accounts Receivable).

How it works. The cash leg posts to the GL in the same transaction as the sub-ledger write, so the control account and the open-item balances are always in agreement at every committed moment. Re-allocating an existing receipt between invoices (changing which invoice the money is applied to) does NOT create a new GL posting — it is a sub-ledger-only change. The receipt amount and every allocation slice must be **positive**, and a receipt may only be allocated to invoices belonging to the **same customer** — an attempt to allocate against another customer's invoice is rejected with a `409 Conflict`. A receipt whose cheque **bounced** has been reversed: it no longer counts as money held on account, and it cannot be re-allocated (record a new receipt when the customer pays).

Navigate to **Accounting > Record Receipt** (`/admin/ar/receipts/record`). Permission required: `AR.RECEIPT.RECORD`.

1. Pick the **customer** by name in the typeahead.
2. Enter the **receipt amount**, pick the **currency** (the Currency Picker — see Chapter 0, *Common UI Patterns*), and set the **receipt date**.
3. Choose the **tender type** (Cash, Mobile Money, Bank Transfer, Cheque, Other). For mobile or bank payments, optionally enter the bank/mobile reference.
4. The customer's open invoices load in the **allocation editor**.
    - Click **Auto oldest-first** to distribute the receipt against invoices starting from the oldest outstanding.
    - Or manually enter allocation amounts against individual invoices.
    - The editor shows the receipt total, allocated total, and unallocated balance. The **Record Receipt** button is disabled if any allocation line exceeds the invoice's outstanding balance.
5. Optionally add a **WHT** amount (see WHT section below).
6. Click **Record Receipt**. The receipt is recorded and the allocated invoices update their outstanding balances. Any unallocated remainder is held **on account** (the receipt shows status UNALLOCATED or PARTIAL).

Receipt statuses:

Status	Meaning
UNALLOCATED	No invoices have been allocated
PARTIAL	Part of the receipt amount is allocated; a balance remains

ALLOCATED	The full receipt amount has been applied

WHT on receipt: if your company withholds tax from customer receipts, select the WHT type (kind = `WHT_ON_RECEIPT`) and enter the WHT amount. The GL posts: Cash DR (amount minus WHT), WHT Receivable DR (WHT amount), AR Control CR (full amount).

Viewing receipts: Navigate to **Accounting > Receipts** (`/admin/ar/receipts`). The list is paged and can be filtered by customer. Click any row to open the receipt detail, which shows the header and allocation lines.

Credit Notes

What it is. A credit note is a document that reduces the amount a customer owes. It is issued when goods are returned, when a billing error has been made, or when a discount is agreed after the fact.

Why it exists. Mistakes happen — an invoice may have been overcharged, or goods may be returned after the invoice was raised. Deleting or editing the original invoice would break the audit trail (the ledger is append-only). A credit note is the correct mechanism: it creates a new, countervailing document that reduces the outstanding balance and posts a contra entry to the GL (reversing the relevant portion of revenue and VAT).

When it is used. By a user with the `AR.CREDITNOTE` permission, initiated from the invoice list when an overcharge or return is identified.

How it works (raise then apply). A credit note has a two-stage lifecycle:

- **Raise** posts the full contra to the GL **once** (DR Sales Revenue, DR VAT Payable, CR Accounts Receivable) at the credit note's exchange rate, and sets an **unapplied amount** equal to the note total. Its status starts at **UNAPPLIED**.
- **Apply** is a sub-ledger move that reduces the chosen invoice's outstanding balance and decrements the note's unapplied amount. Apply posts nothing to the GL except a realized-FX adjustment when the settlement rate differs from the invoice rate. The note's status moves to **PARTIAL** and then **APPLIED** as the unapplied amount falls to zero. A part-applied note can be applied again — to another invoice, or to the same invoice again, in which case the amount is added to the earlier application.

When you raise a credit note directly against an invoice (the usual case from the invoices list), the system raises and immediately applies it in one step, so the invoice outstanding drops right away. Either way the credit note may only be applied to invoices belonging to the **same customer** — a cross-customer application is rejected with a `409 Conflict`. The invoice status updates automatically (OPEN, PARTIAL, or PAID depending on the remaining balance).

A credit note reduces a customer's outstanding balance. It is raised from the invoices list. Permission required: `AR.CREDITNOTE`.

1. On **Accounting > Receivables**, find the target invoice row (OPEN or PARTIAL).
2. Click **Credit note** (visible only when `AR.CREDITNOTE` is held).
3. In the **Raise Credit Note** modal, set the **Note Date**, pick the **Currency** (the Currency Picker — see Chapter 0, *Common UI Patterns*), and enter the **Net Amount**, optional **VAT Amount**, and **Reason**.
4. Click **Raise Credit Note**. The invoice outstanding is reduced and the GL contra posting is made.

Statuses: UNAPPLIED (raised, nothing applied yet), PARTIAL (some of the note applied; an unapplied balance remains), APPLIED (fully applied).

Write-Offs

What it is. A write-off removes an uncollectable balance from AR. When a debt cannot be collected — the customer has gone bankrupt, the debt has been litigated unsuccessfully, or it is simply too old to pursue — the outstanding balance is written off to a Bad Debt Expense account.

Why it exists. Carrying uncollectable balances on the books overstates the company's assets (accounts receivable) and makes financial statements misleading. A write-off acknowledges the economic reality: the money is not coming and the loss should be recognised as an expense. The audit trail is preserved — the original invoice and the write-off record coexist permanently.

When it is used. By a user with the `AR.WRITEOFF` permission, after management has decided a specific debt is uncollectable. Should not be used as a routine alternative to chasing payments.

How it works. The invoice's outstanding amount is set to zero and its status becomes WRITTEN_OFF. The GL posts DR Bad Debt Expense / CR Accounts Receivable for the written-off amount. Both the open item and the write-off record are retained for audit purposes.

A write-off removes an uncollectable balance from AR. Permission required: `AR.WRITEOFF`.

1. On **Accounting > Receivables**, find the OPEN or PARTIAL invoice.
2. Click **Write off**.
3. Enter a reason and confirm the date.
4. Submit. The invoice moves to WRITTEN_OFF status; the outstanding balance is posted to the Bad Debt Expense account.

Invoices already PAID or WRITTEN_OFF cannot be written off again.

AR Opening Balances

What it is. An opening balance is an AR invoice that represents a debt that existed before this system was put into use. When a company migrates from a prior accounting system, the outstanding customer balances that already exist need to be loaded so that the new system shows the correct receivables position from day one.

Why it exists. Without loading opening balances, the new system would show zero receivables even though customers actually owe money. Opening balances are treated as ordinary AR open items — they age, can be receipted against, and appear in customer statements — the only difference is their source is `OPENING_BALANCE` rather than `SALE`.

When it is used. Once, during system go-live or at the start of a new fiscal year, by a user with the `AR.OPENING.SET` permission.

How it works. The opening balance creates an AR invoice (source = OPENING_BALANCE) and posts a GL entry (DR Accounts Receivable / CR Opening Balance Equity) to bring the control account into agreement with the sub-ledger from the first day.

To load balances brought forward from a prior system, navigate to **Accounting > AR Opening Balance** (`/admin/ar/opening-balance`). Permission required: `AR.OPENING.SET`.

1. Pick the customer by name.
2. Enter the original amount, pick the **currency** (the Currency Picker — see Chapter 0, *Common UI Patterns*), set the invoice date, and add an optional due date and document number.
3. Click **Set Opening Balance**. An opening-balance invoice (source = `OPENING_BALANCE`) is created and posted to the AR control account.

Customer Statements and Ageing

What they are. A **customer statement** is a snapshot of a specific customer's full AR position: their outstanding invoices, recent receipts, and ageing breakdown. **Ageing** classifies outstanding balances by how many days they are overdue, providing a practical indicator of collection risk.

Why they exist. AR management is not just about recording receipts — it is about proactively chasing overdue debts. The ageing report identifies which customers are overdue and by how much, allowing the AR team to prioritise collection calls. Customer statements can also be shared with customers as a formal record of what they owe and what they have paid.

When they are used. By AR clerks and finance managers reviewing collections. The statement can be reviewed internally or shared with a customer to resolve a dispute. Requires `AR.STATEMENT.VIEW` for the statement, `AR.VIEW` for the ageing lookup.

How they work. Ageing is calculated dynamically by comparing each open invoice's due date to the current date. The system places each outstanding amount in the appropriate bucket. The balance lookup shows the net AR balance for a specific customer (open invoices minus any unallocated receipt balance).

Customer statement: Navigate to **Accounting > Customer Statement** (`/admin/ar/statement`). Permission required: `AR.STATEMENT.VIEW`. Pick a customer by name to view total outstanding, ageing breakdown, open items, and recent receipts.

Ageing buckets:

Bucket	Days Overdue
Current	0 or not yet due
1–30	1 to 30 days past due date
31–60	31 to 60 days past due date
61–90	61 to 90 days past due date
90+	More than 90 days past due date

Customer balance lookup: on the **Accounting > AR Ageing** screen (`/admin/ar/ageing`), use the balance lookup section to check a specific customer's net balance (outstanding invoices minus unallocated receipts and unapplied credit notes). Permission required: `AR.VIEW`.

The balance is in your **base currency**. A foreign-currency item counts at the rate it was booked at. **Older foreign-currency items with no reliable rate.** Items raised in another currency before multi-currency was switched on were stored with an exchange rate of 1, so their "base" value is really the foreign amount. The system does not add those into a base-currency figure. They are shown underneath, per currency ("Not included above — … USD 30.00"), and the same rule applies to the AR control-account check on the dashboard.

Printable customer statement (to send to the customer). Once a customer is picked on **Customer Statement**, a **Printable Statement** box appears. Choose **From** and **To** dates (From defaults to the first of this month, To to today; leave From empty to start from the customer's very first transaction) and click **Export PDF**, **Export Excel** or **Export CSV**. The statement carries your company letterhead (name, address, phone, TIN, VRN), the customer's name and TIN, the period and currency, then:

- **Balance brought forward** — everything the customer owed before the From date.
- Every movement in the period, oldest first: **Invoice** and **Opening balance** lines in the Debit column; **Receipt**, **Credit note** and **Write-off** lines in the Credit column; a **Reversal** line (Debit) when a customer's cheque came back unpaid — so the customer sees both "you paid" and "it bounced".
- A running **Balance** after every line, and a **Closing balance** row with the period's debit and credit totals.
- At the foot: "Amount due from customer" (or "Customer in credit"), and who printed it and when.

Which currency. The statement covers every currency the customer actually trades in, one section each — a US-dollar customer gets a USD statement, not an empty TZS one. A customer who has transactions in more than one currency gets one block per currency (the customer's own default currency first, then your base currency), each with its own balance brought forward, closing balance and "amount due" line; the amounts are never added together and there is no grand total. Each **Invoice** line's **Reference** is the sales invoice number.

Ageing export. On **AR Ageing**, users with export permission see **Export PDF / Excel / CSV** above the table: one row per customer with the five buckets and their total, and a totals row at the bottom, as at today.

Invoices in another currency. Every ageing figure is in the invoice's own currency. A customer who owes you in TZS and in USD appears on **two rows** — one per currency, each with its own **Ccy** — and the two are never added together. On the customer statement the headline **Total Outstanding** is the base-currency amount only, with any foreign-currency balance shown beside it ("and USD 500.00"), and the ageing table gets one line per currency. In the ageing export, a single currency prints exactly as before; when more than one currency is present the document gains a **Currency** column and a total line per currency instead of one grand total. Amounts are not converted to TZS: the system ages what the customer actually owes, in the currency they owe it.

Both exports need the screen's permission (`AR.STATEMENT.VIEW`) **and** `REPORT.EXPORT`. A user without `REPORT.EXPORT` does not see the buttons.

Accounts Payable

What it is. Accounts Payable (AP) is the module that tracks money your company owes to suppliers. When a supplier invoice is entered and matched against the purchase order and goods receipt, the system creates a **payable** in the AP sub-ledger. Payments against that payable are recorded here. The AP sub-ledger is the supplier-level detail behind the GL control account 2100 Accounts Payable.

Why it exists. Just as AR tracks what customers owe you, AP tracks what you owe suppliers. Without it, the company might pay the same invoice twice, miss a payment, or have no systematic way to match supplier invoices against what was ordered and received. AP also provides the first GL posting for a purchase — unlike the goods receipt (which records a stock movement only), the matched supplier bill is the point at which the purchase cost hits the books.

When it is used. By AP clerks when a supplier invoice arrives. The bill is entered, matched, and eventually paid. Requires AP permissions (by default, ORG_ADMIN holds these).

How it works. A supplier bill goes through a lifecycle: entered as DRAFT, then matched via a 3-way match (bill vs purchase order vs goods receipt). If it matches within tolerance, it posts immediately to the GL (DR Purchases / CR Accounts Payable). If there is a variance, it is held (HELD) for a finance user to review and accept. Payments reduce the outstanding balance and post the cash leg to the GL.

AP tracks amounts your company owes to suppliers. Only users with the appropriate AP permissions can access this module. By default, only the ORG_ADMIN role is granted AP permissions.

Entering a Supplier Bill

What it is. A **supplier bill** (also called an invoice from a supplier) is the formal demand for payment that a supplier sends after goods or services have been delivered. Entering the bill in this system registers it as a payable and triggers the 3-way match.

Why it exists. Entering the bill and running the 3-way match is the control that prevents your company from paying for goods it did not order, did not receive, or was charged incorrectly for. The bill is the third leg of the match: purchase order (what you ordered at what price) + goods receipt (what you actually received) + supplier bill (what the supplier says you owe). Discrepancies are surfaced as variances requiring explicit approval, not silently accepted.

When it is used. By an AP clerk when a supplier's invoice arrives, after the goods receipt has been entered. Requires `AP.BILL.ENTER` permission.

How it works (3-way match). The system compares each bill line against the corresponding purchase order line (price tolerance, default 2%) and the goods receipt line (quantity). If all lines are within tolerance, the bill moves to MATCHED and the GL posts DR Purchases (or the configured purchases account) / CR Accounts Payable. If any line exceeds tolerance, the bill moves to HELD, flagging which lines have a price or quantity variance. A user with `AP.BILL.MATCH` must review and accept each variance before the bill can match and post.

Navigate to **Accounting > Enter Bill** (`/admin/ap/supplier-bills/enter`). Permission required: `AP.BILL.ENTER`.

1. Pick the **supplier** by name in the typeahead.
2. Enter the **Supplier Invoice No.**, **Bill Date**, and **Due Date**.
3. Enter the header **VAT Amount** (0 if none), pick the **Currency** (the Currency Picker — see Chapter 0, *Common UI Patterns*), and, for a PO-matched bill, choose the **Purchase Order** (optional — leave blank for a service bill). For foreign-currency bills, an FX rate for the bill date must exist.
4. Add one or more lines. Each line is a free-text **Description**, a **Billed Qty**, a **Unit Cost**, a computed **Line Net**, and — for goods supplied against a Purchase Order — an optional **PO Line** picker that drives the 3-way match.
5. Click **Enter Bill & Match**. The system runs a **3-way match** automatically:

    - If all lines are within the price and quantity tolerance (default 2%), the bill moves to **MATCHED** and a GL posting is made (DR Purchases / CR AP Control).
    - If any line exceeds tolerance, the bill is **HELD** with a price or quantity variance flag.

    A bill can only be entered against the supplier's own purchase orders — a PO belonging to a different supplier is rejected.

    Behind the scenes. Supplier bill lines can also carry per-line VAT and a per-line GL account override (when a line carries VAT the header VAT becomes the sum of the line VAT amounts). These finer controls are available through the API; the Enter Bill screen above uses a single header VAT field and the default Purchases routing.

Accepting a variance (requires `AP.BILL.MATCH`):

On a HELD bill, each variance line shows the variance amount and percentage. Click **Accept variance** to approve the line. When all variance lines are accepted the bill moves to MATCHED and the GL posts.

Service bills (no PO): leave the PO field blank and enter free-text line descriptions.

Viewing and Navigating Bills

Navigate to **Accounting > Payables** (`/admin/ap/supplier-bills`). The list shows all bills with status, outstanding amount, and source. Click a bill number to open its detail screen, which shows the header, lines, and match result. The header carries **Enter Bill** and **Record Payment** buttons, and a HELD or DRAFT row shows a **Match** action.

Bill statuses:

Status	Meaning
DRAFT	Entered but not yet matched
MATCHED	3-way match passed; GL posted
HELD	Match variance requires acceptance
APPROVED	Explicitly approved for payment
PARTIALLY_PAID	One or more payments made; balance remains
PAID	Fully paid

Payments

What they are. A payment is the settlement of a supplier bill — transferring money from the company's cash or bank account to the supplier. Payments can be made for a single bill or as a **payment run** covering multiple bills for the same supplier.

Why they exist. The payment closes out the payable: it reduces the outstanding balance on the bill and posts the cash leg to the GL (DR Accounts Payable / CR Cash/Bank). Without recording the payment, the AP sub-ledger would continue to show amounts owed even after the supplier has been paid, and the bank/cash accounts would not reflect the outflow.

When they are used. By a user with `AP.PAYMENT.RUN` permission, typically when the company's payment schedule falls due (weekly or monthly payment runs are common). A payment run is a batch operation that pays all selected outstanding bills for a supplier in one action.

How they work. Each payment allocates a specified amount against one or more bills, reducing each bill's outstanding balance. If the payment covers the full outstanding amount, the bill moves to PAID; otherwise it becomes PARTIALLY_PAID. The GL posts immediately in the same transaction as the sub-ledger write (DR Accounts Payable / CR the chosen cash/bank account), keeping the control account and the sub-ledger in agreement.

Single-bill payment: From the **Accounting > Payments** list (`/admin/ap/payments`), use the inline pay form. Permission required: `AP.PAYMENT.RUN`.

1. Select the bill to pay (by bill number).
2. Enter the payment amount (can be partial), payment date, and tender type.
3. Submit. The GL posts DR AP Control / CR Cash.

Payment run (multiple bills): Navigate to **Accounting > Record Payment** (`/admin/ap/payments/record`).

1. Pick the supplier by name.
2. Their payable bills (MATCHED, APPROVED, or PARTIALLY_PAID) load as a checkbox list.
3. Select the bills to pay. Use **Select all** to pay all outstanding bills for that supplier.
4. Set the payment date and tender type.
5. Optionally add a **WHT on payment** amount (see WHT section below).
6. Click **Record Payment** (the button shows the count of selected bills, e.g. **Record Payment (3 bills)**). A payment run record (`PAYRUN-####`) is created covering all selected bills.

WHT on payment: select a WHT type (kind = `WHT_ON_PAYMENT`) and enter the WHT amount. The GL reduces the cash credit by the withheld amount.

Debit Notes

What it is. A debit note is a document that reduces the amount owed to a supplier. It is issued when goods are returned to the supplier, when you were overcharged, or when a credit is agreed after the bill has been matched.

Why it exists. Just as a customer credit note reduces a receivable, a debit note reduces a payable — symmetrically. The supplier has charged too much or goods have been returned, so the amount owed must be reduced. The debit note is the formal, auditable record of that reduction, posting a contra entry to the GL (DR Accounts Payable / CR Purchases).

When it is used. By a user with `AP.DEBITNOTE` permission, when a return or billing dispute is resolved after the bill has been matched.

How it works (raise then apply). A debit note mirrors the AR credit note lifecycle exactly:

- **Raise** posts the full contra to the GL **once** (DR Accounts Payable / CR Purchases, plus CR VAT Input where VAT is present) at the note's exchange rate, and sets an **unapplied amount** equal to the note total. Its status starts at **UNAPPLIED**.
- **Apply** is a sub-ledger move that reduces the chosen bill's outstanding balance and decrements the note's unapplied amount, posting only a realized-FX adjustment when the settlement rate differs from the bill rate. The note's status moves to **PARTIAL** and then **APPLIED** as the unapplied amount falls to zero. A part-applied note can be applied again — to another bill, or to the same bill again, in which case the amount is added to the earlier application.

When you raise a debit note directly against a bill (the usual case from the payables list), the system raises and immediately applies it in one step, so the bill outstanding drops right away. If the reduction brings the outstanding to zero, the bill moves to PAID.

A debit note reduces the amount owed to a supplier. Raised from the payables list. Permission required: `AP.DEBITNOTE`.

1. On **Accounting > Payables**, find a MATCHED, APPROVED, or PARTIALLY_PAID bill.
2. Click **Debit note**.
3. In the **Raise Debit Note** modal, set the **Note Date** and enter the **Net Amount**, optional **VAT**, and **Reason**.
4. Click **Raise Debit Note**. The bill outstanding is reduced and the GL posts DR AP / CR Purchases.

Statuses: UNAPPLIED (raised, nothing applied yet), PARTIAL (some of the note applied; an unapplied balance remains), APPLIED (fully applied).

AP Opening Balances

What it is. An AP opening balance is a supplier bill that represents a debt the company already owed when it started using this system — a balance brought forward from a prior system.

Why it exists. Without loading opening balances, the AP sub-ledger would show no amounts owed to suppliers on day one, even though real debts exist. Opening balances create proper payable records so that subsequent payments are correctly recorded against them.

When it is used. Once, at system go-live, by a user with `AP.OPENING.SET` permission.

Navigate to **Accounting > AP Opening Balance** (`/admin/ap/opening-balance`). Permission required: `AP.OPENING.SET`.

1. Pick the supplier by name.
2. Enter the **Gross Amount**, pick the **Currency** (the Currency Picker — see Chapter 0, *Common UI Patterns*), and set the bill date, due date, and optional supplier invoice number.
3. Click **Set Opening Balance**. An opening-balance supplier bill is created (source = `OPENING_BALANCE`).

Supplier Statement

What it is. The supplier statement shows a specific supplier's full AP position: total outstanding, ageing breakdown, open bills, and a reconciliation between the AP sub-ledger and the GL control account.

Why it exists. Supplier statements serve two purposes. First, they help AP staff track what is owed to each supplier and how overdue the balances are (useful before payment runs). Second, the **reconciliation** section compares the AP sub-ledger total against the GL 2100 Accounts Payable balance — a zero difference confirms the books are in agreement; a non-zero difference is a finance-grade discrepancy that must be investigated and corrected before period close.

When it is used. By AP clerks and finance managers before payment runs, at month-end, or when resolving a supplier query. Requires `AP.VIEW` permission.

Navigate to **Accounting > Supplier Statement** (`/admin/ap/statement`). Permission required: `AP.VIEW`.

Pick a supplier by name to view:

- **Outstanding balance** — total of unpaid bills.
- **Ageing breakdown** — same bucket structure as AR (Current, 1–30, 31–60, 61–90, 90+).
- **Open bills** — all bills with a remaining balance.
- **Reconciliation** — compares the AP sub-ledger total against the GL AP control account. A zero difference confirms the books are in agreement. A non-zero difference is a finance-grade discrepancy requiring investigation.

AP Sub-ledger vs GL Control Account. This box is company-wide, so it shows as soon as the screen opens — before you pick a supplier. It reads **Reconciled** when the supplier balances add up to the GL AP control account, or **Out by TZS X** with which side is higher (the supplier sub-ledger or the GL control account). Click **Re-check** after posting corrections. Permission: `AP.VIEW`. The supplier side is what you owe on open bills, less any payment still held on account, less any **debit note not yet applied** to a bill — a debit note reduces the GL control account in full the moment it is raised, so an unapplied one is credit you already hold against the supplier, not a difference. A payment whose cheque came back unpaid counts again as owed on both sides. The supplier's **Outstanding balance** above is worked out the same way.

Both figures are in your **base currency**; a foreign-currency bill, payment or debit note counts at the rate it was booked at. **Older foreign-currency items with no reliable rate.** Items raised in another currency before multi-currency was switched on were stored with an exchange rate of 1, so their "base" value is really the foreign amount. The system does not add those into a base-currency figure. They are listed as **Unconverted (per currency)** under the reconciliation (and as "Plus, not converted" under the supplier's outstanding balance) and are **excluded from the comparison** — so a difference shown is a real one, and the unconverted list tells you what was left out.

Printable supplier statement. Once a supplier is picked, a **Printable Statement** box appears. Choose **From** / **To** (leave From empty to start from the first transaction) and click **Statement PDF / Excel / CSV**. The statement carries your letterhead, the supplier's name, TIN and VRN, the period and currency, then the **Balance brought forward**, every movement in the period with a running balance, and the **Closing balance** (what you owe):

- **Bill** and **Opening balance** lines in the Credit column — only bills that are on the ledger (matched, approved, part-paid or paid). A bill still **HELD** for a price or quantity variance, or still a **DRAFT**, is not on the statement.
- **Payment** and **Debit note** lines in the Debit column. A payment's line says how much of it was WHT withheld and paid to TRA on the supplier's behalf. For a payment run that paid several suppliers at once, only this supplier's share is shown.
- A **Reversal** line (Credit) when a cheque to the supplier came back unpaid.

As with the customer statement, a supplier you deal with in more than one currency gets one section per currency (the supplier's default currency first), never added together.

Ageing PDF / Excel / CSV prints this supplier's five ageing buckets and the total outstanding, as at today. Bills in a foreign currency are aged in that currency on their own line (and their own total) — they are never added into the TZS figures.

Both exports need `AP.VIEW` **and** `REPORT.EXPORT`.

Cash & Bank

What it is. Cash & Bank is the module that manages the company's named money locations — petty cash boxes, tills, and bank accounts. Each cash/bank account is linked to a specific GL asset account, and every movement through the account (a receipt from a customer, a payment to a supplier, a bank transfer, or a direct entry for interest/charges) is recorded here and posted to the linked GL account in the same operation.

Why it exists. Without a dedicated cash/bank module, the company has no structured way to track the balance of individual accounts, match book records against a bank statement, or manage cheques. The key acceptance criterion is the **reconciliation invariant**: a cash/bank account's book balance must always equal its linked GL asset account balance. Because every movement posts synchronously to both the cash module and the GL, they are always in agreement at every committed moment.

When it is used. Every time money moves into or out of a named account: on recording a customer receipt, running a supplier payment, making a bank transfer, recording a bank charge or interest entry, or performing the monthly bank reconciliation.

Cash and Bank Accounts

What they are. A cash or bank account in this system represents a physical money location (a till, a petty cash box, or a bank account). Each account is linked one-to-one with a GL asset account, so the module balance and the GL balance always track together.

Why they exist. Different money locations need to be tracked separately — the head-office petty cash has a different balance from the main bank account, and the company needs to know the balance of each location independently. Linking each account to its own GL asset account (rather than all sharing a single `CASH` mapping) means the books are accurate at the location level, not just in aggregate.

When they are used. Created by a user with `CASH.ACCOUNT.MANAGE` permission during system setup or when a new physical account is opened. The default account is used as the cash leg when no specific account is selected on a payment or receipt.

Navigate to **Accounting > Cash & Bank Accounts** (`/admin/cash/accounts`). Permission required: `CASH.VIEW` to view; `CASH.ACCOUNT.MANAGE` to create or set the default.

The list shows all cash and bank accounts for the company: code, name, type (CASH or BANK), linked GL account, currency, default flag, and active status.

To create an account (requires `CASH.ACCOUNT.MANAGE`):

1. Click **New account**.
2. Enter the account name and select the account type.
    - For **BANK** accounts, also enter the bank name (required), bank account number, and branch.
3. Select the linked **GL Asset account** from the picker (only ASSET-type accounts are listed).
4. Optionally tick **Set as default account**.
5. Click **Save Account**. The account code is generated automatically.

To set the default account: click **Set default** on any non-default row.

Cash Transfers

What it is. A cash transfer moves funds from one cash or bank account to another within the company — for example, from the main bank account to the petty cash box, or between two bank accounts.

Why it exists. Physically moving cash between accounts needs to be recorded so that the book balances of both accounts update correctly and the GL reflects both the outflow from one account and the inflow to the other. Without recording the transfer, one account would show a higher balance than it actually has and the other would show a lower balance.

When it is used. By a user with `CASH.TRANSFER` permission, whenever funds are moved between two accounts. Common at month-end replenishment of petty cash or when consolidating bank account balances.

How it works. A transfer records one movement OUT of the source account and one movement IN to the destination account, and posts a single balanced GL journal (DR destination account's GL asset / CR source account's GL asset). The transfer is given a unique reference number (`CBT-####`).

To move funds between two accounts, navigate to **Accounting > Cash Transfer** (`/admin/cash/transfers/record`). Permission required: `CASH.TRANSFER`.

1. Select the **Source account** and **Destination account** from the pickers (by code — name). Source and destination must differ.
2. Enter the **amount**, **transfer date**, and an optional **reference**.
3. Click **Record Transfer**. A transfer number (`CBT-####`) is generated. The GL posts a balanced entry covering the two accounts.

View the transfers list at **Accounting > Transfers** (`/admin/cash/transfers`). Click a row to see the transfer detail.

Direct Cash/Bank Entries

What they are. A direct entry records a transaction that moves money into or out of a cash or bank account but does not originate from an AR receipt, AP payment, or inter-account transfer. The most common examples are bank interest credited by the bank, bank charges debited by the bank, and direct income receipts that bypass the AR module.

Why they exist. Not every cash movement is driven by a sales invoice or supplier bill. Bank charges, interest, returned cheque fees, and similar items are imposed by the bank and need to be recorded directly. Without direct entries, these amounts would never appear in the books and the cash account statement would not reconcile to the bank statement.

When they are used. By a user with `CASH.ENTRY.RECORD` permission, when a bank statement item cannot be matched to an AR receipt or AP payment.

How they work. The entry records the direction (IN or OUT), the amount, and a counter GL account (the other side of the double entry — typically an income, expense, or equity account). The GL is posted in the same transaction, so the cash module balance and the linked GL account balance stay in agreement.

Because a direct cash/bank entry is user-driven, its counter account is subject to the **same control-account guard as a manual journal**: a counter GL account that is a locked control account (AR, AP, INVENTORY, TAX, PAYROLL_CLEARING, or FX_CLEARING) or that has its **allow manual posting** flag off is rejected with a `409 Conflict`. Choose a non-control account. CASH and BANK accounts are exempt — that is exactly what this screen is for. The amount must be positive.

For transactions that do not originate from AP, AR, or a transfer (e.g. bank interest, bank charges), navigate to **Accounting > Cash / Bank Entry** (`/admin/cash/entries/record`). Permission required: `CASH.ENTRY.RECORD`.

1. Select the **Cash / Bank Account** by name.
2. Choose the **Direction** (IN for money received by the account, OUT for money leaving the account).
3. Enter the **Amount** and **Transaction Date**.
4. Select a **Counter GL Account** from the picker. The picker lists INCOME, EXPENSE, and EQUITY accounts; locked control accounts are excluded.
5. Enter an optional **Memo**.
6. Click **Record Entry**. A transaction number is generated and the success banner shows the direction and amount.

The entry's currency is the company base currency — there is no currency field on this screen. Direct entries appear in the account statement but are not shown in a separate list screen.

Bank Reconciliation

What it is. Bank reconciliation is the process of comparing the company's book records for a bank account against the bank's own statement. The goal is to confirm that every transaction in the books matches a transaction on the bank statement, and that the closing balance agrees.

Why it exists. The bank's records and the company's records are maintained independently and can differ for legitimate reasons (outstanding cheques not yet presented, deposits in transit, timing differences) or for error reasons (a transaction recorded in the books but not on the bank statement, or vice versa). Reconciliation surfaces those differences. Completing a reconciliation with a zero difference is a strong control that reduces the risk of fraud and ensures the bank balance on the balance sheet is accurate.

When it is used. Monthly, by a user with `CASH.RECONCILE` permission, after the bank statement for the period is received. Only BANK-type accounts can be reconciled (CASH-type accounts do not have a bank statement to match against).

How it works. The reconciliation opens with the account's uncleared book transactions. You mark each transaction as cleared when it appears on the bank statement. The system tracks the cleared book balance and computes the difference against the statement closing balance. When all matched transactions are ticked and the difference reaches zero, the reconciliation can be completed. A completed reconciliation is locked and cannot be modified.

Bank reconciliation matches your book records against your bank statement. Navigate to **Accounting > Bank Reconciliation** (`/admin/cash/reconciliations`). Permission required: `CASH.RECONCILE`.

Opening a reconciliation:

1. Select a **BANK** account (only bank accounts can be reconciled).
2. The account's uncleared transactions load.
3. Click **Open reconciliation**.

4. Enter the **Statement Date** and the **Statement Closing Balance** from your bank statement.
5. Submit. A reconciliation is opened in **DRAFT** status.

Marking transactions cleared:

1. Tick the **Cleared** checkbox against each transaction that appears on the bank statement.
2. The **Cleared book balance** and **Difference** update in real time.

    - Difference = cleared book balance − statement closing balance.

Completing the reconciliation:

1. When the difference is exactly zero, the **Complete** button becomes active.
2. Click **Complete**. The reconciliation moves to **COMPLETED** status and is locked.

A completed reconciliation cannot be modified. The difference must be zero to complete — a non-zero difference means there are unidentified items on either side.

Cheque Register

What it is. The cheque register tracks cheques that have been written and issued against a bank account. Each cheque goes through a simple lifecycle: ISSUED (written and handed out), CLEARED (presented to the bank and cleared), or CANCELLED (voided or stopped).

Why it exists. Issued cheques that have not yet cleared the bank are "outstanding cheques" — they are not on the bank statement yet but are a real liability. Tracking them in the register means they can be identified during bank reconciliation as legitimate outstanding items rather than unexplained differences. Cancelled cheques provide an audit trail of voided instruments.

When it is used. By a user with `CHEQUE.MANAGE` permission whenever a cheque is written. Status is updated when the cheque clears or is cancelled.

Track issued cheques at **Accounting > Cheques** (`/admin/cash/cheques`). Permission required: `CHEQUE.MANAGE`.

Registering a cheque:

1. Click **Register cheque**.
2. Select the **BANK account** (only bank accounts issue cheques).
3. Enter the cheque number, payee, amount, issue date, and value date.
4. Click **Register**. The cheque is recorded with status **ISSUED**.

Cheque lifecycle:

- ISSUED — cheque has been written and handed out.

- Click **Clear** when the cheque has been presented and cleared the bank → status becomes **CLEARED**.
- Click **Cancel** if the cheque is lost, stopped, or voided → status becomes **CANCELLED**.

CLEARED and CANCELLED are terminal states; no further transitions are possible.

> **Behind the scenes.** The cheque model also supports inbound (customer) cheques with a direction flag and a deposit/bounce flow (a bounce posts a reversing GL entry and restores the related receipt's outstanding balance). These inbound actions are available through the API; the Cheque Register screen above currently registers outbound cheques and exposes only Clear and Cancel.

Cash Account Statement

What it is. The cash account statement shows the transaction history of a cash or bank account with a running balance. It also shows a GL reconciliation — a comparison between the account's book balance and the linked GL asset account balance.

Why it exists. A running statement lets treasury staff see the account's full activity in date order, trace individual transactions, and confirm that the cash module and the GL are in agreement. A non-zero GL reconciliation difference is a posting anomaly requiring investigation.

When it is used. By finance staff with `CASH.VIEW` permission, during daily cash management or at month-end review.

Navigate to **Accounting > Cash Statement** (`/admin/cash/statement`). Permission required: `CASH.VIEW`.

Select an account by name to view:

- **Current balance** — the running book balance.
- **Transaction history** — each cash transaction in date order with a running balance column (IN transactions increase the balance; OUT transactions decrease it).
- **GL reconciliation** — compares the account's book balance against the linked GL asset account balance. A zero difference confirms agreement. A non-zero difference requires investigation.

Printing the account statement. Above the balance, choose **From** and **To** and click **Export PDF**, **Export Excel** or **Export CSV**. The statement carries your letterhead, the account (code, name, bank, branch, account number), the period and currency, the **Balance brought forward** on the From date, every transaction in the period with **Money In**, **Money Out** and a running **Balance**, and a **Closing balance** row. For a period that ended before today, the foot also gives today's book balance, so the two are never confused. Leave both dates empty to print the whole history. Permission: `CASH.VIEW` **and** `REPORT.EXPORT`.

End-of-Day Cash Count

What it is. An end-of-day cash count is a reconciliation of a physical **till** (a CASH-type cash/bank account) against its book balance. The cashier counts the drawer note-by-note and coin-by-coin, the system compares that counted total against the amount the books say should be there (the **expected** balance), and any **over** or **short** difference is posted to the ledger.

Why it exists. A till's book balance is only as trustworthy as the physical cash behind it. Counting the drawer at the end of the day catches theft, miscounts at the counter, and un-rung sales while they are still traceable. Posting the over/short variance to the GL means the cash account on the balance sheet always matches the money actually in the drawer, and the loss or gain is recognised in the period it happened.

When it is used. At the close of each business day (or each shift), by a cashier or supervisor with the `CASH.COUNT.MANAGE` permission. Viewing past counts requires `CASH.COUNT.VIEW`.

How it works. A count moves through three states — **OPEN** (started; the expected balance has been captured), **COUNTED** (the denomination breakdown has been entered and the counted total and variance computed), and **RECONCILED** (finalised; any variance posted to the GL and the count locked). Only CASH-type accounts can be counted — a bank account has no physical drawer. When the count is reconciled, an over posts **DR till cash / CR Cash Over (income)** and a short posts **DR Cash Short (expense) / CR till cash**, and a matching cash-book entry is written so the till's cash book and its GL account always move together to the counted figure. A count with **zero variance posts no journal** — nothing needs correcting. A count is locked once reconciled and can never be re-opened or re-counted.

Viewing counts. Navigate to **Accounting > Cash Counts** (`/admin/cash/counts`). Because counts are held per till, pick the **company** (if multi-company) and then the **till** from the selectors; the table then lists that till's counts with count number, business date, expected, counted, variance (green for over, red for short), and status. Click the eye icon to open a count.

Opening a count (requires `CASH.COUNT.MANAGE`):

1. From the Cash Counts list, click **New Count** (or go to `/admin/cash/counts/new`).
2. Choose the **company** (if you belong to more than one), the **Till (Cash Account)** — only CASH-type accounts are listed — and the **Business Date** (defaults to today).
3. Click **Open Count**. The system records the till's **expected** book balance as at that date and opens the count in **OPEN** status. You are taken to the count workspace.

> Only one live count per till per day. If an OPEN or COUNTED count already exists for that till and date, opening another is rejected — finish or reconcile the first.

Entering the count and saving:

1. In the count workspace, the top shows three figures: **Expected** (the book balance), **Counted** (the live total of what you have entered), and **Variance** with an **Over** / **Short** / **Balanced** tag.
2. In the **denomination grid**, enter the **quantity** of each note and coin you counted (the ladder is 10,000 / 5,000 / 2,000 / 1,000 / 500 / 200 / 100 / 50). The line amount and the running counted total update as you type.
3. Click **Save Count**. The count moves to **COUNTED** and the server records the counted total and the variance. You can re-save as many times as you need while the count is not yet reconciled.

Reconciling (posting the variance):

1. Once the count is COUNTED, click **Reconcile**.
2. The count moves to **RECONCILED** and locks. If there is an over or short, the variance is posted to the GL and a **View GL Entry** link appears; a zero-variance count simply locks with no posting.

> The denomination ladder is fixed to the standard TZS notes and coins in this release — a configurable per-currency ladder is a planned follow-up.

Petty Cash

What it is. A petty cash fund is a small **imprest float** — a fixed amount of cash held by a named custodian to pay for minor day-to-day expenses (taxi fares, tea, small stationery) that do not justify a cheque or bank payment. This screen tracks each fund's float, its current balance, and every disbursement and replenishment against it.

Why it exists. Small cash payments still need a record. Without a petty cash ledger, these amounts leave no trail, the custodian cannot be held accountable for the float, and the balance on hand can drift with no way to check it. Tracking each movement keeps the custodian's cash box auditable and shows at a glance how much of the float has been spent and needs replenishing.

When it is used. By a user with the `PETTY_CASH.MANAGE` permission to create funds and record movements; `PETTY_CASH.VIEW` to view. Movements are recorded as they happen — a disbursement when cash is paid out, a replenishment when the float is topped back up.

How it works. Each fund has a **float** (the authorised ceiling) and a **balance** (the cash actually on hand, which starts at zero and moves only through recorded transactions). Three transaction types move the balance: a **Disbursement** decreases it (cash paid out), a **Replenishment** increases it (float topped back up), and an **Adjustment** is a signed correction (a positive amount increases the balance, a negative amount decreases it). The system refuses any transaction that would push the balance below zero. The float is informational — it does not hard-block a disbursement, it simply tells you the ceiling the fund should be topped up to.

> **Record-only.** Petty cash movements are **not posted to the general ledger** in this release. Even the optional expense GL account on a disbursement is captured for reference only — it

> does not create a journal. Reflect petty cash spending in the GL through a manual journal or the periodic replenishment payment.

Viewing funds. Navigate to **Accounting > Petty Cash Funds** (`/admin/petty-cash/funds`). Pick the **company** (if multi-company); the table lists each fund's code, name, custodian, float, current balance, and status (ACTIVE / INACTIVE / ARCHIVED). Click the eye icon to open a fund.

Creating a fund (requires `PETTY_CASH.MANAGE`):

1. From the Petty Cash Funds list, click **New Fund** (or go to `/admin/petty-cash/funds/new`).
2. Enter a unique **Fund Code** (e.g. `PCF-001`) and a **Fund Name** (e.g. "Front Office Petty Cash").
3. Optionally select a **Custodian** (a user in the company) who is responsible for the float.
4. Enter the **Float Amount** (the authorised ceiling) and the **Currency** (defaults to the company base currency).
5. Click **Create Fund**. The fund opens with a balance of zero — record a replenishment to put the opening cash in.

> The company must already have at least one branch before a petty cash fund can be created.

Recording a transaction (requires `PETTY_CASH.MANAGE`):

On the fund detail screen, use the **Record Transaction** panel:

1. Choose the **Type**: **Disbursement** (cash out), **Replenishment** (cash in), or **Adjustment** (a correction).
2. Enter the **Amount** (must be positive for a disbursement or replenishment; for an adjustment, enter a negative amount to decrease the balance) and the **Date**.
3. For a **Disbursement**, optionally pick the **Expense GL Account** the spend relates to (captured for the record only — see the record-only note above).
4. Optionally add a **Reference** and a **Description**.
5. Click **Record Transaction**. The fund balance updates immediately and the movement appears in the **Transaction Ledger** below, showing the transaction number, type, amount, and the running balance after each entry.

> A transaction that would take the balance below zero is rejected. Transactions can be recorded only against an **ACTIVE** fund.

Tax

What it is. The Tax module covers two statutory obligations: the monthly **VAT return** (filed with TRA) and the **WHT (Withholding Tax) register**. Both work from the same underlying transaction data — sales invoices for output VAT, supplier bills for input VAT, and AR/AP payment legs for WHT — but they are separate filings with separate regulatory purposes.

Why it exists. Tanzania (and most countries) requires businesses to collect VAT on sales (output VAT), claim VAT paid on qualifying purchases (input VAT), and remit the net difference to the revenue authority monthly. Without a VAT return module, the company would need to aggregate these figures manually from the GL each month, increasing the risk of error and late filing. The WHT register similarly provides the structured record needed for regulatory compliance.

VAT Returns

What it is. A VAT return is a monthly declaration to TRA of your company's output VAT (collected from customers on sales), input VAT (paid to suppliers on qualifying purchases), and the net amount due (output minus input). A positive net is remitted to TRA; a negative net (input exceeds output) is carried forward as a credit.

Why it exists. VAT is a pass-through tax: the company collects it from customers on behalf of TRA and may recover it from TRA on qualifying business purchases. Without a monthly return, the company has no formal mechanism to net these obligations, report them to TRA, or settle the balance. The return module automates the computation from the system's existing sales and purchase records, produces the net figure, handles manual adjustments for exceptional items, and locks the return once filed to create an auditable record.

When it is used. By a user with `VAT.RETURN.PREPARE` permission each month, after all sales invoices and supplier bills for the period have been entered. The return must be filed with `VAT.RETURN.FILE` permission once ready.

How it works. Output VAT (on account 2200 VAT Payable) accumulates continuously as sales are finalised; input VAT (on the VAT_INPUT control account) accumulates as supplier bills are matched. The return computes both figures from the period's documents, and it already nets the corrections the system knows about:

- **Output VAT** = VAT on sales invoices finalised in the period (including any voided later) − VAT on sales voided in the period − VAT on customer credit notes raised in the period (including sales returns).
- **Input VAT** = VAT on supplier bills in the period − VAT on supplier debit notes raised in the period (including purchase returns).

Each figure is converted to TZS exactly as its own ledger posting was, so filing the return clears VAT Payable and VAT Input to zero. Any prior-period credit is carried forward from the last FILED return. Manual adjustments are only for items that are **not** documents in the system, such as bad debt relief or a prior-period correction. Filing locks the return (FILED), posts a settlement journal to clear both control accounts (and each adjustment) to a dedicated VAT_DUE liability, and records the TRA filing reference.

> **Do not key credit-note or debit-note VAT by hand.** Credit notes, supplier debit notes, sales and purchase returns and voided sales recorded in the system are already in the Output VAT

> and Input VAT figures. Adding a **Credit Note VAT** or **Debit Note VAT** adjustment for them counts them twice. If a DRAFT return already carries such an adjustment for a document in the system, remove it and click **Recompute** before filing.

Navigate to **Accounting > Tax > VAT Returns** (`/admin/tax/vat-returns`). Permission required: `VAT.VIEW`.

The list shows all VAT returns for the company with their return number, period, due date, status, output VAT, input VAT, net VAT, and a result flag (Payable, Credit c/f, or Nil), plus a **New VAT Return** button.

VAT return statuses:

Status	Meaning
DRAFT	Being prepared; editable
FILED	Submitted to TRA; locked

Opening a new VAT return (requires `VAT.RETURN.PREPARE`):

1. Click **New VAT Return**.
2. Select the **year** and **month**.
3. Submit. The system creates a DRAFT return for the period and performs an initial computation. The opening credit (if any) is carried forward from the most recent FILED prior-period return.

VAT return detail: click a return row to open its detail. The detail screen shows:

- **Turnover by rate band** — a table with one row per VAT band showing the **Taxable Base** (the net sales value in that band) and the **Output VAT** on it. All three bands are always listed — **Standard** (18%), **Zero-Rated**, and **Exempt** — with zero shown where a band had no sales in the period, so the breakdown is complete and reconciles to the total every month rather than dropping bands that happened to be empty. A **Total** row sums the taxable base to the **Sales Turnover** and the VAT to the **Output VAT**.
- **Sales Turnover** — the period's total net sales, with **of which Zero-Rated Sales** and **of which Exempt Sales** shown beneath it so the non-standard-rated portion of turnover is visible at a glance (both are subsets of Sales Turnover, not additional amounts). **Purchases Turnover** is shown alongside.
- **Input VAT** — VAT paid on purchases.
- **Manual Adjustments** — optional signed adjustment lines (see below).
- **Opening Credit b/f** — carry-forward from the prior FILED return.
- **Net VAT** — output VAT − input VAT + adjustments − opening credit.
- The net label shows **"Payable to TRA"** (net > 0), **"Credit carried forward"** (net < 0), or **"Nil"** (net = 0).

> Zero-rated and exempt sales carry **no** output VAT (their VAT column is nil) but they **do** count toward turnover — which is why they appear as their own bands and as the "of which" lines.

> This matters for the return: turnover is reported in full even though only the standard band contributes VAT.

Recomputing a DRAFT return (requires `VAT.RETURN.PREPARE`):

Click **Recompute** on the detail screen to re-read the current sales and purchase figures. This is useful after new invoices or bills have been entered for the period.

Printing the return (Export PDF / Excel / CSV). On the VAT return detail, users with `VAT.VIEW` **and** `REPORT.EXPORT` see three export buttons at the top. The document prints the return face as the screen shows it — supplies by tax band (taxable value and VAT), total sales turnover and output VAT, the zero-rated and exempt "of which" lines, purchases turnover, input VAT, adjustments, credit brought forward, and **Net VAT** (Payable to TRA / Credit carried forward / Nil) — under your company letterhead with its **TIN and VRN**. A filed return prints its filing date and TRA reference; a **DRAFT** prints "DRAFT — not yet filed; figures may still change", so it cannot pass for the filed return. Purchases turnover prints blank (not 0.00) when it was not computed. Any penalty or interest recorded is printed at the foot, marked as not included in Net VAT.

VAT Adjustments

What they are. A VAT adjustment is a signed correction line added to a DRAFT VAT return to account for items that do not flow through the standard sales or purchase figures — for example, VAT relief on a bad debt that has been written off, VAT corrections for prior-period errors, or the VAT on a credit or debit note that was issued **outside the system** (on paper, or in another system). Credit notes, supplier debit notes, returns and voids recorded in the system are already in the return (see *VAT Returns* above) and must not be added again as adjustments.

Why they exist. Not every VAT correction can be handled by recomputing the sales and purchase figures. TRA rules allow for specific adjustment types (bad debt relief, prior-period corrections, credit/debit note VAT) to be reflected in the return as signed adjustment lines, each with an identifiable reason and narrative.

When they are used. By a user with `VAT.ADJUST` permission, on a DRAFT return, when a specific regulatory adjustment is identified before filing.

Adjustments can be added to a DRAFT return to correct prior-period errors or to reflect the VAT on credit/debit notes issued outside the system. Permission required: `VAT.ADJUST`.

To add an adjustment:

1. On the DRAFT return detail, click **Add Adjustment**.
2. Choose the **Reason**:

    - Bad Debt Relief
    - Prior Period Correction

- Credit Note VAT — only for a credit note issued outside the system
- Debit Note VAT — only for a debit note issued outside the system
- Other

3. Choose the **Effect** (Increase VAT or Decrease VAT).
4. Enter a positive **Amount** and an optional narrative.
5. Submit. The net VAT recalculates immediately. When the return is filed, each adjustment is posted to the ledger with the settlement journal: Credit Note VAT to VAT Payable, Debit Note VAT to VAT Input, Bad Debt Relief to Bad Debt Expense, and a Prior Period Correction or Other adjustment to VAT Payable (increase) or VAT Input (decrease).

To remove an adjustment, click the remove icon on the adjustment row. Adjustments cannot be added or removed from a FILED return.

Filing a VAT Return

What it is. Filing is the act of submitting the prepared VAT return to TRA and locking it in the system. Filing is irreversible — a filed return cannot be edited.

Why it exists. Filing separates preparation (a workflow step, editable) from submission (a regulatory commitment, locked). Once filed, the return is a permanent record with a TRA reference number. The GL settlement journal it posts clears the VAT control accounts, so the new period starts with only that period's VAT movements on the accounts.

When it is used. By a user with `VAT.RETURN.FILE` permission, after the return has been reviewed, any adjustments added, and the amount payable confirmed. All prior-period returns must be FILED before the current one can be filed.

How it works. Filing runs a final recompute, posts the settlement journal (DR VAT_PAYABLE output amount / CR VAT_INPUT input amount / one line per adjustment / net to VAT_DUE), records the TRA filing reference and date, and sets the return status to FILED. VAT_DUE therefore moves by exactly the filed net amount.

Filing locks the return and posts the settlement journal to the GL. Permission required: `VAT.RETURN.FILE`.

Requirements:

- The return must be in DRAFT status.
- All prior-period returns for the same company must be FILED (you cannot file period N while period N-1 is still DRAFT).
- The GL config keys `VAT_PAYABLE`, `VAT_INPUT`, and `VAT_DUE` must be mapped to active accounts.

Steps:

1. On the DRAFT return detail, click **File Return**.
2. Enter the **TRA Filing Reference** and **Filing Date**.
3. Click **Confirm File**. The system:

    - Runs a final recompute.
    - Posts a settlement journal (DR VAT_PAYABLE output amount / CR VAT_INPUT input amount / balancing leg to VAT_DUE).
    - Sets the return to FILED and records the filing date and reference.
    - Shows a link to the posted journal.

A nil-activity return (output and input both zero) files and locks without posting a journal.

WHT Types and Register

What WHT is. Withholding Tax (WHT) is tax that one party deducts from a payment before remitting the balance to the other party. It works in two directions. When your company **pays a supplier**, it may be required by TRA regulations to withhold a percentage of the payment, remit that withheld amount to TRA, and issue the supplier a WHT certificate (`WHT_ON_PAYMENT`). When a customer **pays your company**, the customer may withhold tax from the receipt; you receive less than the invoice amount and are issued a WHT certificate in return — a tax credit you can use against your own tax liability (`WHT_ON_RECEIPT`).

Why WHT types exist. Different categories of payment attract different WHT rates under Tanzanian tax law (professional fees, rent, interest, etc.). WHT types let you configure the rate for each category once and select the appropriate type on each payment or receipt, ensuring the correct amount is withheld and the correct GL accounts are used.

When they are used. WHT types are maintained by a user with `WHT.MANAGE` permission during initial setup or when a new rate category is needed. WHT is applied optionally on individual AP payments and AR receipts by selecting a WHT type and amount during recording.

What WHT does to the cash. The payment **amount** is what the bill is relieved by; the WHT is held back from it. Paying a 400,000 bill with 20,000 WHT clears the whole 400,000 from the supplier's account, but only **380,000** leaves the bank — that is what the cash book and the GL both record, and the other 20,000 sits on WHT payable until you remit it to TRA. A receipt the customer withheld from works the same way in reverse. The WHT must be less than the amount paid, and an amount withheld needs its WHT type. A payment run can deduct WHT only when it pays a single supplier (the certificate is issued to one supplier) — run each supplier separately.

What the register shows. The WHT register is the period summary of all WHT certificates — how much was withheld on supplier payments (payable to TRA) and how much was withheld by customers from your receipts (a receivable credit against your tax bill). It is the data source for preparing the WHT remittance to TRA.

WHT Types: Navigate to **Accounting > Tax > WHT Types** (`/admin/tax/wht-types`). Permission required: `WHT.VIEW` to view; `WHT.MANAGE` to create, edit, and deactivate.

WHT types define the rates at which tax is withheld. Each type has:

- A unique code and name.
- A kind: **WHT_ON_PAYMENT** (withheld when paying a supplier) or **WHT_ON_RECEIPT** (withheld by the customer from your receipt).
- A rate percentage.

To create a WHT type:

1. Click **New WHT Type**.
2. Enter the code, name, kind, and rate percentage (0 or greater).
3. Save.

The kind is fixed at creation and cannot be changed. To deactivate a type, click **Deactivate** on its row. An inactive type is excluded from the pickers in the AP payment and AR receipt screens.

WHT Register: Navigate to **Accounting > Tax > WHT Register** (`/admin/tax/wht-register`). Permission required: `WHT.VIEW`.

The register shows all WHT certificates in a period, grouped into two sections:

- **WHT Payable to TRA** — certificates from supplier payments (`WHT_ON_PAYMENT`).
- **WHT Receivable** — certificates from customer receipts (`WHT_ON_RECEIPT`).

Select the period by choosing **Month** mode (year + month) or **Range** mode (start and end dates), then click **Load**.

Exporting the register. After loading, users with `WHT.VIEW` **and** `REPORT.EXPORT` see **Export PDF / Excel / CSV**. The export is exactly the period on screen: the WHT Payable section and the WHT Receivable section, each certificate with its date, party, source reference, taxable base and WHT amount, and a subtotal per section, under your company letterhead. There is deliberately no grand total — payable and receivable are different obligations and adding them would mean nothing.

> **Behind the scenes.** A WHT certificate can be marked as remitted to TRA once the withheld tax has been paid over (API: `POST /wht/register/transactions/{uid}/remit`, permission `WHT.REMIT`). This mark-remitted action is not yet exposed on the WHT Register screen above.

Foreign Exchange (FX)

What it is. The FX module enables your company to issue sales invoices, enter supplier bills, and record receipts and payments in foreign currencies (USD, EUR, KES, GBP) while keeping the GL in the

company's **base currency** (TZS). Every foreign-currency document is converted to TZS at the effective exchange rate on the document date; the GL always carries TZS amounts only.

Why it exists. Many businesses transact in foreign currencies — exporting in USD, importing from Europe in EUR — but keep statutory accounts in TZS. Without the FX module, the company would have to manually convert every foreign transaction before posting, with no systematic rate history, no automatic recognition of exchange gains and losses, and no way to revalue open foreign balances at period-end. FX makes multi-currency transacting systematic and auditable while preserving the integrity of the base-currency ledger.

When it is used. Any time a sales invoice, supplier bill, receipt, or payment is denominated in a currency other than TZS. The FX module must be configured first (exchange rates entered) before any foreign-currency document can be posted.

Key concepts:

- **Base currency.** The currency in which the company keeps its books (TZS by default). All GL postings are in the base currency regardless of the document currency. The base currency is a per-company setting and **cannot be changed once any journal entries exist** for the company.
- **Enabled currencies and default document currency.** Each company (and optionally each branch) has an admin-configured allow-list of **enabled currencies** and a configurable **default document currency**. Document currency fields across finance and sales use the filtered **Currency Picker**, which offers only the company's enabled currencies and pre-selects the default — see Chapter 0, *Common UI Patterns*. The one exception is the FX Exchange Rate form below, whose From/To selects list the full seeded currency set (not the enabled-only allow-list) and fall back to a typed 3-letter ISO code if the list fails to load.
- **Exchange rate.** The conversion rate between a foreign currency and TZS, expressed as "1 unit of foreign currency = X TZS" (e.g. 1 USD = 2,500 TZS). Rates are effective-dated: the system uses the most recent SPOT rate on or before the document date.
- **Realized gain/loss.** When a foreign-currency invoice is settled (received or paid), the TZS equivalent at the settlement rate may differ from the TZS equivalent when the invoice was raised. That difference is a **realized FX gain or loss** — it crystallises at the point of settlement and is posted to the books automatically (no manual action).
- **Unrealized gain/loss.** Open foreign-currency balances (unpaid invoices, unsettled bills) gain or lose TZS value as exchange rates move. At period-end, these open balances are **revalued** to the current spot rate. The resulting unrealized gain or loss is posted as a provisional GL entry and reversed at the start of the next period (because it is provisional — it only becomes realized when the invoice is actually settled).

Maintaining Currencies and Rates

What it is. The exchange rate master is a per-company, effective-dated list of rates between each foreign currency and TZS. Rates are entered manually and are append-only — a correction is a new rate row with the correct value, not an edit of the existing row.

Why it exists. Without an accurate, dated rate history, the system cannot convert foreign documents at the right rate, cannot compute realized FX on settlement, and cannot revalue open balances at period-end. The effective-dating ensures that a document dated in the past uses the rate that was in effect on that date, not today's rate.

When it is used. By a user with `CURRENCY.MANAGE` permission whenever an exchange rate needs to be entered or updated — typically daily or at the start of each period.

Navigate to **Accounting > FX > Exchange Rates** (`/admin/fx/rates`). Permission required: `CURRENCY.VIEW` to view; `CURRENCY.MANAGE` to add rates.

A set of currencies (TZS, USD, EUR, KES, GBP) is seeded at system setup. Which of these a company may actually use on *documents* is governed by its admin-configured **enabled-currency allow-list** (with a default document currency); the Currency Picker on every document offers only the enabled currencies (see Chapter 0, *Common UI Patterns*). The **From / To** selects on this rate-entry form are an exception: they list the full seeded currency set, not the enabled-only allow-list. The rate list shows all effective-dated exchange rates for the company, newest first.

To add a new rate (requires `CURRENCY.MANAGE`):

1. Click **New Rate**. The **New Exchange Rate** form opens.
2. Select the **From Currency** and **To Currency** from their dropdowns (each shown as `code — name`). The two must differ; the form does not require the To currency to be the company base currency — to keep the GL in base currency you would normally set To to TZS, but it is not enforced here. (If the currency list fails to load, both fields fall back to a typed 3-letter ISO code.)
3. Enter the **Rate** (expressed as: units of the To-currency per 1 unit of the From-currency), the **Effective Date**, and optionally a **Rate Type** (the dropdown defaults to **— none —**; choose Spot, Forward, or Official) and a **Source**.
4. Click **Save Rate**. The rate is effective from that date for documents and revaluations.

> Rate entry is append-only — there is no edit-in-place. To correct a rate, add a new row with the corrected value and the correct effective date. If a rate for the same currency, date, and type already exists, the entry is rejected. A self-currency rate (From = To) must be exactly 1; any other value is rejected.

The system uses the most recent SPOT rate on or before the document date when converting foreign-currency documents to base TZS.

Foreign-Currency Documents

What they are. A foreign-currency document is any sales invoice, supplier bill, receipt, or payment that is denominated in a currency other than TZS.

Why conversion happens at the document boundary. The GL is strictly base-currency-only. Every journal line must carry TZS amounts. The conversion from foreign currency to TZS therefore happens at the moment of posting — inside the AR/AP/sales services, before the journal lines are built — not inside the GL engine itself. This design means the GL's double-entry integrity rules (debits = credits in TZS) are never weakened or complicated by multi-currency concerns.

How it works. When you enter a sales invoice, supplier bill, or receipt in a foreign currency, the system converts all GL postings to TZS using the effective SPOT rate for the document date. The document stores the face amounts in the foreign currency; all GL ledger entries are in TZS. The conversion rate and the TZS base amount are captured on the document at the point of creation and are immutable — they will not change even if new rates are added later.

When you enter a sales invoice, supplier bill, or receipt in a foreign currency (e.g. USD), the system automatically converts all GL postings to the company base currency (TZS) using the effective SPOT rate for the document date. The document stores the face amounts in the foreign currency; all GL ledger entries are in TZS.

If no rate exists for the document's currency on or before the document date, the posting is rejected with a rate-not-found error.

Period-End Revaluation Run

What it is. A revaluation run is a period-end operation that adjusts the TZS value of open foreign-currency balances (unpaid AR invoices and unsettled AP bills) to the current spot rate. The adjustment is posted as an unrealized FX gain or loss, and a corresponding reversal is automatically scheduled for the first day of the next period.

Why it exists. If a USD invoice was raised when 1 USD = 2,500 TZS and the rate is now 2,600 TZS at period-end, the receivable on the books (2,500 TZS) is understated — the company could receive 2,600 TZS if paid today. The revaluation corrects the book value to 2,600 TZS and recognises the 100 TZS unrealized gain. This is an **accounting standards requirement** (IFRS/IAS 21): period-end statements must reflect current exchange rates on foreign balances. The gain is labelled "unrealized" because the invoice has not yet been paid — it reverses at the start of the next period so the actual settlement computes the real (realized) gain or loss against the original invoice rate, with no double-counting.

When it is used. At period-end, by a user with `FX.REVALUE` permission, after all foreign-currency invoices and bills for the period have been entered and before the period is closed. The run is idempotent — running it twice for the same period produces one run (the second attempt is rejected as already completed).

How it works. A preview (dry run) shows you the would-be adjustments without posting anything. Once you confirm, the system posts a single balanced GL journal (DR/CR the relevant control account / CR/DR the UNREALIZED_FX_GAIN or UNREALIZED_FX_LOSS account). If the next fiscal period is already open, the reversal is posted immediately; otherwise the system records the intent and posts the reversal when that period is opened.

At period end, open foreign-currency balances (AR invoices and AP bills not yet settled) must be revalued at the current spot rate. Navigate to **Accounting > FX > Revaluation Runs** (`/admin/fx/revaluation-runs`). Permission required: `FX.EXPOSURE.VIEW` to view runs; `FX.REVALUE` to preview and post.

Running a preview (dry run):

1. Select the company and click **Preview**.
2. Choose the **fiscal period** from the picker.
3. Optionally enter a **spot rate date** (defaults to the period end date).
4. Click **Run preview**. The system shows each open foreign item with its carrying base amount, revalued base amount, and adjustment (gain or loss). No GL is posted.

Posting the revaluation:

1. After reviewing the preview, click **Post**.
2. Enter the **posting date** and confirm.
3. A revaluation run record is created and a balanced unrealized FX journal posts to the GL.
4. If the next fiscal period is already open, the system automatically schedules and posts a reversal on the first day of the next period.
5. If the next period is not yet open, the run status is **POSTED** and the reversal can be triggered manually later (see below).

Manually reversing a run:

On a POSTED run in the runs list, click **Reverse**, enter the reversal date, and confirm. The reversal journal posts and the run status moves to **REVERSED**.

Realized FX gains and losses are posted automatically when a foreign-currency invoice is settled. The difference between the original invoice rate and the settlement rate is posted to the `REALIZED_FX_GAIN` or `REALIZED_FX_LOSS` accounts configured in GL config. No manual action is needed.

CRM — Customer Relationship Management

The CRM module helps your sales team track every potential customer from first contact through to a closed sale. It is organised around three concepts:

- **Leads** — an initial expression of interest, before you know whether the person will become a customer.
- **Opportunities** — a qualified sales chance with an estimated value and a pipeline stage.
- **Activities** — any interaction logged against a lead or opportunity (calls, emails, meetings, notes, and tasks).

The CRM section also provides a **Pipeline Dashboard** showing deal value across stages, a **Forecast** for a chosen date range, and **Pipeline Stages** settings where an administrator can customise the stage list.

Why CRM exists. Without a systematic way to track potential sales, deals fall through the cracks: a promising contact made at a trade fair is forgotten, a follow-up call that was never made costs the company a contract, and the sales manager has no visibility of what the team is working on or what revenue to expect next quarter. CRM gives every prospect a permanent record, every interaction a logged entry, and every deal a position in the pipeline — so nothing is lost and performance is measurable.

What CRM does and does not do. CRM is a pre-sales layer: it captures prospects, works them through a pipeline, and — on a win — converts the opportunity into a formal sales document (quotation or sales order) that then runs through the standard order-to-cash process. CRM itself posts **no entries to the general ledger, moves no stock, and opens no accounts-receivable balance**. All financial and inventory effects occur in the sales and finance modules once the converted document is processed there.

Navigation: Sidebar **CRM** group — **Leads**, **Opportunities**, **Pipeline Dashboard**, **Pipeline Stages**, **CRM Activities**.

Each item in the CRM nav group is hidden if you do not have the required permission. The sections below state the required permission for each action.

Leads

Navigate to **CRM › Leads** (`/admin/crm/leads`).

View: `CRM.LEAD.VIEW` | **Create / edit / contact / disqualify:** `CRM.LEAD.MANAGE` | **Qualify:** `CRM.LEAD.QUALIFY`

A **lead** is an early-stage record of someone who has expressed interest in your products or services but has not yet been confirmed as a genuine sales prospect. Think of it as a person or company at the "awareness" stage: you know they exist and they are interested, but you have not yet verified that they have a real budget, decision-making authority, or a genuine need. A lead is not a customer — it is a prospect.

Why leads exist as a separate concept from customers. If every enquiry were immediately converted into a customer record, the customer master would fill up with unqualified contacts —

tyre-kickers, wrong numbers, and dead ends — obscuring the real buyers and inflating debtor and pricing reports. Leads are kept separate so that the customer master remains a curated list of verified trading parties. Only after a lead is assessed and confirmed as a real prospect is it **qualified** and linked to a customer record.

When a lead is created. Any member of the sales team (with the `CRM.LEAD.MANAGE` permission) creates a lead when a new enquiry arrives — a website form submission, a referral from an existing client, a walk-in, a cold call, or a trade-show contact. The **Lead Source** field records the origin so the business can later measure which channels generate the most qualified prospects.

How a lead works — lifecycle. A lead starts as **New** and moves through a series of statuses as the sales team engages with it. Once a lead reaches a terminal status (Converted or Disqualified) it is locked and cannot be edited further. The lead is always scoped to the branch where it was created.

Lead status lifecycle

A lead passes through the following statuses:

```
NEW → CONTACTED → QUALIFIED → CONVERTED
               ↓                    ↑
         DISQUALIFIED       (via opportunity)
```

Status	Meaning
New	Freshly captured; no contact made yet.
Contacted	You have made initial contact.
Qualified	Linked to a customer record; ready to become an opportunity.
Converted	An opportunity was created from this lead. Terminal — no further edits.
Disqualified	Ruled out. Terminal — no further edits.

Once a lead reaches **Converted** or **Disqualified** it is locked: you cannot edit it, contact it, qualify it, or disqualify it again.

How to capture a lead

1. Navigate to **CRM › Leads** (`/admin/crm/leads`).
2. Click **New Lead**. An inline form appears.
3. Enter the **Name** (required).
4. Select the **Source** from the dropdown (Website, Referral, Walk-in, Campaign, Cold Call, Existing Customer, or Other).
5. Optionally enter **Company / Org**, **Contact Person**, **Phone**, **Email**, and **Notes**.
6. Click **Create Lead**.

The system assigns a **Lead Number** (for example, `LEAD-0001`) and sets the status to **New**. The lead is stamped with your active branch.

How to mark a lead as contacted

1. Open the lead from the list (`/admin/crm/leads/uid/:uid`).

2. Click **Mark Contacted** (only available when status is New).
3. The status changes to **Contacted**.

How to qualify a lead

Qualification is the process of confirming that a lead represents a real sales opportunity. This step links the lead to a customer record — either an existing customer already in the system, or a newly created one — and moves the lead to **Qualified** status. You need the `CRM.LEAD.QUALIFY` permission.

1. Open a New or Contacted lead (`/admin/crm/leads/uid/:uid`).
2. Click **Qualify**.
3. Choose one of the two modes:

Link an existing customer:

- Select **Link existing customer**.
- Choose the customer by name from the picker. The customer must belong to the same company as the lead.
- Click **Qualify Lead**.

Create a new customer from this lead:

- Select **Create new customer**.
- Enter the new customer's **Customer Name** (required) and select **Kind** (Credit Account or Cash Walk-in).
- Optionally enter Phone, Email, and Address.
- Click **Qualify Lead**. A new customer record is created automatically and the lead is linked to it.

After qualifying, the status badge changes to **Qualified** and the linked customer name is shown on the detail page.

How to convert a qualified lead to an opportunity

Once a lead is **Qualified**, a green **Convert to Opportunity** button appears in the action row on the lead detail page. This button is shown only while the lead is Qualified.

1. Open the qualified lead (`/admin/crm/leads/uid/:uid`).
2. Click **Convert to Opportunity**.
3. The opportunity create form opens with the **Source Lead** and **Customer** already pre-filled from this lead. Complete the remaining fields (Title, Pipeline Stage, Currency, and so on) as described under **How to create an opportunity**, then click **Create Opportunity**.

Creating the opportunity with this lead as its source moves the lead to **Converted** status.

How to disqualify a lead

Disqualification is the formal rejection of a lead — the conclusion that this prospect will not become a customer, at least not from this enquiry. Recording a reason is required so the business can learn which types of leads are typically unsuitable and refine its lead-generation strategy.

1. Open any non-terminal lead (New, Contacted, or Qualified) at `/admin/crm/leads/uid/:uid`.
2. Click **Disqualify**.
3. Enter a **Reason** (required — for example, "Budget too low" or "Not the right fit").
4. Click **Confirm Disqualify**.

The status changes to **Disqualified**. The reason is stored and displayed on the detail page.

Editing a lead

Open the lead detail page and change any editable fields (Name, Source, Company / Org, Contact Person, Phone, Email, Notes) in the **Lead Details** card. Click **Save Changes**. Editing is not available once the lead is Converted or Disqualified — terminal leads show a read-only view instead.

Browsing the leads list

The Leads list has a company selector, a **New Lead** button, and the table of leads. There is no name-search box on this screen. Each row ends with an **Open** button (eye icon) that navigates to the lead detail page; there is no row-click. Pagination controls appear when the list exceeds 20 rows; use the First / Previous / page-number / Next / Last controls to move between pages.

A note on how Source is displayed. When you create or edit a lead you pick the Source from a dropdown with friendly labels (Referral, Walk-in, Cold Call, and so on). In the leads list and on the lead detail page, however, the Source is shown as the stored code — for example `REFERRAL`, `WALK_IN`, or `COLD_CALL` — rather than the friendly label.

Example — Capture a referral lead and qualify it to a new customer:

Sales executive Amina Msangi at Kijenge branch receives a phone call from Juma Banda, who was referred by an existing client and wants to discuss buying office furniture in bulk.

1. Navigate to **CRM › Leads** (`/admin/crm/leads`). Click **New Lead**.
2. Name: `Juma Banda`; Source: `Referral`; Phone: `+255754001122`; Notes: `Referred by Baraka Supplies — bulk office furniture interest`.
3. Click **Create Lead**. System creates `LEAD-0005`, status **New**.
4. Next day, Amina calls Juma. She opens `LEAD-0005` and clicks **Mark Contacted**. Status becomes **Contacted**.
5. After the call confirms he runs a legitimate business, Amina clicks **Qualify**. She selects **Create new customer**, enters Customer Name: `Banda Office Solutions`, Kind: `Credit Account`, Phone: `+255754001122`. Clicks **Qualify Lead**.
6. A new customer record "Banda Office Solutions" is created. Lead status flips to **Qualified**. The linked customer name appears on the detail page.
7. Amina now clicks **Convert to Opportunity** on the qualified lead to start a pre-filled opportunity (see Opportunities section).

Opportunities

Navigate to **CRM › Opportunities** (`/admin/crm/opportunities`).

View: `CRM.OPPORTUNITY.VIEW` | **Create / edit / stage / win / lose:** `CRM.OPPORTUNITY.MANAGE` | **Convert to document:** `CRM.OPPORTUNITY.CONVERT`

An **opportunity** is a specific, identifiable sales deal being pursued with a known customer. Where a lead is a vague expression of interest, an opportunity is a concrete proposal: it has a named customer, an estimated monetary value, an expected close date, and a position in the sales pipeline indicating how far through the sales process the deal has progressed. An opportunity can also carry individual product lines — the specific items and quantities the customer is likely to buy.

Why opportunities exist. Opportunities bridge the gap between the customer master and the order-to-cash process. A sales team may have dozens of active deals at any time; without a systematic record of each one, deals lose momentum, forecasts are guesswork, and management has no way to prioritise effort. The opportunity record is where all of that is centralised: the value, the probability of winning, the stage, the history of interactions, and — at the end — the formal quotation or sales order that results from the win.

When an opportunity is created. A sales representative or manager creates an opportunity when a qualified lead turns into a real, pursuable deal, or directly against a known customer when a sales initiative begins. The opportunity must always be attached to a customer record (not a raw lead contact).

How an opportunity works — lifecycle. An opportunity starts **Open** and has two possible terminal outcomes: **Won** (the deal was closed in your favour) or **Lost** (the deal did not proceed). While Open, the opportunity moves through **pipeline stages** — configurable steps; the default seeded stages are `QUALIFICATION`, `NEEDS_ANALYSIS`, `PROPOSAL`, `NEGOTIATION`, and `CLOSING` — each with a default win probability percentage. The stage drives the weighted pipeline forecast. Once Won, the opportunity can be **converted** to a quotation or sales order in the order-to-cash module.

Opportunity status lifecycle

```
OPEN → WON
OPEN → LOST
```

Once an opportunity is Won or Lost it is closed. Closed opportunities cannot be edited, and lines cannot be added or removed. Conversion to a quotation or sales order is still available on a closed opportunity (with the restrictions described below).

How to create an opportunity

1. Navigate to **CRM › Opportunities** (`/admin/crm/opportunities`).
2. Click **New Opportunity** (or navigate to **CRM › Opportunities › Create** at `/admin/crm/opportunities/create`).

3. Select the **Customer** using the picker. Type part of the customer name to search; select from the results.
4. Select the **Pipeline Stage** from the dropdown. Only active stages are offered, each listed by its stored name (for a default-seeded company these read `QUALIFICATION`, `NEEDS_ANALYSIS`, and so on). The stage's default win probability is applied automatically unless you override it.
5. Enter the **Title** (required).
6. Pick the **Currency** from the Currency Picker. It offers only the currencies enabled for the company and pre-selects the company's default currency — see the **Common UI Patterns** section in *Getting Started* (ch00). The picker is disabled until a company is selected.
7. Optionally enter an **Est. Value**, a **Win % (0–100)** override (blank uses the stage default), and an **Expected Close Date**.
8. Optionally select a **Source Lead** using the picker — only Qualified leads appear in this list. Selecting a source lead converts that lead to **Converted** status. (When you reach this form via the **Convert to Opportunity** button on a qualified lead, the Source Lead and Customer are already pre-filled for you.)
9. Click **Create Opportunity**.

The opportunity is created with status **Open** and an automatically assigned number (for example, `OPP-0001`). You land on the opportunity detail page (`/admin/crm/opportunities/uid/:uid`).

How to add lines to an opportunity

Opportunity lines are the individual products or services the customer is expected to buy. Adding lines serves two purposes: it gives the sales team a precise record of what the deal covers, and it pre-populates the resulting quotation or sales order when the opportunity is later converted — eliminating the need to re-enter every item.

Lines represent the products or services you expect to sell. You can add them while the opportunity is Open.

1. Open the opportunity detail page.
2. In the **Opportunity Lines** panel, type a product name or code into the **Product** search box and select the product (shown as `code — name`).
3. Select the **Unit** from the units dropdown.
4. Enter the **Qty** (must be greater than zero).
5. Optionally enter the **Unit Price** and a **Discount %** (0–100).
6. Click **Add**.

To remove a line, click the trash (Remove) icon on the row.

How to advance the pipeline stage

Advancing the stage moves the opportunity forward in the sales funnel. Each stage represents a milestone in the sales process — for example, moving from `NEEDS_ANALYSIS` to `PROPOSAL` means you have finished diagnosing the customer's requirements and are now ready to present a formal proposal. (Stages appear under their stored names; a default-seeded company shows the code-style names shown here.) The stage's default win probability is suggested automatically; you can override it to reflect the specific circumstances of this deal.

1. Open the opportunity detail page (must be Open).
2. Click **Advance Stage**.
3. Select the **Target Stage** from the active-stages dropdown.
4. Optionally set a **Win % override** to override the stage default.
5. Click **Advance Stage**.

The stage and win probability update immediately.

How to mark an opportunity as Won

1. Open the opportunity detail page (must be Open).
2. Click **Mark Won**.
3. Optionally set **Won At** (defaults to now).
4. Click **Confirm Won**.

Status changes to **Won**. Edit, add-line, advance-stage, win, and lose actions are no longer available. The Convert action remains available.

How to mark an opportunity as Lost

1. Open an Open opportunity.
2. Click **Mark Lost**.
3. Enter a **Loss Reason** (required — for example, "Lost to competitor on price").
4. Click **Confirm Lost**.

Status changes to **Lost**.

How to convert an opportunity to a quotation or sales order

Conversion is the moment a CRM deal becomes a formal commercial document. When you convert an opportunity, the system calls the order-to-cash module to create a quotation or sales order, pre-populated with the opportunity's customer, currency, and all of the lines you entered. The sales team can then take the resulting document through the normal approval, delivery, and invoicing workflow without re-entering any data. Conversion is **idempotent**: clicking Convert a second time returns the document already created rather than making a duplicate.

- A **Quotation** is appropriate when the deal is still being negotiated — you are giving the customer a formal price offer but have not yet received a commitment. An Open or Won opportunity can be converted to a quotation.
- A **Sales Order** is the binding commercial commitment — the customer has agreed to buy. Only a **Won** opportunity can be converted to a sales order, because converting to an SO implies the deal is closed.

Conversion creates a Sales document (Quotation or Sales Order) pre-populated with the opportunity's customer, currency, and lines. You need the `CRM.OPPORTUNITY.CONVERT` permission.

Requirements before converting:

- The opportunity must have at least one line.
- To convert to a **Quotation**: opportunity must be Open or Won.

- To convert to a **Sales Order**: opportunity must be Won.

Steps:

1. Open the opportunity detail page.
2. Click **Convert**.
3. In **Convert To**, choose **Quotation** or **Sales Order (requires WON)**.
4. For a Quotation, optionally set a **Valid Until (optional)** date. The field is blank by default; if you leave it blank, the resulting quotation is given a validity of today + 30 days by the server (this default is not pre-filled in the form).
5. Click **Convert**.

On success the form shows a confirmation with the created document's kind and number, plus a **View Document** button that opens the new Quotation or Sales Order. (Once converted, an info banner on the detail page also links to the document.)

Conversion is idempotent: if you click Convert a second time, the system returns the document that was already created rather than making a duplicate.

Editing an opportunity

Open the detail page (must be Open). In the **Opportunity Details** card you can change the Title, Est. Value, Win % (0–100), Expected Close Date, and Notes; click **Save Changes**. (The pipeline stage is not changed here — use **Advance Stage** for that.) Editing is blocked once the opportunity is Won or Lost, where the detail page shows a read-only summary instead.

Example — Full pipeline journey: lead → opportunity through stages → won → convert to sales order:

Sales manager Benson Kileo at Dar es Salaam branch handles a qualified lead for Banda Office Solutions (created in the lead example above).

1. Navigate to **CRM › Opportunities › Create** (`/admin/crm/opportunities/create`).
2. Customer: `Banda Office Solutions`; Pipeline Stage: `QUALIFICATION` (this company still uses the default seeded stages, so the dropdown lists the stored code names); Title: `Bulk Office Furniture — Q3 2026`; Currency: picked from the company's enabled currencies (here the company default, `TZS`); Estimated Value: `4,500,000`; Expected Close Date: `2026-09-30`; Source Lead: `LEAD-0005 — Juma Banda` (auto-converts that lead to Converted).
3. Click **Create Opportunity**. Opportunity `OPP-0012` created, status **Open**.
4. Add lines to `OPP-0012`:

    - Executive Desk EXD-01, Unit: EA, Qty: 5, Unit Price: 480,000 = TZS 2,400,000.
    - Ergonomic Chair CHR-02, Unit: EA, Qty: 20, Unit Price: 105,000 = TZS 2,100,000.

5. After a needs-analysis call, Benson clicks **Advance Stage**, selects Target Stage `NEEDS_ANALYSIS` (default probability 25%). Clicks **Advance Stage** to confirm.
6. After sending a detailed proposal, Benson advances to `PROPOSAL` (50%). After negotiation the stage moves to `NEGOTIATION` (75%).
7. Juma accepts the quote. Benson opens the opportunity, clicks **Mark Won**, sets Won At: `2026-08-15`, then clicks **Confirm Won**. Status becomes **Won**.
8. Click **Convert**, Convert To: `Sales Order (requires WON)`. System creates `SO-0034` with all lines pre-filled. Benson clicks **View Document** to open the new Sales Order and proceeds with delivery.

Pipeline Dashboard

Navigate to **CRM › Pipeline Dashboard** (`/admin/crm/pipeline`). **Permission:** `CRM.PIPELINE.VIEW`.

The **pipeline dashboard** is a management view that shows the current health of your sales funnel in real time. It answers three questions at a glance: where are your deals right now (the board), how much revenue can you expect in a given period (the forecast), and how effective is the team at closing deals (the KPIs)?

Why the pipeline dashboard exists. A sales manager without visibility of the pipeline is flying blind: they cannot see which stages are bottlenecks, whether the team has enough deals to meet the quarter's target, or whether the win rate has deteriorated. The dashboard distils the raw opportunity data into actionable numbers so management can intervene early, redirect effort, or adjust the forecast before it is too late.

The pipeline dashboard shows the current state of all open opportunities across your sales pipeline. It is scoped to a company and branch — select both to load the data.

Board summary

The **Pipeline Board** shows each active pipeline stage with the count of open opportunities in that stage, their combined **Total Value**, and the **Weighted Value** (value × win probability).

Weighted forecast

The **weighted forecast** is a more realistic estimate of expected revenue than a simple sum of all open deal values. It multiplies each open opportunity's estimated value by its win probability (expressed as a percentage) and sums the results. For example, an opportunity worth TZS 10,000,000 at a 50% probability stage contributes TZS 5,000,000 to the weighted forecast. This gives sales managers a probability-adjusted revenue estimate that accounts for the fact that not all open deals will close.

The Forecast section calculates expected revenue for a date range, weighting each opportunity's estimated value by its win probability. The panel shows two tiles: **Open Opportunities** (the count of open deals in the period) and **Weighted Value** (the probability-weighted total). Set the **From** and **To** dates at the top of the dashboard and click **Refresh**. (This single date range drives both the Forecast and the KPI panels.)

Win-rate and cycle-time KPIs

The **Win-Rate KPIs** panel shows four tiles:

- **Won** — the count of opportunities marked Won in the selected period.
- **Lost** — the count of opportunities marked Lost in the selected period.
- **Win Rate** — the percentage of closed opportunities marked Won in the selected period.
- **Avg Cycle (days)** — the average number of days from opportunity creation to close.

Win Rate measures the sales team's effectiveness at closing deals. A low win rate may indicate that the team is pursuing too many unqualified leads, that the product-market fit is poor, or that competitors are winning on price. **Avg Cycle (days)** measures how long deals take to close — a rising cycle time may indicate bottlenecks in the proposal or approval process. Both KPIs are calculated for the date range you set at the top of the dashboard so that trends over time can be observed.

Adjust the date range and click **Refresh** to recalculate.

Example — Reading the pipeline board and setting a forecast:

Branch manager Zawadi Ngowi opens the **CRM › Pipeline Dashboard** (`/admin/crm/pipeline`), selects company `Kijenge Trading Ltd` and branch `DSM Main`. The board shows (this company still uses the default seeded stages, so the stage names appear in their stored code form):

Stage	Open Opps	Total Value	Weighted Value
QUALIFICATION	3	TZS 8,200,000	TZS 820,000
NEEDS_ANALYSIS	5	TZS 21,500,000	TZS 5,375,000
PROPOSAL	4	TZS 18,750,000	TZS 9,375,000
NEGOTIATION	2	TZS 9,600,000	TZS 7,200,000
CLOSING	1	TZS 4,500,000	TZS 4,050,000

Zawadi sets From: `2026-07-01`, To: `2026-09-30` and clicks **Refresh**. The Forecast panel shows Open Opportunities: 15 and a Weighted Value of TZS 29,340,000 (each deal's estimated value × its win probability). The Win-Rate KPIs panel shows four tiles — Won: 8, Lost: 5, Win Rate: 62%, and Avg Cycle (days): 34 — for deals closed in the selected period.

Pipeline Stages (Settings)

Navigate to **CRM › Pipeline Stages** (`/admin/crm/settings/pipeline-stages`). **Permission to view the settings screen:** `CRM.STAGE.MANAGE` | **Permission to read stages via API:** `CRM.OPPORTUNITY.VIEW`

Pipeline stages are the named milestones in your sales process — the steps a deal must pass through between "new opportunity" and "closed sale." Stages are not universal: a software company might use stages called Discovery, Demo, Evaluation, and Negotiation, while a building-materials distributor might use Route Visit, Sample Sent, Proposal, and Closing. The system therefore makes stages **configurable per company** rather than hard-coding them.

Why stages are configurable. Every business has a different sales process. A fixed, one-size-fits-all set of stages would force companies to map their real process onto arbitrary labels, making the pipeline board meaningless. Configurable stages mean the board reflects the actual milestones the sales team uses, making stage-based reporting and coaching practical.

The default stages. When a company is first created, five stages are seeded automatically. They are stored — and therefore displayed everywhere in the UI — under code-style names: `QUALIFICATION` (10% probability), `NEEDS_ANALYSIS` (25%), `PROPOSAL` (50%), `NEGOTIATION` (75%), and `CLOSING` (90%). These cover the most common B2B sales process and can be used immediately. They can be renamed (for example to a friendlier "Needs Analysis"), reordered, supplemented, or deactivated without affecting historical opportunity records, so once you rename a default stage it appears under the new name.

The default probability. Each stage has a **default win probability** — the system's best guess at the likelihood of closing a deal that has reached this stage. This default is applied automatically when an opportunity is placed at that stage and drives the weighted forecast calculation. Sales reps can override the probability on individual opportunities to reflect the specific situation.

Pipeline stages define the steps in your sales process. Five stages are seeded per company under stored code names: `QUALIFICATION`, `NEEDS_ANALYSIS`, `PROPOSAL`, `NEGOTIATION`, and `CLOSING`. You can add, rename, reorder, change probabilities, and deactivate stages.

How to create a stage

1. Navigate to **CRM › Pipeline Stages** (`/admin/crm/settings/pipeline-stages`).
2. Click **New Stage**.
3. Enter the **Stage Name** (must be unique within the company).
4. Enter the **Order** (a number; must be unique within the company).
5. Enter the **Default Win %** (0–100).
6. Click **Create Stage**.

How to edit a stage

Click the pencil (Edit) icon button at the end of a row — it has no visible "Edit" text (its accessible label is "Edit stage <name>"). The row becomes an inline edit row where you can change the display order, name, default probability, or the **Active** checkbox. Click the check (Save) icon button to save (accessible label "Save changes"), or the **×** button to cancel.

How to deactivate a stage

Deactivating a stage removes it from the stage selection dropdown when creating or advancing an opportunity, while keeping all historical opportunities that were in that stage intact. This is the

correct action when a stage is no longer part of the sales process — for example, if a "Demo" stage is eliminated because demos are now handled differently. Deactivation is reversible.

Click the toggle (Deactivate) icon button on the row — it has no visible "Deactivate" text (its accessible label is "Deactivate stage <name>") and is shown only while the stage is active. Alternatively, clear the **Active** checkbox in the inline edit row and save. The stage record is kept but marked inactive. Inactive stages:

- No longer appear in the stage selection dropdowns when creating or advancing an opportunity.
- Are rejected if you attempt to use them via the API.
- Still appear in historical records.

Deactivation is not permanent — you can reactivate a stage by editing it and switching Active back on.

Stage validation rules

- Name must be unique within the company.
- Display order must be a positive number and unique within the company.
- Default probability must be between 0 and 100 (whole number).

Activities

Navigate to **CRM › CRM Activities** (`/admin/crm/activities`) for the open-task inbox. Activities are also embedded on Lead and Opportunity detail pages.

View activities: `CRM.ACTIVITY.VIEW` | **Log / complete activities:** `CRM.ACTIVITY.MANAGE`

An **activity** is a logged record of an interaction with a prospect or customer in the context of a specific lead or opportunity. Activities capture the history of a deal: the calls made, the emails sent, the meetings held, and the notes taken. They are also the mechanism for assigning follow-up **tasks** — future actions that need to be completed — and for tracking whether those tasks have been done.

Why activities exist. A sales cycle typically involves many touchpoints over days or weeks before a deal closes. Without a structured activity log, the sales team relies on memory and personal notes — which are unreliable, invisible to the manager, and lost when a rep leaves. The activity log on each lead or opportunity gives every team member and manager a complete, timestamped record of what happened and what still needs to happen. The open-task inbox surfaces all outstanding tasks across the whole pipeline so nothing slips through.

When activities are used. A sales representative logs an activity immediately after each interaction — after a call, after sending an email, after a meeting. A follow-up task is created when the next action is identified — for example, "Call back on Thursday to confirm the budget." The task appears in the open-task inbox until it is completed.

How activities work. Every activity is attached to exactly one parent: either a lead or an opportunity — not both, and not neither. There are five activity types. Four (Call, Email, Meeting, Note) are **historical records** — they record something that happened and have no completion state. The fifth (Task) is a **forward-looking action item** with a due date; only Tasks appear in the open-task inbox and only Tasks can be completed.

An activity records an interaction or a task related to a lead or opportunity. Every activity is attached to exactly one parent: either a lead or an opportunity — not both, and not neither.

Activity types

Type	Has due date	Can be completed
Call	No	No
Email	No	No
Meeting	No	No
Note	No	No
Task	Yes (required)	Yes

Only **Task** activities appear in the open-task inbox. Only Tasks can be completed.

How to log an activity on a lead or opportunity

1. Open the lead (`/admin/crm/leads/uid/:uid`) or opportunity (`/admin/crm/opportunities/uid/:uid`) detail page.
2. Scroll to the **Activities** panel.
3. Click **Log Activity**. The **Log New Activity** form appears.
4. Select the **Type** (Call, Email, Meeting, Note, or Task).
5. Enter the **Subject** (required).
6. For Call, Email, Meeting, and Note, optionally set an **Occurred At** date. For a **Task**, the Occurred At field is replaced by a **Due Date** field (required for Tasks).
7. Optionally enter **Notes**.
8. Click **Log Activity**.

The activity appears at the top of the panel list (latest first), and the system assigns an activity number (for example, `ACT-0001`).

Activity panel pagination

The activity panel on a lead or opportunity detail page shows 10 activities per page. Use the paginator controls to move between pages if there are more than 10.

How to complete a task

Completing a task marks it as done and removes it from the open-task inbox. This is the formal acknowledgement that the action was taken — for example, that the follow-up call was made. You cannot undo a completion once recorded.

A task can be completed from the open-task inbox or from the activity panel on the parent lead or opportunity.

1. Find the task (either on the detail page or in **CRM › CRM Activities** at `/admin/crm/activities`).
2. Click the complete control on the task row. The two surfaces look different:

    - In the open-task inbox (**CRM › CRM Activities**), this is a green button labelled **Complete** (with a check icon).
    - In the **Activities** panel embedded on a lead or opportunity detail page, it is an icon-only green check button with no visible text (its accessible label is "Complete task: <subject>"). The button appears only on open Task rows.

The task is marked done and disappears from the open-task inbox. You cannot complete an activity that is not a Task, and you cannot complete a Task that is already done.

Open-task inbox

Navigate to **CRM › CRM Activities** (`/admin/crm/activities`). **Permission:** `CRM.ACTIVITY.VIEW` (view) / `CRM.ACTIVITY.MANAGE` (complete).

The **open-task inbox** is a unified list of all incomplete tasks across every lead and opportunity in the company — a personal and team-wide to-do list for the sales pipeline. It allows a sales manager to see at a glance what follow-up actions are pending, and allows each rep to check what they need to do today without opening every individual lead or opportunity record.

The CRM Activities screen lists all open (not-yet-done) Tasks for the selected company, across all leads and opportunities. It is scoped to the company you select via the company selector at the top of the screen.

The list is paginated (20 per page). Use the paginator controls to browse. When you complete a task, it is removed from the inbox and the list refreshes.

Example — Log activities across the sales journey and manage the task inbox:

Sales rep Farida Hassan is managing opportunity `OPP-0012` (Banda Office Solutions). She logs activities at each step.

1. After the initial qualification call, she opens `OPP-0012` at `/admin/crm/opportunities/uid/:uid`, scrolls to the Activities panel, clicks **Log Activity**: Type `Call`, Subject `Initial qualification call — confirmed budget TZS 4.5M`, Occurred At `2026-07-03`. Clicks **Log Activity** to save. Activity `ACT-0018` appears.
2. She sends a proposal by email: Type `Email`, Subject `Proposal email sent — 5 desks + 20 chairs`, Occurred At `2026-07-10`.
3. After the proposal, she needs a follow-up. She creates a task: Type `Task`, Subject `Follow up on proposal — confirm decision`, Due Date `2026-07-17`. Activity `ACT-0021` created.
4. On 2026-07-17, Farida opens **CRM › CRM Activities** (`/admin/crm/activities`). She sees `ACT-0021` in the open-task inbox. After a productive call, she clicks **Complete**. The task disappears from the inbox.

5. A meeting is later held: back on `OPP-0012`, Type `Meeting`, Subject `Site visit — DSM Main showroom`, Occurred At `2026-07-22`. The activity panel shows all four interactions, newest first.

Reporting and Business Intelligence

What is the Reporting and BI module? Reporting and Business Intelligence (BI) is the read-only analytical layer of the system. It does not create, change, or post anything — it reads what the financial and operational modules have already recorded and presents the results in standard formats that management and external stakeholders (auditors, banks, tax authorities) can read and act on. The four financial statements summarise the company's performance and position in internationally recognised forms; the account ledger lets you drill into the individual transactions behind any figure; the BI dashboard composes key indicators from across all modules into a single at-a-glance view. Because all reports are computed on demand from the live General Ledger, a report run at any moment reflects the current state of the books. Nothing is stored or posted when you run a report (ADR-0018, ADR-0037).

This chapter describes the financial statements, the GL account-ledger drill-down, and the analytics dashboard. All reports are read-only and computed on demand — nothing is stored or posted when you run a report.

Financial Statement Reports

What are financial statements, and why do companies produce them? Financial statements are standardised summaries of a company's financial activity and position. They are the language that businesses, investors, lenders, and regulators use to assess financial health. Every trading company is legally required to produce them at least annually. In this system they are generated directly from the General Ledger and carry a reconciliation bar that confirms the figures tie back to the underlying journal entries — so there is no separate spreadsheet to maintain and no risk of a mismatch between the books and the reports.

The four financial statements are available from the **Accounting** navigation group. Each report requires the relevant permission and a company and period selection before it can be run.

Common controls on every statement screen:

- **Company selector** — if your organisation has more than one company, choose which company to report on by name.
- **Period inputs** — date fields specifying the reporting period.
- **Run button** — computes and displays the statement.

- **Export buttons** (PDF, Excel, CSV) — download the statement in the chosen format. Requires the additional `REPORT.EXPORT` permission. The buttons are hidden if you do not hold that permission.
- **Comparative period** — most statements accept an optional comparative period or date to populate a second column.
- **Branch** (Income Statement, Balance Sheet, Cash-Flow Statement and Financial Ratios) — leave it on *All branches* for the whole company, or pick a branch to see only the journals posted at that branch. The list offers only the branches you are assigned to; asking for another branch is refused with *"You are not assigned to that branch…"*. The statements also offer **Company-level entries (no branch)** — the journals that carry no branch: manual journals, opening balances, FX revaluation runs and the year-end close. The branches plus the company-level entries always add up exactly to the whole-company statement. The statement header names what you are looking at (a branch name, *All branches*, or *Company-level entries*), and so does the company line of every export.
- **Reconciliation indicator** — a green **"Reconciled"** bar confirms the computed figures tie back to the underlying GL movement. A red **data-integrity alarm** means the figures do not agree and the books require investigation; the report is shown but no automatic correction is made.

Profit & Loss (Income Statement)

What is the Profit and Loss statement, and what does it tell you? The Profit and Loss statement (also called the Income Statement) shows how much revenue the company earned and how much it spent over a period of time, arriving at a net profit or loss. Revenue is income from sales and services; Cost of Sales is the direct cost of what was sold (purchases, materials, production); Operating Expenses are the overhead costs of running the business (salaries, rent, utilities). Gross Profit is Revenue minus Cost of Sales — a measure of trading margin. Net Profit is what remains after all operating expenses. The P&L answers the question: "Did the business make money this period, and where did the money come from and go?" It is the report most used for management decisions, bank covenants, and tax assessments. The comparative column lets you benchmark the current period against a prior period (same quarter last year, for example) to spot trends.

Navigate to **Accounting › Income Statement** (`/admin/reporting/income-statement`). Permission required: `REPORT.PL.VIEW`.

1. Select the company by name.
2. Set **Period from** and **Period to** (date inputs).
3. Optionally set a **Comparative from** and **Comparative to** to add a prior-period column.
4. Click **Run**.

The statement shows:

- Revenue lines grouped under **REVENUE**.
- Cost of sales lines under **COST OF SALES**.

- Operating expense lines under **OPERATING EXPENSES**.
- Section subtotals.
- A footer with **Gross Profit** and **Net Profit** rows, each carrying the current period amount and — if a comparative was set — the prior-period amount.

Drill-through to the account ledger: any account name shown as a link in the detail rows can be clicked to open the Account Ledger pre-filtered to that account and period.

Export: after running the statement, click **PDF**, **Excel**, or **CSV** in the export toolbar. The downloaded file is named `income-statement_<from>_<to>.<ext>`.

Year-end close: the closing journal that moves a year's result into Retained Earnings is not income or expense, so the Income Statement leaves it out — a closed year still shows the profit it made.

Example — Run a comparative P&L for two quarters and export to Excel:

Chief accountant Rehema Mwangi needs to compare Q1 2026 performance against Q1 2025 for the board report.

1. Navigate to **Accounting › Income Statement** (`/admin/reporting/income-statement`).
2. Company: `Kijenge Trading Ltd`.
3. Period from: `2026-01-01`; Period to: `2026-03-31`.
4. Comparative from: `2025-01-01`; Comparative to: `2025-03-31`.
5. Click **Run**.

The statement loads. The green **Reconciled** bar confirms net profit ties to the INCOME − EXPENSE GL movement. Results:

Section	Q1 2026	Q1 2025
Revenue	TZS 48,250,000	TZS 39,100,000
Cost of Sales	TZS 29,340,000	TZS 24,600,000
Gross Profit	TZS 18,910,000	TZS 14,500,000
Operating Expenses	TZS 9,720,000	TZS 8,850,000
Net Profit	TZS 9,190,000	TZS 5,650,000

Rehema clicks **Excel** in the export toolbar. The file `income-statement_2026-01-01_2026-03-31.xlsx` downloads with both columns. She forwards it to the board.

To drill into "Sales Revenue", she clicks the account name link — the Account Ledger opens pre-filled with that account and the Q1 2026 period, showing every posted journal line and a running balance.

Balance Sheet

What is the Balance Sheet, and what does it tell you? The Balance Sheet (also called the Statement of Financial Position) shows what the company owns and what it owes at a single point in time. **Assets** are what the company owns — cash, trade receivables, stock, fixed assets, and other resources. **Liabilities** are what the company owes — supplier payables, loans, tax obligations. **Equity** is the residual interest of the owners — the difference between assets and liabilities, representing the net worth of the business. A correctly prepared balance sheet always satisfies the fundamental accounting equation: Assets = Liabilities + Equity. If this equation does not balance, something has been mis-posted. The Balance Sheet answers the question: "What is the company worth, and how solvent is it?" It is used by banks to assess creditworthiness, by investors to evaluate the business, and by management to monitor liquidity. The comparative "as at" date lets you compare financial position at two year-ends side by side.

Navigate to **Accounting › Balance Sheet** (`/admin/reporting/balance-sheet`). Permission required: `REPORT.BS.VIEW`.

1. Select the company by name.
2. Set the **As-at date**.
3. Optionally set a **Compare as-at** date to add a prior-date column.
4. Click **Run**.

The statement shows sections for Current Assets, Non-Current Assets, Current Liabilities, Non-Current Liabilities, and Equity, each with detail lines, subtotals, and four total rows in the footer: **Total Assets**, **Total Liabilities**, **Total Equity**, and a grand-total **Total Liabilities + Equity** row. A balanced set of books shows Total Assets equal to the Total Liabilities + Equity row, confirmed by a green **Balanced** bar (helper text *Assets = Liabilities + Equity.*).

Drill-through: click any real account name link to open the Account Ledger for that account as at the selected date.

Export: file is named `balance-sheet_<asAt>.<ext>`.

A branch Balance Sheet balances on its own, because every journal is posted to exactly one branch (or to none). Read it with two caveats, which the screen also prints under a branch statement:

- **Stock transfers between branches move quantities, not ledger value.** No journal is posted for a transfer, so a branch's *Inventory* reflects what it bought and sold, not what it holds — a branch that sells stock it received by transfer can show low or even negative Inventory, and the sender too much. The company total is right.
- **Company-level journals carry no branch.** After a year-end close, each branch keeps its closed years under *Retained earnings — prior years (unclosed)* because the close itself is company-level; the *Company-level entries* statement carries the matching opposite.

If a journal ever carried lines for different branches, the branch statement would show an explicit **Inter-branch balance (entries spanning branches)** line under Current Liabilities instead of silently failing to balance. Nothing the system posts today produces one.

Example — Run a comparative balance sheet at year-end:

Rehema Mwangi needs the balance sheet as at 30 June 2026 compared with 30 June 2025.

1. Navigate to **Accounting › Balance Sheet** (`/admin/reporting/balance-sheet`).
2. Company: `Kijenge Trading Ltd`; As-at date: `2026-06-30`; Compare as-at: `2025-06-30`.
3. Click **Run**.

The green **Balanced** bar appears, with the helper text *Assets = Liabilities + Equity.* Rehema spots that "Trade Receivables" has grown from TZS 12.4M to TZS 19.7M year-on-year. She clicks the "Trade Receivables" account name to open its ledger for the full fiscal year and reviews each transaction. She then exports to PDF for the audit file.

Cash-Flow Statement

What is the Cash-Flow Statement, and what does it tell you? The Cash-Flow Statement shows how cash moved into and out of the business over a period, organised into three categories. **Operating activities** are cash flows from the company's main trading activities — collecting from customers, paying suppliers, paying wages. **Investing activities** are cash flows from buying or selling long-term assets — purchasing a vehicle or machinery, receiving proceeds from selling an asset. **Financing activities** are cash flows from raising or repaying capital — new loans drawn, loan repayments, equity injections. The statement reconciles the opening and closing cash balance, confirming that the movement in the company's bank accounts is fully explained. The Cash-Flow Statement answers the question: "Where did the cash come from, and where did it go?" It is particularly important for businesses that are profitable on paper but cash-constrained in practice — a common situation when customers pay late or large capital purchases are made. The system uses the **indirect method** (starting from net profit and adjusting for non-cash items), which is the most common format for external reporting.

Navigate to **Accounting › Cash-Flow Statement** (`/admin/reporting/cash-flow`). Permission required: `REPORT.CASHFLOW.VIEW`.

1. Select the company by name.
2. Set **Period from** and **Period to**.
3. Optionally set a comparative period.
4. Click **Run**.

The indirect-method statement shows movements in three sections:

- **OPERATING ACTIVITIES** — cash generated from or used in trading activities.
- **INVESTING ACTIVITIES** — cash spent on or received from capital assets.
- **FINANCING ACTIVITIES** — cash from borrowings, equity, or repayments.

The opening position is shown as a body row at the top of the table (**Opening cash & bank balance**, under an **OPENING CASH POSITION** heading). The footer then shows **Net Change in Cash** and **Closing Cash & Bank Balance**. A green **Cash Tie-out: Reconciled** bar confirms the net change matches the change in cash-equivalent GL account balances.

Export: file is named `cash-flow_<from>_<to>.<ext>`.

A branch Cash-Flow Statement is read from that branch's journals on the company's cash and bank accounts, and ties to the movement on those accounts at that branch.

Example — Cash-flow analysis for H1 2026:

1. Navigate to **Accounting › Cash-Flow Statement** (`/admin/reporting/cash-flow`).
2. Company: `Kijenge Trading Ltd`; Period from: `2026-01-01`; Period to: `2026-06-30`.
3. Click **Run**.

Results show Opening Cash: TZS 6,800,000; Operating inflow: TZS 11,250,000; Investing outflow: TZS −4,200,000 (purchase of delivery van); Financing outflow: TZS −1,500,000 (loan repayment); Net Change: TZS 5,550,000; Closing Cash & Bank Balance: TZS 12,350,000. The green **Cash Tie-out: Reconciled** bar confirms the net change ties to the actual movement in the bank account GL balances.

Statement of Changes in Equity

What is the Statement of Changes in Equity? It is the fourth primary financial statement. Where the Balance Sheet shows the owners' equity at one date, this statement explains how it got from the opening figure to the closing one over a period: the profit earned, capital the owners put in, drawings and dividends they took out, and transfers between equity lines such as the year-end close moving the year's result into Retained Earnings. Auditors and banks expect it alongside the other three.

Navigate to **Accounting › Changes in Equity** (`/admin/reporting/changes-in-equity`). Permission required: `REPORT.BS.VIEW` (the same as the Balance Sheet, because its closing column *is* the Balance Sheet's equity). Export also requires `REPORT.EXPORT`.

1. Select the company by name.
2. Set **Period from** and **Period to**.
3. Click **Run**.

One row per equity component — each equity account (e.g. 3000 Owner's Equity / Capital, 3900 Retained Earnings) plus the two earnings lines the Balance Sheet shows (*Retained earnings — prior years (unclosed)* and *Current-year earnings*) — with these columns:

Column	What it holds
Opening	the Balance Sheet figure the day before the period starts
Profit for the period	the Income Statement's net profit for the same dates (on the current-year earnings line)
Opening balances posted	equity posted by opening-balance journals
Capital introduced & other credits	every other credit to the account — capital injected, and any other credit
Drawings, dividends & other debits	every other debit to the account, shown as a negative amount
Transfers	the year-end close (out of earnings, into Retained Earnings) and, when the period crosses the start of a financial year, last year's "current-year earnings" becoming "prior years". The column always totals zero.
Closing	the Balance Sheet figure on the last day of the period

The ledger records *which journal* moved an equity account, not *why*, so a capital injection and a correcting credit share one column — the column names say so.

A green **Ties to the Balance Sheet** bar confirms that opening plus every movement equals the Balance Sheet's equity at the period end. A component whose movements do not add up is flagged *does not add up* on its row, and the bar turns red with the difference — nothing is adjusted to make it tie.

The statement is company-wide (no branch filter). **Export:** PDF, Excel or CSV, named `changes-in-equity_<from>_<to>.<ext>`; the export footer repeats the tie-out result.

Financial Ratios

What are the financial ratios for? They turn the statements into a handful of numbers a manager or a bank can compare from one period to the next: can the business pay its short-term debts, how much of each sale it keeps, how geared it is, what return the owners earn, and how fast stock, customers and suppliers turn over.

Navigate to **Accounting › Financial Ratios** (`/admin/reporting/ratios`). Permission required: **both** `REPORT.PL.VIEW` and `REPORT.BS.VIEW` (the ratios disclose figures from both statements). Export also requires `REPORT.EXPORT`.

1. Select the company by name.
2. Set **Period from** and **Period to**.
3. Optionally pick a **Branch** (only branches you are assigned to are offered).
4. Click **Run**.

Every ratio is shown with its formula, the statement figures it was worked out from, and the result. The figures come from the Income Statement for the period and the Balance Sheet at its closing date and the day before it opens; "average" means (opening + closing) ÷ 2.

Ratio	Formula
Current ratio	Current assets ÷ Current liabilities
Quick ratio	(Current assets − Inventory) ÷ Current liabilities
Gross margin	Gross profit ÷ Revenue × 100
Net margin	Net profit ÷ Revenue × 100
Debt-to-equity	Total liabilities ÷ Total equity
Return on equity	Net profit ÷ Average equity × 100 (for the period, not annualised)
Inventory turnover	Cost of sales ÷ Average inventory
Inventory days	Average inventory ÷ Cost of sales × Days in period
Debtor days (DSO)	Average receivables ÷ Revenue × Days in period
Creditor days (DPO)	Average payables ÷ Cost of sales × Days in period

Inventory, receivables and payables are the accounts marked as Inventory, Accounts Receivable and Accounts Payable control accounts in the chart of accounts. Debtor days use total revenue because the ledger does not separate credit sales from cash sales (a shop with many cash sales shows fewer days than its credit customers really take); creditor days use cost of sales in place of credit purchases.

A ratio that cannot be worked out — because what it divides by is zero (no revenue, no cost of sales, no current liabilities…) — shows a dash and the reason, never 0.00. If the Income Statement or the Balance Sheet underneath fails its own check, a red warning says so.

For a branch, the ratios are that branch's and inherit the branch caveats above — in particular, inventory ratios are distorted by stock transfers. **Export:** PDF, Excel or CSV, named `financial-ratios_<from>_<to>.<ext>`.

Account-Ledger Drill-Down

What is the Account Ledger, and when do you use it? The Account Ledger shows every individual journal line posted to a single GL account within a date range, with a running balance. It is the most granular view available in the system: while the financial statements show totals and subtotals, the ledger shows the individual transactions behind each total. It is the primary tool for investigating a balance — for example, if Trade Receivables on the balance sheet is higher than expected, you open the ledger for that account to see every invoice and receipt that has been posted. The ledger is also the standard tool for preparing a bank reconciliation (compare the bank account ledger to the bank statement) and for answering auditor queries about specific transactions. The opening balance is the

account's position before the chosen date range, so every line in the report can be traced back to a source document.

Navigate to **Accounting › Account Ledger** (`/admin/reporting/account-ledger`). Permission required: `REPORT.LEDGER.VIEW`.

The account ledger shows every posted journal line for a single GL account within a date range, with a running balance.

1. Select the company by name.
2. In the **Account** picker (placeholder *Select account*), choose an account from the dropdown. Accounts are chosen by name; no uid is typed. The picker is a plain dropdown; a *Type to filter by name…* box appears above it only when the account list exceeds 12 entries.
3. Set **Period from** and **Period to**.
4. Click **Load Ledger** (the button carries a search icon).

The report shows:

- An **opening balance** (the account's balance as at the day before the from date).
- Every journal line in date order under the columns **Date**, **Source**, **Reference**, **Memo**, **Debit**, **Credit**, and **Balance** (the running balance). Negative running balances are shown in red.
- A **closing balance** (the account's balance at the end of the to date).

Pagination: if the account has more than 50 lines in the period, the shared paginator appears at the bottom. Navigate with the chevron icon buttons — first page, previous page, next page, and last page (their text is read out by screen readers via aria-labels) — and the numbered page buttons shown between them. The running balance carries across pages: the first line of each page continues from the last line of the page before, and the last line of the last page equals the closing balance.

Export: the export is bounded at 10,000 rows per download. If the period has more lines than that, the document lists the first 10,000 and then one line "*N further lines not listed (net movement)*" carrying the net of the rest, so the printed figures still add up to the closing balance. For very busy accounts spanning long periods, narrow the date range and export in segments. File is named `account-ledger_<accountCode>_<from>_<to>.<ext>`. Export requires `REPORT.EXPORT`.

Example — Investigate the bank account movements for April 2026:

1. Navigate to **Accounting › Account Ledger** (`/admin/reporting/account-ledger`).
2. Company: `Kijenge Trading Ltd`.
3. Account picker: select **Bank — Main Current (1100)** from the dropdown.
4. Period from: `2026-04-01`; Period to: `2026-04-30`.
5. Click **Load Ledger**.

Opening balance: TZS 12,350,000. The ledger shows 28 lines — 15 customer receipts credited and 13 payments debited, with a closing balance of TZS 14,890,000. The paginator is hidden (fewer than 50 lines). Rehema exports to CSV for the bank reconciliation working paper.

Trial Balance

The Trial Balance is covered fully in the Finance chapter (Accounting › Trial Balance, `/admin/gl/trial-balance`). It can also be reached from the Accounting navigation group. Permission required: `GL.VIEW`. See the General Ledger section for full usage.

Business Intelligence Dashboard

What is the BI Dashboard, and who uses it? The Business Intelligence Dashboard (the page is titled **Analytics Dashboard**) is a single-screen summary that composes key performance indicators (KPIs) from Finance, Operations, and CRM into one view. Rather than opening the income statement, then the AR list, then the stock valuation report separately, a finance director or general manager can open the dashboard and see the essential health indicators at a glance: is the trial balance balanced? Are the AR and AP sub-ledgers in agreement with the GL? How much cash is in the accounts? What is the current pipeline forecast? Each reconciled figure carries a health badge (green `[OK]` / red `[!]`) that instantly signals whether the underlying sub-ledger ties to the GL control account — a critical integrity check the finance team would otherwise have to perform manually. Drill-through links take the reader straight to the relevant detail screen. The dashboard is permission-gated at the panel level: a user with only operations permissions sees the stock panel but not the finance panels, and gets a calm "no permission" message for the panels they cannot access (ADR-0037).

Navigate to **Analytics › Dashboard** (`/admin/dashboard`). Permission required: `BI.VIEW`. Users who hold `BI.VIEW` also see a **Dashboard** card first on their home-page launchpad.

The dashboard is a composite view of key performance indicators drawn from Finance, Operations, and CRM data. Each panel has its own permission. If you hold `BI.VIEW` but lack a panel-specific permission, that panel shows a calm "no permission" message rather than blocking the whole page.

Who sees which panels (standard roles):

Role	Finance, Cash, Working Capital, Trends, Sales by Branch	Inventory	CRM	Export
Finance Director	Yes	Yes	Yes	Yes
Accountant	Yes	—	—	Yes
Branch Manager	Yes	Yes	—	Yes
Sales Manager	—	—	Yes	Yes

Procurement Manager	—	Yes	—	Yes

Root users see everything. A custom role sees exactly the panels whose permission it has been granted (see the panel list below).

Filters at the top of the page

- **Company** — a selector appears only if your organisation has more than one company; switching company reloads its branches, resets **Branch** to "All branches", and re-fetches the dashboard. With a single company it is selected automatically and no selector is shown.
- **Branch** — **All branches** (the default) or one branch, shown as `code — name`. The dashboard re-fetches as soon as you change it. Only the **CRM** panel and the **Sales by Branch** panel are narrowed by this filter. The Finance, Cash Position, Working Capital, Inventory, and Revenue and net profit panels are anchored to the GL at company level and always show company-wide figures — each of those panels carries a **Group-wide** tag, and while a branch is selected a short note under its heading reminds you: *Group-wide — not affected by the branch filter above.* The two panels that *do* follow the filter show a **This branch** tag instead.
- **The list offers only the branches you are assigned to** (in **Administration › Users**, branch assignments); root users see every branch of the company. If an assignment is removed while you have the page open and you then pick that branch, a yellow banner at the top explains: *You are not assigned to that branch. Choose a branch you work in, or clear the branch filter to see the whole company.* Choose one of your branches, or **All branches**.
- **From / To dates** — the reporting date range. **From** defaults to the first day of the current month and **To** defaults to today. Changing a date does **not** refresh on its own: click the circular **Refresh dashboard** button (the arrow-clockwise icon beside the To date) to re-fetch all panels.

The scope line. Once the data loads, a row of three grey tags appears under the filters, showing what the server actually filtered to: the **company name**, **Branch: …** (the branch name, or *All branches*), and the **period** (for example `2026-10-01 – 2026-10-02`). Read these tags rather than the pickers when you want to be sure what a figure covers. If they ever read *Branch: Unknown branch*, the branch you chose does not belong to this company, and the branch-filtered panels will be empty — choose the branch again.

Example — Read the dashboard KPIs and drill through to source screens:

Finance director Gideon Moshi logs in and opens **Analytics › Dashboard** (`/admin/dashboard`). The company `Kijenge Trading Ltd` is selected automatically, **Branch** is on **All branches**, and the dates default to the current month (2026-06-01 to 2026-06-14). The scope line reads `Kijenge Trading Ltd` · `Branch: All branches` · `2026-06-01 – 2026-06-14`.

1. **Health strip** — all five pills (TB, Cash vs GL, AR vs GL 1200, AP vs GL 2100, Stock vs GL 1300) show green `[OK]`. No reconciliation issues, so no `(diff: …)` figures appear.
2. **Finance panel** — Revenue: 9,850,000.00; OpEx: 4,200,000.00; Net Profit (period): 3,480,000.00 (shown green because it is positive). Small trend lines under Revenue and Net Profit show the last 12 fiscal periods at a glance. The Trial Balance card reads **Balanced**. Gideon clicks the drill icon in the **Finance** heading — this opens the Income Statement (`/admin/reporting/income-statement`) where he can run a full P&L; the **View TB** link on the Trial Balance card opens the GL trial balance.
3. **Cash Position panel** — Total Book Balance: 14,890,000.00 (the TZS accounts), with *Not converted: USD 2,500.00* underneath for the company's dollar account, a green **[OK] Cash-GL recon** pill, and a table listing each cash/bank account with its balance in its own currency. He uses the heading drill icon to open the cash & bank accounts list.
4. **Working Capital panel** — AR Outstanding: 19,700,000.00 (green **AR-GL** pill). AP Outstanding: 6,450,000.00 (green **AP-GL** pill). Below AR Outstanding a small line reads *Not converted: USD 1,200.00* — older US-dollar invoices that carry no reliable exchange rate, so they are listed separately instead of being added into the TZS figure. He clicks **View Receivables** to drill into the AR invoices list.
5. **Inventory panel** — Stock Value: 38,250,000.00, with a **Stock-GL (acct 1300)** pill reading **Reconciled**. He uses the heading drill icon to open the stock valuation screen.
6. **CRM panel** — **Pipeline by stage** shows 15 open deals across five stages, one bar per stage with its number of opportunities and value; **Win rate** reads 62.0% with a meter bar, above Won, Lost and Avg cycle; the **Forecast** block shows Open opportunities and a Weighted value of TZS 29,340,000.00. He uses the heading drill icon to open the CRM pipeline dashboard.
7. **Revenue and net profit** — one chart with two smooth lines over the last 12 fiscal periods (labelled `P1 2026`, `P2 2026`, …): Revenue in blue with a light shaded area, Net profit in orange. Gideon moves the pointer across the chart; a vertical line follows it and a small box shows both figures for that period. P6 shows net profit just below the zero line — a loss month.
8. **Sales by Branch panel** — with **Branch** still on "All branches", the panel lists every branch as a bar, in descending order of sales: `DSM — DSM Main` leads with 5,120,000.00 across 34 finalised invoices, followed by `ARU — Arusha Branch` with 2,890,000.00 across 19 invoices and the remaining branches, with a **Total** row of 9,715,000.00 across 61 invoices. (This total is the VAT-inclusive value of finalised sales invoices, so it will not match the Finance panel's Revenue, which is net of VAT and taken from the GL.) The drill icon in the heading opens the sales invoices list.
9. Gideon changes **Branch** to `ARU — Arusha Branch`. The dashboard re-fetches immediately. The scope line now reads `Branch: Arusha Branch`; the CRM and Sales by Branch headings show **This branch**, and the Sales by Branch table shows a single row for Arusha. The other panels still show **Group-wide** with the "not affected by the branch filter" note, and their figures are unchanged.
10. He selects **Excel** in the **Export** dropdown and clicks **Download**. The file `dashboard_2026-06-01_2026-06-14.xlsx` downloads; its first line reads *Branch: Arusha Branch*, so the file states its own scope. (Requires `BI.EXPORT`.)

KPI Panels

What are KPI panels? Each KPI panel on the dashboard is a self-contained summary of one operational or financial domain, sourced from the module that owns that data. The panels display figures that have already been computed by the underlying modules (the AR reconciliation query, the stock valuation query, the CRM pipeline query, etc.); the dashboard simply composes them into one screen. A health badge (`[OK]` or `[!]`) accompanies any figure that has a GL tie-out — it tells you at a glance whether the sub-ledger agrees with the General Ledger. A red badge is a prompt for the finance team to investigate before closing the period.

All amounts are shown with two decimals and no currency symbol unless stated; they are in the company's **base currency** (the scope line names the company).

Each panel has four possible states: a *Loading…* spinner; the figures; an empty message when there is nothing yet for the period (for example *No finance data yet for this period.*); or a message when you cannot see it — either *You do not have permission to view …* (missing panel permission; if the branch filter was refused, the yellow banner above says so) or *Could not load … data.* (a temporary fault — click **Refresh dashboard** to try again).

Health strip — a row of colour-coded status pills under the scope line, showing whether each sub-ledger reconciles with its GL control account. The pills are labelled by what they tie out: **TB** (trial balance), **Cash vs GL**, **AR vs GL 1200**, **AP vs GL 2100**, and **Stock vs GL 1300**. A green pill prefixed `[OK]` means the figures tie; a red pill prefixed `[!]` means there is a discrepancy, and it also shows the difference inline (for example, `[!] AR vs GL 1200 (diff: 1,250.00)`) so the finance team can see how far out the balance is. You only see the pills for the panels you are allowed to view.

Finance panel (requires `BI.FINANCE.VIEW`; Group-wide):

- **Revenue**, **OpEx** (operating expenses, excluding cost of sales), and **Net Profit (period)** for the selected date range — the same figures the Income Statement gives for those dates. Net profit is green when positive and red when negative.
- Under Revenue and Net Profit, a small **trend line** shows the last 12 fiscal periods, with a dot on the latest. It is there for the shape; the full figures are in the *Revenue and net profit* chart.
- A **Trial Balance** card with a status pill showing **Balanced** or **Out of balance** (whether total debits equal total credits), and a **View TB** link to the GL trial balance. The trial-balance check covers all postings to date, not just the selected range.
- The drill icon in the panel heading opens the Income Statement (P&L) report.

Cash Position panel (requires `BI.FINANCE.VIEW`; Group-wide):

- A **Total Book Balance**: the sum of the cash and bank accounts held in the base currency, with a **Cash-GL recon** pill (`[OK]` / `[!]`) showing whether the cash book ties to the GL.
- Accounts held in another currency are **not** added into that total, because a cash transaction records only its own-currency amount. They are summed per currency on a line underneath, for example *Not converted: USD 2,500.00, EUR 300.00*. The line only appears when you have such accounts.

- A table listing each cash/bank account (code and name) with its **Balance** in the account's own currency.
- The drill icon in the panel heading opens the Cash & Bank accounts list.

Working Capital panel (requires `BI.FINANCE.VIEW`; Group-wide):

- **AR Outstanding** with an **AR-GL** reconciliation pill, and a **View Receivables** link into the AR invoices list.
- **AP Outstanding** with an **AP-GL** reconciliation pill, and a **View Payables** link into the AP supplier-bills list.
- Both balances are current (as at today) and in base currency. Foreign-currency items count at the rate they were booked at. **Older foreign-currency items with no reliable rate** (raised before multi-currency was switched on) are not added into the base figure; they appear underneath as *Not converted: USD 1,200.00, EUR 300.00* so nothing is hidden. The same rule applies to the AR and AP reconciliation pills and to the customer and supplier balances described in the Finance chapter.

Inventory panel (requires `BI.OPS.VIEW`; Group-wide):

- **Stock Value** and a **Stock-GL (acct 1300)** pill showing whether the stock sub-ledger ties to the GL inventory account (**Reconciled**, or **Difference: …** with the figure when it does not tie).
- The drill icon in the panel heading opens the stock valuation screen.

CRM panel (requires `BI.CRM.VIEW`; follows the Branch filter):

- **Pipeline by stage**: one bar per stage, in pipeline order, labelled with the stage name, the number of open opportunities and their total value. This is the pipeline as it stands today — it does not follow the From / To dates.
- **Win rate**: the percentage in large type with a meter bar, then **Won**, **Lost**, and **Avg cycle** (days), for the selected date range.
- **Forecast**: **Open opportunities** and **Weighted value** (the probability-weighted pipeline value for the period).
- The drill icon in the panel heading opens the CRM Pipeline Dashboard.

Revenue and net profit (requires `BI.FINANCE.VIEW`; Group-wide):

- One line chart for the company's last 12 fiscal periods, labelled `P<period> <year>` (for example `P3 2026`): **Revenue** as a blue line with a light shaded area, **Net profit** as an orange line. Both are in base currency on the same scale; a darker line marks zero, so a loss month dips below it.
- **Reading a value:** move the pointer over the chart — a vertical line snaps to the nearest period and a box shows Revenue and Net profit for it. With the keyboard, Tab to the chart and use the **left / right arrow** keys (Home / End jump to the first / last period, Esc clears); screen readers announce each period.
- **Show table** (top right of the chart) switches to a table of every period with both figures; **Show chart** switches back.

- The chart does **not** follow the From / To dates — it always shows the last 12 periods set up in the fiscal calendar (so if next year's periods are already open, they appear at zero).

Sales by Branch panel (requires `BI.FINANCE.VIEW`; follows the Branch filter):

- One bar per branch for finalised sales invoices in the selected date range, highest-selling branch first. Each row shows the branch (`code – name`), its number of invoices, and its sales — each invoice's gross total *including VAT*, in base currency. The bar length is relative to the top branch. A **Total** line underneath gives the invoice count and sales for all rows shown.
- With **All branches** the panel shows every branch; selecting one branch narrows it to that branch's row.
- Only **FINALISED** invoices count, dated by when they were finalised; draft or voided invoices are excluded. Invoices raised in another currency count at their base-currency value.
- If no invoices were finalised in the period, the panel shows *No finalised invoices for this period.*
- The drill icon in the panel heading opens the sales invoices list (**Sales › Invoices**, `/admin/sales-invoices`).

Drill-Through

Each panel offers one or more drill links to the relevant detail screen: a small drill icon in the panel heading (Finance → Income Statement, Cash Position → Cash & Bank Accounts, Inventory → Stock Valuation, CRM → Pipeline Dashboard, Sales by Branch → Sales Invoices) plus text links inside the panels (**View TB**, **View Receivables**, **View Payables**). Working Capital and the two trend charts have no heading icon.

The target screen opens with its own default filters — it does not carry over the dashboard's dates or branch filter, so set them again there if needed. It also has its own permission; if you do not hold it, you are taken to an access-denied page.

Exporting the Dashboard

Requires both `BI.VIEW` and `BI.EXPORT`. The Finance Director, Accountant, Branch Manager, Sales Manager, and Procurement Manager roles have it as standard. Without it, no **Export** control is shown. The control sits just under the filters, above the panels.

1. Choose a format from the **Export** dropdown: **PDF** (the default), **Excel**, or **CSV**.
2. Click **Download**. The button reads *Exporting…* while the file is prepared.

The file is named `dashboard_<from>_<to>.<ext>` (for example `dashboard_2026-06-01_2026-06-14.pdf`, or `.xlsx` for Excel). It covers the same company, branch, and date range as the screen, and its first line states the branch (*Branch: All branches*, or the branch name).

What the file contains. Only the panels you are allowed to view, in this order:

- **Finance summary** — Revenue, Operating Expenses, Net Profit, and the trial-balance check.
- **Cash Position** — one line per cash/bank account, labelled with its currency (for example `CB-02 Dollar Account (USD)`); the **Total Cash Book Balance (TZS accounts)**; a *Not included above — USD accounts (in USD)* line for each foreign currency; and the Cash vs GL check.
- **Working Capital** — AR and AP outstanding with their GL checks, plus *AR / AP not converted (in USD)* lines when there are foreign items with no reliable rate.
- **Inventory** — stock value and its GL check.
- **CRM** — the pipeline value per stage, the KPIs, and the weighted forecast.
- **Revenue Trend** and **Net Profit Trend** — the 12 periods.
- **Sales by Branch (finalised invoices, incl. VAT)** — one line per branch with its invoice count, and a total.
- **Health Indicators.**

Amounts are in base currency unless the line names another currency in brackets.

If the export is refused or fails, a short message appears beside the button: *You don't have permission to export the dashboard.*, *The dashboard could not be exported with these filters. Check the dates and try again.*, or *Could not export the dashboard. Please try again.*

Analytical Reports

The following specialised reports sit under the **Budgeting** and **Costing** navigation groups but are described here because they are reporting outputs, not data-entry screens.

Budget Variance Report

What is the Budget Variance Report, and why is it produced? A budget variance report compares what the business planned to spend (or earn) against what actually happened. A variance is the difference: if you budgeted TZS 3,200,000 for fuel but spent TZS 3,850,000, the variance is TZS 650,000 **adverse** (worse than plan). For income accounts, spending more than budgeted is **favourable** (you earned more than expected). This report is the primary tool for **management by exception** — the finance team and department heads review it monthly to identify lines that have gone significantly off-plan and investigate why. It drives conversations about cost control, re-forecasting, and budget reallocation. For the report to show non-zero budget figures, at least one budget version covering the selected fiscal year and scope must have been **approved** (see Part 2 — Budgeting, in the HR, Budgeting, and Platform chapter).

Navigate to **Budgeting › Budget Variance Report** (`/admin/budgeting/variance`). Permission required: `BUDGETING.REPORT.VIEW`.

The report compares GL actuals against an approved budget version.

1. Select the company by name.
2. Choose the **Fiscal Year** from the picker. This is a name-picker dropdown (see *Common UI Patterns › Name pickers* in the Getting Started chapter): each option shows the year code, with

the year's status (`OPEN` / `CLOSED`) as a hint, and the placeholder reads *Select fiscal year*. You select a year code from the list — no UID is typed or pasted. The picker is **reloaded when you switch company**: changing the **Company** selector clears the prior fiscal-year selection and re-fetches the year list for the newly selected company. The field is required; running without a year shows the validation message *Fiscal year is required.*

3. Set the **From Period** and **To Period** (1–12; from must be ≤ to). These are pre-filled with **1** and **12** respectively, so by default the report covers the whole year — change them only to narrow the range.
4. Optionally filter by **Account type** (All account types, Asset, Liability, Equity, Income, Expense) and enter a **Cost Centre UID** (leave blank for all cost centres).
5. Click **Run Report**.

If no **APPROVED** budget version covers the selected scope, a yellow warning banner appears above the results — *No **APPROVED** budget version found for this scope — all budget amounts are zero* — and the budget columns show zero. Approve a budget version first (see Part 2 — Budgeting) to populate them.

The report then shows:

- A **header summary card** restating the Fiscal Year code, the period range (P*from* – P*to*), and the Cost Centre (or — *All* —).
- A **Totals by Account Type** summary table with Budget, Actual, and Variance columns per account type (Asset, Liability, Equity, Income, Expense).
- A **Detail** table of account-level rows: Code, Account, Type, Cost Centre, Budget, Actual, Variance, **Var %**, and an **Assessment** column carrying a **Favourable**, **Adverse**, or **On budget** label. For income accounts, actual > budget is favourable. For expense accounts, actual < budget is favourable.

Example — Run a budget variance report for the first half of the fiscal year:

Management accountant Yasmin Juma navigates to **Budgeting › Budget Variance Report** (`/admin/budgeting/variance`).

1. Company: `Kijenge Trading Ltd`.
2. Fiscal Year: she opens the **Fiscal Year** picker and selects `FY2026` from the dropdown — no UID is copied or pasted.
3. From Period: `1`; To Period: `6` (January through June — overriding the default 1–12).
4. Account type: `Expense` (to focus the board on cost discipline).
5. Click **Run Report**.

Results show that "Fuel & Transport" (actual TZS 3,850,000 vs budget TZS 3,200,000) is marked **Adverse** by TZS 650,000, while "Office Supplies" (actual TZS 480,000 vs budget TZS 600,000) is

Favourable by TZS 120,000. Yasmin notes the fuel over-run for discussion in the monthly management meeting.

Departmental Actuals Report

What is the Departmental Actuals Report? The Departmental Actuals Report shows real GL postings broken down by cost centre and account, without any budget comparison. It answers the question: "How much did each department actually spend on each expense type?" It is useful when a department manager wants to understand their spending in detail, or when the finance team needs to review allocations across departments without the distraction of a budget column. Cost centres are assigned to journal entries when transactions are posted; entries posted without a cost-centre tag appear as **Unallocated**.

Navigate to **Budgeting › Departmental Actuals** (`/admin/budgeting/departmental-actuals`). Permission required: `BUDGETING.REPORT.VIEW`.

Shows actual GL postings broken down by cost centre and account for the chosen fiscal year and period range.

1. Select the company by name.
2. Choose the **Fiscal Year** from the picker — the same name-picker used on the Budget Variance Report (year code in the list, status as a hint, *Select fiscal year* placeholder, no UID typed). It likewise reloads when you switch company, clearing any prior selection. The field is required.
3. Set the **From Period** and **To Period** (1–12; pre-filled with 1 and 12).
4. Click **Run Report**.

There is no Account-type filter or Cost Centre input on this report. The result shows a date-range card followed by a table with columns: **Cost Centre**, **Code**, **Account**, **Type**, **Normal Balance**, and **Actual (TZS)**. Entries posted without a cost-centre tag appear under **Unallocated**. This report has no budget comparison — it shows actuals only, useful for analysing spending by department or cost centre.

Dimension-Sliced Trial Balance

Navigate to **Costing › Sliced Trial Balance** (`/admin/cost-centre/report`). Requires both `COSTING.VIEW` and `GL.VIEW`.

See the General Ledger section (Cost-Centre Dimensions) in the Finance chapter for full usage instructions.

HR & Payroll, Budgeting, and Platform Services

This chapter covers three cross-cutting domains: the Human Resources and Payroll module (departments, employees, contracts, leave, loans, pay components, payroll runs, and statutory setup), the Budgeting module (budget creation, version lifecycle, line entry, and management reports), and the Platform services used by all other modules (document generation, notifications, the approval engine, and the audit trail).

Part 1 — Human Resources and Payroll

What is the HR & Payroll module, and why does it exist? The HR & Payroll module is "Accounts Payable for staff". Just as AP manages what the business owes to suppliers, payroll manages what it owes to employees. It brings together the employee master (who works here, on what terms), the statutory framework (what the government requires to be deducted and remitted — PAYE income tax, NSSF pension, HESLB loan repayments, WCF worker-compensation fund, SDL skills-development levy), voluntary deductions (loans, savings schemes), and the periodic calculation that produces a payslip and a balanced GL journal. Without a formal payroll system, salary payments are unstructured (prone to error and duplication), statutory obligations are hard to track (creating tax and compliance risk), and the cost of labour does not appear correctly in the profit and loss account. The module uses a run lifecycle (DRAFT → CALCULATED → APPROVED → POSTED → PAID) that enforces separation of duties: the person who calculates payroll is not the same person who approves or posts it (ADR-0032).

The HR & Payroll module is accessible from the **HR & Payroll** navigation group. What appears in that group depends on your permissions.

Permission requirements

Screen	View permission	Manage / act permission
Departments	`HR.EMPLOYEE.VIEW`	`HR.EMPLOYEE.MANAGE`
Employees	`HR.EMPLOYEE.VIEW`	`HR.EMPLOYEE.MANAGE`
Employee Contracts	`HR.EMPLOYEE.VIEW`	`HR.EMPLOYEE.MANAGE`
Leave Types	`HR.LEAVE.VIEW`	`HR.LEAVE.MANAGE`
Leave Requests	`HR.LEAVE.VIEW`	`HR.LEAVE.MANAGE` (submit), `HR.LEAVE.APPROVE` (decide)
Employee Loans	`HR.LOAN.MANAGE`	`HR.LOAN.MANAGE`
Pay Components	`HR.PAYCOMPONENT.MANAGE`	`HR.PAYCOMPONENT.MANAGE`
Payroll Runs	`HR.PAYROLL.VIEW`	`HR.PAYROLL.RUN` / `APPROVE` / `POST` / `DISBURSE` / `REVERSE`
Statutory Setup	`HR.STATUTORY.MANAGE`	`HR.STATUTORY.MANAGE`

A user holding none of the HR permissions will not see the **HR & Payroll** nav group and will be blocked from accessing any HR route directly.

Departments

What is a department, and why is it needed? A department is a logical grouping of employees within the company — for example Finance, Operations, or Sales. Departments serve two purposes. First, they appear on payroll reports and payslips, making it easy to see the cost of each part of the business. Second, they act as a cost-centre anchor: when payroll is posted to the General Ledger, the salary expense can be tagged with a department so that management accounts show the labour cost by business unit, not just as one undifferentiated total. Departments are company-level reference data that must be set up before employees can be registered.

Navigate to **HR & Payroll > Departments** (`/admin/hr/departments`).

Departments are company-level reference data. They are assigned to employees and appear on payroll reports.

Creating a department:

1. Click **New Department**.
2. Enter a **Code** (up to 30 characters) and a **Name** (up to 120 characters).
3. Click **Create Department**.

Editing a department: click the **Edit** action on the department row. The row turns into an inline edit form where both the **Code** and the **Name** can be changed; click **Save** to apply.

Deactivating a department: click **Deactivate** on the department row. The record is soft-deactivated, not deleted. Active employees in that department are not affected — the department reference is retained for historical records.

Employees

What is an employee record, and what is it used for? An employee record is the master data entry for a person employed by the company. It holds the information needed to calculate their pay (hire date, department, job title), satisfy statutory reporting requirements (national ID, TIN, NSSF number, HESLB number), and produce payslips. The system assigns an employee number automatically (`EMP-000001` format) that is used throughout HR and payroll screens. The employee record is created when the person joins and is archived (not deleted) when they leave, so that historical payroll records remain intact. Only one status is set at creation (ACTIVE); the only change available through the UI is archiving to TERMINATED.

Navigate to **HR & Payroll > Employees** (`/admin/hr/employees`).

The list shows employee number, name, job title, department name, and employment status. Use the paginator to navigate through large lists.

Creating an employee (minimum required fields):

1. Click **New Employee**.
2. Enter **First Name**, **Last Name**, and **Hire Date**.
3. Optionally fill in **Job Title**, **Gender**, **National ID**, **Department**, and **Branch**. **Department** and **Branch** are dropdowns — pick the department and branch by name from the lists (each defaults to "— none —" to leave unset). Departments must be set up first (see **Departments** above) for them to appear in the list.
4. Click **Create Employee**.

> TIN, NSSF number, HESLB number, and date of birth are not part of the create form; they are added later on the employee detail/edit page.

The system assigns an **employee number** automatically (format `EMP-000001`). The employee's status is set to **ACTIVE** on creation.

Viewing and editing an employee: click the **Open** action on the employee row to open the detail page. If you hold `HR.EMPLOYEE.MANAGE`, you can edit the employee's fields — including **Department** and **Branch**, both dropdowns as on the create form — and save changes.

Archiving an employee: on the employee detail page, click **Archive**. This changes the status to **TERMINATED** and marks the record inactive. The employee record is retained for historical and payroll purposes. There is no way to restore an archived employee through the UI — contact your system administrator if this is needed.

Employment status: only **ACTIVE** (on create) and **TERMINATED** (on archive) are reachable through the HR screens. The statuses ON_LEAVE and SUSPENDED exist in the system but cannot be set from these screens.

Employment Contracts

What is an employment contract, and why are contract types important? An employment contract records the formal terms under which a person is employed: their type of engagement, base salary, start date, and — for fixed-term arrangements — end date. The contract type (PERMANENT, FIXED_TERM, CASUAL, or PROBATION) matters for statutory compliance: permanent and confirmed employees are typically subject to full PAYE and NSSF deductions, while casual workers may be treated differently. The statutory flags on the contract (PAYE Resident, NSSF Member, HESLB Borrower, WCF Covered, SDL Counted) directly control which deductions and employer contributions are calculated during the payroll run. An employee can have at most one active contract at a time; when terms change (a salary review, a change from probation to permanent), the current contract is terminated and a new one is created — preserving the full history of contractual changes.

Navigate to **HR & Payroll > Employee Contracts** (`/admin/hr/contracts`).

The contracts screen is employee-picker driven: you choose an employee first and their contracts are listed in a panel below.

Viewing contracts for an employee:

1. Select the **Company** if shown.
2. In the **employee picker**, start typing the employee's name or number and select them.
3. The panel shows all contracts for that employee: contract type, base salary (TZS), start date, end date, and active status.

Creating a contract:

An employee may have at most one active contract at a time. Creating a second contract while one is active will be rejected.

1. With the employee selected, click **New contract**.
2. Choose the **Contract type**: PERMANENT, FIXED_TERM, CASUAL, or PROBATION.
3. Enter the **Base salary** (stored in TZS) and **Start date**.
4. For FIXED_TERM contracts, enter an **End date**.
5. Set the statutory flags: **PAYE Resident**, **NSSF Member**, **HESLB Borrower**, **WCF Covered**, and **SDL Counted**. These control which statutory deductions and employer contributions are applied during payroll calculation.
6. Click **Create Contract**.

The pay frequency is fixed at MONTHLY (v1). Currency is fixed at TZS.

Terminating a contract: click **Terminate** on the contract row. The contract becomes inactive (`active = false`). Once the active contract is terminated, a new one can be created for the employee.

Leave Types

What is a leave type, and why is it configured? A leave type is the named category of time off an employee can apply for — for example Annual Leave, Sick Leave, Maternity Leave, or Unpaid Leave. Each leave type carries the policy that governs requests of that kind: whether the leave is paid or unpaid, the annual entitlement (how many days an employee accrues per year), how that entitlement accrues, whether days carry forward to the next year, whether the type requires approval, any gender eligibility restriction, and an optional cap on the maximum number of consecutive days. Defining leave types up front means every leave request is applied against a consistent, auditable policy rather than ad-hoc rules. A default set of leave types is seeded for each company.

Where leave types are managed. Leave Types are company-level reference data managed by an administrator. There is no dedicated leave-types screen in this version of the UI — create, edit, and deactivate are performed through the leave-types API (`/api/v1/hr/leave-types`). Viewing requires `HR.LEAVE.VIEW`; creating, editing, and deactivating require `HR.LEAVE.MANAGE`.

Creating or editing a leave type: an administrator supplies:

- **Code** (up to 30 characters) and **Name** (up to 120 characters).
- **Paid** — whether days of this type are paid. If a type is **not** paid (unpaid leave), approved days that overlap a payroll period reduce the employee's basic salary pro-rata (see Leave Requests).
- **Annual entitlement days** and the **accrual method**.
- **Carry forward** — whether unused days roll into the next year.
- **Requires approval** — whether requests of this type must be approved.
- **Gender eligibility** — an optional restriction (for example, maternity leave).
- **Max consecutive days** — an optional cap on the length of a single request.

A leave-type **code** must be unique within the company; a duplicate code is rejected. Deactivating a leave type is a soft-deactivation — the type is retained for historical records but is no longer offered for new requests.

Leave Requests

What is a leave request, and how does it affect payroll? A leave request is the formal record of an employee's application for time off — annual leave, sick leave, maternity leave, or any other type configured by the administrator. The approval workflow (PENDING → APPROVED or REJECTED) ensures that time off is authorised before it is recorded as taken. For **unpaid leave** (where the leave type is flagged as unpaid), the approval has a direct financial consequence: approved unpaid leave days that overlap a payroll period automatically reduce the employee's basic salary pro-rata when the payroll run is calculated (the system uses 22 working days per month as the standard period). This ensures the payroll accurately reflects the actual days worked. Without a formal leave system, unpaid leave deductions would have to be applied manually, risking errors, disputes, and payroll miscalculations.

Navigate to **HR & Payroll > Leave Requests** (`/admin/hr/leave-requests`).

The list shows employee name, leave type, from and to dates, number of days, and status. Use the paginator for large lists.

Submitting a leave request (requires `HR.LEAVE.MANAGE`):

Leave types are managed by an administrator (see **Leave Types** below). When submitting a request you identify the leave type by its numeric ID, which the administrator can supply.

1. Click **Submit Leave Request**.
2. Pick the **Employee** by name.
3. Enter the **Leave Type ID** (the numeric id of the leave type). This is a free-text numeric field, not a dropdown; the resolved leave-type name is shown back to you in the list once the request is saved.
4. Enter **From** date, **To** date, and the number of **Days**.

5. Optionally enter a **Reason**.
6. Click **Submit**. The request status is set to **PENDING**.

Deciding a leave request (requires `HR.LEAVE.APPROVE`):

1. Open the leave request from the list (link goes to `/admin/hr/leave-requests/uid/:uid`).
2. In the **Decision** dropdown, choose **Approve** or **Reject**.
3. Optionally enter a **Decision Note** (the note is optional for both Approve and Reject).
4. Click **Submit Decision**.

The only valid decisions are **APPROVED** or **REJECTED**. PENDING and CANCELLED are not valid decision values and will be rejected.

Leave request statuses:

Status	Meaning
PENDING	Submitted, awaiting a decision
APPROVED	Approved; approved unpaid leave days reduce the employee's basic salary pro-rata in the payroll run
REJECTED	Declined
CANCELLED	Withdrawn before decision

Note on unpaid leave: if the leave type is marked as unpaid (configured by the administrator), approved leave days that overlap a payroll period will reduce that employee's basic salary pro-rata. The system uses 22 working days per month as the standard period.

Employee Loans

What is an employee loan, and how does repayment work? An employee loan is a cash advance made by the company to an employee, to be repaid through regular deductions from their net pay. Examples include salary advances, housing loans, or emergency personal loans. The loan record tracks the original principal, the agreed monthly instalment, and the outstanding balance. A new loan starts in **PENDING** status and is **not** deducted in payroll until it is approved and becomes **ACTIVE**. Once ACTIVE, the payroll calculation engine automatically includes the instalment as a deduction in each payroll run until the balance reaches zero — at which point only the remaining balance is deducted rather than the full instalment. This prevents payroll errors caused by forgetting to stop a deduction. The GL account linked to the loan records the outstanding balance on the balance sheet as an asset (money owed to the company by the employee).

Navigate to **HR & Payroll > Employee Loans** (`/admin/hr/loans`).

The list shows loan number, employee name, principal, installment amount, outstanding balance, start date, and status. Viewing and managing loans both require `HR.LOAN.MANAGE`.

Creating a loan:

1. Click **New Loan**.
2. Pick the **Employee** by name.
3. Enter the **Principal** and the monthly **Installment** (the installment must not exceed the principal).
4. Choose the **Currency** from the Currency Picker. This is the filtered currency picker (only the company's enabled currencies are listed, pre-set to the company default) — see *Common UI Patterns* in Chapter 00 (Getting Started). You no longer type a 3-letter currency code.
5. Enter the **GL Account ID** — the numeric id of the GL account the loan is posted to. (This field is a numeric id entry; obtain the id from your administrator. An unknown id is rejected.)
6. Enter the **Start Date**.
7. Click **Create**. The loan is created in **PENDING** status with its outstanding balance equal to the principal.

Approving a loan:

1. Open the loan from the list (`/admin/hr/loans/uid/:uid`).
2. Click **Approve Loan**. The loan status changes from PENDING to **ACTIVE**.

Approval is only valid for a loan in PENDING status; attempting to approve a loan that is already ACTIVE (or SETTLED/CANCELLED) is rejected. Once ACTIVE, the loan installment is automatically deducted from the employee's net pay during each payroll calculation. If the outstanding balance is less than the installment, only the remaining outstanding amount is deducted.

Loan statuses:

Status	Meaning
PENDING	Created but not yet approved; **not** picked up by payroll
ACTIVE	Approved; the installment is deducted in each payroll run until settled
SETTLED	Fully repaid
CANCELLED	Voided

SETTLED and CANCELLED statuses exist in the system but can only be set by the system administrator — there is no Settle or Cancel button on the UI in this version.

Pay Components

What is a pay component, and why is it needed? A pay component is a named earning or deduction that is applied to employees during payroll calculation — for example "Housing Allowance" (an earning), "Medical Scheme Contribution" (a deduction), or "Transport Allowance" (an earning calculated as a percentage of basic salary). Pay components allow the payroll engine to handle the variety of terms in employment contracts without hard-coding allowances or deductions into the

system. Each component is configured once (with its GL account, its basis — fixed amount or percentage of basic salary — and its tax/pension flags) and then assigned to specific employees as recurring items. This ensures that every employee's payslip is built from a consistent, auditable set of named items rather than ad-hoc adjustments.

Navigate to **HR & Payroll > Pay Components** (`/admin/hr/pay-components`).

Pay components define the earnings and deductions applied to employees during payroll calculation. They are company-level reference data. The list is not paginated. Viewing and managing pay components both require `HR.PAYCOMPONENT.MANAGE`.

Creating a pay component:

1. Click **New Pay Component**.
2. Enter a **Code** and a **Name**.
3. Set the **Kind**: EARNING (adds to gross) or DEDUCTION (reduces net).
4. Set the **Basis**: FIXED (a fixed amount per run) or PERCENT_OF_BASIC (a percentage of the employee's basic salary).
5. Enter the **GL Account ID** — earnings and deductions post to this account. This is a free-text numeric-id entry (placeholder "Numeric id"), not a name picker; obtain the id from your administrator.
6. Check **Taxable** if this component is subject to PAYE.
7. Check **Pensionable** if this component is included in the pension-contribution base.
8. Click **Create**.

Editing and deactivating: open the component by clicking the **Open** action on its row (`/admin/hr/pay-components/uid/:uid`). Edit the fields and save, or click **Deactivate** to soft-deactivate the component (it becomes inactive and will no longer appear in payroll calculations going forward).

Per-employee recurring items (the amounts for PERCENT_OF_BASIC components and any fixed amounts applied to specific employees) are configured by the administrator directly in the system. These are applied automatically during payroll calculation and do not have a separate UI screen.

Payroll Runs

What is a payroll run, and what does it produce? A payroll run is the process of computing every employee's pay for a given month and producing the payslips, the GL journal, and the cash disbursement that physically pays the employees. The run gathers all relevant inputs — base salaries from contracts, deductions from approved unpaid leave, loan instalments, voluntary pay-component items — and applies the current statutory rates (PAYE bands, NSSF rates, HESLB rates, WCF, SDL) to produce a balanced journal entry and a payslip for every employee. The lifecycle (DRAFT → CALCULATED → APPROVED → POSTED → PAID) enforces a four-eyes check: one person prepares and calculates, a second person approves, a third posts to the books, and a fourth authorises the actual

payment. A POSTED run can be reversed if an error is found after posting. Only one run can be active per period — you cannot accidentally pay the same month twice.

Navigate to **HR & Payroll > Payroll Runs** (`/admin/hr/payroll-runs`).

A payroll run computes gross pay, statutory deductions, voluntary deductions, and loan repayments for all employees with an active contract in a given period. The list shows each run's number, period, pay date, status, and gross and net totals.

Payroll run lifecycle:

```
DRAFT → CALCULATED → APPROVED → POSTED → PAID
                                          ↓
                                      REVERSED
```

Each step requires a different permission. Only one active run can exist per period.

Step 1 — Create a run (requires `HR.PAYROLL.RUN`):

1. Click **New Payroll Run**.
2. Enter the **Year**, choose the **Month** from the dropdown, and enter the **Pay Date**. (The company is taken from your active session.)
3. Optionally enter a **Branch ID** if the run covers a specific branch. This is a free-text field (placeholder "Optional"), not a name picker.
4. Click **Create**. The run is created in **DRAFT** status with zero totals.

Step 2 — Calculate (requires `HR.PAYROLL.RUN`):

1. Open the run (`/admin/hr/payroll-runs/uid/:uid`).
2. Click **Calculate**. The system builds one payroll line per ACTIVE employee who has an ACTIVE contract:

    - Basic salary earning.
    - PAYE income tax (from the effective PAYE band set for the pay date, if `payeResident = true`).
    - NSSF deduction (employee share, if `nssfMember = true`).
    - HESLB deduction (if `heslbBorrower = true`).
    - Employer contributions (NSSF/WCF/SDL employer shares from the effective statutory rate sets).
    - Any voluntary pay-component recurring items configured for the employee.
    - Loan repayment deductions for any ACTIVE loans with an outstanding balance.
    - Pro-rata reduction for any approved unpaid leave overlapping the period.

3. The run status moves to **CALCULATED** and the **Payroll Lines** table populates.

You can recalculate from DRAFT, CALCULATED, or APPROVED status — recalculation rebuilds all lines from scratch.

Reviewing lines: the **Payroll Lines** table lists each employee's line. A line showing a **FLAGGED** badge means the line needs attention before approval — typically because the employee's net pay is negative after deductions, or because a payment target (payee/bank details) is missing for that employee. The reason is shown in the line's **Flag Reason** column. You must resolve flagged lines before the run can be approved — for example by reducing a loan installment, supplying the missing payee details, and then recalculating.

Step 3 — Approve (requires `HR.PAYROLL.APPROVE`):

1. With the run in CALCULATED status and zero FLAGGED lines, click **Approve**.
2. Status moves to **APPROVED**.

Step 4 — Post (requires `HR.PAYROLL.POST`):

1. With the run in APPROVED status, click **Post to GL**.
2. Status moves to **POSTED**. The GL journal is written asynchronously via the payroll posting handler. Payslips are generated (one per employee line).

Step 5 — Disburse (requires `HR.PAYROLL.DISBURSE`):

1. With the run in POSTED status, click **Disburse**.
2. Enter the **Cash / Bank Account UID** of the account from which the net wages will be paid. (This is a UID text field on this screen; obtain the account UID from your administrator or the Chart of Accounts.)
3. Optionally enter a **Transaction Date** (defaults to the run's pay date).
4. Click **Disburse**. Status moves to **PAID**. A Cash & Bank OUT entry is recorded (debit Net Wages Payable, credit the chosen bank/cash account), booked to the payroll run's branch. Net Wages Payable (2550) is a locked payroll control account — you cannot post to it with a manual journal or a direct cash entry — but disbursement is a system settlement and is allowed to clear it.

Reversing a run (requires `HR.PAYROLL.REVERSE`):

A POSTED or PAID run can be reversed if needed (for example, a posting error). Click **Reverse** on the run detail. The status moves to **REVERSED** and a reversing GL journal is posted.

Legal action matrix:

From status	Calculate	Approve	Post	Disburse	Reverse
DRAFT	Allowed	Blocked	Blocked	Blocked	Blocked
CALCULATED	Allowed (recalc)	Allowed (if no FLAGGED)	Blocked	Blocked	Blocked
APPROVED	Allowed (recalc)	Blocked	Allowed	Blocked	Blocked
POSTED	Blocked	Blocked	Blocked	Allowed (if net > 0)	Allowed
PAID	Blocked	Blocked	Blocked	Blocked	Allowed
REVERSED	Blocked	Blocked	Blocked	Blocked	Blocked

Download bank file (requires `HR.PAYROLL.DISBURSE`):

Once a run is **POSTED** (and still after it is **PAID**), the run detail shows **Download bank file**. It downloads a CSV with one row per employee — employee number and name, payment method, bank name, account name, account / mobile-money number, net pay and currency — ready to upload to your bank. The button is not shown before posting (net pay can still change) or for a reversed run, and the system itself refuses the file for any other status — so a bank file can never be produced for an unapproved or reversed run, even outside this screen. If the download is refused (for example because the run was reversed in another window after you opened it), a short message explains why; refresh the page to see the run's current status.

Statutory Summary on the run (requires `HR.PAYROLL.VIEW`):

From CALCULATED onwards the run detail shows a **Statutory Summary**: totals for **PAYE** and **SDL** (paid to TRA), **NSSF** employee + employer, **WCF** and **HESLB**, and a per-employee table with each employee's **TIN** and **NSSF number** (blank where none was captured), gross, each deduction/contribution and net. While the run is not yet approved the summary is marked **Provisional** — do not file it; a reversed run is marked likewise. Holders of `REPORT.EXPORT` also see **Export PDF / Excel / CSV**.

Payroll Statutory Report

Navigate to **HR & Payroll > Payroll Statutory Report** (`/admin/reports/payroll-statutory`). Requires `HR.PAYROLL.VIEW`; exporting also requires `REPORT.EXPORT`.

Choose a **Pay date from / to** range and click **Run report**. The report lists one row per payroll run whose **pay date** falls in the range — period, pay date, status, number of staff, gross, PAYE, SDL, NSSF (employee and employer), WCF, HESLB and net — with totals, plus three summary figures: **Due to TRA (PAYE + SDL)**, **Due to NSSF (employee + employer)** and **Employer cost (NSSF + WCF + SDL)**.

Only **APPROVED**, **POSTED** and **PAID** runs are counted, because their figures are final. Runs still in DRAFT or CALCULATED, and REVERSED runs, are left out; a note under the table says how many, so nothing is silently missing. Click a run number to open that run.

> **Note for administrators:** the seeded *HR Payroll Manager* role holds `HR.PAYROLL.VIEW` but not `REPORT.EXPORT`, so it sees the reports but not the export buttons until `REPORT.EXPORT` is granted.

Statutory Setup

What is the Statutory Setup, and why are the rates held in updatable tables? Statutory setup holds the tax bands and levy rates mandated by Tanzanian law: PAYE (Pay As You Earn income tax), NSSF (National Social Security Fund pension contributions), HESLB (Higher Education Students' Loans Board repayments), WCF (Workers' Compensation Fund), and SDL (Skills and Development Levy).

These rates are set by the government and change periodically with each budget announcement. Because they are stored as **effective-dated data** in the system — not hard-coded in software — the administrator can add a new rate set with a future effective date when a budget announcement is made, and the payroll engine will automatically apply the correct rates when the pay date falls in the new period. This means a payroll run always reproduces exactly what the law required on that pay date, regardless of subsequent rate changes. Without effective-dated rate tables, every budget announcement would require a software update to change hard-coded constants.

Navigate to **HR & Payroll > Statutory Setup** (`/admin/hr/statutory`). Requires `HR.STATUTORY.MANAGE`.

The statutory setup screen shows two sections: **PAYE band sets** and **Statutory rate sets**. These sets determine how income tax and levies are calculated during payroll calculation.

Creating a PAYE band set:

What is a PAYE band set? A PAYE band set is a schedule of income tax rates that applies a progressive rate to different slices of monthly income. For example, the first TZS 270,000 per month might be tax-free, the next slice taxed at 9%, the next at 20%, and so on. Each band defines the lower income threshold at which the rate starts and the cumulative tax already payable on income up to that threshold (to avoid re-computing all lower bands for every employee). The system selects the most recently effective band set whose effective date is on or before the payroll run's pay date, ensuring the correct bands apply to each period.

1. Click **New Band Set**.
2. Enter an **Effective From** date, a **Tax-Free Threshold** (the monthly income amount below which no PAYE applies), and an optional **Description**.
3. Add one or more bands. Each band requires: band number (ascending), lower bound (monthly income where this rate starts), **Marginal Rate (%)** (entered as a percentage, e.g. `20` for 20%; the field accepts 0–100), and cumulative fixed tax (the tax already accumulated on income up to this band's lower bound).
4. Click **Create Band Set**.

The system uses the **most recently effective** band set whose effective date is on or before the payroll run's pay date.

Creating a statutory rate set:

What is a statutory rate set? A statutory rate set holds the percentage rates for one of the non-PAYE levies: NSSF, WCF, SDL, or HESLB. Each set records the employee rate, the employer rate (where applicable), the basis for the calculation (gross salary or basic salary), and — for SDL — a headcount threshold (SDL only applies to companies above a minimum employee count). Like PAYE band sets, rate sets are effective-dated so that rate changes can be scheduled in advance without software updates.

1. Click **New Rate Set**.

2. Choose the **Type**: NSSF, WCF, SDL, or HESLB.
3. Enter the **Effective From** date and the **Basis** — a free-text field (placeholder "e.g. GROSS").
4. Enter the applicable rates: **Employee Rate (%)** and/or **Employer Rate (%)**, entered as percentages (e.g. `20` for 20%; each field accepts 0–100).
5. Optionally enter a **Ceiling Amount** (the income cap above which the rate no longer applies).
6. For SDL, enter a **Headcount Threshold** (SDL applies only when the company headcount equals or exceeds this number).
7. Click **Create Rate Set**.

Contract statutory flags control which rate sets apply to each employee:

- `NSSF Member` → NSSF deductions apply.
- `PAYE Resident` → PAYE income tax applies.
- `HESLB Borrower` → HESLB deduction applies.
- `WCF Covered` → the employer WCF contribution is included for this employee.
- `SDL Counted` → the employee is counted towards SDL (the employer SDL levy applies to the run if an effective rate set exists and the SDL headcount threshold is met).

Part 2 — Budgeting

What is a budget, and how does the module work? A budget is a forward-looking financial plan: it states how much the business expects to earn and spend in a future period, account by account. It exists to give management a target to work towards, a benchmark to compare against actual results, and a tool for anticipating cash needs. Budgets in this system are **planning records only** — they never post to the General Ledger. Instead, the approved budget lines are held separately and compared at report time against actual GL postings, producing the **Budget Variance Report** (how far actuals diverged from plan). The system supports multiple **versions** of a budget so that the business can revise the plan during the year without losing the original, and each version goes through an approval lifecycle (DRAFT → SUBMITTED → APPROVED) to ensure the plan is authorised before it is used as a benchmark (ADR-0034).

The Budgeting module is accessible from the **Budgeting** navigation group. Budgets are planning tools only — they do not post to the General Ledger. GL actuals are read at report time for comparison purposes.

Permission requirements

Action	Permission
View budgets, versions, lines	`BUDGETING.BUDGET.VIEW`
Create and edit budgets, versions, lines	`BUDGETING.BUDGET.MANAGE`
Submit and recall versions	`BUDGETING.BUDGET.SUBMIT`
Approve and reject versions	`BUDGETING.BUDGET.APPROVE`
Run variance and actuals reports	`BUDGETING.REPORT.VIEW`

Creating a Budget

Navigate to **Budgeting > Budgets** (`/admin/budgets`). Requires `BUDGETING.BUDGET.MANAGE`.

What is a budget header? The budget header is the container for all the planning work. It identifies the fiscal year being budgeted and the scope — either the whole company, or a specific cost centre (a department or business unit). You create one budget per fiscal year per scope, and within it you manage one or more versions as the plan evolves. A company-wide budget covers all income and expense accounts; a cost-centre-scoped budget covers only the activity attributed to that centre.

A budget covers a specific fiscal year and may be scoped to a specific cost centre (dimension value) or set as company-wide.

1. Click **New Budget**.
2. Enter a **Name** for the budget.
3. Choose the **Fiscal Year** from the Fiscal-Year picker. This is a dropdown of the company's fiscal years (each option shows the year code with its status); select the year you are budgeting for. You no longer type a Fiscal Year UID.
4. Optionally enter a **Version label** (the label for the first version) and **Notes**.
5. Click **Create Budget**.

The system creates the budget and automatically creates **Version 1** in DRAFT status. There can be only one budget per fiscal year and cost-centre scope combination.

Cost-centre scope. The Create Budget screen creates company-wide budgets only — it does not expose a cost-centre field. A cost-centre-scoped budget cannot be created from this screen in this version; if you need one, contact your system administrator.

The budget list shows each budget's number, name, fiscal year, cost centre, latest version status, and the number of versions. A **Status filter** at the top narrows the list, and a pager appears at the bottom for long lists.

Budget Versions and the Version Lifecycle

What is a budget version, and why are multiple versions needed? A budget version is a specific iteration of the plan. The first version (V1) is the original budget prepared at the start of the year. If actual events require the plan to be revised — a new product launch, an unexpected cost increase, a change in strategy — a new version (V2, V3, etc.) is created. The version lifecycle ensures that each revision is authorised before it replaces the previous plan: the preparer submits the version for approval, the approver reviews and approves or rejects it, and only one version is APPROVED (active) at any time. All prior approved versions are moved to SUPERSEDED (kept for reference) when a new one is approved. Rejected versions are kept but cannot be used as a benchmark; a new version must

be created to revise after a rejection. Lines can only be edited on DRAFT versions — once submitted, the plan is locked.

Each budget can have multiple versions representing revisions to the plan. Versions go through an approval cycle before becoming the active plan.

Version statuses:

Status	Meaning
DRAFT	Under construction — lines can be edited
SUBMITTED	Submitted for approval — lines are locked
APPROVED	The active approved plan; supersedes any prior approved version
REJECTED	Declined — create a new version to revise
SUPERSEDED	A previously approved version replaced by a newer APPROVED version

Version lifecycle transitions:

```
DRAFT → SUBMITTED (submit, requires ≥1 line)
SUBMITTED → DRAFT (recall)
SUBMITTED → APPROVED (approve — also supersedes the prior APPROVED version)
SUBMITTED → REJECTED (reject, reason required)
```

APPROVED, REJECTED, and SUPERSEDED are terminal — no further edits or lifecycle actions can be taken on a version in these states. To re-plan after a rejection, create a new version.

Opening the budget detail:

Click the **Open** action on the budget row to open its detail (`/admin/budgets/uid/:uid`). The detail shows the budget header and all versions, listed newest first. Each version row shows its version number ("V1", "V2", etc.), label, status badge, and line count.

Creating a new version (Re-plan):

1. On the budget detail, click **New Version / Re-plan**.
2. Optionally enter a **label** for this version.
3. To start from a prior version's lines, pick the source version from the **Seed from version** picker by its version label and status (e.g. "V1 — FY2026 base"). Leave blank to start with an empty version.
4. Click **Create**. The new version is created in DRAFT status.

Entering Budget Lines

What is a budget line? A budget line is the atomic planning unit: it links one GL account to one fiscal period and states the planned amount for that account in that period. For example, a line might say

"Account: 5400 Fuel & Transport, Period: March 2026, Amount: TZS 3,200,000". The sum of all lines for an account across all periods is that account's annual budget. Lines are stored at the period grain (month by month) so that the variance report can show monthly deviations, not just annual totals. Lines can only be added, changed, or deleted when the version is in DRAFT status.

Open the version detail by clicking **View Lines** on the version (`/admin/budget-versions/uid/:uid`). The lines table shows account, period, amount (TZS), and memo. Lines are editable only when the version is in DRAFT status.

Click **Edit Lines (Replace All)** to open the line editor. Choose one of three entry modes:

DIRECT — enter each line individually:

1. Click **Add line**.
2. In the **Account** picker, choose the GL account by name.
3. In the **Period** picker, choose the fiscal period. Each option is labelled in the format `P{number} (start date – end date)` — for example "P3 (2026-03-01 – 2026-03-31)" — with the period's status shown as a hint.
4. Enter the **Amount** in TZS (must be ≥ 0).
5. Optionally enter a **Memo**.
6. Repeat for additional lines.
7. Click **Replace Lines**. The new lines replace all prior lines for this version.

ANNUAL_SPREAD — enter an annual total and spread it evenly across 12 periods:

When is this useful? When the budget planner knows the full-year target for an account but does not want to apportion it manually month by month. The system splits the annual amount into 12 equal monthly lines, using HALF_UP rounding and adding any cent-level residual to the last period so that the 12 lines sum exactly to the annual total.

1. In the **Account** picker, choose the GL account by name.
2. Enter the **Annual amount** in TZS.
3. Click **Replace Lines**. The system creates 12 lines (one per period), spreading the annual amount as evenly as possible (HALF_UP rounding; any remainder is added to the last period so the sum equals the annual total exactly).

The fiscal year must have exactly 12 periods to use ANNUAL_SPREAD mode.

SEED — copy lines from another version:

When is this useful? When creating a revised budget version that starts from the same lines as a prior version. Rather than re-entering all lines from scratch, you seed from V1 and then edit only the accounts that have changed. This also works across fiscal years: you can seed V1 of the FY2027 budget from the approved V2 of FY2026 as a starting point.

1. In the **Seed from version** picker, choose the source version by its label and status.

2. Click **Replace Lines**. All lines from the source version are copied to this version, replacing any existing lines.

Note: editing lines is blocked when the version is not in DRAFT status. If you need to edit a SUBMITTED version's lines, recall it to DRAFT first.

Submitting, Approving, and Rejecting Versions

Submit (requires `BUDGETING.BUDGET.SUBMIT`):

1. On the budget detail, click **Submit for Approval** next to the DRAFT version.
2. The version must have at least one line. If it has no lines, submission is rejected.
3. Status moves to SUBMITTED. Lines are locked.

Recall (requires `BUDGETING.BUDGET.SUBMIT`):

If you need to revise a SUBMITTED version, click **Recall to Draft** to return it to DRAFT. The submission timestamp is cleared and lines become editable again.

Approve (requires `BUDGETING.BUDGET.APPROVE`):

1. Click **Approve** next to the SUBMITTED version.
2. Optionally enter an approval note.
3. Confirm.

The version status moves to APPROVED. If there was a previously APPROVED version for the same scope, it is automatically moved to SUPERSEDED. Only one version can be APPROVED at any time per scope.

Reject (requires `BUDGETING.BUDGET.APPROVE`):

1. Click **Reject** next to the SUBMITTED version.
2. Enter a **rejection reason** (required).
3. Confirm.

The version status moves to REJECTED. The rejection reason is recorded for reference. To re-plan, create a new DRAFT version.

Budget Reports

Both reports require `BUDGETING.REPORT.VIEW` and are accessible from the **Budgeting** nav group.

Budget Variance Report (`/admin/budgeting/variance`):

Compares the approved budget lines against actual GL postings for the selected period range.

1. Select the **Company**.
2. Choose the **Fiscal Year** from the Fiscal-Year picker (a dropdown of the company's fiscal years; no UID is typed). Switching company clears the fiscal-year selection and reloads the picker for the newly selected company.
3. Set the **From Period** and **To Period** (1–12; from must be ≤ to). These default to 1 and 12.
4. Optionally filter by **Account type** (Income, Expense, Asset, Liability, Equity) and enter a **Cost Centre UID** to limit results to a specific centre.
5. Click **Run Report**.

The report shows account-level rows with budget amount, actual amount, variance (actual − budget), a variance percentage, and a Favourable/Adverse assessment, plus a Totals-by-Account-Type summary. For income accounts, actual > budget is favourable. For expense accounts, actual < budget is favourable.

If no APPROVED version exists for the selected scope, the report is returned with all budget amounts as zero and an on-screen "No APPROVED budget version found for this scope" warning banner — the report is never silently wrong.

Departmental Actuals Report (`/admin/budgeting/departmental-actuals`):

Shows actual GL postings grouped by cost centre and account, with no budget comparison. Useful for monitoring departmental spending.

1. Select the **Company**.
2. Choose the **Fiscal Year** from the Fiscal-Year picker (a dropdown of the company's fiscal years; no UID is typed). As on the Variance report, switching company clears and reloads the picker.
3. Set the **From Period** and **To Period**.
4. Click **Run Report**.

A null cost centre (transactions posted without a cost-centre dimension) appears as an **Unallocated** row.

Part 3 — Platform Services

What are Platform Services? Platform services are cross-cutting capabilities that every other module uses — they are not specific to Finance, HR, or Operations. Document generation produces the printable PDFs from data that already exists in the system. Notifications tells users what has happened that they need to act on. The approval engine intercepts high-value actions and routes them through a human authorisation chain. The audit trail records every state-changing action so that nothing can be silently altered. These services are the governance and communication spine of the ERP.

Platform services provide cross-cutting functionality used by all modules: document generation and management, notifications, the approval engine, and the audit trail.

Documents

What is the Document Generation module, and what does it produce? The Document Generation module renders formally formatted, branded PDF documents from transactions that already exist in the system. A sales invoice stored in the AR module, a purchase order in the Procurement module, or a goods receipt in the Inventory module all contain the data for a printable document, but that data is not yet in the layout a customer or supplier expects to receive. This module reads the source transaction as a read-only snapshot, merges it with the company's branding (logo, address, bank details, footer text), applies the chosen template, and produces a download-ready PDF. The generated document is stored in a log for audit purposes — you can re-download a document issued months ago without regenerating it. Every render is append-only: the log is never edited, and rendering a document never changes the source transaction. The six supported types in v1 are: Invoice, AR Statement, Purchase Order, Goods Receipt, Delivery Note, and Credit Note (ADR-0023).

Generated Documents Log

Navigate to **Documents > Generated Documents** (`/admin/documents`). Requires `DOCUMENT.VIEW`.

The log lists every document that has been rendered for the active company, with document number, type badge, source, and generated-at timestamp. Use the **Type** filter dropdown to narrow results by document type (Invoice, AR Statement, Purchase Order, Goods Receipt, Delivery Note, Credit Note). The **Render Document** button opens the render form, and each row carries **View** and **Download** actions.

Rendering a Document

Requires `DOCUMENT.RENDER`. The render form is on the same Generated Documents screen.

Six document types can be rendered in v1:

Document type	Source
Invoice	A finalised sales invoice
AR Statement	A customer (with from/to date range)
Purchase Order	A confirmed purchase order
Goods Receipt	A received GRN
Delivery Note	A delivery record
Credit Note	A sales return/credit

To render a document:

1. Click **Render document**.
2. Choose the **Document type** from the dropdown.
3. For all types except AR Statement: pick the **source record** by its number (invoice number, PO number, etc.) in the source picker.

4. For AR Statement only: enter the **customer** (chosen by name) and the **from date** and **to date** in the params section.
5. Click **Render**. A new row appears in the log.

To download a rendered document, click the **Download** button on the log row or detail page (also requires `DOCUMENT.RENDER`).

Document Templates

What is a document template? A document template controls the layout and structure of a rendered document type. The system ships with a default template for each of the six renderable types. The template can be activated or deactivated — deactivating it prevents new renders of that type. Template content (the actual layout formatting) is maintained by the system administrator; the UI allows you to toggle the template's status and update its display title.

Navigate to **Documents > Document Templates** (`/admin/document-templates`). Requires `DOCUMENT.TEMPLATE.MANAGE`.

The template registry lists one row per renderable document type. You can change the template's **title** and toggle its **status** (ACTIVE or INACTIVE) by clicking the row and saving. Deactivating a template does not delete it.

Document Branding

What is Document Branding? Document branding is the per-company configuration that controls what appears in the header and footer of every rendered PDF. Without branding, a PDF would carry no company name, address, tax ID, or bank details — it would be unacceptable as a formal document. The branding profile is a single set of settings per company (a "singleton"): there is no list to navigate, just one form that you edit and save. Changes take effect immediately on all subsequent renders; previously generated documents are not retroactively changed (the log is append-only).

Navigate to **Documents > Document Branding** (`/admin/document-branding`). Requires `DOCUMENT.BRANDING.MANAGE`.

The branding profile is a per-company singleton (one set of settings per company, no list). It controls what appears in the header and footer of rendered PDF documents.

1. Open the screen. The current branding values load into the form.
2. Edit: **Display name**, **Legal name**, **Tax ID**, **Address**, **Contact phone**, **Contact email**, **Website**, **Footer terms text**, and **Bank details**.
3. Click **Save**.

Changes take effect on all subsequent document renders. Previously generated documents are not changed (the log is append-only).

Notifications

What is the Notifications module, and how does it work? The Notifications module is the system's alerting spine. It listens for events that other modules emit — a payment received, an approval submitted, a payroll posted — and delivers an in-app message (and optionally an email) to the users who need to know. It also runs a scheduled background scanner for time-based conditions that have no single event trigger (such as an invoice becoming overdue overnight, or stock falling below its reorder level). Without notifications, users must actively poll every module to find out what has happened; notifications inverts this by pushing relevant information to the right person at the right time. Each notification type has an audience defined by permission (for example, an approval-submitted notification goes to all users who hold the approver role), and each user can customise their preferences — muting types they do not need, or disabling email delivery for types they prefer to see only in-app (ADR-0024).

Notification Inbox

Navigate to **Notifications > Inbox** (`/admin/notifications`). Requires `NOTIFICATION.VIEW`.

The inbox shows notifications sent to you within the active company, with title, message body, severity badge (INFO / WARNING / CRITICAL), and created-at timestamp.

- Toggle **Unread only** to filter to unread items only.
- Click **Mark read** on an individual row to mark it as read.
- Click **Mark all read** to clear the unread count at once.

The shell navigation bar shows a badge with the count of unread notifications. The badge reflects `GET /api/v1/notifications/unread-count`.

Notification Preferences

Navigate to **Notifications > Preferences** (`/admin/notification-preferences`). Requires `NOTIFICATION.PREFERENCE.MANAGE`.

Each notification type listed here can be tuned per user:

- **Muted** — suppress all delivery for this type for you.
- **Channels enabled** — choose which channels (IN_APP, EMAIL) should deliver this type to you.

Click a preference row, adjust the settings, and save.

Notification Type Catalogue (Admin)

Navigate to **Notifications > Type Catalogue** (`/admin/notification-types`). Requires `NOTIFICATION.ADMIN`.

The catalogue lists all notification types registered in the system for the active company. Each type shows display name, audience permission, severity, default channels, and the company-level enabled/disabled toggle.

To disable a notification type company-wide (all users in the company stop receiving it), click the row, toggle **Company enabled** to off, and save. Any delivery attempts while the type is disabled will be recorded in the delivery log with suppression reason **COMPANY_TYPE_OFF**.

Notification Delivery Log (Admin)

Navigate to **Notifications > Delivery Log** (`/admin/notification-deliveries`). Requires `NOTIFICATION.ADMIN`.

The delivery log shows every notification delivery attempt with its outcome and, if suppressed, the reason. Use the **Channel** and **Outcome** filters to diagnose delivery problems.

Suppression reason	Meaning
MUTED	The recipient muted this type
CHANNEL_DISABLED	The channel (EMAIL/SMS) is not active
NO_EMAIL	The recipient has no email address configured
COMPANY_TYPE_OFF	The type is disabled for this company
NO_AUDIENCE	No eligible recipients found

Approvals

What is the Approval Engine, and why is it a shared platform service rather than module-specific? The approval engine is a generic governance layer that intercepts certain high-value actions across the system and requires one or more human sign-offs before the action proceeds. Examples include confirming a large purchase order, posting a payroll run, or approving a budget version. Rather than each module building its own approval screen (which would lead to inconsistent behaviour and duplicate maintenance), the approval engine is a single shared service that any module can delegate to. A policy defines when approval is needed (which document type, above what monetary threshold, at which branch) and who must approve (which role). When a matching action is submitted, the engine creates an approval request, routes it to the appropriate approvers in sequence, and releases the action only when all steps are completed. If no policy matches, the action is auto-approved instantly. The engine posts nothing to the books; its sole purpose is to gate other modules' actions (ADR-0022).

Approval requests are created automatically by the relevant modules — there is no "create approval request" screen.

Approval Policies

What is an approval policy? An approval policy defines the rule that triggers human approval: it says "for documents of type X, in the amount band [min, max), at scope Y, require approval from role Z". A policy can be company-wide (applies to all branches) or scoped to a specific branch (a branch-scoped policy takes priority when both match). The amount bands within a policy type must not overlap, and there can only be one active policy per (type, scope, band) combination — this ensures that every submission matches exactly one policy, making the outcome deterministic.

Navigate to **Approvals > Approval Policies** (`/admin/approvals/policies`). Requires `APPROVALS.POLICY.VIEW`.

Policies define when human approval is required and who must approve. Each policy targets one document type and a monetary band.

Creating a policy (requires `APPROVALS.POLICY.MANAGE`):

1. Click **New policy**.
2. Enter a **Name** and choose the **Document type** (e.g. Purchase Order).
3. Set the **Branch scope**:
    - **COMPANY_WIDE** — applies to the entire company. Do not enter a branch.
    - **BRANCH** — applies to one specific branch. Pick the branch by name from the branch picker. A branch-scoped policy takes priority over a company-wide policy when both match the same request.
4. Set **Min amount** and **Max amount** (TZS). Leave **Max amount** blank for an unbounded top band (applies to all amounts ≥ Min amount).
5. Add one or more **Approval steps**. Each step has a **sequence number** (dense from 1) and an **Approver role code** (the permission role whose holders will see this request in their inbox). Click **Add step** to add another level of approval.
6. Click **Save**.

Policy changes only affect future submissions. In-flight PENDING requests continue under the policy that existed when they were created.

Deactivating a policy: open the policy detail (`/admin/approvals/policies/uid/:uid`), click **Deactivate**. Status moves to INACTIVE. Inactive policies are not matched on new submissions.

Approval Inbox (My Inbox)

What is the Approval Inbox? The inbox shows every approval request that is currently waiting for your decision — specifically, requests where the current open step is routed to one of your permission roles. It is the daily working screen for managers, finance directors, and senior staff who hold approver roles. You see only the requests assigned to your role; you do not see requests routed to other roles. Approving moves the request to its next step (or resolves it if it was the final step); rejecting ends the entire request immediately and marks all remaining steps as skipped.

Navigate to **Approvals > My Inbox** (`/admin/approvals/inbox`). Requires `APPROVALS.DECIDE`.

The inbox shows PENDING requests whose current open step is routed to one of your roles. These are the requests waiting for your decision. When nothing is awaiting you, the screen shows an "Your inbox is empty" message.

1. Click a request to open its detail.
2. Review the request: document type, amount, submitter, submission date, and step chain.
3. Click **Approve** (and optionally add a comment) or **Reject** (comment required).

4. Confirm.

When you approve a step:

- If this was the last step in the chain, the request moves to **APPROVED** and the originating document is released.
- If there are further steps, the request remains PENDING and the next step becomes the active one.

When you reject a step, the whole request moves to **REJECTED** and all remaining steps are marked **SKIPPED**.

All Requests

Navigate to **Approvals > All Requests** (`/admin/approvals/requests`). Requires `APPROVALS.REQUEST.VIEW`.

Shows all approval requests for the active company regardless of who the approver is. Filter by status (PENDING / APPROVED / REJECTED / RECALLED / CANCELLED) to narrow the list.

Request statuses:

Status	Terminal?	Meaning
PENDING	No	Awaiting a decision on the current open step
APPROVED	Yes	All steps approved (or auto-approved)
REJECTED	Yes	A step was rejected; remaining steps skipped
RECALLED	Yes	Withdrawn by the submitter
CANCELLED	Yes	Cancelled by an administrator

Terminal requests cannot be acted on further — all action buttons are hidden on a terminal request detail.

Recalling a request (requires `APPROVALS.REQUEST.VIEW`, submitter only):

The original submitter can recall their own PENDING request from the All Requests list. Open the request detail and click **Recall**. Status moves to RECALLED.

Cancelling a request (requires `APPROVALS.ADMIN`):

An administrator can cancel any non-terminal PENDING request. Open the request detail and click **Cancel**. Status moves to CANCELLED.

Audit Trail

What is the Audit Trail, and why is it append-only? The audit trail is the system's immutable record of every state-changing action — who did what, to which record, when, and from where. It is the primary tool for investigating a suspicious change, resolving a dispute ("who approved this payment?"), and satisfying auditor and regulatory requirements for a documented chain of custody. The audit trail is append-only: no record can be deleted or edited, not even by a system administrator. This property is fundamental to its integrity — an editable audit trail is no audit trail at all. Every module writes to the same audit trail so that you can search across the entire system in one place.

Navigate to **Audit** (`/admin/audit`). Requires `AUDIT.VIEW`.

The audit trail is a read-only, append-only log of all state-changing actions in the system. No records can be added, edited, or deleted from the UI.

The list shows:

- **Actor** (username)
- **Action** (event code, e.g. `USER.CREATE`, `PAYROLL.POST`, `APPROVAL.STEP.DECIDE`)
- **Target type and identifier**
- **Company and branch**
- **Timestamp** and IP address

Rows are sorted newest first (default page size 50, maximum 200 per page).

Filters:

- **Actor** — choose a user from the user picker by name.
- **Action** — enter or select an action code.
- **Target type** and **Target UID** — filter to records affecting a specific resource.
- **From date** and **To date** — filter by time range.

Non-root users are confined to their own active company's audit rows. The system administrator (`rootadmin`) can view audit records across all companies.